O'REILLY®

# Transformers

## The Definitive Guide

### Applications Beyond NLP

Nicole Koenigstein

"Most transformer books stop at language. Nicole Koenigstein keeps going—through vision, audio, video, time series, and RL—and makes the shared architecture feel inevitable rather than incidental. The production insights on KV caching and deployment trade-offs are especially hard-won."

**Mike Erlihson**
Head of AI, DriveNets

"The focus on reasoning, alignment, and system-level optimization captures exactly where the next wave of AI capabilities is heading. This book stands out for making these topics both accessible and highly actionable."

**Alma Thomas, MBA**
Enterprise data architect and principal data engineer, Americo Financial Life and Annuity Insurance Company

# Transformers: The Definitive Guide

The vast potential of AI technology remains untapped in areas like audio, video, and complex data analysis. In fact, many of today's professionals find it challenging to apply AI innovations across these diverse domains due to a lack of guidance and practical implementations.

This comprehensive guide, tailored especially for intermediate to advanced ML engineers, data scientists, and researchers, fills the gap. Author Nicole Koenigstein guides readers through the versatile applications of transformer models, not only deepening theoretical understanding but also emphasizing actionable strategies for real-world applications. The book equips you with a grand unified theory for transformers—foundational insights that keep you on the cutting edge regardless of state-of-the-art model evolutions.

You'll discover how to apply:

- **Transformers in nontext domains like image, video, and music generation**
- **Reasoning models, coding agents, and multi-agent architectures**
- **Training-time and test-time optimization strategies**
- **Production deployment, runtime engineering, and hardware efficiency**

**Nicole Koenigstein** is an AI researcher and practitioner in agentic systems, working across research, consulting, teaching, and direct system implementation to build reliable, production-ready AI systems. She is the author of several books, including *AI Agents: The Definitive Guide* (O'Reilly, 2026), and has served as an external evaluator for a European Commission AI Grand Challenge.

DATA

US $79.99 CAN $99.99
ISBN: 978-1-098-16701-1

O'REILLY®

# Transformers: The Definitive Guide

*Applications Beyond NLP*

*Nicole Koenigstein*

O'REILLY®

**Transformers: The Definitive Guide**
by Nicole Koenigstein

Copyright © 2026 Nicole Koenigstein. All rights reserved.

Published by O'Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*http://oreilly.com*). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

**Acquisitions Editor:** Nicole Butterfield
**Development Editor:** Sarah Grey
**Production Editor:** Elizabeth Faerm
**Copyeditor:** Piper Content Partners
**Proofreader:** Dave Awl
**Indexer:** BIM Creatives, LLC
**Cover Designer:** Susan Brown
**Cover Illustrator:** José Marzan Jr.
**Interior Designer:** David Futato
**Interior Illustrator:** Kate Dullea

April 2026: First Edition

**Revision History for the First Edition**
2026-03-25: First Release

See *http://oreilly.com/catalog/errata.csp?isbn=9781098167011* for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Transformers: The Definitive Guide*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-16701-1

[LSI]

# Table of Contents

**Preface** ..... ix

**1. From First Principles to State-of-the-Art Transformers** ..... 1
Transformer Basics 2
Tokenizer: Text Representation in the Transformer 2
Token and Positional Embeddings 3
Attention Mechanism 5
Encoder and Decoder Parts 7
Enhancements in Transformer Design:
Longer Context and Attention Variations 9
Longer Context Windows with Better Performance 9
Attention Mechanism Variations 16
Conclusion 25

**2. Transformers for Time Series** ..... 27
Understanding the Intricacies of Time Series Data 28
Autocorrelation and Partial Autocorrelation 28
Cointegration 29
Cross-Correlation 30
Stationarity 31
Trend and Seasonality 31
Preparing a Dataset 32
Time Series Modeling in Various Application Domains 32
Tokenizing Time Series Data 33
Chronos: Learning the Language of Time Series 38
Fine-Tuning Chronos 40
PatchTST: A Time Series Is Worth 64 Words 43
Fine-Tuning PatchTST on Historical IBM Stock Prices 45

TimesFM: A Decoder-Only Time Series Foundation Model 48
Fine-Tuning TimesFM on Hourly Energy Consumption Data 53
AnomalyBERT for Self-Supervised Anomaly Detection 55
Conclusion 58

**3. Transformers for Vision Tasks. . . . . 59**
Overview of Different Vision Tasks 60
Embeddings and Tokenization for Vision Models 61
Key Strategies for Improving the Robustness and Effectiveness of Vision Tasks 64
Swin Transformer V2 66
Image classification with Swin Transformer V2 68
Segment Anything 72
Fine-Tuning SAM on a Custom Dataset 76
Segment Anything in Images and Videos 83
Segment Videos and Images with Concept Prompts 88
Conclusion 95

**4. Transformers for Image Generation. . . . . 97**
Introduction to Generative Image Models 98
Diffusion Models: What's That Noise About? 99
Classifier-Free Guidance in Diffusion Models 101
Scalable Diffusion Models with Transformers 102
Generating Images with the DiT 106
PIXART-α 108
PixArt-Σ 111
Generating Images with PixArt-Σ 115
Diffusion Vision Transformers for Image Generation 120
Interpretable Features with Diffusion Transformers 123
Conclusion 127

**5. Transformers for Video Generation. . . . . 129**
Hidden Effectiveness of Latent Diffusion 130
LTX-Video: Video in Realtime 132
Latte: Structured Detail, Poured into Every Video Frame 137
Tora: From Trajectory to Storyline, One Frame at a Time 140
Conclusion 148

**6. From Sound to Token and Back: Transformers in the Audio Domain. . . . . 149**
From Waveforms to Spectrograms:
Understanding the Structure of Audio Data 151
Audio as a Waveform 151

Sampling Rate and the Nyquist Theorem 152
Amplitude, Bit Depth, and Quantization 152
The Frequency Domain and Fourier Transform 153
Spectrograms and the Short-Time Fourier Transform 154
The Mel Spectrogram and Perceptual Scaling 155
Phase, Reconstruction, and Vocoders 155
Audio Modeling in Various Application Domains 159
Transformer Architectures for Audio:
From Perception to Foundational Intelligence 160
The Rise of Speech Transformers: The Impact of Whisper 160
Audio Foundation Models: Unifying Understanding, Generation, and Conversation 163
Qwen2-Audio 164
Transcribing a Meeting with Kimi-Audio 168
Segment Anything in Audio 174
Beyond Text and Speech: Transformers as Music Composers 177
Conclusion 181

**7. Reinforcement Learning Transformers. . . . . 183**
Getting Started with Reinforcement Learning 183
Foundational Concepts in Reinforcement Learning 185
Online and Offline Reinforcement Learning 185
Model-Based and Model-Free Approaches 186
On-Policy Versus Off-Policy Reinforcement Learning 186
Temporal Difference Learning 188
World Models in Reinforcement Learning 188
Transformers in Reinforcement Learning 189
Decision Transformer 190
Going Live: Online Decision Transformer 196
A Brave New World: Stochastic Transformer-Based World Model 199
TWISTER: Transformer-Based World Models with Contrastive Predictive Coding 204
Conclusion 211

**8. Embracing the Era of Experience: Transformers for Planning, Reasoning, and Coding 213**
From Human Data to Lived Experience 214
Learning to Reason: From Pretraining to Reinforcement Learning 216
DeepSeek-R1: Reinforcing Reasoning Capabilities 217
Qwen3: Unified Reasoning with Dynamic Control 220
Qwen3-Coder: Agentic Reasoning for Open-Ended Coding 225
Kimi K2: Open Agentic Intelligence at Scale 227
Muon: Scaling Optimization for the Agentic Era 228

Inference with Kimi K2 229
Scaling Reasoning at Test-Time: Smarter, Not Just Bigger 230
Adaptive Branching Monte Carlo Tree Search (AB-MCTS) 232
The RethinkMCTS Framework for Code Generation 238
The *S** Framework for Code Generation 239
Conclusion 241

**9. From Scripts to Thinking: AI Agents for Complex Tasks. . . . . 243**
Autonomy: What's Possible at the Moment? 244
Designing Agent Workflows 246
Multi-Agent Architectures 249
Agentic Communication: The Right Context Is All You Need 255
Beyond Context: How to Help Agents Remember 258
Agent Memory Types 258
Going Global and Lifelong 259
The Human Factor: Steering Agent Actions 261
Common Patterns for Human-in-the-Loop 261
Solving GitHub Issues with Coding Agents 268
Conclusion 271

**10. Smarter, Better, Faster, Stronger: Optimizing LLMs and AI Agents. . . . . 273**
Training-Time Intelligence: Reinforcement Learning for Agents 274
Beyond Hand-Crafted Rewards: How RULER Works 275
Training in Practice: ART in a Market Scenario 276
Reason Smarter, Not Harder: Adaptive Compute Allocation 285
The Delta Incentive: Enforcing Efficiency 285
Open Innovation: Community-Driven RL Frameworks 287
The Checkpoint Engine: Systems-Level Optimization for LLM Policy Updates 297
Conclusion 299

**11. Deploying Transformer Models. . . . . 301**
Choosing Between Open and Closed Source 302
Understanding the Architecture You're Deploying 303
Deploying Decoder-Only Models 305
Runtime Engineering for Decoder-Only Models 306
Security Considerations for Decoder-Only Deployments 307
Building Applications with Coding Models 308
Evaluating LLM Deployments in Production 313
Cost Efficiency and Hardware Comparison 314
Quantization 320
Test-Time Low-Rank Adaptation in Vision-Language Models 326
Conclusion 328

**12. Where to Go Next: From Models to Intelligent Systems. . . . . 331**
Combining Capabilities: SAM 3 Agent 333
The Science of Scaling Agentic Systems 339
Conclusion 341

**Index. . . . . 343**

# Preface

When I first started working with transformers in 2019, I was immediately drawn to them. Two years later, I built my own deep learning architecture based on attention, and that same year I taught my first university lecture on vision transformers. What struck me then, and continues to do so today, is just how versatile this architecture really is. Once you understand how transformers work, patterns begin to emerge. You start to see how the same principles apply across images, video, audio, and beyond. The deeper insight is not found in treating each domain as a separate problem, as is often the case in deep learning, but in recognizing how naturally the transformer architecture extends across domains.

As it turns out, it doesn't matter whether you're modeling discrete tokens, image patches, or spectrogram frames. The core logic remains the same. You may change the axis of representation, but the underlying architecture stays intact. That's the elegance of transformers, at least to me, and the reason I decided to write this book: to share my passion for transformers and let you discover that elegance at your own pace.

What also continues to fascinate me is what happens when transformers are used as part of a larger system. Their real power shows up when you combine multiple models into an agent that can reason, act, and iterate on a task. Beyond simply connecting models, my recent focus has been on pairing transformers with test-time compute and reinforcement learning. Together, these ideas make it possible to build systems that adapt, improve, and learn from their own behavior. That's what led me to focus my PhD research on self-improving AI agents.

As you read through this book, you'll follow both the development of transformers across different domains and my own path working with them, up to my most recent work on building AI agents. Along the way, I'll share how I personally think about these models—the mental models I use to reason about architectures, trade-offs, and system behavior and how those models shape the design decisions I make in practice. My goal isn't just to show you what works but to help you see why it works.

By the end of the book, my hope is that you arrive at the same place I did: seeing transformers not as isolated models but as building blocks inside larger systems. Transformers aren't just another architecture. They're a framework for abstraction. That's why they aren't a passing trend but a shift in how we generalize and connect modeling across domains. Arthur C. Clarke once said:

> Any sufficiently advanced technology is indistinguishable from magic.

I hope, as you read through the book, that it gives you the tools to turn the "magic" of attention into a reliable, engineered reality.

## What This Book Is About

This book is a practical systems guide for building intelligent transformer-powered applications across modalities, written by someone who has designed, debugged, and deployed these architectures in real products.

You'll start with first principles: tokenization, embeddings, attention, encoder and decoder design, and modern architectural enhancements for long-context reasoning. From there, the book expands across domains, showing how the same core transformer abstraction applies to time series forecasting, computer vision, image and video generation, audio understanding, reinforcement learning, and reasoning-driven coding systems.

Rather than treating each modality in isolation, the chapters build on shared structure: how tokens become patches, frames, or spectrogram slices; how attention adapts across domains; and how architectural choices propagate into performance, stability, and cost.

Later chapters move into reinforcement learning, test-time compute, reasoning models, and AI agents, connecting transformers with world models, planning algorithms, and multi-agent architectures. The goal is to help you understand how transformers evolve from standalone predictors into components of adaptive systems.

Beyond individual models, this book focuses on systems-level thinking. You'll learn how transformers behave when deployed in production, how to optimize inference and training pipelines, how to manage memory and compute, and how to integrate models into agentic workflows that can plan, reflect, and improve over time.

Throughout the book, theory is paired with concrete implementations, production considerations, and design trade-offs drawn directly from real deployments.

## What This Book Is Not

This is not an introductory deep learning book or an introduction to large language models. I assume you already understand neural networks, backpropagation, basic machine learning workflows, and large language models. While core concepts are explained, the focus is on how transformers operate across domains and in practice.

This is also not a prompt engineering guide or a collection of model or API recipes. Although modern foundation models appear throughout the book, the emphasis is on understanding fundamentals. Model names are ephemeral. Instead, this book teaches the abstraction behind multimodal systems. My goal is to give you a kind of grand unified theory for transformers that remains relevant regardless of which model happens to be state of the art. Along the way, it focuses on training dynamics, optimization, and deployment, not on crafting prompts or consuming hosted endpoints.

This book is also not an academic exercise. While I include math where it's needed, I ensure not to overindulge. You'll see equations and architectural details throughout the book; they're always paired with practical engineering explanations. If you're looking for exhaustive theoretical proofs or benchmark-only discussions, this book takes a different, more applied approach.

## Who This Book Is For

This book is tailored for intermediate to advanced machine learning (ML) engineers, data scientists, and AI architects who are ready to look beyond language models. It's for those who find themselves at the intersection of research and engineering: people who need to understand the why of an architecture to solve the how of a production problem.

Specifically, I wrote this for:

- The architect who needs to design systems that handle data beyond just text, such as video and image generation, image and audio classification, or time-series forecasting
- The engineer tasked with moving a model from a local notebook into a high-performance production environment and handling the production "gotchas," such as spiking memory usage while optimizing key-value (KV) cache in decoder-only models
- The systems thinker interested in how individual transformers can be coordinated into AI agents that learn from their own test-time experience

Throughout these chapters, I don't shy away from the complexity and math that a technical book demands. However, I am a firm believer that technical complexity should always serve a practical purpose and be accompanied with clear coding examples. I share production insights: hard-won lessons I've gathered from building attention-based architectures and deploying them in the wild, so you don't have to. These are the nuances that research papers often leave out but that make the difference between a model that works in a demo and one that survives in production.

## Navigating This Book

The book is organized to move from foundations to increasingly complex systems.

Chapter 1, "From First Principles to State-of-the-Art Transformers", establishes transformer fundamentals, including tokenization, embeddings, attention, encoder and decoder architectures, and modern enhancements for long-context modeling.

Chapter 2, "Transformers for Time Series", applies transformers to time series, covering domain-specific challenges such as stationarity and autocorrelation, and introduces foundation models for forecasting and anomaly detection.

Chapter 3, "Transformers for Vision Tasks", focuses on vision tasks, including classification and segmentation, with practical examples using modern vision transformers.

Chapter 4, "Transformers for Image Generation", and Chapter 5, "Transformers for Video Generation", move into generative modeling for images and video, exploring diffusion transformers and latent representations for scalable generation.

Chapter 6, "From Sound to Token and Back: Transformers in the Audio Domain", covers audio, from waveform fundamentals to speech, multimodal audio models, and music generation.

Chapter 7, "Reinforcement Learning Transformers", introduces reinforcement learning with transformers, including decision transformers and world models.

Chapter 8, "Embracing the Era of Experience: Transformers for Planning, Reasoning, and Coding", shifts toward reasoning and coding, examining how transformers learn to plan, reason, and solve open-ended problems using reinforcement learning and test-time compute.

Chapter 9, "From Scripts to Thinking: AI Agents for Complex Tasks", builds on this by introducing AI agents, multi-agent architectures, memory systems, and human-in-the-loop workflows.

Chapter 10, "Smarter, Better, Faster, Stronger: Optimizing LLMs and AI Agents", focuses on optimization, including reinforcement learning for agents, adaptive compute, and systems-level training strategies.

Chapter 11, "Deploying Transformer Models", addresses deployment: runtime engineering, quantization, evaluation, security, and cost-aware production design.

Chapter 12, "Where to Go Next: From Models to Intelligent Systems", looks forward, integrating models into intelligent systems and discussing how agentic architectures scale.

Each chapter is largely self-contained, so you can read selectively depending on your interests. However, the material builds conceptually, and reading sequentially provides the strongest mental model.

Now it's your turn. From first principles and multimodal foundations to reinforcement learning, agentic systems, optimization, and deployment, the system is yours to design. Transformers are tools, and the chapters ahead are built around one goal: reliability beyond the demo stage.

The difference between a model that impresses in a notebook and one that survives in production lies in the details, in architectural choices, memory constraints, training dynamics, evaluation strategy, and system design. This book gives you those details. What you build next is where your own journey with transformers begins.

## Conventions Used in This Book

The following typographical conventions are used in this book:

*Italic*
: Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`
: Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

This element signifies a tip or suggestion.

This element signifies a general note.

This element indicates a warning or caution.

## Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at *https://oreil.ly/github-transformers*.

If you have a technical question or a problem using the code examples, please send email to *support@oreilly.com*.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Transformers: The Definitive Guide* by Nicole Koenigstein (O'Reilly). Copyright 2026 Nicole Koenigstein, 978-1-098-16701-1."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at *permissions@oreilly.com*.

## O'Reilly Online Learning

For more than 40 years, O'Reilly Media has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit *https://oreilly.com*.

## How to Contact Us

Please address comments and questions concerning this book to the publisher:


O'Reilly Media, Inc.
141 Stony Circle, Suite 195
Santa Rosa, CA 95401
800-889-8969 (in the United States or Canada)
707-827-7019 (international or local)
707-829-0104 (fax)
*support@oreilly.com*
*https://oreilly.com/about/contact.html*


We have a web page for this book, where we list errata and any additional information. You can access this page at *https://oreil.ly/transformers-the-definitive-guide*.

For news and information about our books and courses, visit *https://oreilly.com*.

Find us on LinkedIn: *https://linkedin.com/company/oreilly*.

Watch us on YouTube: *https://youtube.com/oreillymedia*.

## Acknowledgments


Writing a book or pursuing a career in a field such as AI is never truly a solitary effort. This feels especially true today, as the pace of progress in AI continues to accelerate, and the exchange of ideas becomes increasingly important, even though much of the work itself still happens during long, solitary days of research, coding, and writing. I would like to take this moment to thank the people who supported and inspired me at important moments throughout this journey.

In this spirit, I would like to thank colleagues, peers, students, readers, and course participants who engage with my work. Through discussions, questions, collaborations, and a shared curiosity about AI, these exchanges provide perspective and inspiration that shape how ideas develop and mature over time.

Special thanks go to Nicole Butterfield for her continued support during some of the more challenging moments while finishing this book. I would also like to express my appreciation to the rest of the team at O'Reilly who helped guide this book from manuscript to publication. In particular, I am grateful to Sarah Grey and Liz Faerm for their thoughtful support throughout the development process and for making my work as an author both easier and genuinely enjoyable. I would also like to thank the many members of the production team whose work behind the scenes helps bring a book like this to life.

Finally, I would like to thank the technical reviewers, Chris Fregly, Al Krinker, and Dibyendu Roy Chowdhury. Your careful review, questions, and feedback helped clarify important points and strengthen the overall quality of the book.

CHAPTER 1

# From First Principles to State-of-the-Art Transformers

Since its introduction in 2017, the transformer architecture[1] has revolutionized the field of natural language processing (NLP), marking a paradigm shift toward models capable of natural language understanding (NLU). This shift was possible because transformers process sequential data in parallel, enabling a deeper and more contextual understanding of language than was achievable with previous sequential models, like long short-term memory (LSTM) networks.

In recent years, transformers have evolved to impact a wide array of domains, including computer vision, speech recognition, reinforcement learning, and mathematical operations, moving beyond their initial usage within NLP. Their adaptability has led to significant advancements in machine translation, allowing for context-aware translations, and in scientific research, notably in predicting protein structures with remarkable accuracy.

Among the most exciting developments are reasoning models, which are advanced large language models (LLMs) trained with reinforcement learning to perform complex, multi-step reasoning. They generate internal chains of thought before answering, which is inspired by the human thought process. This technique first solves intermediate steps before getting to the final answer.

I assume in this book that you have at least some familiarity with the transformer architecture. Perhaps you're read the book *Natural Language Processing with Transformers* (O'Reilly), or a similar work. Moreover, I take it you're not just curious about

1 Ashish Vaswani et al. "Attention Is All You Need" (2017).

transformers. You're here because you want to build real applications with transformers, and you want to do it right.

This chapter provides a focused review of the transformer architecture to set the stage for the more advanced and complex models beyond NLP that I'll cover in later chapters.

I'll begin with the basic transformer architecture, then explain how longer context becomes possible, and finish with a tour of various attention mechanisms. Throughout this chapter, and the ones that follow, I'll share practical insights from real deployments so that you can benefit from my experience and learn the patterns, pitfalls, and principles that matter when theory meets the hard surface of production.

# Transformer Basics

This section explains the main architectural components of the original transformer model, such as encoder and decoder, positional embeddings, and attention mechanism.

The transformer architecture was originally developed for machine translation, a challenging sequence-to-sequence task in which the concept of tokenization plays a critical role. *Tokenization* breaks down sequences like sentences into manageable units, or tokens, that the transformer can effectively process. For example, in the sentence:

```
The Transformer has revolutionized NLP.
```

The word *the* represents a single word-level token.

Before we dive into the architectural components, understanding tokenization is crucial, as it facilitates the transformer's ability to interpret text. And it sets the foundation for its application to other sequences.

## Tokenizer: Text Representation in the Transformer

A *tokenizer* is used to tokenize the text. This is the first step to make natural language digestible for the model, before applying token embeddings and finally positional embeddings. The different types of tokenization are:

*Character-level tokenization*
: *Character-level tokenization* splits the underlying alphabet into each existing character in the sequence. If you used character-level tokenization for:

  ```
  "The Transformer has revolutionized NLP."
  ```

  it would yield:

  ```
  [T, h, e ,' , … 'N, L, P, .]
  ```

This will lead to very long sequences, which can increase computational complexity. It can also be challenging for the model to learn long-term dependencies. Nonetheless, this can be helpful if your task requires a fine-grained understanding.

*Word-level tokenization*
: *Word-level tokenization* would split the example sentence as follows:

  `[The, Transformer, … NLP, ., ]`

  That is, the sequence will be split into its words, plus punctuation. The downside is that this requires a large vocabulary, and if the language changes, this tokenization will not be able to understand new words.

*Subword tokenization*
: Most modern LLMs use *subword tokenization*, in which the word is split into smaller parts. For instance, a subword tokenizer would split the word *hiking* into:

  `[h, ik, ing ]`

  and the word *cooking* into:

  `[cook, ing]`

  So subword tokenization splits a word (or sequence) into smaller, commonly occurring chunks, like:

  `[ing]`

  Single-character words are also included.

Now that you understand the basics of tokenization, let's move on to token and positional embeddings.

## Token and Positional Embeddings

A part of the transformer architecture that contains learnable parameters is the token and positional embeddings (PEs). The token embedding is tasked with encoding each vocabulary element into a $d$-dimensional vector in the space of $\mathbb{R}$ of $d_e$ ($\mathbb{R}^{d_e}$). The token embedding can mathematically be presented as follows:

Let $V$ be the vocabulary with $|V| = N_v$, where each word $w$ in $V$ is assigned a unique token ID, $v \in \{1, 2, \ldots, N_v\}$. The token embedding is a function $e{:}\mathbb{N} \rightarrow \mathbb{R}^{d_e}$ that maps each token ID to a $d_e$-dimensional vector. This is achieved through a token embedding matrix $W_e \in \mathbb{R}^{d_e \times N_t}$, where $d_e$ is the dimensionality of the embeddings. Here's how to do it using bidirectional encoder representations from transformers (BERT):

```
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased') ❶
model = AutoModel.from_pretrained('bert-base-uncased')

sentence = "The Transformer has revolutionized NLP."
inputs = tokenizer(sentence, return_tensors='pt') ❷
input_ids = inputs['input_ids'] ❸

print(input_ids)
outputs = model(input_ids)

embeddings = outputs.last_hidden_state ❹
print(embeddings)
```

❶ Load tokenizer and model from Hugging Face.

❷ Tokenize the sentence.

❸ Get the input IDs and pass them through the model to get the embeddings.

❹ Get the last hidden state, to access the embeddings of the tokens.

This will result in the following output for the input IDs of the sentence:

```
tensor([[ 101, 1996, 10938, 2121, 2038, 4329, 3550, 17953, 2361, 1012,
102]])
```

For the corresponding embeddings, the output is:

```
tensor([[[-0.5249, -0.2210,  0.2696,  ..., -0.4204,  0.2605,  0.6457],
         [-0.6665, -0.4994,  0.4651,  ..., -0.2517,  0.2334,  0.0176],
         [ 0.8416, -2.0561,  0.8323,  ..., -0.2709, -0.1999, -0.1918],
         ...,
         [-0.4018, -0.6402,  0.7791,  ..., -0.0290, -0.4070,  0.2974],
         [-0.3327, -0.8091, -0.0304,  ...,  0.4745,  0.3230, -0.5991],
         [ 0.4928, -0.0878, -0.0971,  ...,  0.1629, -0.7012, -0.3848]]],
       grad_fn=<NativeLayerNormBackward0>)
```

This representation lacks the position of the word in the sequence. And since the transformer does not have *recurrence*, meaning that it doesn't need to process the data sequentially as it was originally represented, you need a function to represent the position. This is why you need to add positional embeddings: without them the model treats sequences as unordered collections of words.

The positional-embedding function learns to encode a token's location within a sequence into a vector in the space $\mathbb{R}^{d_e}$. The original transformer uses for position $p_i$:

$$p_{i,\,2t} = \sin(k/10000^{2t/d})$$
$$p_{i,\,2t+1} = \cos(k/10000^{2t/d})$$

Here, $p_{i,\,2t}$ is the $2t^{th}$ element of the $d$-dimensional vector $p_i$. This means that the position of the first token is captured by a vector, $p[1]$, while the position of the second token is captured by a different learned vector, $p[2]$, and so on.

This technique enables transformer models to understand the order of words. In the next section you'll see how the transformer uses this vector representation to understand and learn from the text.

## Attention Mechanism

The attention mechanism is at the core of the transformer's ability to understand and interpret text. It gives the model the ability to analyze the relevance of a word in a sequence on a token-to-token basis.

In that context, you'll often hear the term *attribution matrix*, which is computed from the input embeddings. Here the term *attribution* refers to the significance between different parts of the input. The attribution matrix is computed with the $Q$ (query) and the $K$ (key) matrices. The resulting scores form the $Q$ and $K$ interaction to determine the attention weights, which are then applied to the $V$ (value) matrix to produce the output of the attention mechanism:

$$\text{Attention}\,(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

This attribution matrix is crucial for understanding how the model interprets and processes the corresponding input sequences. For instance, by analyzing these scores, you can gain insights into the model's decision-making process, such as which tokens it sees as more relevant than others when generating the output token. Libraries such as Captum help make this decision-making process visible.

However, despite the specific roles of $Q$, $K$, and $V$, the initial computation for each of these matrices follows a similar process: a *linear projection* of the input embeddings. This means that for each of these matrices, the input embeddings are multiplied by a weight matrix. This process can be mathematically described as follows:

- Query matrix Q: $Q = W_q E$
- Key matrix K: $K = W_k E$
- Value matrix V: $V = W_v E$

Here, $E$ represents the input embeddings, and $W_q$, $W_k$, and $W_v$ are the weight matrices for the query, key and value projections, respectively. Take the dot product of the query and key matrices, followed by the Softmax function and the scaling factor (for scaled dot attention). The result will be a matrix of scores representing self-attention, or how much focus each token should put on each other token by considering its relationship with every other element in the sequence. These scores are then used to weight the values in the $V$ matrix, producing the final weighted-sum output of the attention mechanism:

$$Output = AttentionScores \times V$$

This dynamic process allows the model to focus on different parts of the input sequences for each input token, making it possible to understand each token's contextual relevance and information.

### Multi-head attention

The attention mechanism you've seen so far represents the computation performed by a single attention head, which is the component responsible for calculating attention in the transformer. However, the original transformer, as well as state-of-the-art (SOTA) models, applies multiple attention heads simultaneously. Each individual attention head has its own learnable parameters, which are then combined into a single output. This allows the model to integrate information from the same sequence and capture a variety of relationships between its words or elements. This approach enhances the model's ability to understand and represent complex dependencies in the data.

In technical terms, given input sequences $A$, $B$, and $C$, . . ., the multi-head attention mechanism computes new representations for the elements in $A$ by considering information from $B$, $C$, and so on. This process involves several steps: each head computes its own attention scores and output vectors based on the input, then concatenates and linearly transforms these outputs to produce the final output vector $V$.

This process consolidates the contextual information captured by the individual attention heads into one unified output that encapsulates all critical information across the entire input sequence. Since each attention head might focus on different relationships within the input sequence, this is crucial to the model gaining a better language understanding.

### Bidirectional and unidirectional attention

As I mentioned, the first transformer model was used for machine translation. That's why it uses two distinct types of attention mechanism within the architecture: one for the encoder and another for the decoder.

First, the encoder applies bidirectional self-attention, not just left-to-right processing, as traditional sequence processing methods do. This means it treats all tokens as context, applying attention to each token in the sequence. This gives the model a full understanding of the entire input sequence when it generates representations for each token.

The decoder's attention is masked (also called *causal attention*) to prevent the model from attending to future tokens (subsequent positions). In practice, this means that for the prediction $i$, the model can attend to the position $< i$. With that method in place, the model generates each token based only on the tokens previously created, from left to right, thus preventing it from using future tokens in the sequence. This is important for all tasks where the model must generate one token at a time, as, for instance, for translation.

Now that you understand the two distinct variations of attention used with the first transformer, let's look at the encoder and decoder.

## Encoder and Decoder Parts

The first transformer model's architecture (Figure 1-1) was characterized by its encoder-decoder structure. Some subsequent models leverage a decoder-only framework, such as GPT, LLaMA, Mistral, and Falcon.

The encoder itself is composed of six identical layers, each containing two principal components: a multi-head self-attention mechanism and a pointwise fully connected feed-forward network. The term *pointwise* refers to applying the same linear transformation to each sequence element. These components are further refined with residual connections and layer normalization.

The decoder interprets the encoded information, mirroring the encoder's layered structure, but introduces an essential feature: *masked multi-head self-attention*. This added feature in the decoder prevents the model from accessing subsequent positions in the sequence.

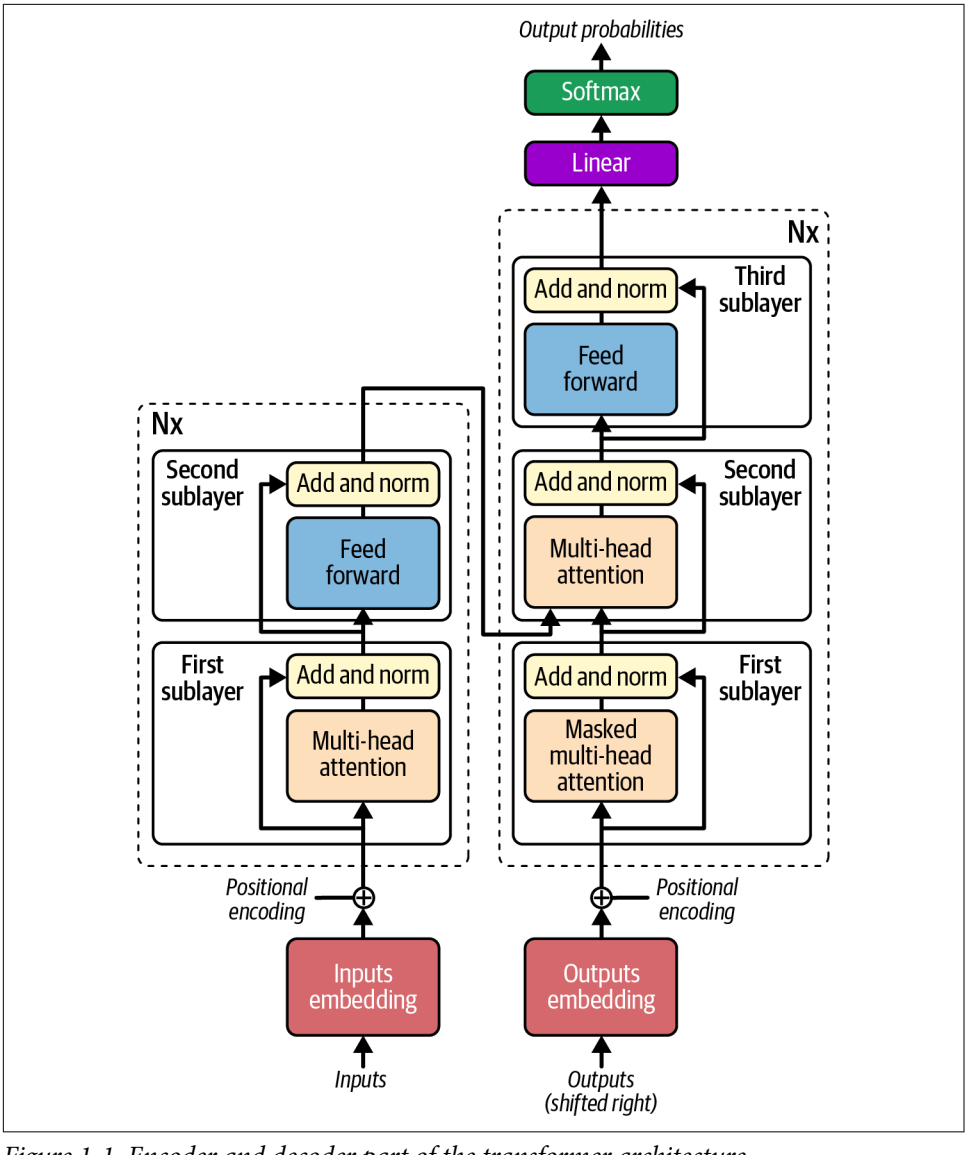


*Figure 1-1. Encoder and decoder part of the transformer architecture.*

The model maintains a consistent output dimension of 512 across all sub-layers, including the embedding layers, meaning its maximum sequence length is 512 tokens. This limitation comes mostly from the specific architectural setup of the first transformer model, which made it hard to process longer sequences on the available hardware efficiently.

# Enhancements in Transformer Design: Longer Context and Attention Variations

Now it's time to look into methods by which modern transformer models, like GPT-4.5 and Qwen3, achieve higher levels of performance and flexibility—in particular, the ability to process more information at once, through longer context windows. Attention-mechanism variations such as multi-query and flash attention also increase the efficiency and accuracy of SOTA transformer models.

## Longer Context Windows with Better Performance

A model's *context window* refers to the portion of text it can process when making predictions or generating text. A longer context window allows the model to understand more complex narratives and capture nuances better than it could using a chunked version of a text with a small context window.

However, simply extending the context length results in quadratic increases in time complexity and memory usage, which can constrain improvements. Therefore, recent enhancements, such as *rotary positional embedding* (RoPE),[2] *position interpolation* (PI),[3] and *Yet another RoPE extensioN method* (YaRN),[4] are designed to more effectively manage longer contexts during inference.

RoPE brings *absolute* and *relative* PEs together. But before I dive deeper into how RoPE works, let's first look at the key differences between absolute and relative PEs:

- With absolute PEs, for each token embedding, the model adds information about the absolute position of the token. Absolute PEs are simpler and faster to compute.
- Relative PEs consider distances between sequence elements and can be shared across sequences, which helps the model understand and interpret the relationships and distances between different tokens within a sequence. Relative PEs result in an increase in performance but are computationally more complex.

RoPE combines absolute and relative PEs, representing a significant advancement in the design of transformer models. These models process longer sequences of text more naturally and accurately while maintaining efficiency.

2 Jianlin Su et al. "RoFormer: Enhanced Transformer with Rotary Position Embedding" (2021).

3 Shouyuan Chen et al. "Extending Context Window of Large Language Models via Positional Interpolation" (2023).

4 Bowen Peng et al. "YaRN: Efficient Context Window Extension of Large Language Models" (2023).

Specifically, RoPE integrates a rotation matrix, $R_{\Theta,m}$, to encode the absolute positions of tokens, incorporating the explicit dependency of relative positions into the self-attention mechanism. To illustrate RoPE's implementation more concretely, consider a model with dimension $d = 6$, which then can be computed as follows:

$$R^{6}_{\Theta,m} = \begin{pmatrix} cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & 0 & 0 \\ 0 & 0 & 0 & 0 & \cos m\theta_3 & -\sin m\theta_3 \\ 0 & 0 & 0 & 0 & \sin m\theta_3 & \cos m\theta_3 \end{pmatrix}$$

Higher dimensions are divided into $d/2$ subspaces, so the dimension number has to be even. Let's put the math into code to make the theoretical concept more clear:

```
def simple_rotary_matrix(d, m, max_len):
    assert d % 2 == 0, "Embedding dimension must be even."❶

    theta = 10000 ** (-2 * torch.arange(d // 2).float() / d) ❷
    theta *= m

    cos_theta = torch.cos(theta)❸
    sin_theta = torch.sin(theta)

    R = torch.zeros((d, d)) ❹

    R[torch.arange(0, d, 2), torch.arange(0, d, 2)] = cos_theta ❺
    R[torch.arange(0, d, 2), torch.arange(1, d, 2)] = -sin_theta
    R[torch.arange(1, d, 2), torch.arange(0, d, 2)] = sin_theta
    R[torch.arange(1, d, 2), torch.arange(1, d, 2)] = cos_theta

    return R
```

❶ To ensure that the dimension $d$ is even (required by the formulation).

❷ Compute thetas.

❸ Compute sine and cosine values for rotation.

❹ Initialize the rotation matrix.

❺ Compute the rotation matrix.

To use the function, you can simply do the following:

```
d = 6 ❶
max_len = 10 ❷
R_matrix = simple_rotary_matrix(d, m=1, max_len=max_len) ❸
print(R_matrix)
```

❶ Define the embedding dimension $d$.

❷ Define the sequence length.

❸ Create the rotation matrix.

This creates the following rotary matrix:

```
tensor([[ 0.5403, -0.8415,  0.0000,  0.0000,  0.0000,  0.0000],
        [ 0.8415,  0.5403,  0.0000,  0.0000,  0.0000,  0.0000],
        [ 0.0000,  0.0000,  0.9989, -0.0464,  0.0000,  0.0000],
        [ 0.0000,  0.0000,  0.0464,  0.9989,  0.0000,  0.0000],
        [ 0.0000,  0.0000,  0.0000,  0.0000,  1.0000, -0.0022],
        [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0022,  1.0000]])
```

Figure 1-2 illustrates the RoPE process.

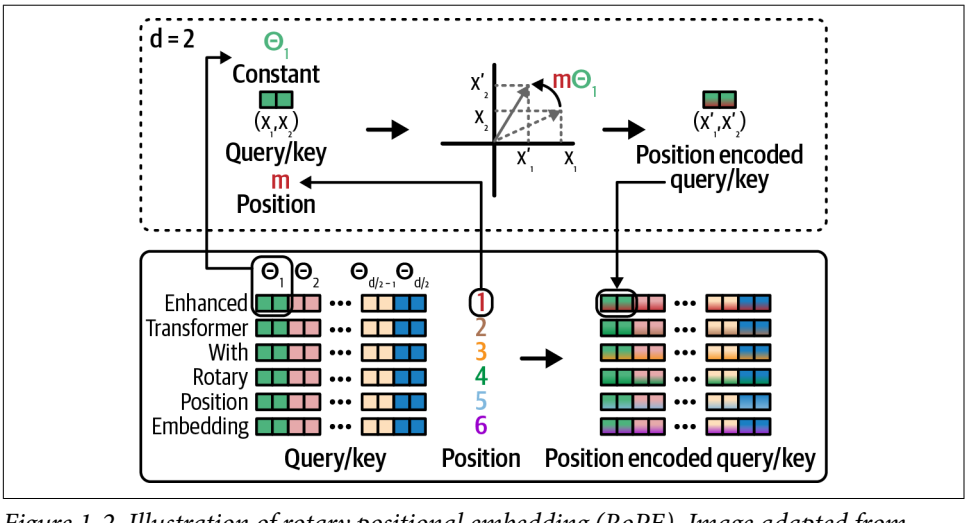


*Figure 1-2. Illustration of rotary positional embedding (RoPE). Image adapted from Jianlin Su et al. (2021).*

To apply RoPE in the context of self-attention, define the relationship between the $q_m$ in position $m$ and key $k_n$ in position $n$ as:

$$q_m^T k_n = (R^d_{\Theta,m} W_q x_m)^T (R^d_{\Theta,m} W_k x_n) = x^T W_q R^d_{\Theta,n-m} W_k x_n$$

Here $R^{d}_{\Theta, n-m} = (R^{d}_{\Theta, m})^{T} R^{d}_{\Theta, n}$ represents the rotary matrix adapting the relative positions.

RoPE enhances efficiency and accuracy, so it's used in SOTA models like Qwen3. Even SOTA LLMs have a maximum number of tokens they can process at once. For instance, the Qwen3 models can handle up to 32,768 tokens in a single input.

This limitation becomes a problem in use cases that involve long prompts or extensive document summaries, where LLMs capable of managing more extensive contexts are desirable. However, it would take substantial computational resources to create a new LLM with an expanded context capability from the ground up. This raises an important question: is it possible to increase the context window size of an already pretrained LLM? The good news is: yes! PI and YaRN can extend these pretrained LLMs with minimal fine-tuning. Figure 1-3 demonstrates the PI technique for a LLaMA model with a 2048 context window.

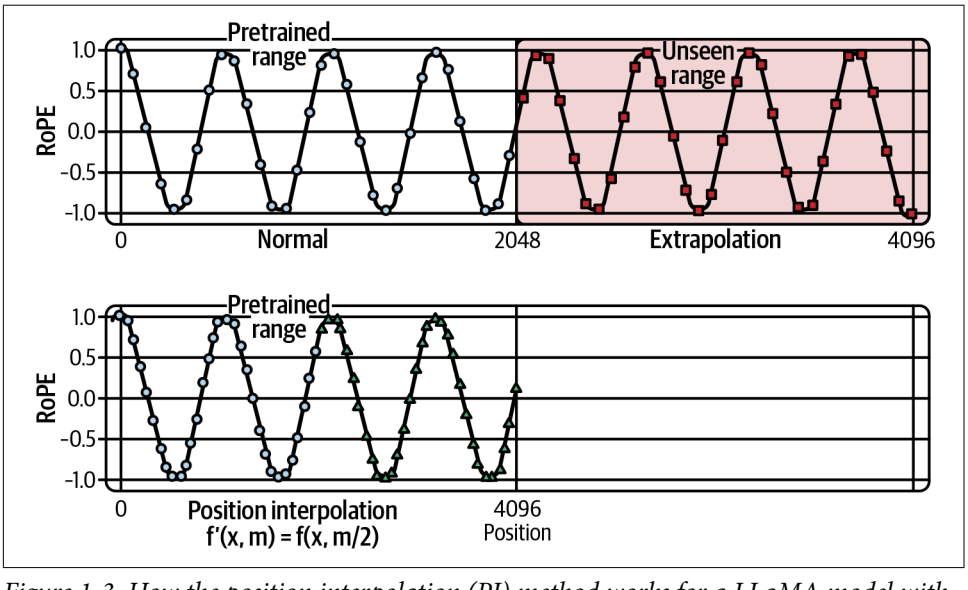


*Figure 1-3. How the position interpolation (PI) method works for a LLaMA model with a 2048 context window. The dots stand for the training limit of LLMs; the squares illustrate how models adapt to new positions. The dots and triangles demonstrate how PI scales down from [0, 4096] to [0, 2048] to keep them within the trained range. Image adapted from Shouyuan Chen et al. (2023).*

Normally, LLM models use input positions (dots) within their trained range. For length extrapolation, models handle new positions (squares) up to 4096. Position interpolation downscales these indices (dots and triangles) from [0, 4096] to [0, 2048], ensuring that they stay within the pretrained range.

To extend the context window, PI interpolates the position indices within the pretrained limit, with a small set of fine-tuning applied.

That is, PI extends RoPE's function $f$ by $f'$ as follows:

$$f'(x, m) = f(x, \frac{mL}{L'})$$

Here $L' > L$ is a new context window beyond the pretrained one.

Let me take a short step back and explain an important way to evaluate the performance of a model $L$ *perplexity* (PPL). This is a measure of how "surprised" or "perplexed" a model is about context. That is, perplexity measures how well a probability model predicts a sample, with lower values indicating better predictive accuracy. Let me illustrate this with a concrete coding example:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
model = AutoModelForCausalLM.from_pretrained("tiiuae/falcon-7b")
tokenizer = AutoTokenizer.from_pretrained("tiiuae/falcon-7b")

wiki_text = tokenizer("Apple Inc. is an American multinational " +
                      "corporation and technology company headquartered " +
                      "in Cupertino, California, in Silicon Valley. ",
                      return_tensors = "pt")

loss = model(input_ids = wiki_text["input_ids"],
             labels = wiki_text["input_ids"]).loss
ppl = torch.exp(loss)
print(ppl)

input_text = tokenizer("A Falcon is a generative transformer "+
                       "model and it can't fly.", return_tensors = "pt")

loss = model(input_ids = input_text["input_ids"],
             labels = input_text["input_ids"]).loss ❶
ppl = torch.exp(loss)
print(ppl)
```

❶ Compute loss.

The `wiki_text`

input yields a score of 5.08, while the

`input_text`

yields 121.19. This significantly higher perplexity score indicates that the model finds this sentence quite surprising or unlikely. This is because the model was most likely just trained on data indicating that a falcon is a bird known for its remarkable flying abilities, not a transformer model.

For evaluating LLM performance with longer context windows, you'll use *sliding window perplexity*. This metric calculates perplexity over a fixed-size window of tokens, moving across the text, to better handle and evaluate large texts and datasets.

One downside of RoPE is that it expands token positional information into a multidimensional complex vector. It struggles with encoding high-frequency components, because its one-dimensional input limits its ability to distinguish between very similar and proximate tokens.

#### Softmax and the Haystack Problem

The attention distributions in transformers are computed using the Softmax function. As the context window grows, Softmax tends to produce flatter distributions. This happens because the denominator (the sum of exponentials across all tokens) increases with context size, while each numerator (the exponential of a token's score) remains fixed. As a result, the output probabilities shrink, and the model struggles to focus on important tokens.

This is often referred to as the *haystack problem*: relevant signals get diluted among many irrelevant ones. Even with advanced techniques like RoPE, the model's ability to prioritize key elements across long contexts weakens. To address this, SOTA models like LLaMA 4 apply post-training optimization on long contexts, use inference-time temperature scaling of attention,[5] and introduce architectural changes such as interleaved attention layers without positional embeddings (iRoPE).[6] The usage of these methods increases the supported context to up to 10 million tokens while still performing well on "retrieving the needle in the haystack."

To address this, practitioners of *neural tangent kernel* (NTK)[7] theory developed *NTK-aware interpolation*, adjusting the scaling of frequencies differently across dimensions to preserve high-frequency information. One of the applications of NTK theory is identifying and mitigating issues related to training neural networks, such as difficulties in learning high-frequency components or patterns in data with low *intrinsic dimensionality*, as is the case with RoPE. Intrinsic dimensionality refers to the minimum number of parameters needed to accurately describe a dataset without losing significant information, representing the dataset's inherent complexity.

5 Ken M. Nakanishi. "Scalable-Softmax Is Superior for Attention" (2025).

6 Amirhossein Kazemnejad et al. "The Impact of Positional Encoding on Length Generalization in Transformers" (2023).

7 Arthur Jacot et al. "Neural Tangent Kernel: Convergence and Generalization in Neural Networks" (2018).

However, NTK-aware interpolation can stretch some dimensions beyond their bounds, potentially degrading the model's performance. Additionally, *NTK-by-parts interpolation* and *dynamic NTK interpolation* were introduced as refined strategies, focusing on preserving relative local distances and adapting scale factors dynamically for varying sequence lengths, respectively.

Building upon these NTK techniques, YaRN introduces a temperature $t$ to the attention scores before the attention Softmax, uniformly affecting perplexity across different data samples and token positions. This approach modifies attention weight computation and utilizes a length-scaling technique that adjusts both $q_m$ and $k_n$ by a constant factor, enhancing the attention mechanism without altering its underlying code. RoPE embeddings, pregenerated and reused, facilitate this process with no additional computational cost during inference or training. When combined with *NTK-by-parts interpolation*, YaRN performs effectively in models like LLaMA and LLaMA 2 (see Figure 1-4).

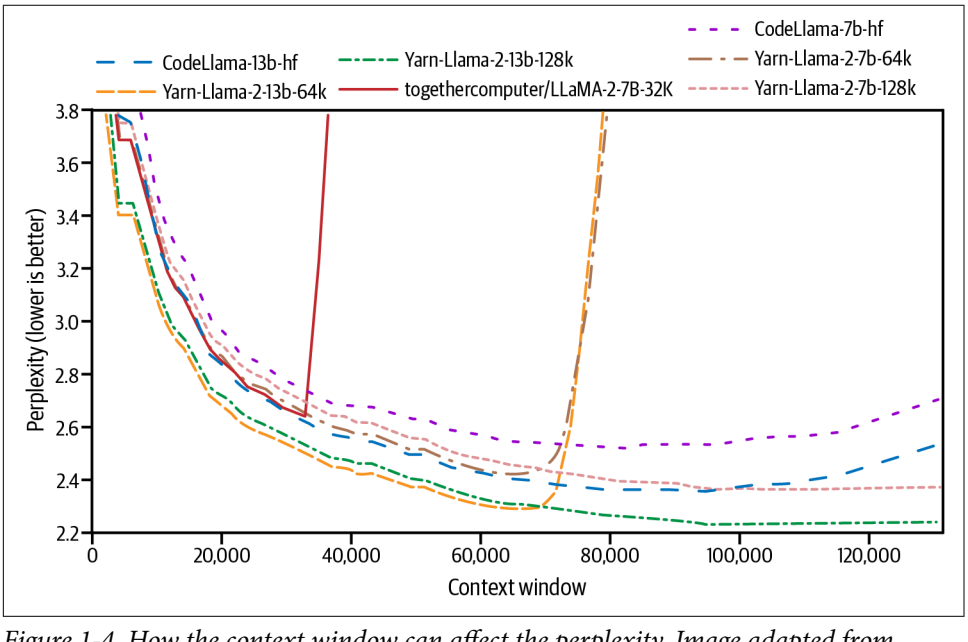


*Figure 1-4. How the context window can affect the perplexity. Image adapted from Bowen Peng et al. (2023).*

As you've seen, the lower the perplexity score, the better the model performs. For instance, LLaMA 7b with YaRN and 128k extrapolation performs well in comparison to LLaMA 7b without YaRN.

I'm sure you would now love to know how you can actually apply techniques like RoPE or YaRN to enhance the context length, to ensure optimal performance on

lengthy texts. The great news is that most frameworks allow for easy activation of longer context windows; for instance, vLLM supports YaRN, which can be configured as:

```
vllm serve Qwen3/Qwen3-8B --rope-scaling {"rope_type":"yarn","factor":
4.0,"original_max_position_embeddings":32768} --max-model-len 131072
```

Next, let's move to different attention variations and how they improve the performance.

## Attention Mechanism Variations

Today's transformers are more efficient than previous models, like LSTMs. That is, the first transformer model achieved a high Bilingual Evaluation Understudy (BLEU) score similar to LSTMs, which needed to be trained for months, after only 3.5 days of training. However, transformers can still be considered memory-hungry, since the time and memory complexity of self-attention grows quadratically with the sequence length. This section explores various improvements on the attention mechanisms used in high-performing SOTA LLMs, including:

- Cross-attention[8]
- Multi-query attention (MQA)[9]
- Grouped-query attention (GQA)[10]
- FlashAttention[11]
- FlashAttention-2[12]
- FlashAttention-3[13]

It's common for models to combine different attention variations: for instance, Falcon uses multi-query attention and FlashAttention.

8 Mozhdeh Gheini et al. "Cross-Attention Is All You Need: Adapting Pretrained Transformers for Machine Translation" (2021).

9 Noam Shazeer. "Fast Transformer Decoding: One Write-Head Is All You Need" (2019).

10 Joshua Ainslie et al. "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints" (2023).

11 Tri Dao et al. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness" (2022).

12 Tri Dao. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning" (2023).

13 Jay Shah et al. "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision" (2024).

### Cross-attention

In *cross-attention*, the inputs from two sequences are combined. Usually this means that the queries come from the decoder and the keys and the values come from the encoder. So, in essence, cross-attention enables the interaction between a set of embeddings. This is important for applications where you want to attend to a source sequence while generating a target sequence, such as translation or question-answering tasks. Let me explain the concept further with code:

```
def CrossAttention(x_1, x_2, W_query, W_key, W_value):

    scaling_factor = W_query.shape[1]**0.5

    Q = torch.einsum('bd,dk->bk', x_1, W_query)
    K = torch.einsum('bd,dk->bk', x_2, W_key)
    V = torch.einsum('bd,dv->bv', x_2, W_value)

    attn_scores = torch.einsum('bk,mk->bm', Q, K)
    attn_weights = F.softmax(attn_scores / scaling_factor, dim=-1)

    Y = torch.einsum('bm,mv->bv', attn_weights, V)

    return Y
```

In this code, you can see that the input for $Q$ comes from $x_1$ and for $K$ and $V$ from $x_2$, demonstrating the information flow between sequences. Using multiple information sources, the LLM gets a more sophisticated understanding and better generation results.

### Multi-query attention

*Multi-query attention* (MQA) uses only a single key-value head, whereas multi-head attention (MHA) uses $h$ number of heads for query, key, and value heads, respectively. Thus, MQA significantly speeds up the decoder's inference time. Figure 1-5 compares the two.

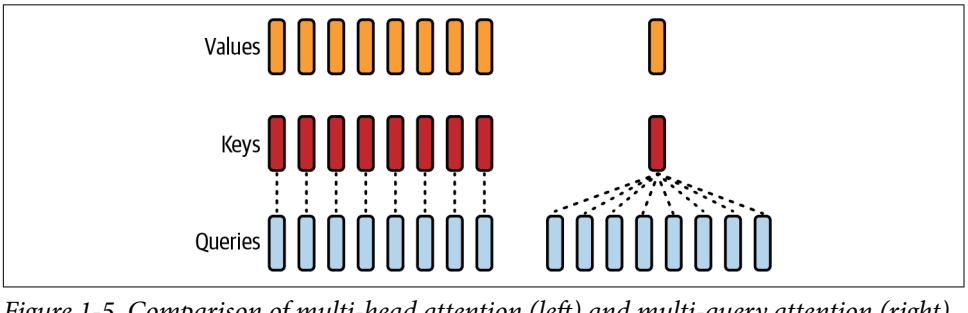


*Figure 1-5. Comparison of multi-head attention (left) and multi-query attention (right). Where multi-head attention has* h *number of query, key, and value heads, multi-query shares a single key and value head across all query heads.*

To make this difference more tangible, read the following code that computes MHA. Note that there is a letter *h* for each *Q*, *K*, and *V* to represent the head's dimension:

```
def MultiheadAttention(x, M, W_query, W_key, W_value, P_o):

    scaling_factor = W_key.shape[1]**0.5

    ❶
    Q = torch.einsum('d,hdk->hk', x, W_query)
    K = torch.einsum('md,hdk->hmk', M, W_key)
    V = torch.einsum('md,hdv->hmv', M, W_value)

    ❷
    attn_scores = torch.einsum('hk,hmk->hm', Q, K) / scaling_factor

    ❸
    attn_weights = F.softmax(attn_scores, dim=-1)

    ❹
    o = torch.einsum('hm,hmv->hv', attn_weights, V)
    y = torch.einsum('hv,hdv->d', o, P_o)

    return y
```

❶ Weight matrices.

❷ Compute attribution matrices using the scaling factor for scaled dot-product attention.

❸ Softmax applied to attention scores.

❹ Final attention weights (context vectors) computed.

With MQA, the letter *h* is omitted from the *K* and *V* matrices:

```
def MultiqueryAttention(X, M, mask, W_query, W_key, W_value, P_o):

    scaling_factor = W_key.shape[1]**0.5

    Q = torch.einsum('bnd,hdk->bhnk', X, W_query)
    K = torch.einsum('bmd,dk->bmk', M, W_key)
    V = torch.einsum('bmd,dv->bmv', M, W_value)

    attn_scores = torch.einsum('bhnk,bmk->bhnm', Q, K) / scaling_factor
    attn_weights = F.softmax(attn_scores + mask, dim=-1)

    O = torch.einsum('bhnm,bmv->bhnv', attn_weights, V)
    Y = torch.einsum('bhnv,hdv->bnd', O, P_o)

    return Y
```

These two examples make it clear that MQA is identical to MHA, except that in MQA the different *Q* heads share a single set of keys and values. This modification speeds up computation in the decoder but can lead to loss of quality, though it's still more performant than MHA. GQA was developed to address this.

### Grouped-query attention

*Grouped-query attention* (GQA) organizes query heads into *G* number of groups, with each group sharing one key and one value head. Figure 1-6 compares MHA (left) and GQA (right).

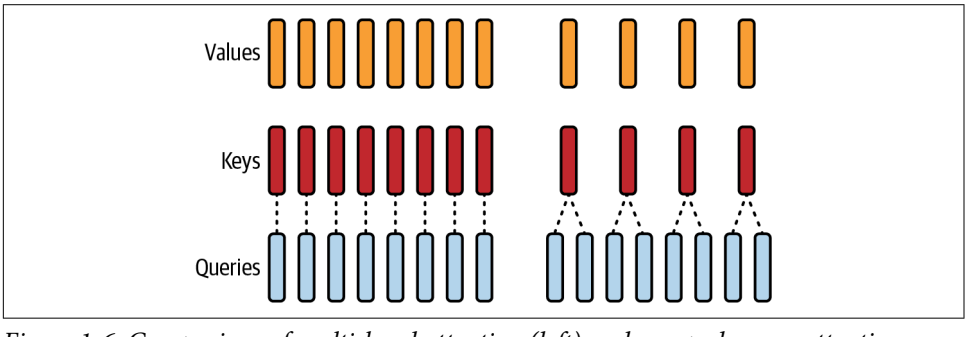


*Figure 1-6. Comparison of multi-head attention (left) and grouped-query attention (right). Where multi-head attention has* h *number of query, key, and value heads grouped-query attention instead shares one key and value head for each group of query heads, interpolating between multi-head and multi-query attention.*

Comparing MHA to GQA, you can see that GQA consolidates multiple key and value heads into a single key and value head, effectively reducing the key-value (KV) size.

### KV Caching

*KV caching* optimizes *inference latency* by storing the computed key and value tensors for previously generated tokens during autoregressive decoding. That is, instead of recalculating the full attention context at every step, the model appends only the new keys and values, which significantly reduces the computational cost of the attention mechanism. However, although KV caching provides substantial improvements in inference speed, it increases memory usage proportionally with the sequence length and the number of layers. In scenarios where memory is a limiting factor, you may have to reduce the model size or limit the context window, which can lead to a drop in model accuracy. Deploying KV caching in large-scale production systems also introduces complexity in managing the cache lifecycle. This includes implementing strategies for cache eviction, dynamic memory allocation, and evaluating strategies for cache reuse across requests or sessions.

This means significantly lesser data to load into memory during computation, decreasing the required bandwidth and capacity by a factor of *h*. The following code illustrates this setup:

```
def GroupedQueryAttention(Q, K, V, num_heads, group_size):

    batch_size, seq_len, embed_dim = Q.shape
    scaling_factor = (embed_dim // num_heads) ** 0.5

    Q = rearrange(Q, 'b s (h d) -> (b h) s d', h=num_heads)
    K = rearrange(K, 'b s (h d) -> (b h) s d', h=num_heads)
    V = rearrange(V, 'b s (h d) -> (b h) s d', h=num_heads)

    attn_scores = torch.einsum('bid,bjd->bij', Q, K) / scaling_factor
    attn_weights = F.softmax(attn_scores, dim=-1)
    attn_output = torch.einsum('bij,bjd->bid', attn_weights, V)

    Y = rearrange(attn_output, '(b h) s d -> b s (h d)',
    b=batch_size, h=num_heads)

    return Y
```

GQA is specifically beneficial for larger models as they usually expand the number of heads. That said, employing GQA substantially reduces both memory bandwidth and capacity while maintaining performance as models scale up.

Thus, memory bandwidth overhead from attention has less impact in larger models. This is because the key-value cache size increases linearly with the model dimension, whereas the model's floating-point operations per second (FLOPs) and parameters increase quadratically with the model dimension.

Even given these improvements, there's still room to optimize how attention leverages the GPU memory. This is where FlashAttention and FlashAttention-2 come in.

### FlashAttention

*FlashAttention* uses *tiling* to rearrange how attention calculations are performed. By doing so, it avoids creating an $N \times N$ attention matrix. Tiling involves transferring chunks of input data from GPU high bandwidth memory (HBM) and GPU on-chip SRAM (speedy cache). FlashAttention iterates over sections of the *K* and *V* matrices, transferring them to the "speedy cache." Within each section, it cycles through portions of the *Q* matrix, moving them to SRAM, then saves the results of the attention process back to the HBM (illustrated in Figure 1-7).

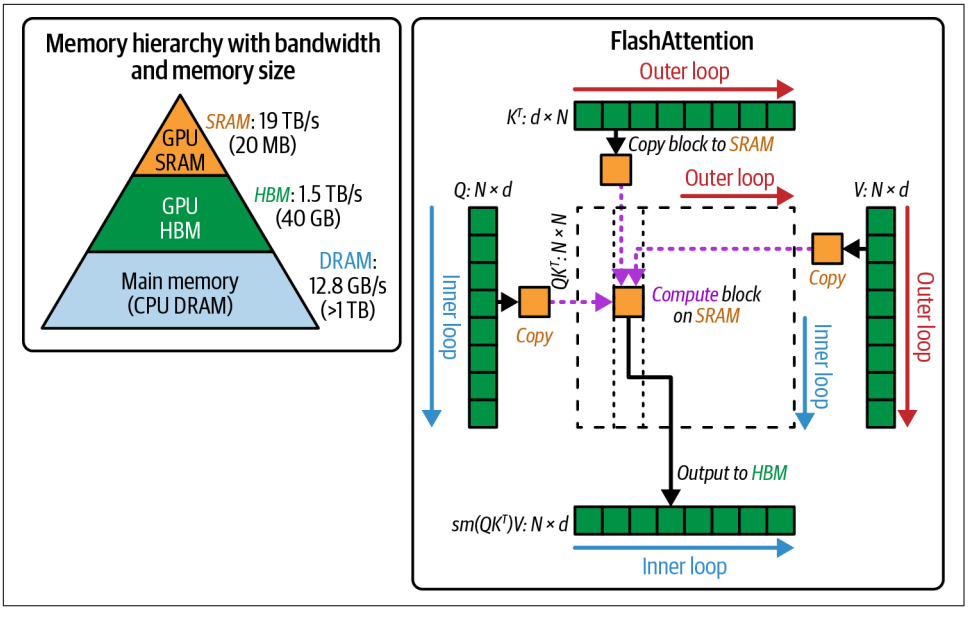


*Figure 1-7. FlashAttention uses tiling to eliminate the large* N × N *attention matrix. It works by cycling through segments of the* K *and* V *matrices in its outer loop (indicated with red arrows), loading these segments into the fast on-chip SRAM. For each segment, FlashAttention also processes chunks of the* Q *matrix (denoted by light gray arrows), loading them into SRAM, then saving the attention output back to HBM. Image adapted from Tri Dao et al.*

This enhances computation speed while decreasing memory consumption from quadratic to linear, relative to the sequence length. FlashAttention avoids saving the large intermediate attention matrices in HBM, minimizing memory operations and doubling or even quadrupling processing speed. In addition, FlashAttention enables longer context windows in transformers, resulting in better perplexity scores and therefore higher-quality models.

This is impressive, but there's still room for improvement. The number of non-matmul FLOPs operations can be further reduced, as you'll see in the next section.

### FlashAttention-2

I mentioned earlier that it's difficult to increase context window size in transformers. The core attention layer's runtime and memory demands grow quadratically with the input sequence length. RoPE, PI, and YaRN help improve efficiency and lower the perplexity, as you saw.

FlashAttention-2 reduces the amount of non-*matmul* FLOPs while not changing the output. Although these non-matrix multiplication FLOPs amount to only a minor portion of the total FLOPs, they are slower to execute. GPUs have specialized units that make matrix multiplication operations run up to 16 times faster than non-matrix multiplication operations. Therefore, minimizing non-matrix multiplication FLOPs and maximizing the time spent on matrix multiplication FLOPs is crucial for speeding up your computations.

FlashAttention-2 achieves this by optimizing GPU resource utilization. It minimizes shared memory access through parallel computation across different thread blocks and work partitioning among warps within a single thread block. A *warp* is a group of threads that execute computations. These adjustments contribute to a 2–3× speedup.

This approach involves inverting the *loop hierarchy*, focusing first on row segments in the outer loop and column segments in the inner loop. This reverses the original method presented in the FlashAttention and introduces parallel processing along the sequence length dimension. Figure 1-8 illustrates this.

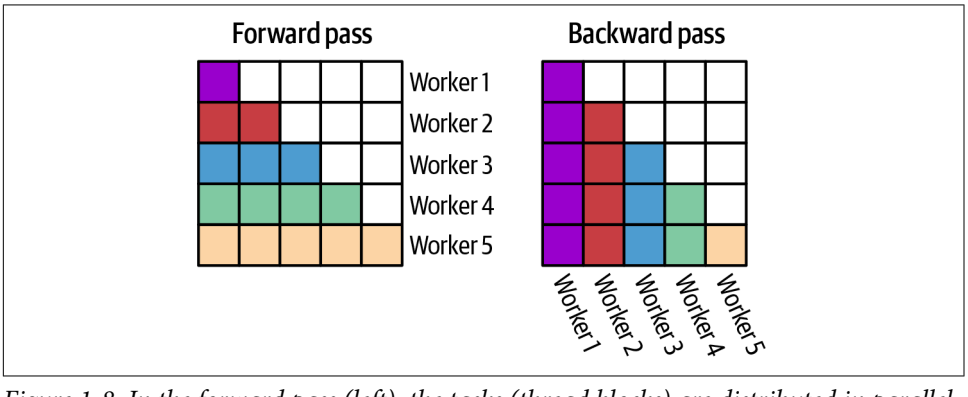


*Figure 1-8. In the forward pass (left), the tasks (thread blocks) are distributed in parallel, with each task handling a segment of rows from the attention matrix. In the backward pass (right), each task is responsible for a segment of columns within the attention matrix. Image adapted from Tri Dao et al. (2022, 2023).*

Figure 1-9 compares the work partitioning between different warps in the forward pass in FlashAttention and FlashAttention-2. Efficiently dividing work among warps can significantly impact the performance of parallel computing tasks, including those in deep learning models like transformers.

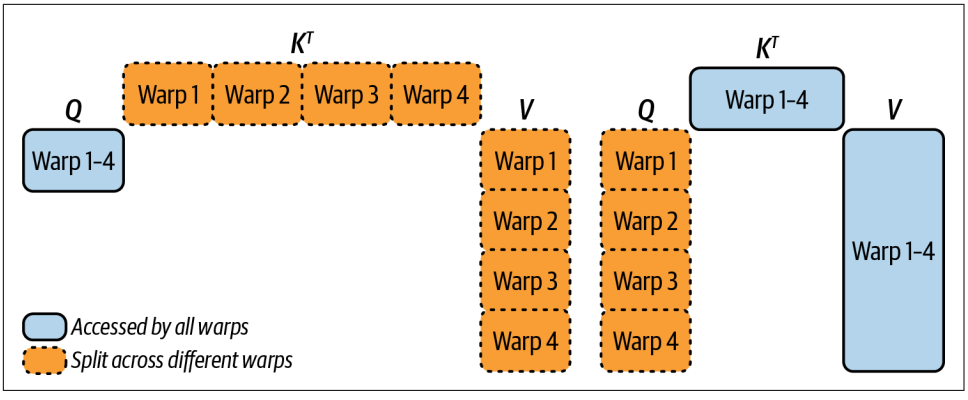


*Figure 1-9. Comparison of work partitioning between different warps in the forward pass in FlashAttention (left) and FlashAttention-2 (right). Image adapted from Tri Dao et al. (2022, 2023).*

### FlashAttention-3

*FlashAttention-3* introduces new programming techniques that take full advantage of the Hopper[14] GPU architecture, specifically the NVIDIA H100, to accelerate attention computation beyond the limits of previous methods. While FlashAttention-2 performs well for most GPUs, on newer architectures such as H100, FlashAttention-2 achieves only 35% GPU utilization.

While FlashAttention and FlashAttention-2 focused on reducing memory bandwidth usage and optimizing compute schedules, FlashAttention-3 advances performance by leveraging hardware asynchrony and low-precision formats such as FP8. One of its key innovations is the use of producer-consumer asynchrony, where separate GPU warps are assigned distinct roles: some act as producers loading data ($Q$, $K$, $V$) via the Tensor Memory Accelerator (TMA), while others act as consumers performing matrix multiplications on Tensor Cores. This strategy, often referred to as *pingpong scheduling*, allows data transfer and computation to run concurrently, effectively hiding latency and maximizing throughput.

14 The architecture is named after Grace Hopper, a pioneer in computer programming who famously popularized the term *bug* in 1947. She was known for carrying a piece of wire to illustrate how far light travels in a nanosecond, which she used as a playful response when asked to make things faster. Now she gets her revenge, as we name GPUs after her that can perform nearly two quadrillion operations per second.

### PagedAttention for Higher Throughput

While FlashAttention-3 introduces techniques that fully leverage the Hopper architecture and low-precision formats like FP8, it's optimized for H100 GPUs only. But these GPUs can be expensive to run on cloud services. Therefore, for most teams and production environments, PagedAttention offers a more accessible and cost-effective solution to increase inference throughput without needing specialized hardware. *PagedAttention* is a memory-efficient attention variant designed to improve throughput during LLM inference. I'm sure you've read my note on KV caching earlier in this chapter, and you might have to evaluate strategies to optimize KV caching. This is exactly what PagedAttention[15] does. PagedAttention stores the KV cache in noncontiguous memory blocks, similar to virtual memory paging in operating systems. These blocks can be dynamically allocated, shared across sequences, and reused with copy-on-write semantics.

PagedAttention is built into the vLLM serving system and achieves up to 4× higher throughput by minimizing KV cache waste and enabling batching of more requests. PagedAttention is especially beneficial for workloads with long sequences, variable decoding lengths, and complex algorithms like beam search or parallel sampling. Note that PagedAttention is only available on vLLM. Moreover, vLLM can struggle if you have a lot of concurrent requests, so your throughput could still be better with Hugging Face's Text Generation Inference (TGI), as it's very reliable on many concurrent requests. I suggest that you use a TGI benchmarking tool to validate this for your application.

Another innovation is GEMM-Softmax pipelining. *General matrix-matrix multiplication* (GEMM) is a fundamental operation in deep learning that multiplies two matrices to produce a third, and is heavily optimized on GPUs using specialized hardware like Tensor Cores. In transformers, the Softmax operation depends on the output of GEMM, introducing a sequential dependency. FlashAttention-3 breaks this bottleneck by pipelining GEMM and Softmax across iterations so that while one block performs Softmax, the next GEMM operation can already begin. This overlapping is essential to exploit Hopper's asynchronous compute capabilities.

FlashAttention-3 also introduces low-precision attention with FP8, which nearly doubles throughput compared to FP16. To achieve this without sacrificing accuracy, it adapts the memory layout of $Q$, $K$, and $V$ to meet Hopper's FP8 GEMM constraints and applies two techniques to reduce quantization error: block quantization and incoherent processing. The latter involves multiplying $Q$ and $K$ with a random orthogonal matrix constructed from Hadamard transforms before quantization.

15 Woosuk Kwon et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention" (2023).

*Hadamard transforms* refer to a mathematical operation that maps a vector into a new space using only additions and subtractions. It relies on the *Hadamard matrix*, which is made up entirely of +1 and −1 entries. This transformation is efficient to compute and helps spread information across dimensions, which is useful for reducing the impact of outliers in low-precision quantization.

# Conclusion

This chapter has taken you from the foundational ideas of the original transformer to some of the most powerful architectural and inference-time innovations that define today's SOTA models. From tokenization and multi-head attention to rotary embeddings, longer context windows, and advanced memory optimizations like PagedAttention and FlashAttention, you've seen how the architecture has evolved over time to meet the ever-growing demands of real-world applications.

This progression is a testament to the fact that the transformer is no longer a static blueprint confined to language tasks. It's a dynamic and extensible framework that continues to improve in both accuracy and efficiency. In the next chapters, we'll move beyond language and explore how these models, along with the architectural advances introduced here, enable breakthroughs in domains such as vision, time series, reinforcement learning, and structured reasoning. You'll learn how to apply these tools in practice and how to make architectural choices based on the specific demands of each problem space.

# CHAPTER 2
# Transformers for Time Series

Transformers have gained popularity for time series modeling due to their ability to capture long-sequence interactions more effectively than recurrent neural network (RNN) models. The self-attention mechanism in transformers reduces the maximum length of network signal travel paths to the theoretical minimum of O(1) by allowing each element in the sequence to directly attend to every other element. This eliminates the need for recurrent structures and allows for more efficient information flow. As a result, transformers show great potential for long sequence time-series forecasting (LSTF) and other domains such as anomaly detection and spatiotemporal time series prediction.

Many real-world applications, such as weather forecasting, traffic prediction, industrial process controls, and electricity consumption planning, require the prediction of long sequence time series. LSTF demands a model with high prediction capacity, capable of efficiently capturing precise long-range dependencies between input and output.

In addition, probabilistic time series forecasting is an important task in many practical applications, from finance and weather forecasting to managing computer system performance. Accurate probabilistic forecasts are essential for making informed decisions in these areas. *Probabilistic time series forecasting* includes a range of probabilities for all possible future outcomes, rather than identifying a single specific outcome as "the forecast."

During your exploration of this chapter, you'll learn key considerations for using time series data. You'll also acquire a basic understanding of different domains for time series, such as time series forecasting, anomaly detection, and spatiotemporal time series prediction. As the last steps in your journey through this time series chapter, you'll examine different transformer architectures for these domains, including

Chronos, PatchTST, and AnomalyBERT, and learn how you can fine-tune the first two models for your own data.

# Understanding the Intricacies of Time Series Data

When working with time series data, it's fundamental to consider certain key properties that can significantly impact your analysis and forecasting. This section offers a brief overview of the most common properties you should examine when handling time series data.

## Autocorrelation and Partial Autocorrelation

*Autocorrelation* is the internal correlation between observations in a time series, often represented as a function of the time interval between observations. The mathematical definition of the autocorrelation at lag $k$, $\gamma(k)$, is as follows:

$$\gamma(k) = \frac{E(X_t - \mu)(X_{t+k} - \mu)}{E(X_t - \mu)^2},$$

Here, $X_t, t = 0, \pm 1, \pm 2, \ldots$ represents the series' values, while $\mu$ represents the series' mean. The expected value is denoted by the symbol $E$. The sample statistic for this sample can be computed by the following equation:

$$\hat{\gamma}(k) = \frac{\sum_{i=1}^{n-k}(x_t - \overline{x})(x_{t+k} - \overline{x})}{\sum_{i=1}^{n}(x_t - \overline{x})^2},$$

$\overline{x}$ is the mean of the observed values, $x_1, x_2, \ldots, x_n$, of the time series.

The *partial autocorrelation function* for $k$ lags is defined as the autocorrelation between $y_t$ and $y_{t-k}$, and the effect of other variables is shifted by $k$ lags $(y_{t-1}, y_{t-2}, \ldots, y_{t-k+1})$. The autocorrelation function and partial autocorrelation function are often referred to as a correlogram. Figure 2-1 shows a *correlogram* for *the Amazon stock price*.

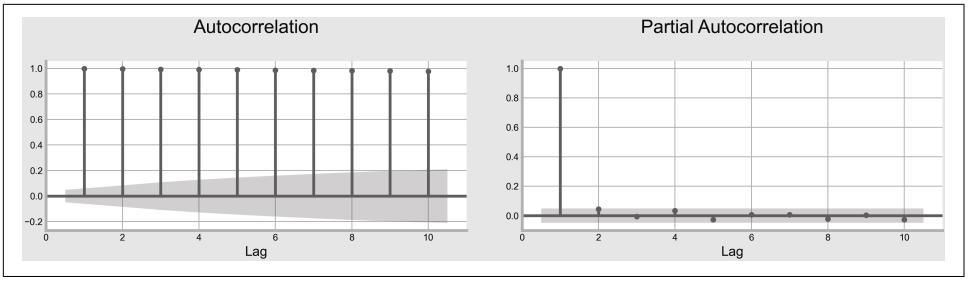


*Figure 2-1. Correlogram of Amazon stock.*

The horizontal axis of the autocorrelation plot (left plot) shows the size of the lag between the elements of the time series. For instance, the autocorrelation with lag 2 is the correlation between the time series and the corresponding elements that were observed two time periods earlier. Examining the autocorrelation plot reveals a high autocorrelation between the actual and the prior timestep of the series, with no decline. Additionally, the partial autocorrelation (right plot) is significant for the first lag. The gray areas are the *confidence bands*, indicating whether the correlations are statistically significant.

## Cointegration

*Cointegration* refers to a real relationship between two time series. A commonly used example is a drunk pedestrian and their dog. Their individually measured walks might appear random if taken alone, but they never stray too far from each other.

In the case of cointegration, you expect to observe high correlations. The difficulty lies in assessing whether two processes are indeed cointegrated or if you're looking at a *spurious correlation*. A fun example of a spurious correlation is shown in Figure 2-2.

The important difference between the two states is that in the case of a spurious correlation, there may not be any relationship between the processes, whereas cointegrated time series are strongly related to one another. You might remember from your statistics class the expression "Correlation is no proof of causation!"

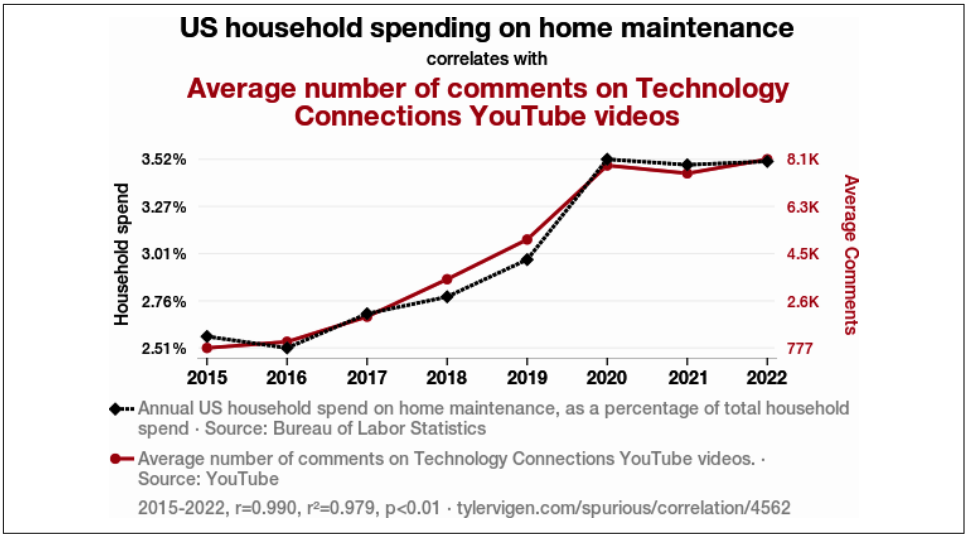


*Figure 2-2. Example of spurious correlation. The figure is from Tyler Vigen's website about spurious correlations. You can find more funny spurious correlations on his site.*

## Cross-Correlation

When working with multivariate time series, it's essential to understand the (inter)dependency between the series. *Cross-correlation* (also called *cross covariances*) is a measure of the degree of linear dependency between two time series processes. The cross-correlation between two processes $X_1$ and $X_2$ is shown in the following equation:

$$\gamma_{12}(k) = \text{Cov}\left(X_{1,t+k}, X_{2,t}\right), \gamma_{21}(k) = \text{Cov}\left(X_{2,t+k}, X_{1,t}\right)$$

However, high cross-correlation between features can cause issues in predictive modeling, such as multicollinearity, which can distort the model's estimates and reduce interpretability. *Multicollinearity* is a statistical phenomenon where multiple independent variables in a model exhibit correlation with one another. It may be beneficial to either remove one of the cross-correlated features or create new features that combine the information from the correlated ones.

## Stationarity

The statistical features of a stationary time series, such as mean, variance, and autocorrelation, are *period-independent*; that is, they don't change over time. Thus, *stationarity* implies that a time series is lacking trends or seasonal influences and that descriptive statistics such as the mean or standard deviation are constant or exhibit little variation over time when computed for different rolling windows. Strict stationarity implies that the joint distribution of any subset of time series observations is independent of time in all moments.

## Trend and Seasonality

Industrial time series, such as the time series of stock prices, are expected to exhibit trends or seasonality. To explore these properties, you can use *decomposition*, which involves considering a series in terms of its level, trend, seasonality, and noise components. Decomposition can provide valuable insights for understanding time series analysis and forecasting difficulties. The Statsmodels package offers functions to automatically decompose a given time series. Figure 2-3 shows an example of such a decomposition for Amazon stock pricing data, clearly indicating a continuous upward trend and a seasonal component.

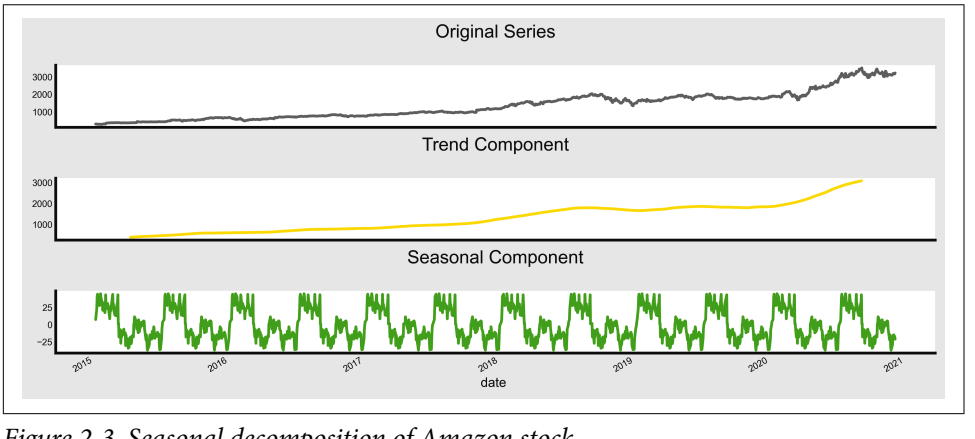


*Figure 2-3. Seasonal decomposition of Amazon stock.*

Typically, you begin by conducting an *exploratory data analysis* (EDA) to thoroughly examine your data before moving on to time series prediction. This initial step helps identify patterns, trends, and potential anomalies in the data, providing a solid foundation for accurate forecasting. By understanding the underlying structure of your data, you can make more informed decisions on the next steps, such as removing irrelevant features or addressing data quality issues.

## Preparing a Dataset

Data preparation is always a necessary step before using your data for modeling. When working with time series data, it's essential to consider their sequential nature; you can't just randomly shuffle your data. You also need to be careful about how you split and scale your data. Standardizing your data is crucial to avoid potential scaling difficulties when fitting your model. If you have more than one feature (covariate), scaling ensures that one feature doesn't dominate another due to different scales.

To avoid introducing look-ahead bias into your prediction, the training data must be scaled without knowledge of the test set. *Look-ahead bias* occurs when future information is used to predict past data, leading to overly optimistic and unrealistic model performance. To avoid this, you should standardize the training set using its own mean and standard deviation, not those of the whole dataset. For the validation and test sets, you should apply the same normalization: use the mean and standard deviation of the training set. This approach helps avoid systematic bias. *Systematic bias* happens when the model consistently overestimates or underestimates due to improper handling of the data during preparation.

Additionally, depending on your data, you might consider performing log transformations to reduce data variability and bring the data closer to a normal distribution. By preparing your dataset in these ways, you can improve the performance and reliability of your time series models.

# Time Series Modeling in Various Application Domains

Time series modeling is an important application area for machine learning. It underpins many aspects of modern operations across various domains, including inventory control, customer management, production, distribution, finance, and marketing. Improvements in these areas can have significant financial impacts, often amounting to millions of dollars for each point of forecasting accuracy gained. Tasks may include anomaly detection, probabilistic time series modeling, spatiotemporal time series prediction, and classification. In this section, I'll briefly cover the main differences between these tasks and give some examples of their applications:

*Time series forecasting*

> *Time series forecasting* involves predicting future values based on previously observed data in a time series. This can be applied to both short-term and long-term predictions, each with its own set of challenges and methodologies. *Short-term forecasting* focuses on immediate future values, so it's often used for tasks such as inventory management and demand planning. *Long-term forecasting*, on the other hand, aims to predict trends and seasonal patterns over extended periods, which is vital for financial markets, electricity demands, and weather prediction.

*Spatiotemporal forecasting*
: In *spatiotemporal forecasting*, both temporal and spatial dependencies are considered for accurate predictions. *Temporal* refers to time, while *spatiotemporal*, or *spatial-temporal*, is used when data is collected across both space and time. This approach is essential for describing phenomena at specific locations and times. Examples of spatiotemporal forecasting applications include traffic flow forecasting, wind speed forecasting, weather and climate forecasting, and air quality forecasting.

*Event forecasting*
: *Event forecasting* aims to predict the timing and characteristics (marks) of future events based on the history of past events, often modeled by *temporal point processes* (TPP). In TPP, the goal is to predict the times and marks of future events given the sequence of past occurrences. Recommender systems are a recent application area for TPPs, leveraging the temporal dimension of user behavior to provide time-sensitive recommendations, such as optimal timing for promotions. Other applications include clinical event prediction, which involves forecasting sequences of patient interactions with the healthcare system, human activity prediction for assisted living, and demand forecasting.

*Anomaly detection*
: *Anomaly detection* involves identifying abnormalities in time series data and is a key task in time series analysis. It's used to detect defects in machines and prevent potential harm, making it essential for many industrial environments, such as monitoring machines, IT devices, spacecrafts, and engines. However, this task is challenging because most datasets don't provide ground truth labels for the training set. That is, it's unknown whether any given point is anomalous or not in the training data. As a result, anomaly detection is typically approached as an unsupervised learning task.

*Time series classification*
: *Time series classification* involves analyzing multiple labeled classes of time series data to determine the class to which a new dataset belongs. This is important in many environments, such as analyzing sensor data or financial data to support business decisions. For instance, in healthcare, time series classification can be used to diagnose medical conditions based on patient data over time. In finance, it can classify stock price movements, where labels like 0 and 1 indicate down and up movements, respectively, to inform trading strategies.

# Tokenizing Time Series Data

Time series forecasting aims to understand the relationships between data points across different timesteps. Unlike words in a sentence, individual timesteps lack inherent semantic meaning, making it important to extract local semantic

information to analyze their connections effectively. Some transformer models treat each timestep as a separate token, processing them point by point. This method makes it difficult for the model to learn temporal dependencies when considering only one timestep at a time.

One way to address this is patching. *Patching* involves grouping multiple time points together before tokenizing and embedding them. This process creates subseries-level patches that serve as input tokens to the transformer. Figure 2-4 illustrates this process.

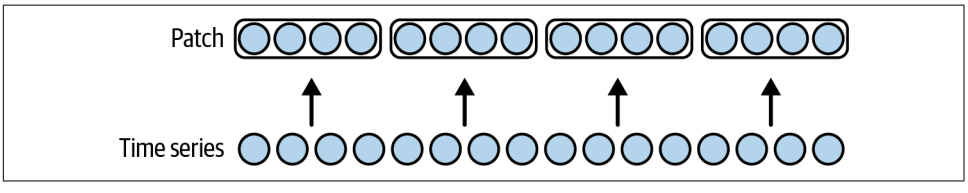


*Figure 2-4. Example of time series patching.*

Using patches significantly extends the historical time range of the input while maintaining the same token length. This method is similar to the approach used in vision transformers, which you'll learn about in the next section, but adapted for time series data. Patching offers three key benefits: it preserves local semantic information within the embedding, significantly reduces the computation and memory usage of the attention given the same lookback window, and allows the model to consider a longer historical context. PatchTST[1] introduced this method and significantly improved long-term forecasting accuracy.

The tokenization scheme of Lag-Llama[2] constructs lagged features from prior values of the time series, using specified lag indices (quarterly, monthly, weekly, daily, hourly, and second-level frequencies). Given a sorted set of positive lag indices $\mathscr{L} = \{1, \ldots, L\}$, where $\mathscr{L}$ refers to the list of lag indices, $L$ is the last lag index in the sorted list $\mathscr{L}$. The lag operation on a particular time value $x_t$ is defined as $x_t \mapsto \mathbf{k}_t \in \mathbb{R}^{|\mathscr{L}|}$, where each entry $j$ of $k_t$ is given by

$$\mathbf{k}_t[j] = x_{t - \mathscr{L}[j]}$$

1 Yuqi Nie et al. "A Time Series Is Worth 64 Words: Long-Term Forecasting with Transformers" (2023).

2 Kashif Rasul et al. "Lag-Llama: Towards Foundation Models for Probabilistic Time Series Forecasting" (2024).

To create lag features for a context-length window $x_{1:C}$, a larger window with $L$ more historical points is sampled. Figure 2-5 illustrates this tokenization scheme.

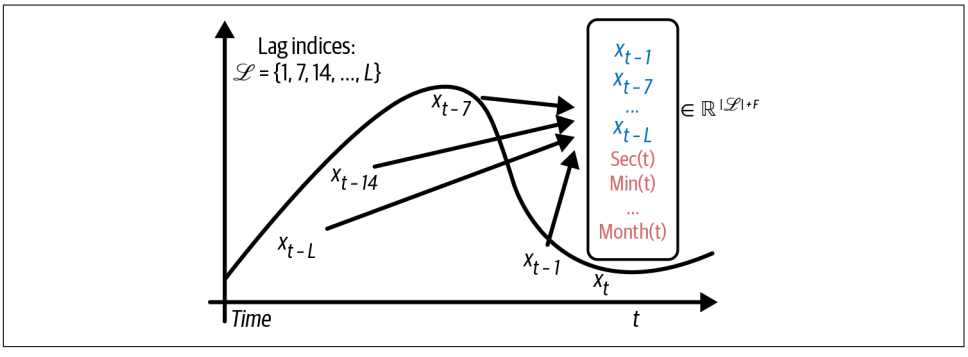


*Figure 2-5. For a time series, tokenization at timestep* t *of the value includes lag features based on an example set of lag indices* L*, where each value in the vector is from the past of* $x_t$*, and* F *possible temporal covariates are constructed from timestamp* t*. Image adopted from Kashif Rasul et al. (2024).*

In addition to these lagged features, date-time features of various frequencies (second-of-minute, hour-of-day, etc., up to quarter-of-year) from the time index $t$ are added. While these date-time features provide additional information, for any time series, all except one date-time feature remain constant from one timestep to the next, allowing the model to implicitly understand the frequency of the time series. With a total of $F$ date-time features, each token is then of size $|\mathscr{L}| + F$.

In 2024, LLMTime[3] introduced a new tokenization method that converts real-valued data into strings of digits after fixing numerical precision and scaling the data.

As you know, tokenization is needed as it impacts the formation of patterns within tokenized sequences and the operations that language models can learn. However, common methods like byte-pair encoding (BPE) can tokenize numbers in a way that complicates arithmetic; for instance, the number

```
42235630
```

might be tokenized as

```
[422, 35, 630]
```

causing different tokenizations with minor changes.

3 Nate Gruver et al. "Large Language Models Are Zero-Shot Time Series Forecasters" (2024).

Newer LLMs like LLaMA tokenize numbers into individual digits by default. To improve tokenization in GPT models, digits in LLMTime are separated with spaces, and commas are used to separate each timestep. Decimal points are dropped for fixed precision. For example:

```
0.123, 1.23, 12.3, 123.0
```

becomes

```
"1 2, 1 2 3, 1 2 3 0, 1 2 3 0 0".
```

This encoding prevents unusual tokens in GPT models. However, in LLaMA models, added spaces can be counterproductive, since each digit and space already has its token. Adding spaces can increase sequence length unnecessarily and potentially make the sequence out-of-distribution. Figure 2-6 compares the tokenization of GPT-based and LLaMA models.

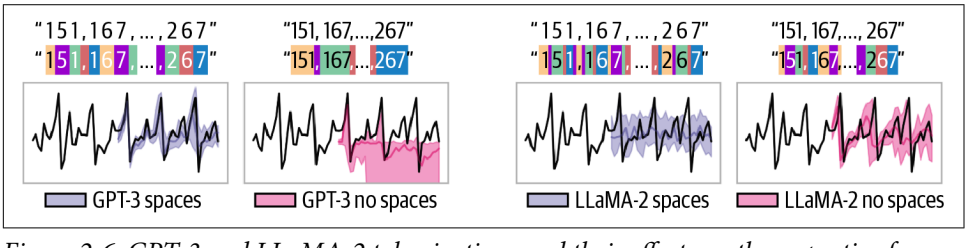


*Figure 2-6. GPT-3 and LLaMA-2 tokenizations and their effects on the respective forecasting performance. Adding spaces helps GPT-3 by creating one token per digit, enhancing performance. In contrast, LLaMA-2 tokenizes digits individually, so adding spaces reduces its performance. Image adopted from Nate Gruver et al. (2024).*

Chronos[4] tokenizes time series values by scaling and quantizing them into a fixed vocabulary, then trains existing transformer-based language models on these tokenized series using *Cross-Entropy Loss*. Figure 2-7 shows this process.

*Cross-Entropy Loss* quantifies the difference between the predicted probability distribution (also called *input logits*) and the true distribution of the target variables.

4 Abdul Fatir Ansari et al. "Chronos: Learning the Language of Time Series" (2024).

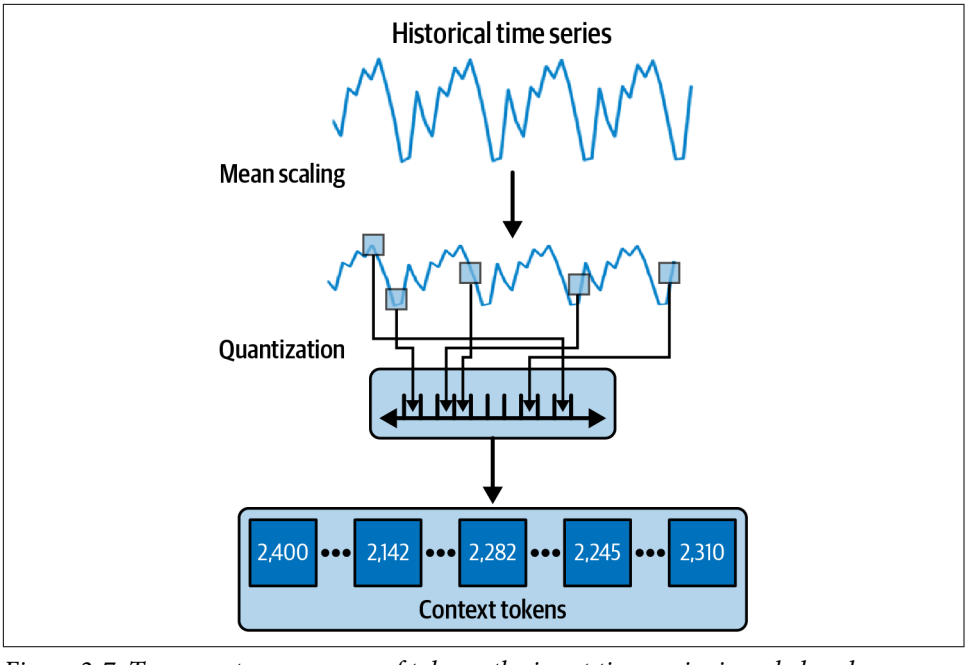


*Figure 2-7. To generate a sequence of tokens, the input time series is scaled and quantized. Image adapted from Abdul Fatir Ansari et al. (2024).*

Chronos is a framework that adapts existing language model architectures and training procedures to probabilistic time series forecasting. This requires specific tokenization modifications to adapt language models for time series data because while both language and time series are sequential, they differ in representation; natural language uses a finite vocabulary, while time series are continuous. To make this more concrete, consider a time series:

$$x_{1:C+H} = \left|x_1, \ldots, x_{C+H}\right|$$

Here the initial $C$ steps provide the historical context, and the next $H$ steps represent the forecast horizon. Since language models use a finite vocabulary, adapting them for time series data involves converting observations $x_i \in \mathbb{R}$ into a finite set of tokens. Chronos achieves this by first scaling the observations and then quantizing them into a fixed number of bins.

Now that you know about the complexity of tokenizing time series data and how some time series transformers handle this challenge, let's move on to the next sections and see how different transformers integrate this into their overall architecture.

## Chronos: Learning the Language of Time Series

Traditionally, forecasting has been dominated by statistical models such as *autoregressive integrated moving average* (ARIMA) models. However, the zero-shot learning capabilities of LLMs have sparked interest in creating foundation models for time series. Chronos trains standard language models on time series without altering the model architecture. Tokens, as described in the previous section, are input into a language model, which can be either an encoder-decoder or a decoder-only model.

Training is done using Cross-Entropy Loss. The following equations show the loss function for a single tokenized time series including the EOS tokens:

$$\ell(\theta) = -\sum_{h=1}^{H+1}\sum_{i=1}^{|V_{ts}|}\mathbf{1}_{(z_{C+h+1}=i)}\log p_\theta(z_{C+h+1}=i \mid z_{1:C+h})$$

Here, $p_\theta(z_{C+h+1}=i \mid z_{1:C+h})$ is the categorical distribution predicted by the model parameterized by $\theta$. $\ell(\theta)$ is the Cross-Entropy Loss function parameterized by $\theta$. The outer summation runs over the prediction horizon and the inner summation over the vocabulary of time series tokens $V_{ts}$. This loss is then averaged over a batch of time series during training.

During inference, the model autoregressively generates tokens and maps them back (dequantizes) to numerical values, sampled from the predicted distribution:

$$p_\theta(z_{C+h+1} \mid z_{1:C+h}), \text{ for } h \in \{1, 2, \ldots, H\}$$

Multiple trajectories are sampled to derive a predictive distribution. This process is illustrated in Figure 2-8.

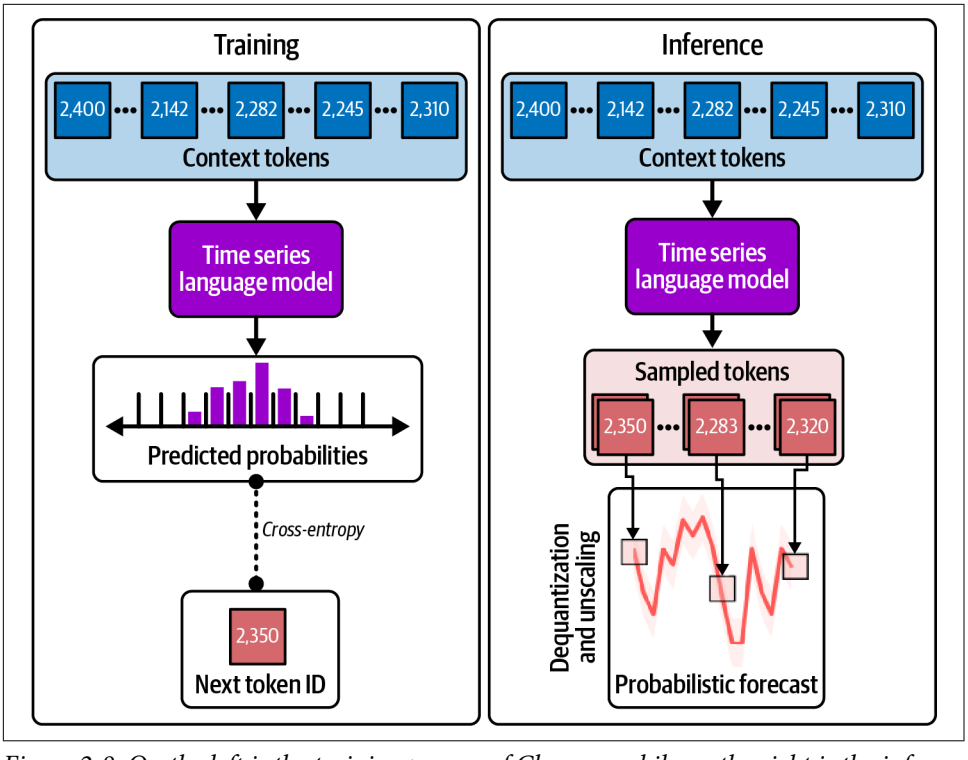


*Figure 2-8. On the left is the training process of Chronos, while on the right is the inference of Chronos producing a probabilistic forecast. Image adapted from Abdul Fatir Ansari et al.*

Chronos is trained on 13 datasets from various application domains, including nature, energy, transport, and the web, with sampling frequencies ranging from five minutes to monthly. Chronos is based on T5,[5] a text-to-text encoder-decoder transformer. Chronos can perform zero-shot time series prediction. An example performance is shown in Figure 2-9.

5 Colin Raffel et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" (2019).

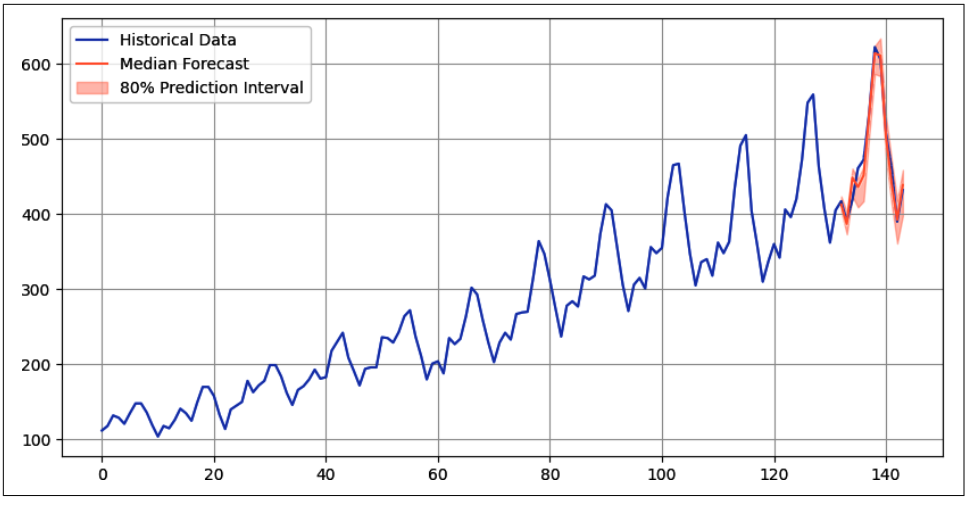


*Figure 2-9. Historical data and zero-shot forecast for air passengers time series. Data is from the book Practical Time Series Analysis (O'Reilly).*

Depending on your data, you'll want to fine-tune the model on your own dataset, which you'll be doing in the next section.

## Fine-Tuning Chronos

For fun, I'll be using the historical stock price data of Amazon, since this model was developed by Amazon. While the code for fine-tuning and training Chronos is available, I recommend using my modified version, which addresses some errors and includes a *requirements.txt* file. I won't show you how to download the stock pricing data here, but you can find all code in the accompanying notebook.

If you want to use Chronos with your time series, you have to convert the data into the Arrow format. Example 2-1 shows a Python function to do this conversion.

*Example 2-1. Function to convert a DataFrame into Arrow format for Chronos*

```
def convert_to_arrow(
    path: Union[str, Path],
    time_series: Union[List[np.ndarray], np.ndarray],
    start_times: Optional[Union[List[np.datetime64], np.ndarray]] = None,
    compression: str = "lz4",
):
    assert len(time_series) == len(start_times)

    dataset = [
        {"start": start, "target": ts} for ts, start in zip(time_series, start_times)
    ]
    ArrowWriter(compression=compression).write_to_file(
```

```
        dataset,
        path=path,
    )
```

Now you can use the function to convert the time series data, as shown in Example 2-2.

*Example 2-2. Convert the time series data to Arrow format*

```
convert_to_arrow("./amazon_train_data.arrow",
        time_series=train_time_series_list, start_times=train_start_times)
convert_to_arrow("./amazon_test_data.arrow",
        time_series=test_time_series_list, start_times=test_start_times)
```

After converting your data, you can set up your training configurations. The `yaml` file to specify your configuration is located in the configs folder of the repo in GitHub. Here you can define the training parameters, such as:

```
context_length
```

Others might include the path to your training data. Alternatively, you can also alter the parameters when you pass your commands into your terminal, as you see in Example 2-3.

*Example 2-3. Command line arguments for fine-tuning Chronos*

```
CUDA_VISIBLE_DEVICES=0 python /chronos-forecasting/scripts/training/train.py
--config /chronos-forecasting/scripts/training/configs/chronos-t5-base.yaml \
--model-id amazon/chronos-t5-base \
--no-random-init \
--max-steps 44600 \
--learning-rate 0.001
```

This command will use one GPU and will train the T5 base model. The training will take around 1 hour in Google Colab and save the model in a folder called

```
output
```

which you can also modify in your `yaml` file.

If you train the model on your own data, you might want to push your model to Hugging Face, so you can use it later for your predictions. Example 2-4 shows you how you can do this.

*Example 2-4. Push fine-tuned model to Hugging Face hub*

```
os.environ["HF_TOKEN"] = "your_token" ❶

pipeline = ChronosPipeline.from_pretrained(
"/content/output/run-3/checkpoint-final")❷
pipeline.model.model.push_to_hub("your-model-name"
, use_auth_token=os.getenv("HF_TOKEN")) ❸
```

❶ Set your Hugging Face token.

❷ Add your path to your Chronos model.

❸ Add your model name.

Now, you can just load your model from Hugging Face, as shown in Example 2-5, and use it for the forecast.

*Example 2-5. Chronos pipeline*

```
pipeline = ChronosPipeline.from_pretrained(
    "your-model-name",
    use_auth_token=os.getenv("HF_TOKEN"),
    device_map="cuda",
    torch_dtype=torch.bfloat16,
)
```

If you just want to test the model on the stock price dataset, you can simply use my fine-tuned version of Chronos. To access the model, use the model path *nicolepcx/chronos-t5-base-fine-tuned-AMZN-EOD*.

The model is fine-tuned with a context length of 21 days and has a prediction horizon of 5 days. The result of the forecast is in the circle in Figure 2-10.

Overall, the result is not bad for an LLM-based time series prediction.

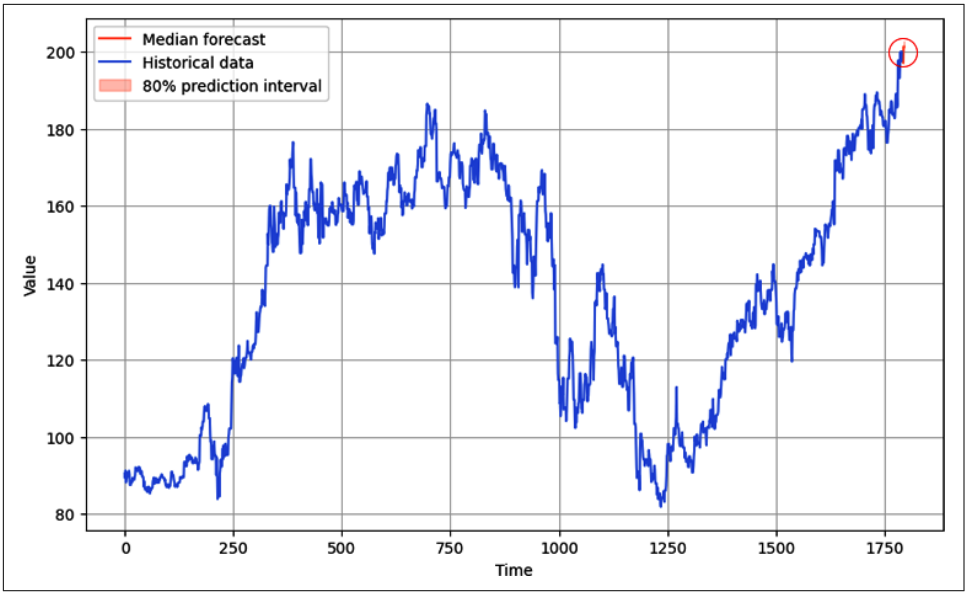


*Figure 2-10. Plot of the results for the 5-day stock-price prediction.*

# PatchTST: A Time Series Is Worth 64 Words

PatchTST[6] introduced not only patching but also *channel independence*. This means that for multivariate time series, each channel consists of a single univariate time series that uses the same embedding and transformer weights across the series. A *multivariate time series* is a multi-channel signal where each transformer input token can represent data from either a single channel or multiple channels.

A previous method for multivariate time series was *channel mixing*, which refers to cases where the input token consists of the vector of all time series features and projects it to the embedding space to mix information. With *channel independence*, each input token contains information from only a single channel. Mathematically, you can represent this as follows:

$$x \in \mathbb{R}^{M \times L}$$

6 Yuqi Nie et al. "A Time Series Is Worth 64 Words: Long-Term Forecasting with Transformers" (2023).

Here, $L$ is the lookback window $x_1, \ldots, x_L$, where each $x_t$ at timestep $t$ is a vector of dimension $M$. Next, you can represent the $i$th univariate series as follows:

$$x^{(i)} \in \mathbb{R}^{1 \times L}$$

Here, the series starts with length $L$ at time index 1, which results in $x_{1:L}^{(i)} = \left(x_1^{(i)}, \ldots, x_L^{(i)}\right)$, where $i = 1, \ldots, M$. This means you split the input, $x_1, \ldots, x_L$, into $M$ univariate series and feed it separately into the transformer. The transformer then provides prediction results in the following form:

$$\hat{x}^{(i)} = \left(\hat{x}_{L+1}^{(i)}, \ldots, \hat{x}_{L+T}^{(i)}\right) \in \mathbb{R}^{1 \times T}$$

This process is illustrated in Figure 2-11.

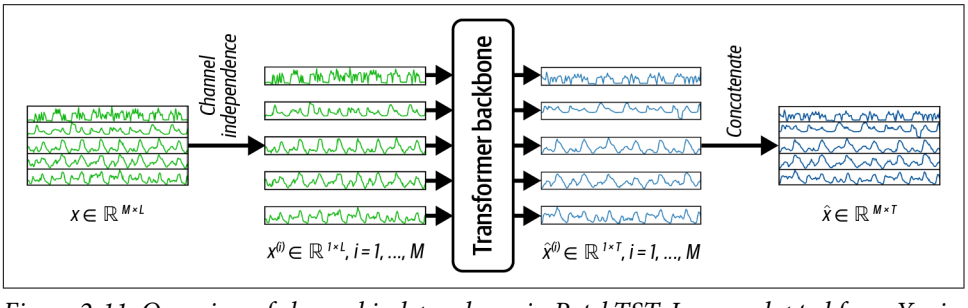


*Figure 2-11. Overview of channel independence in PatchTST. Image adapted from Yuqi Nie et al. (2023).*

PatchTST uses a transformer encoder to map the observed signals to latent representations. The patches are mapped to the transformer latent space of dimension $D$ via a trainable linear projection and a learnable additive positional encoding, to keep the temporal order of the patches. Each multi-head attention head then transforms these inputs into query, key, and value matrices. The multi-head attention block uses BatchNorm and a feed-forward network with residual connections. The overall architecture is illustrated in Figure 2-12.

In the next section, you'll try out PatchTST with your own data to see how this works.

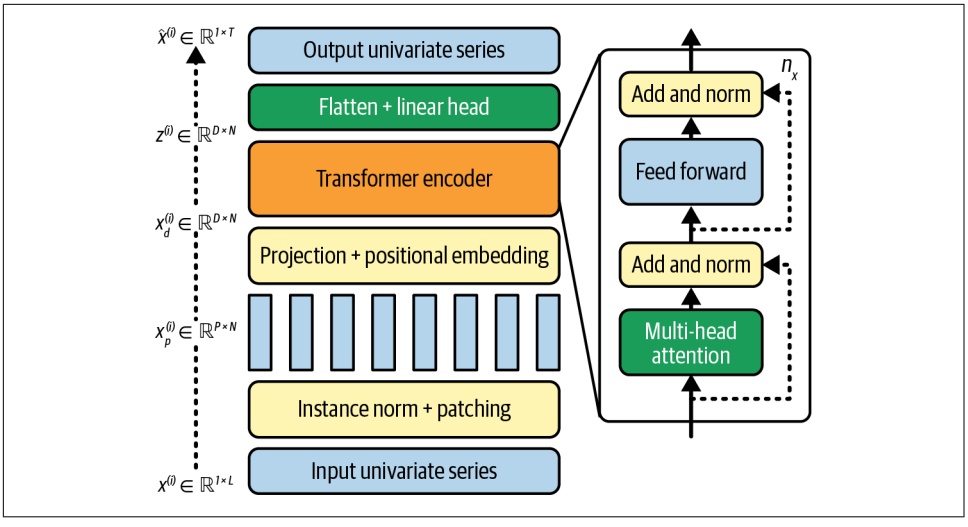


*Figure 2-12. Architecture overview of PatchTST. Image adapted from Yuqi Nie et al. (2023).*

## Fine-Tuning PatchTST on Historical IBM Stock Prices

Again, I thought it would be fun to predict IBM stock prices using PatchTST, since the model was primarily developed by IBM. I'll omit downloading the dataset and preparing the dataset here, but it can all be found in the accompanied notebook for this section. I used the following setup for my model parameter initialization, Optuna hyperparameter setup, and trainer construction.

*Example 2-6. PatchTST setup*

```
context_length = 32
forecast_horizon = 10
patch_length = 8
num_workers = 16
batch_size = 128
```

As a next step, to do the hyperparameter search, you need to set up your study object and specify the direction of the optimization with Optuna. Optuna is a framework for hyperparameter optimization.

*Example 2-7. Optuna hyperparameter trial setup*

```
def optuna_hp_space(trial: optuna.Trial):
    return {
        "learning_rate": trial.suggest_loguniform(
        "learning_rate", 1e-8, 1e-2),
        "per_device_train_batch_size": trial.suggest_categorical(
        "per_device_train_batch_size", [16, 32, 64, 128]),
        "num_train_epochs": trial.suggest_int(
        "num_train_epochs", 50, 300, step=20),
        "dataloader_num_workers": trial.suggest_int(
        "dataloader_num_workers", 0, 16, step=4),
        "weight_decay": trial.suggest_float(
        "weight_decay", 0.0, 0.3, step=0.05),
        "per_device_eval_batch_size": trial.suggest_categorical(
        "per_device_eval_batch_size", [16, 32, 64, 128]),
    }
```

Now you need to initialize the trial.

*Example 2-8. Initialize model for trial*

```
def model_init(trial):
    return PatchTSTForPrediction(config)
```

As the next step, you can initialize your training arguments, as you're used to with the Hugging Face trainer class.

*Example 2-9. Initialize training arguments and trainer class*

```
training_args = TrainingArguments(
    output_dir="./checkpoint/output_dir",
    overwrite_output_dir=True,
    do_eval=True,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    logging_strategy="epoch",
    save_total_limit=3,
    logging_dir="./checkpoint/logging_dir",
    load_best_model_at_end=True,
    metric_for_best_model="eval_loss",
    greater_is_better=False,
    num_train_epochs=200,
    label_names=["future_values"],
)

trainer = Trainer(
    model=None,
    args=training_args,
    train_dataset=train_dataset,
```

```
    eval_dataset=valid_dataset,
    model_init=model_init,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=30,
    early_stopping_threshold=0.00001)]
)
```

As the final step, to start your trial with Optuna, you have to start the trial run, as shown in Example 2-10.

*Example 2-10. Start the hyperparameter search*

```
best_run = trainer.hyperparameter_search(
    backend="optuna",
    n_trials=30,
    direction="minimize",
)
```

After your hyperparameter search is done, you can access the found hyperparameter and fine-tune your PatchTST on your data by defining the training arguments from your best run. Example 2-11 shows how you can do this.

*Example 2-11. Use hyperparameter for fine-tuning*

```
best_hyperparameters = best_run.hyperparameters

training_args = TrainingArguments(
    output_dir="./checkpoint/output_dir",
    overwrite_output_dir=True,
    learning_rate=best_hyperparameters['learning_rate'],
    per_device_train_batch_size=int(
    best_hyperparameters['per_device_train_batch_size']),
    do_eval=True,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    logging_strategy="epoch",
    save_total_limit=3,
    logging_dir="./checkpoint/logging_dir",
    load_best_model_at_end=True,
    metric_for_best_model="eval_loss",
    greater_is_better=False,
    num_train_epochs=200,
    label_names=["future_values"],
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=valid_dataset,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=30,
```

```
    early_stopping_threshold=0.00001)]
)

trainer.train()
```

Now you can evaluate your model and print out your results, as detailed in Example 2-12.

*Example 2-12. Evaluate and print results*

```
results_valid_dataset = trainer.evaluate(valid_dataset)
print("Valid Results:", results_valid_dataset)
results_test_dataset = trainer.evaluate(test_dataset)
print("Test Results:", results_test_dataset)
```

I get the results as shown in Example 2-13.

*Example 2-13. Validation results*

```
Valid Results: {'eval_loss': 0.01785971038043499, 'eval_runtime': 1.0838,
'eval_samples_per_second': 1384.057, 'eval_steps_per_second': 173.468,
'epoch': 34.0}
Test Results: {'eval_loss': 0.04794887453317642, 'eval_runtime': 2.8294,
'eval_samples_per_second': 1086.082, 'eval_steps_per_second': 136.069,
'epoch': 34.0}
```

The results are measured in mean squared error (MSE), and achieving a test set MSE of 0.0479 for a 10-day forecast horizon for a stock can be considered a good result.

# TimesFM: A Decoder-Only Time Series Foundation Model

I'm sure you've heard of the scaling law.[7] The *scaling law* states that model performance improves in a predictable power-law fashion as you increase the amount of data, the number of parameters, and the compute used. Meaning, the sheer volume of data helps the model to generate accurate predictions without using any additional data from the target time series.

This very fact motivated the researchers behind TimesFM[8] to ask themselves a key question: "Can large pretrained models trained on massive amounts of time series data learn temporal patterns that can be useful for time series forecasting on previously unseen datasets?"

7 Jared Kaplan et al. "Scaling Laws for Neural Language Models" (2020).

8 Abhimanyu Das et al. "A Decoder-Only Foundation Model for Time-Series Forecasting" (2017).

Let me just spill the beans here, without making you read through the whole section: the answer is yes! TimesFM demonstrates strong zero-shot capabilities. In some cases, only minimal fine-tuning is required to achieve results comparable to SOTA time series models. This is what you'll be doing in this section, fine-tuning TimesFM on an hourly energy consumption dataset. But first, let me explain the architecture in more detail.

The researchers introduce two pivotal ideas. First, they propose a large-scale, heterogeneous time series corpus that includes both real-world data sources, such as Google Trends and Wikipedia page views, and synthetic datasets. In time series, *heterogeneous* refers to data composed of different types, sources, or temporal patterns, such as varying frequencies, units, or modalities across the series. This variety is essential for learning cross-domain temporal patterns and generalizing across different granularities, from 10-minute weather logs to yearly retail sales. Second, they use a decoder-only transformer with input patching, designed to scale across varying context and forecast lengths while maintaining training efficiency. The key parts of the decoder-only architecture are:

- Patch-based tokenization of time series inputs
- Residual multilayer perceptron (MLP) blocks for embedding
- Positional encodings to preserve temporal order
- Causal self-attention for autoregressive forecasting
- Decoupled input and output patch lengths to allow long-horizon forecasting in fewer steps

Let me break this down for you and explain each part.

After reading the section on PatchTST, this will already feel familiar. As you learned in "PatchTST: A Time Series Is Worth 64 Words" on page 43, PatchTST introduced the concept of patching in time series forecasting, treating consecutive timesteps as a single token, similar to how vision transformers treat image patches. This allowed PatchTST to capture local temporal structure, reduce attention complexity, and enable longer historical lookbacks. TimesFM builds on this idea.

Just like in PatchTST, TimesFM treats segments of the time series as patches. The difference is that while PatchTST uses an encoder-based transformer, TimesFM adopts a decoder-only architecture, more similar to language models like GPT or Llama. That is, each input time series is divided into contiguous, nonoverlapping patches of length $p$.

These patches are passed through a residual MLP block with one hidden layer that has a skip connection. The patches are then transformed into fixed-length vectors of model dimension $D$. Positional encodings are added to these patch embeddings to

maintain their order in time. This results for the $j$-th input token to the subsequent transformer layers as:

$$\mathbf{t}_j = \text{InputResidualBlock}\left(\tilde{\mathbf{y}}_j \odot \left(1 - \tilde{\mathbf{m}}_j\right)\right) + \text{PE}_j$$

$\tilde{y}_j$ is the $j$-th input patch, defined as $\tilde{\mathbf{y}}j = \mathbf{y}p(j-1) + 1{:}pj$. $\tilde{\mathbf{m}}_j$ is the corresponding mask, and $\odot$ denotes element-wise multiplication used to zero out masked values in the input patch, enabling the model to simulate different context lengths during training. This is critical for zero-shot generalization and robustness. $PE_j$ is the positional encoding added to retain the temporal order of the patch token $\mathbf{t}_j$.

This transforms a time series of length $L$ into $N = \lfloor L/p \rfloor$ tokens, just like in PatchTST. However, instead of channel independence and encoder mapping, TimesFM processes these tokens autoregressively (left to right) using causal attention.

The code in Examples 2-14 and 2-15 shows how the classes work together to support the architectural innovation TimesFM introduces for patching and input layers.

*Example 2-14. Residual block in TimesFM*

```
class ResidualBlock(nn.Module):

def __init__(self, input_dims, hidden_dims, output_dims):
    super(ResidualBlock, self).__init__()
    self.input_dims = input_dims
    self.hidden_dims = hidden_dims
    self.output_dims = output_dims

# Hidden nonlinear transformation
self.hidden_layer = nn.Sequential(
    nn.Linear(input_dims, hidden_dims),  ❶
    nn.SiLU(),                           ❷
)

# Output projection
self.output_layer = nn.Linear(hidden_dims, output_dims)  ❸

# Residual connection to project input directly to output_dims
self.residual_layer = nn.Linear(input_dims, output_dims)  ❹

def forward(self, x):
    hidden = self.hidden_layer(x)          ❺
    output = self.output_layer(hidden)     ❻
    residual = self.residual_layer(x)      ❼
    return output + residual               ❽
```

❶ Projects the input patch (for instance, 32 timesteps) to a hidden representation.

❷ Applies a non-linear activation to enhance learning capacity.

❸ Maps the hidden vector to the model's transformer dimension $D$.

❹ Direct residual path ensures that information from the raw patch can bypass transformation.

❺ Transforms each patch into a learnable high-dimensional embedding for inputs and autoregressive outputs.

❻ Nonlinear path models complex local dependencies.

❼ Ensures that even short input patches or sparse data contribute signal.

❽ Adds the residual and MLP outputs to decode longer output patches with fewer autoregressive steps.

This block is used in two places:

- To convert input patches $\tilde{y}_j$ into transformer tokens $t_j$
- To map the decoder output tokens into multi-step predictions over output patches (e.g., 128-step forecasts from 32-step contexts)

*Example 2-15. MLP block in TimesFM*

```
class TransformerMLP(nn.Module):

  def __init__(
      self,
      hidden_size: int,
      intermediate_size: int,
  ):
    super().__init__()
    self.gate_proj = nn.Linear(hidden_size, intermediate_size) ❶
    self.down_proj = nn.Linear(intermediate_size, hidden_size) ❷
    self.layer_norm = nn.LayerNorm(normalized_shape=hidden_size, eps=1e-6) ❸

  def forward(self, x, paddings=None):
    gate_inp = self.layer_norm(x) ❹
    gate = self.gate_proj(gate_inp) ❺
    gate = F.relu(gate) ❻
    outputs = self.down_proj(gate) ❼
    if paddings is not None:
      outputs = outputs * (1.0 - paddings[:, :, None]) ❽
    return outputs + x ❾
```

❶ Projects the normalized input token to an intermediate feature space.

❷ Compresses the expanded representation back to the model's original size *D*.

❸ Layer normalization ensures stable learning even across variable-length patch sequences.

❹ Applies normalization before feeding tokens into the MLP (pre-norm variant).

❺ The first linear transformation expands model capacity.

❻ *Rectified Linear Unit* (ReLU) introduces nonlinearity to model complex intra-token patterns.

❼ Projects the output back to the original embedding space.

❽ Padding-aware masking ensures that masked or padded patches (e.g., partial windows) don't affect the MLP computation.

❾ Adds a residual connection from the input to the output, preserving information across layers.

This MLP is used inside every decoder layer to process each token after causal attention, ensuring that TimesFM can handle:

- Masked input patches during training (to generalize across context lengths)
- Long output patches during decoding (to avoid step-by-step generation when possible)

These two blocks help avoid full-step autoregressive generation while still maintaining decoder-style generalization and ensuring compatibility with variable-length, masked patch inputs. These input tokens are processed through a stack of $n_l$ transformer layers using causal self-attention. This ensures that each output token attends to patches only from the past or present, not the future. This can be mathematically formulated as:

$$\mathbf{o}_j = \text{StackedTransformer}\left(\left(\mathbf{t}_1, \dot{m}_1\right), \cdots, \left(\mathbf{t}_j, \dot{m}_j\right)\right)$$

This design enables autoregressive forecasting where each prediction depends only on previously observed data, mimicking the LLM-style decoding.

After processing, each output token is mapped to a forecast patch using a second residual MLP block. Importantly, TimesFM supports different lengths for input and

output patches (e.g., input length = 32, output length = 128), allowing it to predict longer sequences in fewer steps. This relationship is expressed by the following equation:

$$\hat{y}_{pj+1:pj+h} = \text{OutputResidualBlock}(o_j)$$

This structure enables efficient long-horizon forecasting: rather than generating one step at a time, TimesFM can jump ahead by larger windows. To ensure robustness across varying context lengths, a random patch masking strategy is used during training. This forces the model to learn from incomplete histories, improving its ability to generalize in zero-shot scenarios.

Let's say patch length $p = 32$ and max context length is 512. During training, the model might randomly mask the first 4 points in the first patch, making it see only 28 steps instead of 32. Repeating this across all windows ensures that the model sees every possible context length up to 512.

## Fine-Tuning TimesFM on Hourly Energy Consumption Data

The TimesFM paper comes with an extensive code repository and even with checkpoints for different model sizes on HuggingFace. I cloned the repository and adjusted the provided fine-tuning code for you a bit. Let's look at the important steps in Example 2-16.

*Example 2-16. Fine-tuning TimesFM on hourly energy consumption data*

```
def finetune_model():
    model, hparams, tfm_config = get_model(load_weights=True) ❶

    config = FinetuningConfig(
        batch_size=64,
        num_epochs=5,
        learning_rate=1e-4,
        use_wandb=True, ❷
        freq_type=1, ❸
        log_every_n_steps=10,
        val_check_interval=0.5,
        use_quantile_loss=True ❹
    )

    train_dataset, val_dataset = get_data( ❺
        context_len=192,
        horizon_len=tfm_config.horizon_len,
        freq_type=config.freq_type
    )

    finetuner = TimesFMFinetuner(model, config) ❻
```

```
print("Starting finetuning...")

results = finetuner.finetune( ❼
    train_dataset=train_dataset,
    val_dataset=val_dataset
)

print(f"Finetuning completed after {len
(results['history']['train_loss'])} epochs.")

mae, ctx, future, preds = evaluate_model(model, val_dataset) ❽
plot_forecast(ctx, future, preds, save_path="timesfm_predictions.png") ❾

print(f"Validation MAE: {mae:.4f}")
```

❶ Loads the pretrained TimesFM model along with its hyperparameters and configuration. Set `load_weights=True` to start from the foundation model checkpoint.

❷ Enables logging to Weights & Biases (optional for experiment tracking).

❸ Specifies the frequency encoding type for the time series (e.g., hourly, daily).

❹ Uses quantile loss instead of MSE, which is common in probabilistic forecasting tasks.

❺ Loads the training and validation datasets, specifying context (lookback) length and forecast horizon.

❻ Initializes the finetuner object that wraps the model and training loop logic.

❼ Starts the fine-tuning loop using the specified datasets and config. Returns a history object with training stats.

❽ Evaluates the fine-tuned model on the validation dataset and computes MAE (Mean Absolute Error).

❾ Plots a forecast using the last validation sample and saves it to disk for visual inspection.

Figure 2-13 shows the result of the training.

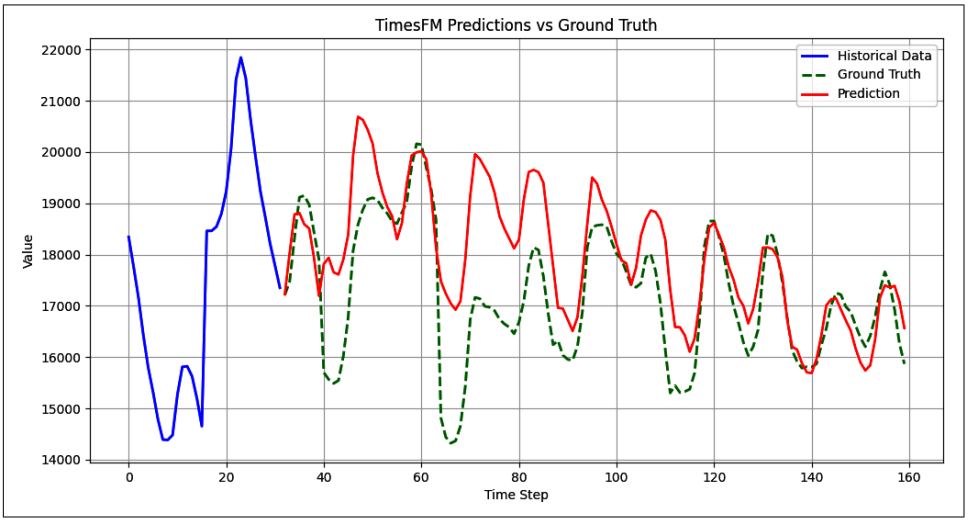


*Figure 2-13. Fine-tuning result after five epochs.*

The result of the fine-tuning doesn't look so bad considering the fact that I fine-tuned the model for less than 30 minutes on an A100 GPU.

# AnomalyBERT for Self-Supervised Anomaly Detection

AnomalyBERT[9] is capable of detecting anomalies in complex time series data. The model is inspired by BERT[10] from natural language processing (NLP), but modifies the masked language modeling by replacing a random portion of the input data with degraded data and training the model to identify the degraded part. Figure 2-14 shows the degradation examples, consisting of soft replacement, uniform replacement, peak noise, and length adjustment.

*Soft replacement* involves substituting the sequence with one sequence fetched from outside the window, representing a weighted sum of the original interval and an external interval. The *uniform replacement* substitutes the sequence with a constant value, while *length adjustment* refers to lengthening or shortening the sequence. *Peak noise* adds a single peak value. This data degradation technique helps in detecting various unnatural patterns in real-world time series.

9 Yungi Jeong et al. "AnomalyBERT: Self-Supervised Transformer for Time Series Anomaly Detection Using Data Degradation Scheme" (2023).

10 Jacob Devlin et al. "BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding" (2018).

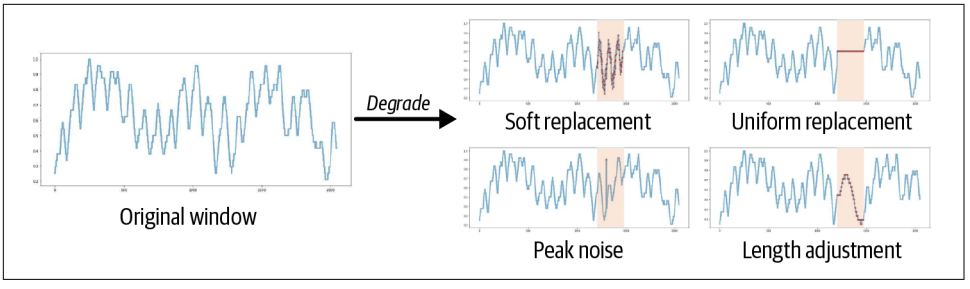


*Figure 2-14. Degradation examples from the training data for AnomalyBERT. Image adapted from Yungi Jeong et al. (2023).*

AnomalyBERT consists of three components: a linear embedding layer, a transformer body, and a prediction block. A window of multivariate time series $X = x_{t0:t1} \in \mathbb{R}^{N \times D}$ is input into the model, and the linear embedding layer projects each data patch $x_{t:t+p}$ in the window $X$ to an embedded feature. The transformer body processes all embedded features from $X$, producing latent features that share information and reflect the temporal context. The prediction block outputs anomaly scores for the data points in the window, with higher scores indicating more anomalous points. This architecture is shown in Figure 2-15.

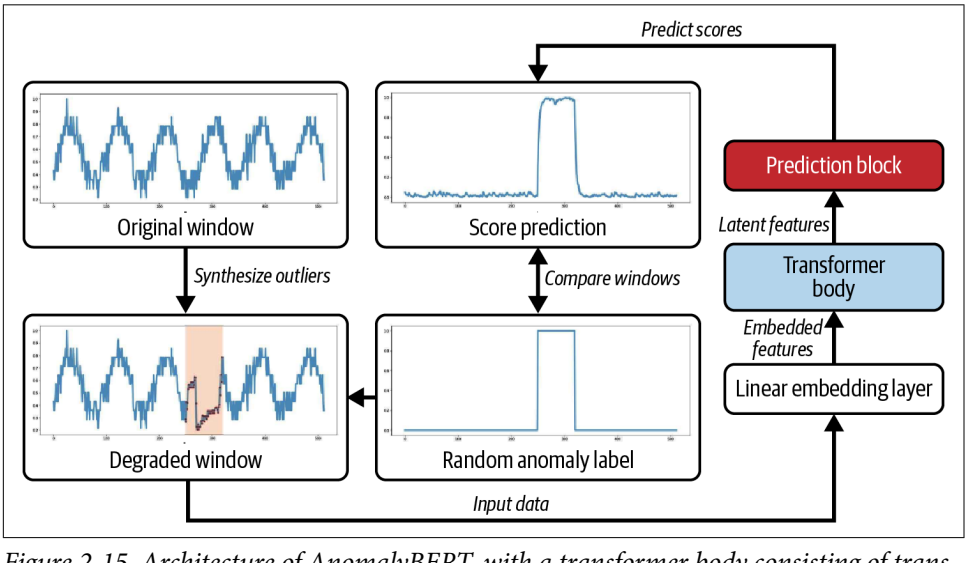


*Figure 2-15. Architecture of AnomalyBERT, with a transformer body consisting of transformer layers that use 1D relative position bias. Image adapted from Yungi Jeong et al. (2023).*

The transformer encoder serves as the main body, with each layer containing a multi-head self-attention (MSA) module and an MLP block. Each module is preceded by a LayerNorm (LN) layer, and the Gaussian Error Linear Unit (GELU) is used for activation. GELU smooths the output using a Gaussian cumulative distribution by retaining input values close to zero and scaling larger values non-linearly. Instead of using sinusoidal positional encodings or absolute position embeddings, one-dimensional relative position bias is added to each attention matrix to consider the relative positions between features. Self-attention in each head with the relative bias is computed as:

$$\text{Attention}\ (Q, K, V) = \ \text{Softmax}\left(\frac{QK^T}{\sqrt{d}} + B\right)V$$

Here, $Q$, $K$, and $V$ are the query, key, and value of input features, respectively, and $d$ is the dimension of features in an attention head. $B$ is the relative position bias:

$$B = \left[b_{i,j}\right] \in \mathbb{R}^{M \times M}$$

Each element $b_{i,j} = \hat{b}_{j-i}$ is derived from a learnable bias table. A different position bias is applied to each MSA module.

AnomalyBERT was tested on five widely used benchmark datasets, each of which contains an unlabeled training dataset and a labeled test set:

*Secure water treatment*
: *Secure water treatment* (SWaT) is a dataset generated from a water treatment testbed, designed to simulate a real-world water treatment process. It includes various sensor and actuator data.

*Water distribution*
: *Water distribution* (WADI) is a dataset collected from a water distribution testbed, capturing the behavior of a water distribution system over time. Like SWaT, it includes sensor and actuator data.

*Soil moisture active passive*
: *Soil moisture active passive* (SMAP) is a dataset from NASA's soil moisture active passive satellite mission. It includes time series data from satellite measurements related to soil moisture.

*Mars science laboratory*
: *Mars science laboratory* (MSL) is a dataset collected from NASA's Mars rover capturing various telemetry data points to monitor the rover's health and detect anomalies.

*Server machine dataset*
: *Server machine dataset* is a dataset collected from servers, recording metrics like CPU usage, memory usage, and network traffic, to simulate real-world issues in server performance.

AnomalyBERT outperforms the previous methods on all datasets.

# Conclusion

In this chapter, you explored the transformative role of transformers in time series modeling and discovered their superiority in capturing long-sequence interactions compared to traditional RNN models.

You delved into key time series concepts such as autocorrelation, cointegration, cross-correlation, and stationarity, understanding their importance in analyzing time series data. Additionally, you learned techniques for handling trends and seasonality and gained insights into the significance of proper data preparation to avoid biases and improve model performance.

You examined different use cases for time series modeling, from forecasting and classification to anomaly detection and spatiotemporal prediction. Along the way, you learned how transformers adapt to these challenges by tokenizing continuous input through techniques like lagged features (Lag-Llama), quantized vocabularies (Chronos), or patch-based segmentation (PatchTST, TimesFM).

You explored how to fine-tune Chronos, PatchTST, and TimesFM. You also looked at how AnomalyBERT uses degraded data to learn robust representations for unsupervised anomaly detection.

From autoregressive decoders to masked transformer encoders, and from quantized digits to continuous patches, this chapter has given you a deep understanding of how transformer-based models are adapted to structured, sequential, and highly temporal data for various domains.

In Chapter 3, you'll learn about transformers for vision tasks, another domain that greatly benefits from foundation models and their few-shot and zero-shot capabilities.

# CHAPTER 3
# Transformers for Vision Tasks

One of the great benefits of transformers in this domain is their few-shot and zero-shot learning capabilities. Labeling images is expensive, and often there are not enough labels available, as is the case with cancer detection. Having an image classification or segmentation model capable of performing well with few samples is a significant leap forward. This makes transformers particularly valuable for tasks where data scarcity is a critical issue, a challenge where traditional convolutional neural network (CNN)-based models often fall short.

Transformers have clearly revolutionized NLP. The next domain that transformers are conquering is vision tasks. The vision transformer (ViT)[1] is a landmark in demonstrating the full potential of transformers in computer vision.

In the sections to come, I'll show you how you can leverage transformers for vision tasks such as classification, image segmentation, instance segmentation, and panoptic segmentation. I'll explain the challenges you might encounter, how to mitigate them when working with images, and how to monitor the training of your model.

Additionally, I'll discuss the impact of different loss functions on the quality of the results. For example, in medical imaging, compound loss functions are often the most robust, as they're better suited for highly imbalanced segmentation tasks. This careful selection of loss functions can significantly enhance model performance, particularly in critical applications like cancer detection.

1 Alexey Dosovitskiy et al. "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale" (2020).

## Overview of Different Vision Tasks

When it comes to vision, there are several distinctions that are important for understanding the various applications and techniques in computer vision. In this section, I'll distinguish between the most commonly used ones: classification, image segmentation, instance segmentation and panoptic segmentation. In later chapters, I'll cover object detection, pose estimation, image captioning, and visual question answering, as they fit better into the video and multimodal transformer model cases:

*Classification*
: *Classification* is the process of predicting the category or class of an object within an image. This task involves assigning a label to the entire image based on the dominant object or feature present. For instance, in a dataset of animal images, a classification model might categorize each image as "cat," "dog," "bird," etc. The main goal here is to identify the object as a whole without considering its location or specific parts.

*Semantic image segmentation*
: *Semantic image segmentation* goes a step further into the image by dividing it into multiple segments or regions, each presenting a different object. This approach classifies each pixel in the image into a category. For example, in an image of a street scene, pixels might be classified as "road," "car," "pedestrian," or "building." The primary objective is to understand the image at a pixel level, where each segment represents a class label.

*Instance segmentation*
: This method dives even deeper, dividing the image not only into multiple segments or regions but even into different parts of an object within an image. That is, *instance segmentation* distinguishes between different objects of the same category. In the same street-scene example, instance segmentation would not only label pixels as "car" but also differentiate between individual cars. This provides a more detailed understanding of the image by identifying separate instances of objects.

*Panoptic segmentation*
: *Panoptic segmentation* combines both instance segmentation and semantic image segmentation to offer a comprehensive understanding of an image. It labels each pixel in the image with the class of the object (e.g., "car," "road," "pedestrian") and also distinguishes between different instances of those objects. This approach provides a detailed segmentation of the scene, capturing both the objects and their individual instances coherently. Figure 3-1 highlights the differences between semantic image, instance, and panoptic segmentation.

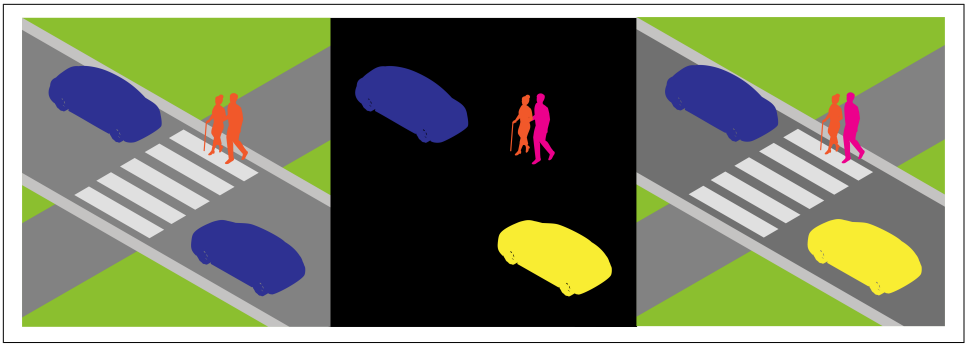

*Figure 3-1. The left image shows semantic image segmentation, the middle image illustrates instance segmentation, and the right image displays panoptic segmentation, which clearly combines the two segmentation variations into one comprehensive approach.*

Now that you have an understanding of the differences in these image tasks, let's explore how transformers are designed to understand and process images. In the next section, you'll learn how embeddings and tokenization, which are foundational for enabling transformers to effectively handle various vision tasks, are implemented in vision models.

## Embeddings and Tokenization for Vision Models

Convolutional neural networks (CNNs) have dominated vision models from 2011 to 2020, starting with the combination of GPUs and CNNs winning a series of competitions between 2011 and 2012. However, convolutions usually operate on regular grids, making it challenging to integrate elements like tokens or positional embeddings into these networks. This architectural limitation has been overcome with the introduction of vision transformers in 2020. But what is the difference between vision and language, and why do the tokens and positional embeddings need to be different?

To answer this question, you need to understand the significant difference in information density between language and vision. Languages are human-generated signals that are highly semantic and information-dense. The task of training a model to predict only a few missing words per sentence requires a sophisticated understanding of language.

In contrast, images are natural signals with substantial spatial redundancy. A missing patch in an image can often be reconstructed from neighboring patches with little need for high-level understanding of parts, objects, and scenes.

This difference directly influences why ViTs use patches to process images: to capture the global context of an image rather than relying solely on local pixel continuity. By breaking the image into patches, ViT can process each patch as a token, similar to how language models process words in NLP tasks. This approach allows the model to

learn relationships between different parts of the image, effectively "disconnecting" neighboring pixels and enabling a more comprehensive understanding of the image as a whole. Example 3-1 demonstrates how ViT breaks images into patches.

*Example 3-1. Patch embedding in ViT*

```
class PatchEmbedding(nn.Module):
    def __init__(self, image_size=28, patch_size=7, channels=1, dim=64):
        super().__init__()
        assert image_size % patch_size == 0, """The image dimension must
        be evenly divisible by the patch size, e.g., image_size=28, patch_size=7."""
        num_patches = (image_size // patch_size) ** 2
        patch_dim = channels * patch_size ** 2

        self.patch_size = patch_size

        self.pos_embedding = nn.Parameter(torch.randn(1, num_patches + 1, dim))
        self.patch_to_embedding = nn.Linear(patch_dim, dim)
        self.cls_token = nn.Parameter(torch.randn(1, 1, dim))

    def forward(self, img):
        p = self.patch_size

        # Rearrange the image into patches
        x = rearrange(img, 'b c (h p1) (w p2) -> b (h w) (p1 p2 c)', p1=p, p2=p) ❶
        x = self.patch_to_embedding(x) ❷

        # Add classification token and positional embedding
        cls_tokens = self.cls_token.expand(img.shape[0], -1, -1)
        x = torch.cat((cls_tokens, x), dim=1) ❸
        x += self.pos_embedding ❹
        return x
```

❶ The image is divided into patches using the `rearrange` function from the einops library.

❷ Each patch is flattened and passed through a linear layer to obtain patch embeddings.

❸ A learnable classification token (`cls_token`) is prepended to the sequence of patch embeddings.

❹ Positional embeddings are created and added to the patch embeddings to retain information about the position of each patch within the image.

I use the well-known Modified National Institute of Standards and Technology (MNIST) handwritten digit dataset to demonstrate how this works. You can try it

yourself by using the plotting function provided in the notebook for this section. You'll get the output shown in Figure 3-2.

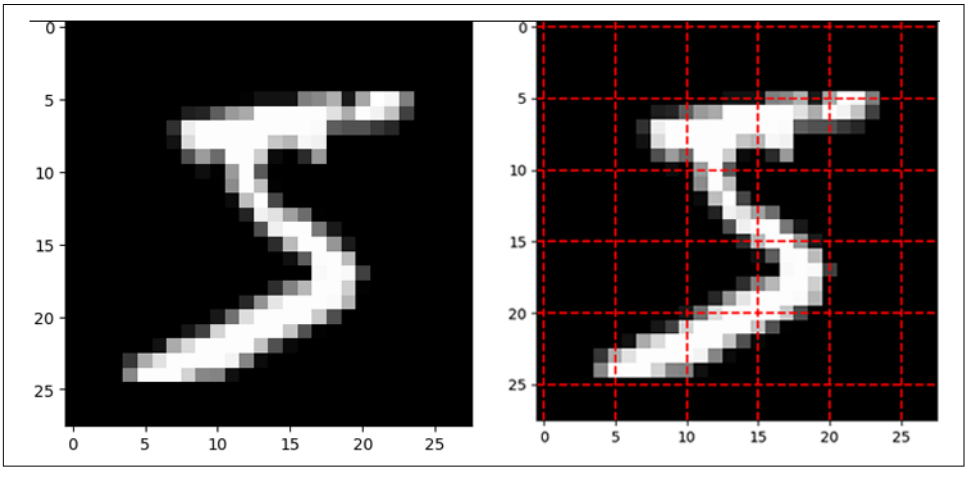


*Figure 3-2. Left is the original image; on the right, with the dashed lines, is the patched image.*

For handling, the ViT reshapes the 2D images, which are $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$, into a sequence of flattened 2D patches, $\mathbf{x}_p \in \mathbb{R}^{N \times \left(P^2 \cdot C\right)}$, where $(H, W)$ represents the resolution of the original image and $C$ denotes the number of channels. The transformer maintains a constant latent vector size $D$ across all its layers, so the patches are flattened and mapped to $D$ dimensions using a trainable linear projection:

$$\mathbf{z}_0 = \left[\mathbf{x}_{\text{class}}; \mathbf{x}_p^1\mathbf{E}; \mathbf{x}_p^2\mathbf{E}; \cdots; \mathbf{x}_p^N\mathbf{E}\right] + \mathbf{E}_{pos}, \quad \mathbf{E} \in \mathbb{R}^{\left(P^2 \cdot C\right) \times D}, \mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D}$$

Similar to BERT's `[class]` token, a learnable embedding is appended to the sequence of embedded patches ($z_0^0 = x_{class}$), with its state at the output of the transformer encoder ($z_0^L$) serving as the image representation ($y$) in $\mathbf{y} = \text{LN}\left(\mathbf{z}_L^0\right)$.

During both pretraining and fine-tuning, a classification head is attached to $z_0^L$. This classification head is implemented as an MLP with one hidden layer during pretraining and a single linear layer during fine-tuning. To retain positional information, the model adds standard learnable 1D positional embeddings to the patch embeddings. The resulting sequence of embedding vectors serves as input to the encoder.

The methods introduced by the ViT, for handling embeddings or tokens, are still used in vision transformers. In fact, as of now, there are no widely known

transformer-based vision models that don't leverage the techniques established by ViT. Figure 3-3 shows an overview of this architectural setup.

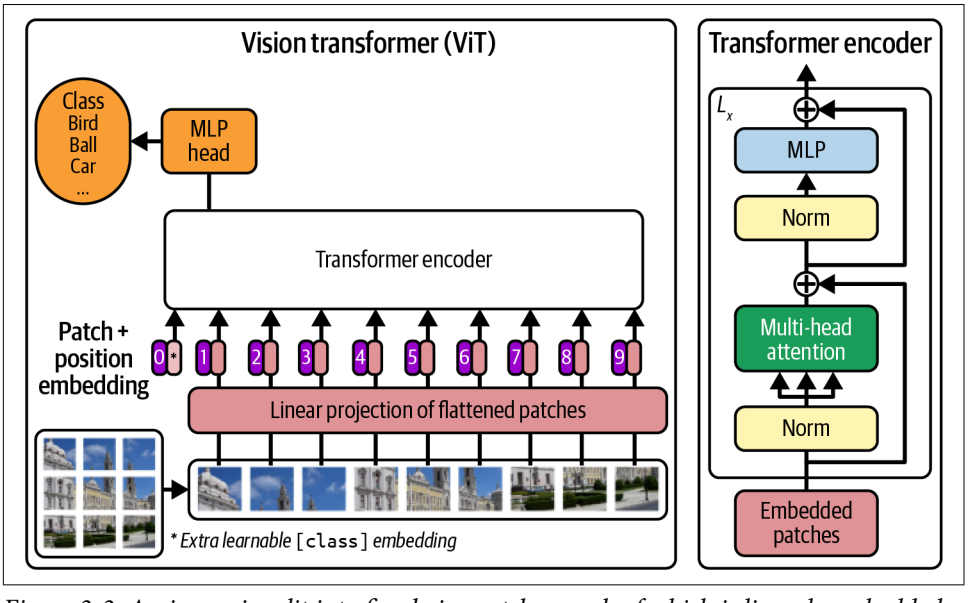


*Figure 3-3. An image is split into fixed-size patches, each of which is linearly embedded. Position embeddings are then added, and the resulting sequence of vectors is fed into a standard transformer encoder. For classification, an extra learnable "classification token" is added to the sequence. Image adopted from Alexey Dosovitskiy et al.*

The next section takes a short detour to show how you can enhance the generalization of your vision model, before you dive deeper into using vision transformers.

## Key Strategies for Improving the Robustness and Effectiveness of Vision Tasks

When performing vision tasks, the underlying data plays a crucial role. Not only is the variety of the data important but also the quality of the images and the specific task at hand. For instance, segmenting roofs from satellite images is fundamentally different from detecting cancer. Satellite imagery often covers large areas with varying resolutions and requires handling complex backgrounds, whereas medical images demand high precision and are usually high resolution to capture fine details. Depending on your task, you need to choose the appropriate loss function, augmentation techniques, and regularization methods to achieve the best results. Therefore, this section aims to offer some ideas and resources to help you get started and inspire your own projects. To make the differences clearer, I'll use roof segmentation and cancer detection as examples for comparison:

*Data augmentation*

Standard methods use data augmentation to flip, rotate, or scale images. These methods help the model to generalize better and are usually sufficient for tasks like segmenting a roof from a satellite image. However, when it comes to cancer detection, you have to apply advanced data-augmentation techniques such as elastic deformations, random cropping, and intensity variations. These techniques help the model generalize better by simulating the variability found in real medical images.[2]

*Regularization techniques*

Common regularization techniques can be effective for tasks like segmenting roofs from satellite images but may not be as suitable for cancer detection.

*Dropout* prevents overfitting by ensuring that the model doesn't rely too heavily on any individual neurons. This method is effective for tasks where the dataset might not be extremely large and overfitting is a common concern.

*L2 regularization* (weight decay) adds a penalty proportional to the sum of the squared weights of the model. This is particularly helpful if the model has a large number of parameters, because it helps to prevent the weights from becoming too large and thereby reduces overfitting, which is typical in high-resolution image segmentation tasks. *Early stopping* is another effective regularization technique. It involves monitoring the model's performance on a validation set and stopping training when performance stops improving, thus preventing overfitting.

*Loss functions*

Choose loss functions[3] to focus on spatial accuracy and boundary precision, such as *Intersection over Union* (IoU) loss or *Dice Loss*. These loss functions are tailored to handle large-scale, high-resolution images where the primary challenge is accurately delineating roof boundaries amid varying background conditions. However, medical image datasets are often highly imbalanced: for instance, cancerous regions are rare compared to healthy tissue. Loss functions like *Focal Loss* or *Compound Loss* (a combination of Dice Loss and Cross-Entropy Loss) are typically used to address this imbalance and focus the model's learning on the difficult, minority-class instances. Focal Loss is designed to address the classification of rare classes by introducing a modulation term to the standard Cross-Entropy Loss. This modulation term down-weights the loss assigned to well-classified examples, thereby focusing more on hard, misclassified examples. Dice Loss

2 Manuel Cossio. "Augmenting Medical Imaging: A Comprehensive Catalogue of 65 Techniques for Enhanced Data Analysis" (2023).

3 Jun Ma et al. "Loss Odyssey in Medical Image Segmentation" (2021).

measures the overlap between predicted and true regions by computing the Dice Coefficient, which ranges from 0 to 1. Dice Loss is 1 minus the Dice Coefficient, emphasizing correct predictions for both foreground and background classes and effectively handling class imbalances by focusing on the overlap between predicted and true regions. *Dice Focal Loss* puts high emphasis on the segmentation boundaries, ensuring accurate boundary delineation, which can be helpful to distinguish healthy from cancerous tissue.

#### Data Augmentation Libraries

For data augmentation tasks like random cropping and flipping, I recommend the torchvision.transforms library. Additionally, libraries such as Albumentations and Kornia perform various augmentation tasks efficiently.

As you've learned, you must tailor your choice of loss functions and regularization techniques to the specific characteristics and challenges of the task at hand. The nature of the images, the distribution of the data, and the specific requirements of accuracy and precision dictate these choices, highlighting the need for a customized approach in each scenario.

## Swin Transformer V2

Since the introduction of AlexNet,[4] architectures have grown deeper and larger, significantly advancing various visual tasks and driving the deep learning wave in computer vision, with notable examples such as VGG[5] and ResNet.[6] However, even though CNN architectures have been scaled up to around 1 billion parameters as of this writing in 2025, this increase in size has not necessarily led to proportionate improvements in performance.

Swin Transformer V2[7] is a dense vision model with 3 billion parameters. Its developers had to address several issues. For example, to address instability in training large vision models, they introduced a new normalization configuration called *res-post-norm*. In this approach, each residual block's output is normalized before merging back into the main branch, preventing amplitude accumulation and incorporating an additional layer normalization every six transformer blocks for further stabilization in large model training.

4 Alex Krizhevsky et al. "ImageNet Classification with Deep Convolutional Neural Networks" (2012).

5 Karen Simonyan et al. "Very Deep Convolutional Networks for Large-Scale Image Recognition" (2014).

6 Kaiming He et al. "Deep Residual Learning for Image Recognition" (2015).

7 Ze Liu et al. "Swin Transformer V2: Scaling Up Capacity and Resolution" (2022).

*Scaled cosine attention*, which computes the attention logit of a pixel pair $i$ and $j$, is used to stabilize attention values. Compare this approach with the original self-attention computation, where similarity terms of pixel pairs are calculated as a dot product of the query and key vectors. This often results in attention maps dominated by a few pixel pairs in large visual models, especially in the *res-post-norm* configuration. The formula for scaled cosine attention is as follows:

$$\text{Sim}\left(\mathbf{q}_i, \mathbf{k}_j\right) = \cos\left(\mathbf{q}_i, \mathbf{k}_j\right)/\tau + B_{ij},$$

Here, $B_{ij}$ is the relative position bias between $i$ and $j$, and $\tau$ is a learnable scalar set larger than 0.01 and not shared across heads and layers. This cosine function is naturally normalized, resulting in more stable attention values.

Additionally, Swin Transformer V2 uses a log-spaced continuous position bias (Log-CPB) to handle variations in window size between low-resolution pretraining and high-resolution fine-tuning. More precisely, continuous relative position bias employs a small meta network on the relative coordinates:

$$B(\Delta x, \Delta y) = G(\Delta x, \Delta y)$$

Here, $G$ is a small network, such as a 2-layer MLP with ReLU activation. *ReLU* outputs the input directly if it's positive; otherwise, it outputs zero, introducing non-linearity into the model while being computationally efficient. This network generates bias values for arbitrary coordinates, allowing seamless transfer to fine-tuning tasks with varying window sizes. You can precompute and store bias values for efficient inference. Additionally, the network uses log-spaced coordinates to address the extrapolation needed for varying window sizes:

$$\widehat{\Delta x} = \text{sign}\,(x) \cdot \log(1 + |\Delta x|),$$
$$\widehat{\Delta y} = \text{sign}\,(y) \cdot \log(1 + |\Delta y|),$$

Here, $\Delta x$, $\Delta y$, and $\widehat{\Delta x}$, $\widehat{\Delta y}$ are the linear-scaled and log-spaced coordinates, respectively.

Using log-spaced coordinates reduces the extrapolation ratio significantly compared to linear-spaced coordinates, enhancing the model's performance across different window resolutions. These adaptations enhance the model's scalability and effectiveness in transferring across different window resolutions, leading to improved performance and flexibility. Figure 3-4 shows this architecture.

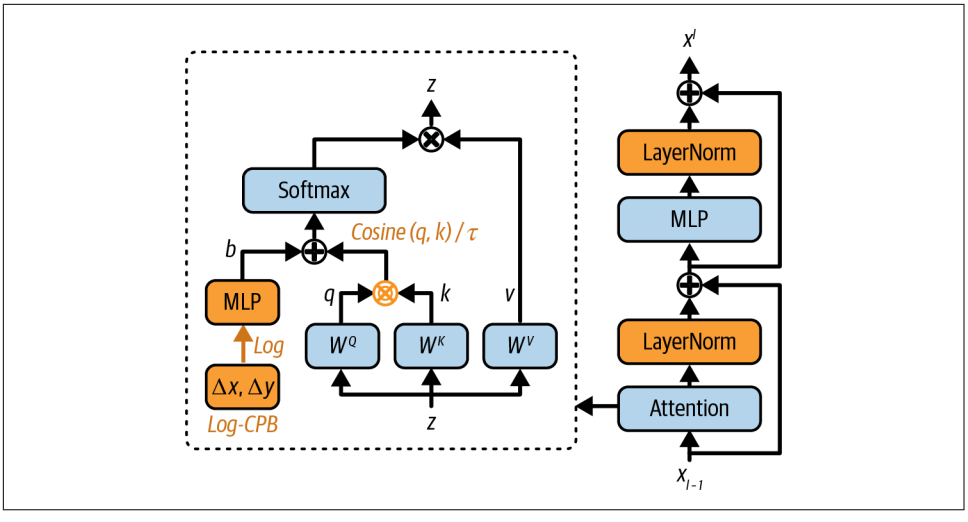


*Figure 3-4. The Swin Transformer V2 architecture incorporates several adaptations for better scaling of model capacity and window resolution: a res-post-norm configuration, scaled cosine attention, and log-spaced continuous relative position bias approach. Image adapted from Ze Liu et al. (2022).*

## Image classification with Swin Transformer V2

Now that the theoretical foundations have been laid out, it's time to see Swin Transformer V2 in action. This section will guide you through the steps to harness its power for your image-classification projects. I'll use the snacks dataset, which is a dataset of 20 different types of snack foods. Feel free to replace it with any other image-classification dataset.

To get an idea what your features are, let's print the labels in your dataset:

```
print(dataset["train"].features['label'].names)
```

This will result in:

```
['apple', 'banana', 'cake', 'candy', 'carrot', 'cookie', 'doughnut', 'grape',
'hot dog', 'ice cream', 'juice', 'muffin', 'orange', 'pineapple', 'popcorn',
'pretzel', 'salad', 'strawberry', 'waffle', 'watermelon']
```

To display an image from the dataset, you can do the following:

```
dataset['test'][1]['image']
```

This will show you the following image:

It's useful to create dictionaries to decode the `ids` and labels in your dataset (see Example 3-2).

*Example 3-2. Create dictionaries to decode the `ids` and labels*

```
labels = dataset["train"].features["label"].names
num_labels = len(dataset["train"].features["label"].names)
label2id, id2label = dict(), dict()
for i, label in enumerate(labels):
    label2id[label] = i
    id2label[i] = label
```

Next, let's load the image processor and model (see Example 3-3).

*Example 3-3. Initialize processor and model*

```
MODEL_PATH = "microsoft/swinv2-tiny-patch4-window8-256"
image_processor  = AutoImageProcessor.from_pretrained(MODEL_PATH)

model = AutoModelForImageClassification.from_pretrained(
    MODEL_PATH,
    label2id=label2id,
    id2label=id2label,
    ignore_mismatched_sizes = True,
)
```

Before you can use the dataset to train your model, you have to preprocess the data (see Example 3-4).

*Example 3-4. Custom image preprocessing and dataset transforms*

```
class ImageProcessor:
    def __init__(self, image_processor):
```

```
        self.normalize = Normalize(mean=image_processor.image_mean,
                                   std=image_processor.image_std)

        if "height" in image_processor.size:
            self.size =
            (image_processor.size["height"],
             image_processor.size["width"])
            self.crop_size = self.size
            self.max_size = None
        elif "shortest_edge" in image_processor.size:
            self.size = image_processor.size["shortest_edge"]
            self.crop_size = (self.size, self.size)
            self.max_size = image_processor.size.get("longest_edge")

        self.transforms = Compose([
            Resize(self.size),
            CenterCrop(self.crop_size),
            ToTensor(),
            self.normalize,
        ])

    def preprocess(self, example_batch):
        example_batch["pixel_values"] = [
            self.transforms(image.convert("RGB")) for image in example_batch["image"]
        ]
        return example_batch

processor = ImageProcessor(image_processor)

train_ds.set_transform(processor.preprocess)
val_ds.set_transform(processor.preprocess)
```

As the next step, you can define your training arguments (see Example 3-5).

*Example 3-5. Define training arguments*

```
args = TrainingArguments(
    f"{model_name}-finetuned-snacks",
    remove_unused_columns=False,
    evaluation_strategy = "epoch",
    save_strategy = "epoch",
    learning_rate=5e-5,
    per_device_train_batch_size=BATCH_SIZE,
    gradient_accumulation_steps=4,
    per_device_eval_batch_size=BATCH_SIZE,
    num_train_epochs=5,
    warmup_ratio=0.2,
    logging_steps=10,
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
)
```

Now pass this to the `trainer` class to train your model (see Example 3-6).

*Example 3-6. Initialize Trainer and start training*

```
trainer = Trainer(
    model,
    args,
    train_dataset=train_ds,
    eval_dataset=val_ds,
    tokenizer=image_processor,
    compute_metrics=compute_metrics,
    data_collator=collate_fn,
)

trainer.train()
```

The model achieves an accuracy of 91.41%, using this setup. To evaluate your results, you can do the following (see Example 3-7).

*Example 3-7. Evaluate results*

```
trainer.evaluate()
```

To test the model at inference time, you can use the pipeline functionality from Hugging Face (see Example 3-8).

*Example 3-8. Run inference with the Hugging Face image classification pipeline*

```
image_processor = AutoImageProcessor.from_pretrained(name)
model = AutoModelForImageClassification.from_pretrained(name)

pipe = pipeline("image-classification", model=model, image_processor=image_processor)
pipe(image)
```

This will give you the following classification results:

```
    [{'label': 'apple', 'score': 0.9996001124382019},
     {'label': 'watermelon', 'score': 8.05784366093576e-05},
     {'label': 'banana', 'score': 7.75724183768034e-05},
     {'label': 'juice', 'score': 5.8855093811871484e-05},
     {'label': 'pineapple', 'score': 4.748155697598122e-05}]
```

Looks like the model correctly classified the snacks in the image as apples.

#### Finding the Right Labeling Tool

I've spent a fair share of my time searching for the optimal labeling tool for segmentation tasks. As a rule of thumb, you should decide whether you want easy integration within your workflow and an extensive Python SDK for your labeling tool. Additionally, consider how many people will be working on labeling the data and whether you want to use the tool to create a feedback loop. This involves completing initial labeling, training the model, and then having the labeling team review the predicted masks to retrain the model based on this feedback.

For the first use case, I recommend looking into Segments.ai's labeling platform. For the latter use case, consider Label Studio. Label Studio is available as open source software and offers various installation methods, including via Docker.

Now that you know how to use transformers for image-classification tasks, let's move on to the next section and leverage transformers for segmentation tasks.

## Segment Anything

You learned that image segmentation involves identifying which pixels in an image belong to an object, making it a core task in computer vision. However, developing an accurate segmentation model is challenging due to the need for technical expertise, access to AI training infrastructure, and large volumes of carefully annotated in-domain data, which can be very costly.

This is where the *Segment Anything Model* (SAM)[8] comes in—it's a foundation model for image segmentation that can be prompted, similar to how you would prompt a language model. These prompts can be bounding boxes, points, text, or basic masks. The model is then trained to output the appropriate mask for the image and the prompt. Besides being a promptable image segmentation model, it excels at zero-shot segmentation tasks.

SAM has three components: an image encoder, a flexible prompt encoder, and a mask decoder. It's built on top of different transformer-based vision models. The image encoder, for instance, is motivated by Masked Autoencoders Are Scalable Vision Learners (MAE)[9] and ViT with minimal adaption.

The prompt encoder uses two types of prompts: *sparse* (points, boxes, text) and *dense* (masks). Points and boxes are represented by positional encodings combined with

8 Alexander Kirillov et al. "Segment Anything" (2023).

9 Kaiming He et al. "Masked Autoencoders Are Scalable Vision Learners" (2021).

learned embeddings for each prompt type, while free-form text is encoded using a text encoder from Contrastive Language-Image Pretraining (CLIP).[10] Dense prompts, such as masks, are embedded with convolutions and combined element-wise with the image embedding.

The *mask decoder* maps the image embedding, prompt embeddings, and an output token to a mask. Inspired by previous designs, it uses a modified transformer decoder block followed by a dynamic mask prediction head. This decoder block uses prompt self-attention and cross-attention in both directions (prompt-to-image embedding and vice versa) to update all embeddings. After two blocks, the image embedding is upsampled, and an MLP maps the output token to a dynamic linear classifier, which computes the mask foreground probability at each image location. Figure 3-5 illustrates the model's architecture.

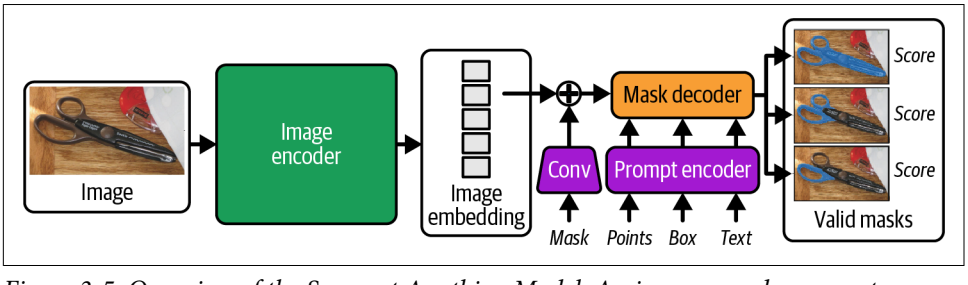


*Figure 3-5. Overview of the Segment Anything Model. An image encoder generates an image embedding, which can be queried using various input prompts (mask, points, box, text) through the prompt encoder. The mask decoder processes these embeddings to produce object masks along with their associated confidence scores. Image adapted from Alexander Kirillov et al. (2023).*

Example 3-9 demonstrates how feeding the model a set of 2D points can predict a segmentation mask. The more 2D points you provide, the more accurate the resulting mask will be.

*Example 3-9. Use 2D points to predict a mask with SAM*

```
input_points = [[[300, 250]]]
show_points_on_image(raw_image, input_points[0])

inputs = processor(raw_image, input_points=input_points,
         return_tensors="pt").to(device)
inputs.pop("pixel_values", None)
inputs.update({"image_embeddings": image_embeddings})
```

10 Alec Radford et al. "Learning Transferable Visual Models from Natural Language Supervision" (2021).

```
with torch.no_grad(): outputs = model(**inputs)

masks = processor.image_processor.post_process_masks(outputs.pred_masks.cpu(),
        inputs["original_sizes"].cpu(), inputs["reshaped_input_sizes"].cpu())
scores = outputs.iou_scores
```

As illustrated in the final part of the SAM overview, this will produce the following output for the scores:

```
tensor([[[1.0003, 0.9940, 0.6576]]], device='cuda:0')
```

You can then use the best fitting score as a basis to create the mask for your segmentation task. The following code shows how you can show the mask on the images (see Example 3-10).

*Example 3-10. Show masks on images*

```
def show_masks_on_image(raw_image, masks, scores):
    if len(masks.shape) == 4:
        masks = masks.squeeze() ❶
    if scores.ndim > 0 and scores.shape[0] == 1:
        scores = scores.squeeze() ❷

    image_array = np.array(raw_image)
    nb_predictions = scores.shape[0] if scores.ndim > 0 else 1
    fig, axes = plt.subplots(1, nb_predictions, figsize=(15, 5 * nb_predictions))
    if nb_predictions == 1:
        axes = [axes]

    for i, mask in enumerate(masks):
        mask = mask.cpu().detach().numpy() ❸

        contours, _ = cv2.findContours((mask * 255).astype(np.uint8),
                    cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE) ❹

        for cnt in contours:
            epsilon = 0.01 * cv2.arcLength(cnt, True)
            approx = cv2.approxPolyDP(cnt, epsilon, True)
            cv2.polylines(image_array, [approx], True, (255, 0, 0), 3) ❺

        axes[i].imshow(image_array)

        if scores.ndim == 0:
            score_text = f"{scores.item():.3f}"
        elif scores.ndim > 0 and scores.numel() == 1:
            score_text = f"{scores.item():.3f}"
        elif scores.ndim > 0:
            score_val = scores[i].item() if scores[i].numel() == 1 else scores[i]
            score_text = f"{score_val:.3f}" if
                isinstance(score_val, float) else "Multiple"
```

```
        axes[i].set_title(f"Mask {i+1}, Score: {score_text}")
        axes[i].axis("off")
```

❶ Remove any extra dimensions from the masks.

❷ Remove any extra dimensions from the scores.

❸ Convert the mask tensor to a NumPy array.

❹ Find contours in the mask.

❺ Draw the contours on the image.

I use OpenCV, which is an open source computer vision library, to get the contours of the mask and draw the polygon lines. If you want to refine your segmentation, I suggest the segmentation refinement library.

### Variations of SAM

There are several variations of SAM that can help you develop your model more quickly for your specific task. One version is tailored for medical images, with the code freely available.[11] Additionally, a Python package exists for segmenting geospatial data with SAM. Grounding DINO aims to detect and segment anything using text inputs.

HQ-SAM[12] enhances SAM with the ability to accurately segment any object while preserving its original promptable design, efficiency, and zero-shot generalizability. Another variant is ClassWise-SAM-Adapter (CWSAM),[13] which adapts the high-performing SAM for landcover classification on satellite images. This model could be particularly useful for classifying disaster areas after weather damage.

Now it's time to dive in and try SAM out for yourself in the next section.

11 Jun Ma et al. "Segment Anything in Medical Images" (2023).

12 Lei Ke et al. "Segment Anything in High Quality" (2023).

13 Xinyang Pu et al. "ClassWise-SAM-Adapter: Parameter Efficient Fine-Tuning Adapts Segment Anything to SAR Domain for Semantic Segmentation" (2024).

## Fine-Tuning SAM on a Custom Dataset

In this section, you'll learn how you can fine-tune SAM on your custom dataset. I'll show you how you can create a study object and specify the direction of the optimization with Optuna. In addition, you'll log sample images and masks to Weights & Biases. This helps you to compare the ground truth mask and the predicted mask at specified intervals and gives you an additional way to track and improve your model's performance. I'll use a medical image dataset designed to help detect breast cancer.

### Preparing your data for SAM

It's important to understand how to properly prepare your data. Each example in your dataset should include the following components:

*Pixel values*
: This is the image data formatted and ready for the model.

*Prompt*
: A mask, points, bounding box, or text that serves as the input prompt for the model.

*Ground truth segmentation mask*
: The actual segmentation mask for validation.

Example 3-11 presents a function that shows how you can generate a bounding box prompt based on the ground truth segmentation.

*Example 3-11. Get bounding box*

```
def get_bounding_box(mask):
    y_coords, x_coords = np.nonzero(mask > 0) ❶

    x_start, x_end = np.min(x_coords), np.max(x_coords) ❷
    y_start, y_end = np.min(y_coords), np.max(y_coords)

    height, width = mask.shape ❸
    x_start = max(0, x_start - np.random.randint(0, 20))
    x_end = min(width, x_end + np.random.randint(0, 20))
    y_start = max(0, y_start - np.random.randint(0, 20))
    y_end = min(height, y_end + np.random.randint(0, 20))

    bounding_box = [x_start, y_start, x_end, y_end] ❹

    return bounding_box
```

❶ Find the indices of nonzero elements in the mask.

❷ Determine the minimum and maximum x and y coordinates.

❸ Apply random perturbation to the bounding box coordinates. *Random perturbation* applies small, random modifications to data or parameters to enhance robustness and prevent overfitting.

❹ Create the bounding box.

Now, to generate the actual dataset, you can leverage the `Dataset` class from PyTorch (see Example 3-12).

*Example 3-12. Create the dataset*

```
class CustomDataset(Dataset):
    def __init__(self, data, transformer):
        self.data = data
        self.transformer = transformer

    def __len__(self):
        return len(self.data)

    def __getitem__(self, index):
        data_item = self.data[index]
        img = data_item["image"]
        mask = np.array(data_item["label"])

        bounding_box = extract_bounding_box(mask) ❶

        transformed_inputs = self.transformer(img,
        input_boxes=[[bounding_box]], return_tensors="pt") ❷

        transformed_inputs = {key: val.squeeze(0) for key, val in
                              transformed_inputs.items()} ❸

        transformed_inputs["ground_truth_mask"] = mask ❹

        return transformed_inputs
```

❶ Generate the bounding box from the mask.

❷ Prepare the image and bounding box for the model.

❸ Remove the batch dimension added by the transformer.

❹ Include the ground truth mask.

Next, you can use the `DataLoader` class from PyTorch to retrieve batches from the dataset:

```
    train_dataloader = DataLoader(train_dataset, batch_size=2, shuffle=True)
```

### Setting up the model and logging with Weights & Biases

As a next step, you have to load the SAM model. To ensure that gradients are computed only for the mask decoder, freeze the parameters of the vision encoder and prompt encoder (see Example 3-13).

*Example 3-13. Load SAM and freeze the vision and prompt encoders*

```
model = SamModel.from_pretrained("facebook/sam-vit-base") ❶

for param_name, parameter in model.named_parameters(): ❷
    if param_name.startswith(
        "vision_encoder") or param_name.startswith("prompt_encoder"):
        parameter.requires_grad = False
```

❶ Load SAM from Hugging Face.

❷ Ensure that gradients are computed only for the mask decoder.

Now initialize a new Weights & Biases project (see Example 3-14).

*Example 3-14. Initialize a Weights & Biases project*

```
wandb.init(project='image segmentation')
```

### Preparing the hyperparameter tuning

Now it's time to set up the hyperparameter tuning with Optuna (see Example 3-15).

*Example 3-15. Define the Optuna objective and training loop*

```
def objective(trial):
    lr = trial.suggest_float("lr", 1e-6, 1e-3, log=True) ❶
    weight_decay = trial.suggest_float("weight_decay", 0, 1e-3)
    num_epochs = trial.suggest_int("num_epochs", 10, 50)
    sigmoid = trial.suggest_categorical("sigmoid", [True, False])
    squared_pred = trial.suggest_categorical("squared_pred", [True, False])

    model.to(device) ❷
    optimizer = Adam(model.mask_decoder.parameters(), lr=lr,
                     weight_decay=weight_decay)
    seg_loss = monai.losses.DiceFocalLoss(sigmoid=sigmoid,
                squared_pred=squared_pred, reduction='mean')
```

```
    model.train()
    for epoch in range(num_epochs):
        epoch_losses = []
        for batch_idx, batch in enumerate(tqdm(train_dataloader)):
            # Forward and backward passes
            outputs = model(pixel_values=batch["pixel_values"].to(device),
                            input_boxes=batch["input_boxes"].to(device),
                            multimask_output=False)
            predicted_masks = outputs.pred_masks.squeeze(1)
            ground_truth_masks = batch["ground_truth_mask"].float().to(device)
            loss = seg_loss(predicted_masks, ground_truth_masks.unsqueeze(1))

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            epoch_losses.append(loss.item())

        trial.report(np.mean(epoch_losses), epoch)

        if trial.should_prune(): ❸
            raise optuna.exceptions.TrialPruned()

    return np.mean(epoch_losses)
```

❶ Suggest values for the hyperparameters.

❷ Set up the model, optimizer, and loss functions.

❸ Handle pruning based on the intermediate value.

After defining the function with your desired hyperparameters, create a new study to run your trials (see Example 3-16).

*Example 3-16. Create and run an Optuna study*

```
study = optuna.create_study(direction="minimize") ❶
study.optimize(objective, n_trials=5)

print("Best trial:")
trial = study.best_trial
print(f"  Value: {trial.value}")
print("  Params: ")
for key, value in trial.params.items():
    print(f"    {key}: {value}")
```

❶ Create a study object and specify the direction of the optimization.

### Use the best hyperparameters to fine-tune SAM

When the hyperparameter tuning is finished, which usually takes around 4–5 hours, you can access the best parameters directly to fine-tune your model (see Example 3-17).

*Example 3-17. Configure optimizer and loss with best trial parameters*

```
optimizer = Adam(model.mask_decoder.parameters(),
                 lr=trial.params.get("lr"),
                 weight_decay=trial.params.get("weight_decay")) ❶

seg_loss = monai.losses.DiceFocalLoss(sigmoid=trial.params.get("sigmoid"),
                       squared_pred=trial.params.get("squared_pred"),
                       reduction='mean') ❷
```

❶ Use the `trial.params.get ("hyperparameter_name")` to get each needed hyperparameter directly for tuning the model.

❷ Use Dice Focal Loss to make sure the model puts high emphasis on the segmentation boundaries, ensuring accurate boundary delineation.

With the optimizer and loss in place, you now can move on to fine-tuning the model (see Example 3-18).

*Example 3-18. Fine-tune SAM with the selected hyperparameters and log results*

```
num_epochs = trial.params.get("num_epochs")
device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)
model.train()

for epoch in range(num_epochs):
    epoch_losses = []
    for batch_idx, batch in enumerate(tqdm(train_dataloader)):
        outputs = model(pixel_values=batch["pixel_values"].to(device),
                        input_boxes=batch["input_boxes"].to(device),
                        multimask_output=False) ❶

        predicted_masks = outputs.pred_masks.squeeze(1)
        ground_truth_masks = batch["ground_truth_mask"].float().to(device)
        loss = seg_loss(predicted_masks, ground_truth_masks.unsqueeze(1)) ❷

        optimizer.zero_grad() ❸
        loss.backward()

        optimizer.step() ❹
        epoch_losses.append(loss.item())
```

```
        if batch_idx % 5 == 0: ❺
            image_to_log = batch["pixel_values"][0].permute(1, 2, 0).cpu().numpy()❻
            predicted_mask_to_log = predicted_masks[0].cpu().detach().numpy() ❼
            ground_truth_mask_to_log = ground_truth_masks[0].cpu().detach().numpy()

            wandb.log({
                "Input Image": wandb.Image(image_to_log, caption="Input Image"),
                "Predicted Mask": wandb.Image(predicted_mask_to_log,
                                caption="Predicted Mask"),
                "Ground Truth Mask": wandb.Image(ground_truth_mask_to_log,
                                caption="Ground Truth Mask")
            }, commit=False) ❽

    wandb.log({'epoch': epoch, 'mean_loss': mean(epoch_losses)}) ❾

    print(f'EPOCH: {epoch}')
    print(f'Mean loss: {mean(epoch_losses)}')
```

❶ Run forward pass.

❷ Compute loss.

❸ Run backward pass (compute gradients with respect to loss).

❹ Optimize.

❺ Log sample images and masks to Weights & Biases (wandb) at a specified interval.

❻ Select the first sample in the batch for logging.

❼ Convert to Height, Width, Channels (HWC) format for wandb.

❽ Log using wandb, and use `commit=False` to accumulate logs.

❾ Log mean loss for the epoch outside the inner loop.

Logging to Weights & Biases will result in two graphs: one for the mean loss and one for the epochs, as shown in Figure 3-6.

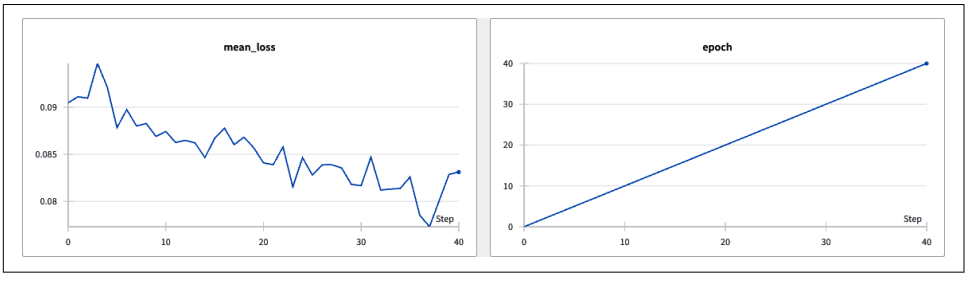


*Figure 3-6. Mean loss and epochs of the training.*

Logging these metrics is beneficial for several reasons. First, tracking the mean loss and epochs helps you to monitor training progress and identify potential issues, such as overfitting or underfitting. By visualizing these metrics, you can make more informed decisions about adjusting hyperparameters or training duration.

In addition to the graphs, you can create a combined panel for the original image, the predicted mask, and the ground truth mask from the logged training data, as shown in Figure 3-7.

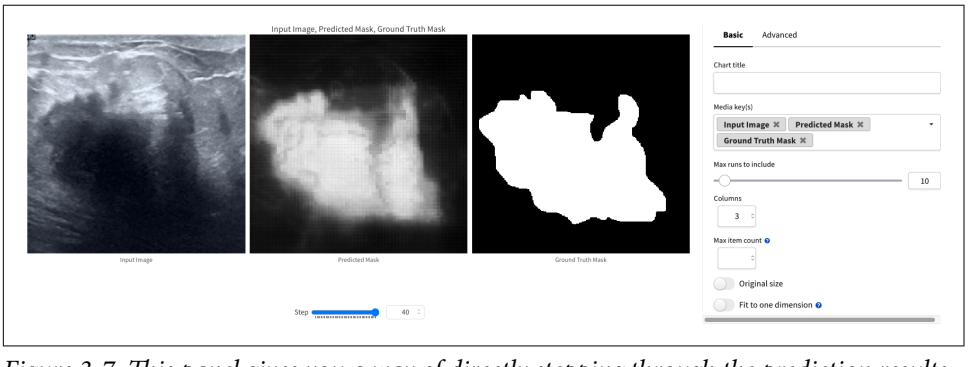


*Figure 3-7. This panel gives you a way of directly stepping through the prediction results of your model.*

This combined panel allows you to qualitatively assess the model's performance. By directly comparing the predictions to the ground truth, you can identify specific areas where the model excels or struggles. This can provide insights into potential improvements in data preprocessing, augmentation strategies, or loss function. Additionally, being able to step through different prediction results helps to ensure that the model's performance is consistent across various samples in the dataset.

In addition, you can very easily create a report from the logged data and share it with team members and other stakeholders.

## Segment Anything in Images and Videos

Even though this chapter focuses on image-based tasks, we can view videos as sequences of images. From this perspective, models that generalize across both domains can be seen as natural extensions of image-based architectures. Segment Anything Model 2 (SAM 2)[14] is exactly such a case: it builds upon the original SAM architecture I introduced you to in the previous section. It extends the model into the temporal domain, enabling promptable segmentation not only in images but also across video frames. Figure 3-8 shows the architecture of SAM 2.

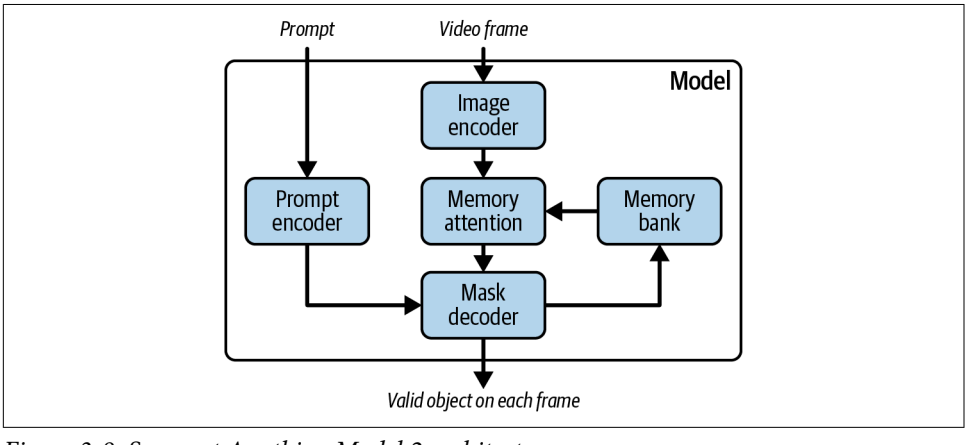


*Figure 3-8. Segment Anything Model 2 architecture.*

The core innovation of SAM 2 is the introduction of a *Promptable Visual Segmentation* (PVS) task, which unifies the segmentation process across both modalities. In this task, you provide a prompt, such as a click, bounding box, or mask, on any frame of a video. The model then generates a corresponding segmentation mask for that frame and propagates this information temporally to generate a consistent segmentation across the entire video sequence. To achieve this, SAM 2 uses a streaming transformer architecture equipped with a memory. The model processes video frames one at a time. For each frame, the model uses a memory attention mechanism that refers back to previous predictions and prompts to maintain temporal consistency. This memory system is composed of spatial memories from past frames, prompted frame memories with corrective information, and object pointers that summarize the object identity. These are stored in fixed-size memory queues, allowing the model to operate efficiently even on long video sequences.

14 Nikhila Ravi et al. "SAM 2: Segment Anything in Images and Videos" (2024).

The core components of SAM 2 resemble those of the original SAM model. The image encoder is based on a hierarchical transformer trained with a masked autoencoding objective. It processes each video frame and produces multiscale feature embeddings. The prompt encoder converts user inputs into embeddings using either positional encoding for sparse prompts or convolutional layers for dense prompts like masks. The mask decoder receives the image features, prompt embeddings, and memory context to produce the segmentation mask for the current frame. This decoder uses two-way attention blocks, similar to the ones in SAM, to refine both prompt and image features. If the object is ambiguous or partially visible, the model predicts multiple candidate masks and ranks them using predicted intersection-over-union scores.

SAM 2 adds a memory encoder that transforms predictions and image features into a representation suitable for storage. These memory entries are used later by the memory attention module to provide context to future frames. When the model encounters frames where the object is no longer visible due to occlusion or motion, it can explicitly predict the absence of the object, avoiding spurious mask generation.

But enough of the theory—let me show you how you can actually use SAM 2 for video segmentation. In the book's repo you find a Jupyter notebook called `segment_videos_with_sam2.ipynb` with all the code. Here, I'll focus on the important parts.

SAM 2 is available in four model sizes, starting from the lightweight "`sam2_hiera_tiny`" with 38.9 million parameters up to the more powerful "`sam2_hiera_large`" with 224.4 million parameters. In the code example, I'll use `sam2.1_hiera_large.pt`, which is the checkpoint for the larger model. How to properly load the model is shown in Example 3-19.

*Example 3-19. Load SAM 2*

```
sam2_checkpoint = "../checkpoints/sam2.1_hiera_large.pt"
model_cfg = "configs/sam2.1/sam2.1_hiera_l.yaml"

sam2_model = build_sam2_video_predictor(model_cfg, sam2_checkpoint, device=device)
```

I have a video prepared for you to test the model out. It's important for you to know that you need to split the video in separate images before you can use SAM 2. Figure 3-9 shows the first frame of the video.

*Figure 3-9. First frame of the video.*

To convert a video into frames of images, you can use FFmpeg. FFmpeg is a multimedia framework that handles nearly any video format. It supports decoding, encoding, streaming, and filtering, and it runs on Linux, macOS, and Windows. We'll run it in Google Colab—that is, in a Linux environment (see Example 3-20).

*Example 3-20. Run in Linux environment*

```
!apt-get update
!apt-get install ffmpeg

!mkdir -p frames
!ffmpeg -i movie_players.mp4 -q:v 2 -start_number 0 frames/%05d.jpg
```

This code first installs the library and then creates a new folder and converts the video into *jpg* files.

To segment with SAM 2, we need to initialize an inference state for the video, using stateful inference for interactive video segmentation. During this initialization, all *jpg* frames from your `frames_path` are loaded, and their pixel data is stored in `inference_state = sam2_model.init_state(video_path=video_dir)`.

To make it easier to select an object from the image frame in a Jupyter notebook, you can use Jupyter BBox Widget. To use this library, you need to create an `object` class, and then you can easily annotate your frame with bounding boxes and convert them into points for SAM 2. This is shown in Example 3-21.

*Example 3-21. Use BBox Widget for easily creating points*

```
OBJECT = ['ball']

widget = BBoxWidget(classes=OBJECT)
widget.image = encode_image("/content/frames/00000.jpg")

box = widget.bboxes[0] if widget.bboxes else default_box[0]
points = np.array([[box['x'], box['y']]], dtype=np.float32) ❶
points
```

❶ Convert box to point format expected by the model.

To apply point prompts across all video frames, you need to use the `propagate_in_video` generator. Each iteration yields `frame_idx`, the index of the current frame; `object_ids`, the IDs of detected objects; and `mask_logits`. The associated logit values can then be converted into masks through thresholding. Example 3-22 shows the first step of the process.

*Example 3-22. Add points to predictor*

```
ann_frame_idx = 0 ❶
ann_obj_id = 1  ❷

labels = np.array([1], np.int32)
_, out_obj_ids, out_mask_logits = sam2_model.add_new_points_or_box(
    inference_state=inference_state,
    frame_idx=ann_frame_idx,
    obj_id=ann_obj_id,
    points=points,
    labels=labels,
)
```

❶ The frame index to interact with

❷ Unique ID for each object

Next, you can propagate through the frames, as demonstrated in Example 3-23.

*Example 3-23. Add points to predictor*

```
video_segments = {} ❶
for out_frame_idx, out_obj_ids,
    out_mask_logits in sam2_model.propagate_in_video(inference_state):
    video_segments[out_frame_idx] = {
        out_obj_id: (out_mask_logits[i] > 0.0).cpu().numpy()
        for i, out_obj_id in enumerate(out_obj_ids)
    }
```

❶ Contains the per-frame segmentation results.

After that, you can simply select frames from the image to plot. The results of the segmentation of the ball in the video at frame 0 and frame 120 is shown in Figures 3-10 and 3-11, respectively.

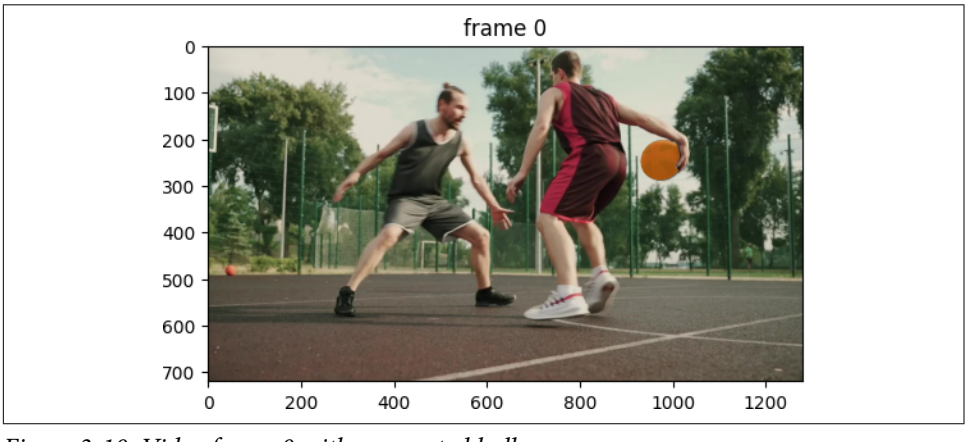


*Figure 3-10. Video frame 0 with segmented ball.*

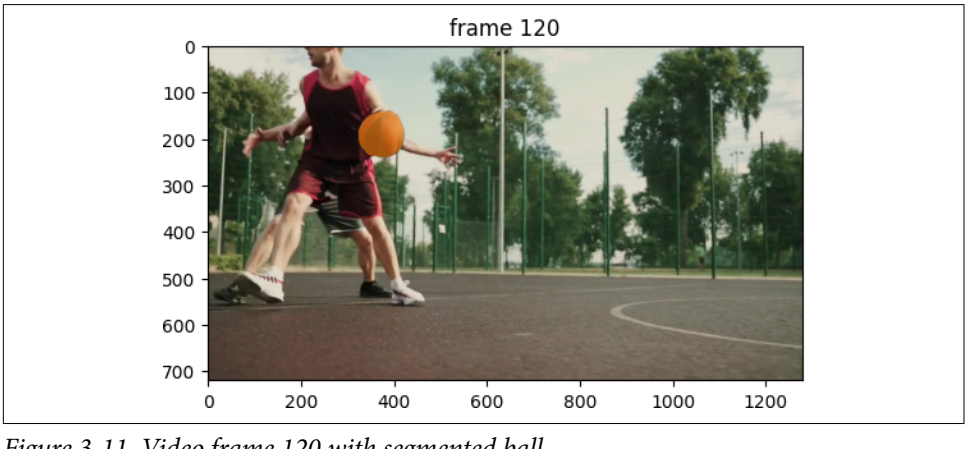


*Figure 3-11. Video frame 120 with segmented ball.*

The result is very good. SAM 2 successfully segmented the ball even in front of one of the players.

However, SAM 2 improved over SAM, and not just in terms of adding video segmentation. It segments images six times faster than SAM and requires three times fewer interactions to reach the same quality. To achieve this performance, the model was trained on a large-scale dataset called *SA-V*. The dataset includes over 35 million masks from more than 50 thousand videos. It was collected using a model-in-the-loop annotation engine that integrates human feedback and iterative prompting to refine annotations.

**SAM 2 Integrations**

For a seamless experience, you can try SAM2 Studio, a native macOS app developed by Hugging Face that makes image segmentation fast and intuitive. Popular labeling platforms like Label Studio also offer built-in support for SAM 2. Label Studio is available both as an open source tool and an enterprise solution.

With its universal visual segmentation system, SAM 2 is well-suited for real-world applications such as augmented reality (AR), robotics, autonomous navigation, and video editing. It can be used as a plug-and-play solution for segmentation or fine-tuned for domain-specific tasks.

## Segment Videos and Images with Concept Prompts

SAM 3[15] introduces a new way of segmenting images and videos with concept prompting. The Segment Anything models evolved across three generations, each building on the strengths of the previous one. Because these capabilities stack rather than replace one another, this chapter covers all three to show how they connect, how they differ, and how to apply them in practical media workflows. Table 3-1 illustrates this evolution.

15 Nicolas Carion et al. "SAM 3: Segment Anything with Concepts" (2025).

*Table 3-1. SAM model evolution*

| Capability/feature | SAM 1 | SAM 2 | SAM 3 |
|---|---|---|---|
| Core purpose | Segment any object from a single click | Segment and track any object in image/video | Detect, segment, and track instances of any category using text or examples |
| Input modalities | Click | Click, box, or mask prompts | Text prompts, example images, clicks, and follow-up guidance |
| Segment an object from a click | ✓ | ✓ | ✓ |
| Track segmented objects in videos | ✗ | ✓ | ✓ |
| Refine prediction with follow-up clicks | ✓ | ✓ | ✓ |
| Detect and segment matching instances from text | ✗ | ✗ | ✓ |
| Refine detection with visual examples (prompt by example) | ✗ | ✗ | ✓ |
| Example-based concept matching | ✗ | ✗ | ✓ |
| Category-agnostic segmentation | ✓ | ✓ | ✓ (plus text-driven, instance-level detection) |

SAM 3 generalizes SAM 2 by supporting both the traditional *promptable visual segmentation* (PVS) tasks and the new *promptable concept segmentation* (PCS) task for concept-based segmentation. It accepts concept prompts like noun phrases or image exemplars, as well as visual prompts such as points, boxes, and masks, to define objects that are segmented across space and time. Prompts can be iteratively added to refine targets, remove false positives, or recover missed objects. The architecture builds on SAM and modulated end-to-end detector ((M)DETR),[16] combining a dual encoder-decoder detector for image segmentation with a tracker and memory module for video. These components run on top of a shared perception encoder (PE)[17] backbone that aligns vision and language inputs. Figure 3-12 shows an overview of the architecture, as well as the new components of SAM 3.

16 Nicolas Carion et al. "End-to-End Object Detection with Transformers" (2020).

17 Daniel Bolya et al. "Perception Encoder: The Best Visual Embeddings Are Not at the Output of the Network" (2025).

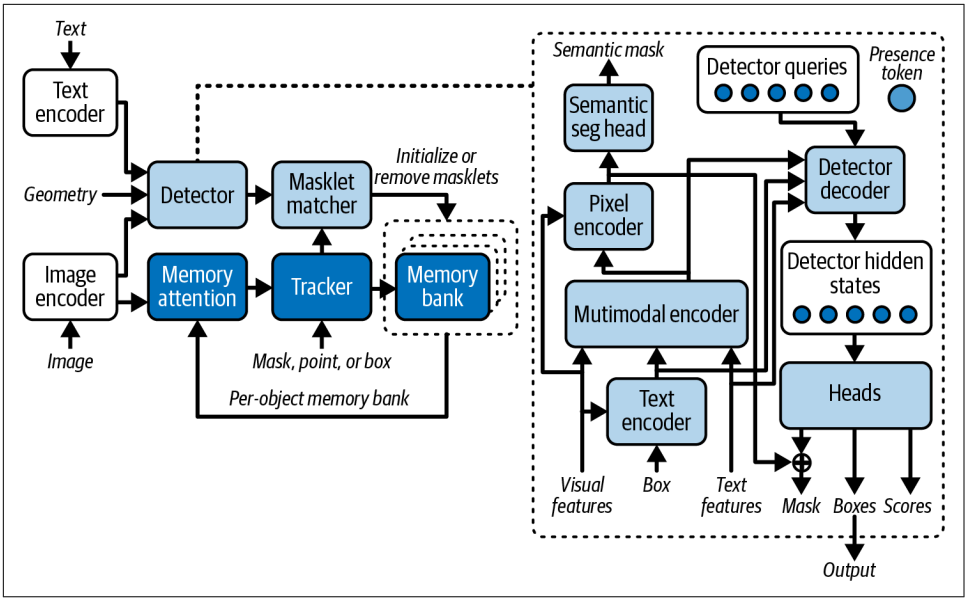


*Figure 3-12. SAM 3 architecture. New components are in light gray, SAM 2 components are in dark gray, and components from the perception encoder are in white. Image adapted from Nicolas Carion et al. (2025).*

The following section explains each component in more detail, describing how they contribute to the overall system and how SAM 3 extends the SAM 2 pipeline to support PCS:

*Detector architecture*
: The detector follows a DEtection TRansformer (DETR)-style design where image features and prompt tokens are fused to condition object detection on text or exemplar inputs. Learned queries attend to these conditioned features to classify and localize objects, predicting both presence and bounding box adjustments. A mask head and semantic segmentation head generate object masks and pixel labels aligned to the prompt.

*Presence token*
: The presence token separates recognition from localization by predicting whether the target concept exists in the image before localization occurs. This prevents detection queries from being overloaded with both global understanding and local positioning, improving reliability when concepts appear ambiguously or sparsely.

*Image exemplars and interactivity*
: Image exemplars provide a bounding box and positive or negative label to guide detection and can be used with or without text prompts. They allow the model to detect all matching instances rather than just one and can be interactively added to correct false positives or false negatives during refinement.

*Tracker and video architecture*
: For video, SAM 3 combines the detector with a tracker and memory module to maintain consistent object identities across frames. New detections are introduced, while tracked objects are propagated forward as masklets, forming spatial-temporal masks that persist through the sequence.

*Tracking an object with SAM 2–style propagation*
: Propagation predicts updated masklet positions for tracked objects using a mechanism similar to SAM 2, sharing the same image encoder and drawing on a memory bank for appearance cues. The tracker updates masks frame-by-frame based on past context, user prompts, and conditioning frames without rerunning full detections each time.

*Matching and updating based on detections*
: Propagated masklets and current detections are aligned with an *intersection over union* (IoU) based matching function to maintain identity. IoU measures object detection performance by calculating how much the predicted bounding box overlaps with the ground truth bounding box. Masklets that fail to match consistently are suppressed, while new detections spawn new masklets. High-confidence detections periodically reset the tracker to correct drift from occlusions or distractors.

*Instance refinement with visual prompts*
: After initial segmentation, users can refine masks with positive or negative clicks. These prompts guide the mask decoder to adjust object boundaries, and in video the refinement propagates across frames to update the corresponding masklet throughout the sequence.

Using SAM 3 is straightforward. Note that I'll omit some code here, but you'll find all code in the notebook of the book's repository. Example 3-24 walks you through loading the model and processor and how to segment an object with a text prompt. Here, I decided to have the model select the cats in the image.

*Example 3-24. SAM 3 for image segmentation*

```
model = Sam3Model.from_pretrained("facebook/sam3").to(device)
processor = Sam3Processor.from_pretrained("facebook/sam3")

image_path = "/content/cats_dog.jpeg" ❶
image = Image.open(image_path).convert("RGB")

inputs = processor(images=image, text="cats",
return_tensors="pt").to(device) ❷

with torch.no_grad():
    outputs = model(**inputs)

results = processor.post_process_instance_segmentation(
    outputs,
    threshold=0.5,
    mask_threshold=0.5,
    target_sizes=inputs.get("original_sizes").tolist()
)[0]
```

❶ Load local image.

❷ Segment using text prompt.

Figure 3-13 shows the image and the masks selected by SAM 3.

*Figure 3-13. Image with mask overlay.*

For SAM 2, I showed you the example of selecting an object in a video and propagating this throughout the video. You can do the same with SAM 3, now just by using the concept prompt "ball." To demonstrate how you can use SAM 3, let's reuse the same video, the two basketball players. Example 3-25 walks you through the important steps to achieve this.

*Example 3-25. Use SAM 3 for video segmentation*

```
model = Sam3VideoModel.from_pretrained(
    "facebook/sam3").to(device, dtype=torch.bfloat16)
processor = Sam3VideoProcessor.from_pretrained("facebook/sam3")

video_url = "/content/movie_players.mp4"
video_frames, _ = load_video(video_url) ❶

inference_session = processor.init_video_session(
    video=video_frames,
    inference_device=device,
    processing_device="cpu",
    video_storage_device="cpu",
    dtype=torch.bfloat16,
)

text = "active ball"
inference_session = processor.add_text_prompt(
    inference_session=inference_session,
    text=text,
)

outputs_per_frame = {}
for model_outputs in model.propagate_in_video_iterator(
    inference_session=inference_session, max_frame_num_to_track=50
):
    processed_outputs = processor.postprocess_outputs(
        inference_session, model_outputs)
    outputs_per_frame[model_outputs.frame_idx] = processed_outputs

print(f"Processed {len(outputs_per_frame)} frames")

frame_idx = 0 ❷
frame_outputs = outputs_per_frame[frame_idx]
masks = frame_outputs["masks"] ❸
frame = video_frames[frame_idx] ❹

overlay = overlay_masks_on_frame(frame, masks, alpha=0.5)
```

❶ List of frames

❷ Visualization of one frame (for example, frame 0)

❸ Shape (`num_objects`, H, W)

❹ Python Imaging Library (PIL) image or NumPy array

Figure 3-14 depicts one video frame of the segmentation.

*Figure 3-14. Segmented video frame.*

If you look more closely at this frame, you can see a potential issue. When you prompt the model with something simple like "ball," it will segment every ball it recognizes, including the one lying quietly in the background. If your goal was to select only the active ball the two players are using, the model has no way to infer that nuance from such a minimal prompt. However, there is a way to handle this. By moving beyond basic prompts, you can guide the model with more specific instructions. If you want to see how to take this further, "Combining Capabilities: SAM 3 Agent" on page 333 demonstrates how to use richer prompts for segmentation, such as "leftmost orange yarn ball."

# Conclusion

In this chapter, you learned about the game-changing impact of vision transformers, such as Swin Transformer V2 and the Segment Anything Model, and the latest SAM 2 for video segmentation.

You explored the key modifications that enhanced the scalability and performance of Swin Transformer V2, including the introduction of the res-post-norm configuration, scaled cosine attention, and log-spaced continuous position bias. These innovations address the challenges of training instability, effective attention computation, and varying window sizes, leading to better model performance.

You then saw how SAM enables prompt-based segmentation with zero-shot capabilities and how SAM 2 builds on this by extending segmentation into the video domain. With a memory-augmented transformer architecture and a unified promptable visual segmentation task, SAM 2 ensures temporal consistency across frames and supports real-time processing. It significantly improves efficiency, requiring fewer interactions and delivering faster predictions, making it ideal for tasks like AR, robotics, and video editing.

SAM 3 takes this evolution one step further by adding concept prompting. Instead of reacting only to visual cues like clicks or boxes, the model can now respond to text instructions, example images, and combined prompts to segment specific instances, categories, or semantic concepts. This expands the promptable visual segmentation task into promptable concept segmentation, where the user defines what to find and the model aligns language, exemplars, and vision features to execute it. The architecture builds on SAM 2 but replaces simple interaction loops with a detector that fuses prompts and image features, a presence token that separates existence from localization, and a tracker that maintains identities across time.

Throughout the chapter, you also worked with Optuna for hyperparameter tuning, and Weights & Biases for tracking and visualizing experiments. These tools streamline your development workflow and provide insights that support more effective model development and evaluation.

In Chapter 4, we stay in the image domain, but now we don't segment or classify them; we generate new ones based on a prompt input.

# CHAPTER 4
# Transformers for Image Generation

A new era of photorealistic image creation has started with the development of text-to-image (T2I) generative models such as DALL·E, Imagen, Midjourney and Stable Diffusion. These models can generate highly detailed and contextually accurate images and artwork from textual inputs, enabling artists, designers, and even non-creative individuals to bring their ideas to life with unprecedented ease.

The core building block of these models is *diffusion*, a process in physics that describes how molecules move from high-concentration to low-concentration areas. In the context of image generation, diffusion models leverage this principle to progressively transform random noise into coherent and detailed images through a series of iterative refinements.

This diffusion-based approach has become the backbone of SOTA models, replacing earlier methods such as Generative Adversarial Networks (GANs),[1] which previously dominated image generation. This shift occurred because diffusion models have proven to be more effective in producing high-quality and diverse outputs. Notably, while the original DALL·E model was autoregressive, DALL·E 2 and DALL·E 3 have since adopted diffusion-based methods as their approach.

Given the current research interest in diffusion models, this chapter will focus on diffusion models such as the first diffusion transformer (DiT), PIXART-α, PixArt-Σ, and DiffiT for image-generation tasks. Additionally, I'll focus on open source models and how you can efficiently generate images with them or fine-tune these models using your own image data.

1 Ian J. Goodfellow et al. "Generative Adversarial Nets" (2014).

## Introduction to Generative Image Models

Denoising diffusion probabilistic models (DDPMs)[2] and score-based generative models[3] have been particularly successful as generative models for images, with convolutional U-Nets[4] serving as the de facto backbone architecture for these approaches. However, these are not the only methods used in generative image modeling. Two other key approaches are GANs and autoregressive models, each with its unique strengths and challenges. The following section highlights the main characteristics and differences between these generative modeling techniques.

*GANs* operate through a game-theoretic framework where two neural networks, the generator and the discriminator, work together. The *generator* creates images, while the *discriminator* attempts to distinguish between real and generated images. Over time, the generator improves its output to the point where the discriminator can no longer tell the difference. However, GANs are extremely difficult to train due to issues like mode collapse, where the generator produces a limited variety of outputs.

*Autoregressive models*, such as the first DALL·E model, generate images in a sequential manner, predicting each pixel or patch of pixels one step at a time based on the preceding pixels. These models, often based on transformer architectures, are powerful in their ability to model dependencies in data. However, they can be computationally expensive, especially for high-resolution images, because they require a vast number of predictions to generate a complete image. In addition, the pixel-by-pixel approach can be slow and may struggle with capturing global structures or coherency in complex images.

*Diffusion models* represent a newer approach that synthesizes images through an iterative denoising process. These models start with random noise and gradually refine it into a coherent image by reversing a diffusion process. Diffusion models offer several advantages over GANs and autoregressive models. They are more stable during training, as they don't rely on adversarial training, and they cover the latent space more comprehensively. Additionally, diffusion models are easier to invert, making them more flexible for tasks like image editing.

2 Jonathan Ho et al. "Denoising Diffusion Probabilistic Models" (2020).

3 Yang Song and Stefano Ermon. "Generative Modeling by Estimating Gradients of the Data Distribution" (2019).

4 Olaf Ronneberger et al. "U-Net: Convolutional Networks for Biomedical Image Segmentation" (2015).

## Diffusion Models: What's That Noise About?

*Diffusion models* add noise, usually Gaussian, during the forward process and are trained to reverse these corruptions, effectively learning to restore the original data. More specifically, a diffusion probabilistic model is a parameterized Markov chain trained through variational inference to generate samples that closely match the data after a finite number of steps. The model learns transitions in this chain that reverse a diffusion process, where noise is gradually added to the data in the opposite direction of sampling until the original signal (image) is completely obscured. The following equation shows how the noise is gradually added to the original data:

$$x_0{:}q(x_t \mid x_0) = \mathcal{N}\left(x_t; \sqrt{\overline{\alpha}_t}x_0, (1-\overline{\alpha}_t)\mathbf{I}\right)$$

Here, $x_0$ is the uncorrupted data point, $x_t$ the data at timestep $t$, which is a noisier version of $x_0$. $\overline{\alpha}_t$ is a hyperparameter that controls the amount of noise added at each step. And $\mathcal{N}$ is the Gaussian distribution, where $x_t$ is normally distributed around a mean value with a specified variance.

The process is designed such that as $t$ increases, more noise is added until the data is completely corrupted; that is, at $t = T, x_T$ it's almost pure noise. The goal of the model is to learn the reverse process: starting from pure noise, the model gradually denoises the data to recover a sample that resembles the original data distribution. Figure 4-1 illustrates this process.

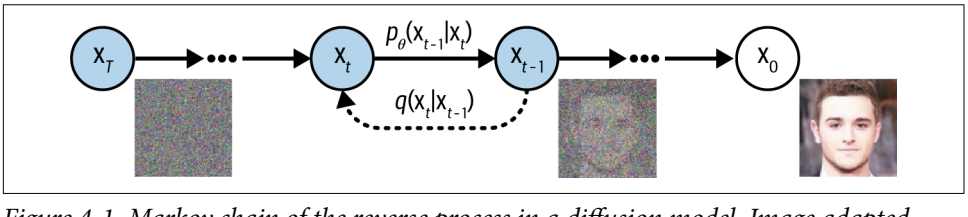


*Figure 4-1. Markov chain of the reverse process in a diffusion model. Image adapted from Jonathan Ho et al. (2020).*

To achieve this, the reverse process model is trained using the variational lower bound of the log-likelihood of the original data $x_0$. The *reparameterization trick* allows for efficient sampling at each timestep $t$ by representing the noised data as:

$$x_t = \sqrt{\overline{\alpha}_t}x_0 + \sqrt{1-\overline{\alpha}_t}\epsilon_t$$

Here, $\epsilon_t$ is sampled from a standard normal distribution $\mathcal{N}(0, \mathbf{I})$.

### Benefits of the Reparameterization Trick

One of the key advantages of the reparameterization trick is that it allows you to avoid using Monte Carlo methods to compute gradients of expectations with respect to the model parameters $\theta$. Monte Carlo estimation introduces stochastic noise into the gradient estimates. This can be inefficient and leads to high variance in the gradient estimates. The reparameterization trick reformulates the expectation in such a way that the randomness is isolated in a variable that is independent of the model parameters $\theta$. This allows you to compute the gradient directly with respect to *theta*, using standard backpropagation, without needing to rely on stochastic gradient estimation via Monte Carlo methods. Diederik Kingma and Max Welling introduced this trick in 2013 in their paper "Auto-Encoding Variational Bayes."[5]

One of the key components in this process is the Kullback-Leibler (KL) divergence. KL divergence allows diffusion models to measure how one probability distribution diverges from a second, expected probability distribution. In this context, it quantifies the difference between the actual transitions of data in the model and the transitions predicted by the model during the reverse process. Essentially, KL divergence provides a mathematical way to assess how well the model's predictions align with the actual changes observed in the data as it moves through various stages of the diffusion process. By minimizing these divergences, the model can more accurately reverse the noise and produce high-quality samples.

Because both the forward process $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ and the reverse process $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ are Gaussian, the KL divergences can be computed analytically using closed-form expressions, instead of high-variance Monte Carlo estimates, which enhances the efficiency and stability of training. This closed-form expression for the KL divergence between two Gaussians is given by:

$$D_{\mathrm{KL}}\left(\mathcal{N}\left(\mu_1, \sigma_1^2\right) \| \mathcal{N}\left(\mu_2, \sigma_2^2\right)\right) = \frac{1}{2}\left(\frac{\sigma_1^2}{\sigma_2^2} + \frac{\left(\mu_2 - \mu_1\right)^2}{\sigma_2^2} - 1 + 2\log\frac{\sigma_2}{\sigma_1}\right)$$

5 Diederik P. Kingma and Max Welling. "Auto-Encoding Variational Bayes" (2013).

This rather complex looking mathematical equation can be very efficiently translated into simple code, as shown in Example 4-1.

*Example 4-1. Code to compute KL divergence in closed-form*

```
kl_divergence = 0.5 * (
    (th.exp(logvar1 - logvar2)) ❶
    + ((mean1 - mean2) ** 2) * th.exp(-logvar2) ❷
    - 1.0  ❸
    + logvar2  ❹
    - logvar1 ❺
)
```

❶ This corresponds to the term $\sigma 1^2/\sigma 2^2$.

❷ This corresponds to the term $(\mu 2 - \mu 1)^2/\sigma 2^2$.

❸ This is the $-1$ in the formula.

❹ This corresponds to the first part of $2 \cdot log(\sigma 2/\sigma 1)$.

❺ This corresponds to the second part of $2 \cdot log(\sigma 2/\sigma 1)$.

This framework of gradually adding and then reversing noise allows diffusion models to capture complex data distributions and generate high-quality samples with better training stability and fewer artifacts compared to other generative models like GANs.

## Classifier-Free Guidance in Diffusion Models

In conditional diffusion models, extra information like a class label ($c$) is used to guide the generation process. This means that during the reverse process, the model tries to generate a sample $x_{t-1}$ based not only on the noisy data at timestep $t$ but also conditioned on the class label $c$. Mathematically, this can be expressed as:

$$p_\theta(x_{t-1} \mid x_t, c)$$

However, instead of strictly following this conditioning, *classifier-free guidance* introduces a technique to guide the sampling process more strongly toward generating samples that are highly likely to belong to a particular class. This is achieved by modifying the reverse process to balance between the conditioned and unconditioned model outputs.

The key idea is to compute a weighted combination of the class-conditioned and unconditioned predictions. If you denote the score (or gradient) of the class-conditioned model as $s(x_t;c)$ and the unconditioned model as $s(x_t; \varnothing)$, the classifier-free guidance modifies the sampling step as follows:

$$\hat{s}(x_t;c) = s(x_t; \varnothing) + \alpha \cdot (s(x_t;c) - s(x_t; \varnothing))$$

Here, $\alpha$ is the guidance scale, where $\alpha > 1$ pushes the model to focus more on the class-conditioned output. When $\alpha = 1$, the model returns to standard sampling.

To train the model for classifier-free guidance, the class label $c$ is randomly dropped out during training and replaced with a learned "null" embedding $\varnothing$. This helps the model learn to generate samples both with and without explicit conditioning, enabling the flexibility of guidance during sampling.

Classifier-free guidance improves the quality of generated samples and helps in producing more accurate and class-consistent images compared to models that don't use this technique.

## Scalable Diffusion Models with Transformers

*Diffusion transformers* (DiTs)[6] successfully replace the U-Net backbone with a transformer. In a February 2024 interview with TechCrunch, Saining Xie, coauthor of the DiT paper and an assistant professor of computer science at NYU Courant, stated, "What transformers contribute to the diffusion process is akin to an engine upgrade."

The training process of the DiT starts with the input image—for instance, in the form of `256 × 256 × 3`. This image is passed through a pretrained variational autoencoder (VAE) with KL loss[7] and the VAE encoder downscales the image by a factor of 8, resulting in a latent representation $z$ of size `32 × 32 × 4`, corresponding to `I × I × C`, that is `image size × image size × channels`. Note that the channels relate to the latent space where noise is added and encode features that include both content and noise information.

6 William Peebles and Saining Xie. "Scalable Diffusion Models with Transformers" (2023).

7 Diederik P Kingma and Max Welling. "Auto-Encoding Variational Bayes" (2013).

After encoding the image into the latent space $z$ with the VAE, it's divided into smaller patches of size $p \times p$, which are then *patchified* into a sequence of tokens, where the number of tokens $T$ is calculated as $T = (I/p)^2$. This process is shown in Figure 4-2.

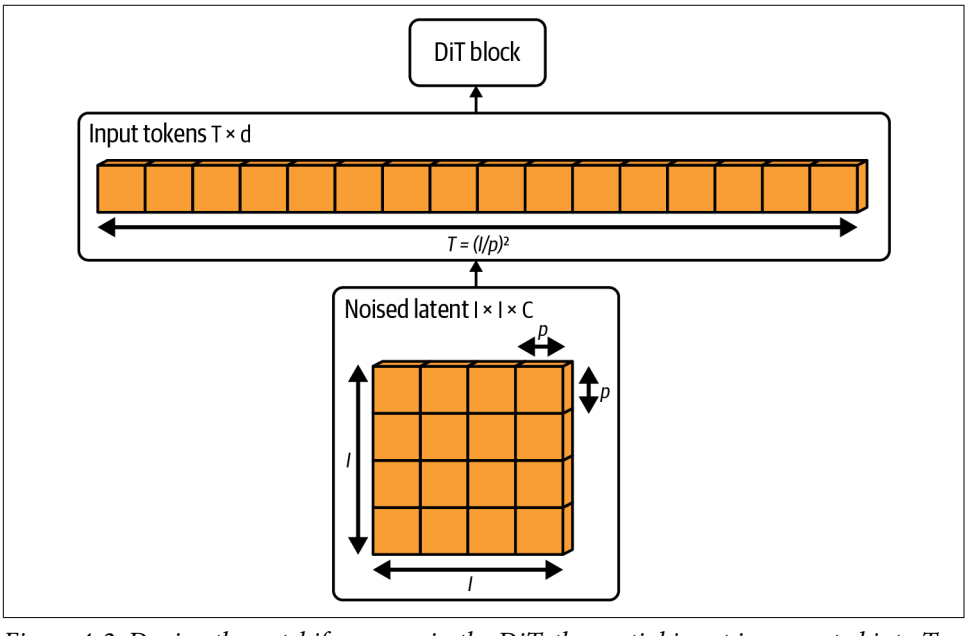


*Figure 4-2. During the patchify process in the DiT, the spatial input is converted into T tokens. Image adapted from William Peebles and Saining Xie (2023).*

To the sequence of patches, which are now tokens, positional embeddings are added and then fed into the transformer backbone of the DiT. This transformer block is composed of multiple layers of attention and feed-forward networks and learns the representations of the tokens. DiTs are based on the ViT architecture you learned about in Chapter 3. The DiT preserves many best practices from ViTs but introduces some small modifications to the standard ViT design. These modifications introduce the additional conditioning inputs like noise timesteps and class labels:

*In-context conditioning*
: In-context conditioning incorporates conditioning inputs as additional tokens in the sequence, allowing for minimal alteration to the transformer's structure, which efficiently processes these inputs with negligible overhead.

*Cross-attention blocks*
: These add a separate multi-head cross-attention layer to process conditioning inputs, increasing the model's computational complexity by roughly 15% but enhancing its flexibility in handling diverse data types.

*Adaptive Layer Normalization (adaLN) blocks*
: Derived from techniques used in GANs and diffusion models with U-Net backbones, adaLN layers adapt the normalization parameters based on conditioning inputs.

*adaLN-Zero blocks*
: This variant initializes the block to function as an identity map at the start of training, drawn from ResNet methodologies. It helps accelerate the training by stabilizing early learning dynamics, with negligible computational overhead.

This overall architecture is illustrated in Figure 4-3.

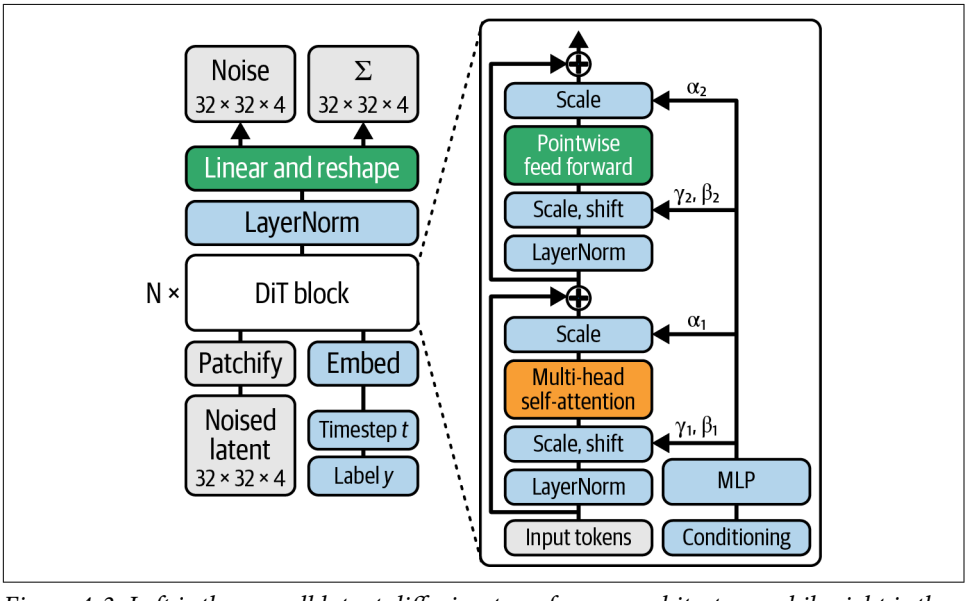


*Figure 4-3. Left is the overall latent diffusion transformer architecture, while right is the transformer block. Image adapted from William Peebles and Saining Xie (2023).*

The DiT block with adaLN-Zero conditioning implementation from the original paper's code is shown in Example 4-2.

*Example 4-2. DiT Block*

```
class DiTBlock(nn.Module):
    def __init__(self, hidden_size, num_heads, mlp_ratio=4.0, **block_kwargs):
        super().__init__()
        self.norm1 = nn.LayerNorm(hidden_size, elementwise_affine=False, eps=1e-6)
        self.attn = Attention(hidden_size, num_heads=num_heads,
                              qkv_bias=True, **block_kwargs)
        self.norm2 = nn.LayerNorm(hidden_size,
                                  elementwise_affine=False, eps=1e-6)
```

```
        mlp_hidden_dim = int(hidden_size * mlp_ratio)
        approx_gelu = lambda: nn.GELU(approximate="tanh")
        self.mlp = Mlp(in_features=hidden_size,
                       hidden_features=mlp_hidden_dim, act_layer=approx_gelu, drop=0)
        self.adaLN_modulation = nn.Sequential(
            nn.SiLU(),
            nn.Linear(hidden_size, 6 * hidden_size, bias=True)
        )

    def forward(self, x, c):
        shift_msa, scale_msa, gate_msa, shift_mlp, scale_mlp, gate_mlp =
        self.adaLN_modulation(c).chunk(6, dim=1)
        x = x + gate_msa.unsqueeze(1) * self.attn(modulate(self.norm1(x),
            shift_msa, scale_msa))
        x = x + gate_mlp.unsqueeze(1) * self.mlp(modulate(self.norm2(x),
            shift_mlp, scale_mlp))
        return x
```

The `DiTBlock` applies two main components: multi-head self-attention and an MLP. Both of these components are modulated by adaLN-Zero, which adjusts the normalization parameters (shift and scale) based on the conditioning vector *c*. The first part of the block involves a self-attention mechanism (`self.attn`). Before applying attention, the input tensor $x$ is normalized using LayerNorm (without learned affine parameters), and the result is modulated using the `shift_msa` and `scale_msa` parameters derived from the conditioning vector. The second part of the block involves an MLP, which also operates on the normalized input. Similar to the attention mechanism, the MLP's input is modulated using `shift_mlp` and `scale_mlp` before being passed through the MLP.

This code is fed into the transformer architecture as part of the DiT model's forward pass. Each `DiTBlock` receives the input tensor $x$ and a conditioning vector $c$, which combines timestep embeddings and class label embeddings. Additionally, classifier-free guidance is used within the DiT model to enhance generation quality by steering the model toward more desired outputs without the need for explicit class labels.

DiTs come in multiple configuration, DiT-S, DiT-B, DiT-L, and DiT-XL, allowing for flexibility depending on the application's requirements. In the next section, you'll be using DiT-XL to generate some images.

## Generating Images with the DiT

The DiT models are trained with `256 x 256` and `512 x 512` image resolution on the ImageNet dataset. Therefore, you can choose any category out of these image categories. To make it easier, I've written a Python function that maps the names with the numerical class labels. I omit the code here, but you can find the function in the accompanying notebook in the book's repo for this chapter. First, set the variables for your application, as shown in Example 4-3.

*Example 4-3. Setting up variables for model*

```
IMAGE_SIZE = 512
VAE_MODEL = "stabilityai/sd-vae-ft-ema"  ❶
LATENT_SIZE = IMAGE_SIZE // 8
SEED = 42
NUM_SAMPLING_STEPS = 200
CFG_SCALE = 4.0
CLASS_LABELS = get_label_ids()
SAMPLES_PER_ROW = 4
OUTPUT_IMAGE = "sample.png"
MODEL_NAME = f"DiT-XL-2-{IMAGE_SIZE}x{IMAGE_SIZE}.pt"
```

❶ Options are `stabilityai/sd-vae-ft-mse` or `stabilityai/sd-vae-ft-ema`.

As the next step, load the VAE model and the checkpoint of the DiT (see Example 4-4).

*Example 4-4. Load DiT and VAE*

```
def load_models(device, MODEL_NAME):
    model = DiT_XL_2(input_size=LATENT_SIZE).to(device)
    state_dict = download_model(MODEL_NAME)
    model.load_state_dict(state_dict)
    model.eval()
    vae = AutoencoderKL.from_pretrained(VAE_MODEL).to(device)
    return model, vae
```

Next, you need to create the diffusion object, the sampling noise, and the setup for the classifier-free guidance to generate the sample images, as shown in Example 4-5.

*Example 4-5. Generate image samples*

```
def generate_samples(model, vae, device):
    torch.manual_seed(SEED)

    diffusion = create_diffusion(str(NUM_SAMPLING_STEPS)) ❶

    n = len(CLASS_LABELS) ❷
    z = torch.randn(n, 4, LATENT_SIZE, LATENT_SIZE, device=device)
    y = torch.tensor(CLASS_LABELS, device=device)

    z = torch.cat([z, z], 0) ❸
    y_null = torch.tensor([1000] * n, device=device)
    y = torch.cat([y, y_null], 0)
    model_kwargs = dict(y=y, cfg_scale=CFG_SCALE) ❹

    samples = diffusion.p_sample_loop(
        model.forward_with_cfg, z.shape, z, clip_denoised=False,
        model_kwargs=model_kwargs, progress=True, device=device
    )
    samples, _ = samples.chunk(2, dim=0) ❺
    samples = vae.decode(samples / 0.18215).sample
    return samples
```

❶ Create diffusion object.

❷ Create sampling noise.

❸ Set up classifier-free guidance.

❹ Set classifier-free guidance scale.

❺ Remove null class samples.

Now you can initialize these functions to load the models and generate your samples, as shown in Example 4-6.

*Example 4-6. Initialize models and functions*

```
model, vae = load_models(device, MODEL_NAME)
samples = generate_samples(model, vae, device)
save_and_display_samples(samples)
```

The `save_and_display_samples` function will save and display the images, as shown in Figure 4-4.

*Figure 4-4. The generated images from the DiT.*

While DiT marked a significant advancement in image generation models, it does have some limitations. Nonetheless, it's important to recognize that DiT has been instrumental in laying the groundwork for more advanced models, which you'll explore in the next sections. These newer models offer the capability to generate images without being restricted to a single class, further expanding the possibilities of image generation.

# PIXART-α

*PIXART-α*[8] is a transformer-based T2I diffusion model that delivers high-quality image generation comparable to top-tier models like Imagen while significantly reducing training costs and environmental impact. PIXART-α has lower financial and ecological costs compared to other SOTA T2I models. It supports high-resolution image synthesis up to 1024 × 1024 pixels with a training efficiency that surpasses existing large-scale models.

To achieve this, the model decouples the training process into three distinct stages, each optimized for different aspects of image synthesis. The first stage, *pixel dependency learning*, leverages a class-conditional model pretrained on ImageNet, enhancing the initial training efficiency. The second stage focuses on *text-image alignment*, where a cross-attention layer is added between the self-attention and feed-forward layers of the base DiT architecture, allowing the model to interact flexibly with text embeddings (see Figure 4-5 for reference). The final stage fine-tunes the model using high-resolution, aesthetically rich data, with adaptation converging faster due to the strong priors established in earlier stages.

8 Junsong Chen et al. "PixArt-α: Fast Training of Diffusion Transformer for Photorealistic Text-to-Image Synthesis" (2023).

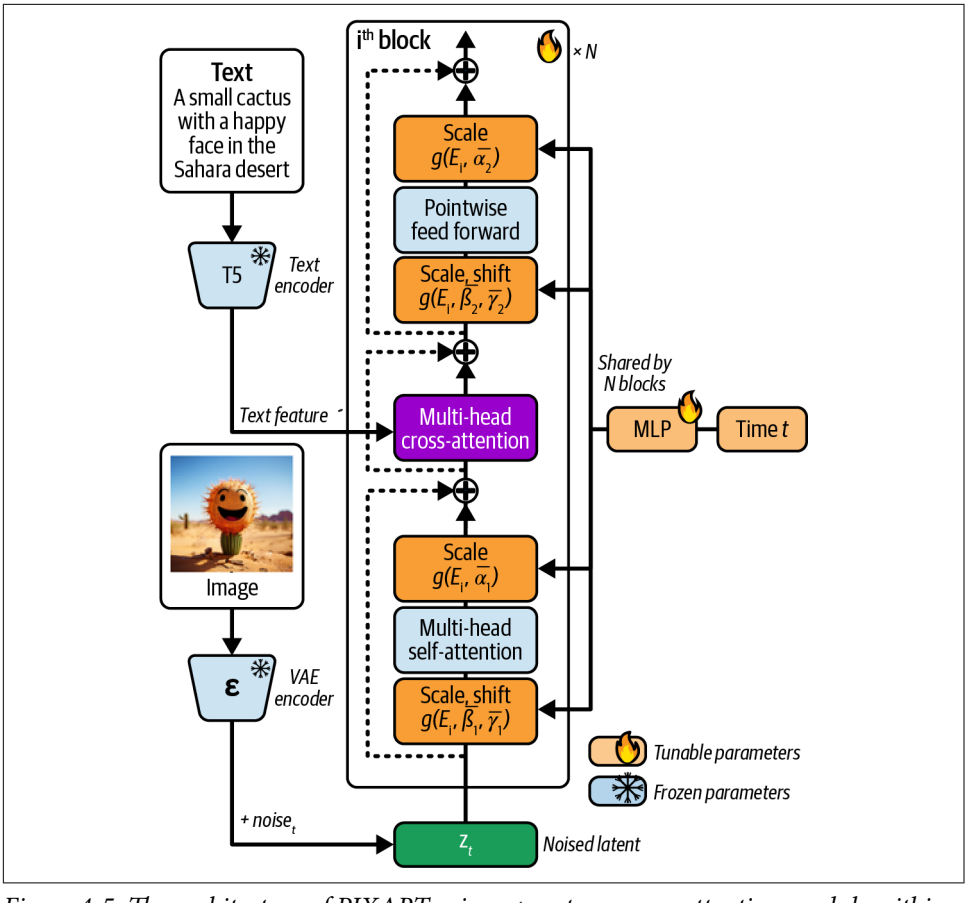


*Figure 4-5. The architecture of PIXART-α incorporates a cross-attention module within each block to seamlessly integrate textual conditions. Image adapted from Junsong Chen et al. (2023).*

PIXART-α leverages the DiT as its base architecture but introduces the following design changes:

*Cross-attention in PIXART-α*

The *cross-attention* layer is positioned between the self-attention and feed-forward layers, allowing the model to integrate text conditions effectively. This layer enables the model to handle diverse text-image pairings with high semantic accuracy by injecting textual conditions into the image-generation process. For efficiency, the output projection layer is initialized to zero, maintaining the input's identity while the model trains.

*AdaLN-single*

PIXART-α introduces the *adaLN-single module* to reduce computational complexity by cutting down the number of parameters compared to the standard adaLN module used in DiT. In DiT, the adaptive normalization layers involve a large number of parameters because they compute scales and shifts using both class conditions and time embeddings, but adaLN-single does not use class conditions. Instead, adaLN-single uses only the time embeddings to compute a single global set of shift and scale parameters in the first block, which is then shared across all transformer blocks. Each block then adjusts these global parameters with a layer-specific trainable embedding, ensuring both efficiency and consistency in the training process.

*Reparameterization*

To make use of the pretrained weights, the layer-specific trainable embeddings in adaLN-single are initialized in a way that replicates the same scale and shift parameters as the original DiT model when class conditions are absent (in the paper $t = 500$ is used), focusing solely on the selected time embedding. This approach replaces the block-specific MLPs with a global MLP and layer-specific embeddings, allowing PIXART-α to streamline its architecture.

In addition, PIXART-α tackles the limitations of existing datasets by using a vision-language model (LLaVA)[9] to auto-label image-text pairs, enhancing the concept density and quality of training data. By applying LLaVA to the SAM dataset,[10] which features rich and diverse object imagery, PIXART-α ensures a high degree of text-image alignment, reducing the need for extensive iterations to achieve stable model performance. Additionally, the final fine-tuning stage incorporates data from JourneyDB[11] and an internal dataset, further refining the model's ability to generate aesthetically pleasing, high-resolution images.

9 Haotian Liu et al. "Visual Instruction Tuning" (2017).

10 Alexander Kirillov et al. "Segment Anything" (2023).

11 Keqiang Sun et al. "JourneyDB: A Benchmark for Generative Image Understanding" (2017).

# PixArt-Σ

PIXART-α serves as the foundation for a more powerful version, PixArt-Σ.[12] This process, referred to as *weak-to-strong training*, involves transitioning from a relatively weaker baseline to a stronger model through efficient training methods.

The first step in this enhancement process was the collection of a new high-quality dataset. The new dataset focuses on two key aspects: the quality of the images and the precision of the captions. The dataset comprises 33 million high-resolution images, all exceeding 1K resolution, with a subset of 2.3 million images at approximately 4K resolution. These images are chosen for their high aesthetic value and diversity in artistic styles. Furthermore, to improve text-image alignment, the LLaVA image captioner used in PIXART-α was replaced with a more powerful alternative, *ShareCaptioner*,[13] which generates detailed and correct captions, reducing the occurrence of hallucinations. Additionally, the average caption length is increased to 180 words, providing more descriptive power and improving text-image alignment.

To mitigate the computational challenges due to the increased number of tokens for ultra-high-resolution image generation, a self-attention module with key and value token compression was integrated into the DiT framework. This process is illustrated in Figure 4-6. This design effectively reduces training and inference time by approximately 34% for high-resolution image generation while maintaining the model's performance. To be able to scale from low to high resolution, the VAE was also replaced with a more powerful version, evolving from a model without key-value (KV) token compression to one with this feature.

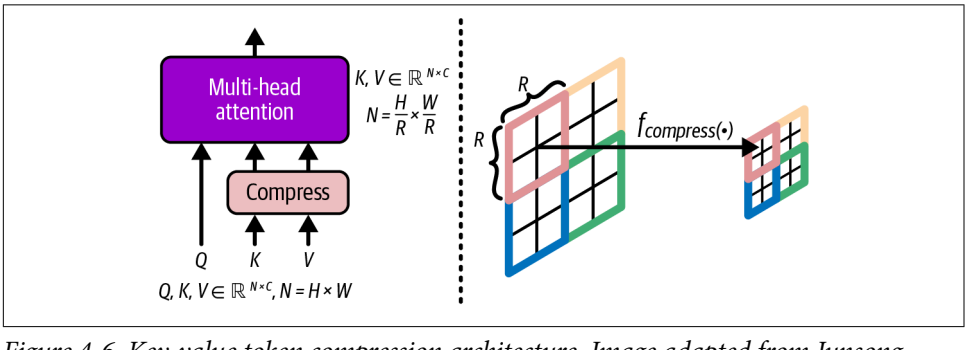


*Figure 4-6. Key-value token compression architecture. Image adapted from Junsong Chen et al. (2024).*

12 Junsong Chen et al. "PixArt-Σ: Weak-to-Strong Training of Diffusion Transformer for 4K Text-to-Image Generation" (2024).

13 Lin Chen et al. "ShareGPT4V: Improving Large Multi-Modal Models with Better Captions" (2023).

Example 4-7 shows the code of the simplified `AttentionKVCompress` class from PixArt-Σ. The function `downsample_2d` demonstrates how the reshaping of the input tensor and permutations is applied to reduce the tensor's dimensions.

*Example 4-7. Key-value compression code example*

```
class AttentionKVCompress(nn.Module):
    def __init__(self, dim, sr_ratio):
        super(ExampleModel, self).__init__()
        self.sr = nn.Conv2d(dim, dim, groups=dim,
            kernel_size=sr_ratio, stride=sr_ratio)
        self.norm = nn.LayerNorm(dim)

    def downsample_2d(self, tensor, H, W, scale_factor, sampling=None):
        if sampling is None or scale_factor == 1:
            return tensor, tensor.shape[1]

        B, N, C = tensor.shape
        print(f"Original Tensor Shape: {tensor.shape}")

        if sampling == 'uniform_every':
            tensor = tensor[:, ::scale_factor]
            print(f"Shape after 'uniform_every': {tensor.shape}")
            return tensor, int(N // scale_factor)

        tensor = tensor.reshape(B, H, W, C).permute(0, 3, 1, 2)
        print(f"Shape after Reshape and Permute: {tensor.shape}")
        new_H, new_W = int(H / scale_factor), int(W / scale_factor)
        new_N = new_H * new_W

        if sampling == 'ave':
            tensor = F.interpolate(
                tensor, scale_factor=1 / scale_factor, mode='nearest'
            ).permute(0, 2, 3, 1)
        elif sampling == 'uniform':
            tensor = tensor[:, :, ::scale_factor, ::scale_factor].permute(0, 2, 3, 1)
            print(f"Shape after 'uniform' downsampling: {tensor.shape}")
        elif sampling == 'conv':
            tensor = self.sr(tensor).reshape(B, C, -1).permute(0, 2, 1)
            tensor = self.norm(tensor)
            print(f"Shape after 'conv' downsampling: {tensor.shape}")
        else:
            raise ValueError

        tensor = tensor.reshape(B, new_N, C).contiguous()
        print(f"Final Shape after Reshape: {tensor.shape}")
        return tensor, new_N
```

Depending on the sampling method specified, that is, `'conv'`, `'ave'`, `'uniform'`, or `'uniform_every'`, the function reduces the spatial dimensions of the tensor,

effectively compressing the tokens. The tensor initially has the shape (`B, N, C`), where `B` is the batch size; `N` is the number of tokens, which can be viewed as spatial positions; and `C` is the channel dimension. Here's how the different compression versions work:

`conv`
: Applies a convolutional layer (`self.sr`) to downsample, followed by normalization.

`ave`
: Uses average pooling (`F.interpolate`) to downsample the tensor.

`uniform`
: Selects every nth element, reducing the spatial resolution.

`uniform_every`
: Helps compress the number of tokens by regularly sampling tokens at intervals defined by the `scale_factor`, making it a straightforward approach to token compression within the model, as it operates directly on the flattened sequence.

To see how this affects the tensor shape for the attention mechanism, you can create an example input tensor and feed it into the class, as shown in Example 4-8.

*Example 4-8. Apply compression to example tensor*

```
B, H, W, C = 1, 4, 4, 1  ❶
input_tensor = torch.arange(1, 17).view(B, H * W, C).float() ❷

compressed_KV = AttentionKVCompress(dim=C, sr_ratio=2)

print("=== Uniform Every Downsampling ===")
output_tensor, new_N = model.downsample_2d(input_tensor, H, W,
                                           scale_factor=2, sampling='uniform_every')
print(f"New number of tokens: {new_N}\n")

print("=== Average Pooling Downsampling ===")
output_tensor, new_N = model.downsample_2d(input_tensor, H, W,
                                           scale_factor=2, sampling='ave')
print(f"New number of tokens: {new_N}\n")

print("=== Uniform Downsampling ===")
output_tensor, new_N = model.downsample_2d(input_tensor, H, W,
                                           scale_factor=2, sampling='uniform')
print(f"New number of tokens: {new_N}\n")

print("=== Convolution Downsampling ===")
output_tensor, new_N = model.downsample_2d(input_tensor, H, W,
                                           scale_factor=2, sampling='conv')
print(f"New number of tokens: {new_N}\n")
```

❶ Batch size, height, width, channels

❷ Creating a tensor [1, 16, 1]

For the preceding code, it will print out the following new tensor shapes:

```
=== Uniform Every Downsampling ===
Original Tensor Shape: torch.Size([1, 16, 1])
Shape after 'uniform_every': torch.Size([1, 8, 1])
New number of tokens: 8

=== Average Pooling Downsampling ===
Original Tensor Shape: torch.Size([1, 16, 1])
Shape after Reshape and Permute: torch.Size([1, 1, 4, 4])
Final Shape after Reshape: torch.Size([1, 4, 1])
New number of tokens: 4

=== Uniform Downsampling ===
Original Tensor Shape: torch.Size([1, 16, 1])
Shape after Reshape and Permute: torch.Size([1, 1, 4, 4])
Shape after 'uniform' downsampling: torch.Size([1, 2, 2, 1])
Final Shape after Reshape: torch.Size([1, 4, 1])
New number of tokens: 4

=== Convolution Downsampling ===
Original Tensor Shape: torch.Size([1, 16, 1])
Shape after Reshape and Permute: torch.Size([1, 1, 4, 4])
Shape after 'conv' downsampling: torch.Size([1, 4, 1])
Final Shape after Reshape: torch.Size([1, 4, 1])
New number of tokens: 4
```

Example 4-9 illustrates how this compression is integrated in the forward process of the model.

*Example 4-9. KV compression in forward function*

```
# Rest of code omitted
k, new_N = self.downsample_2d(k, H, W, self.sr_ratio, sampling=self.sampling)
v, new_N = self.downsample_2d(v, H, W, self.sr_ratio, sampling=self.sampling)
```

These optimizations are the reason why PixArt-Σ achieves high-quality 4K resolution image generation with minimal training costs and model parameters. Most importantly, if you want to fine-tune PixArt-Σ on your own images, it requires only a fraction of the GPU needed by other top-tier models because it uses significantly fewer parameters while delivering comparable aesthetic quality and same-level text-image alignment.

## Generating Images with PixArt-Σ

Generating images with PixArt-Σ is straightforward: you can simply use the diffusers library from Hugging Face, as shown in Example 4-10.

*Example 4-10. Image generation with Diffusers library*

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

pipe = PixArtSigmaPipeline.from_pretrained(
    "PixArt-alpha/PixArt-Sigma-XL-2-1024-MS", torch_dtype=torch.float16
).to(device)

prompt = "Cute animated tabby with big eyes"
image = pipe(prompt).images[0]
image.save("./tabby.png")
```

For me, this prompt created the image shown in Figure 4-7.

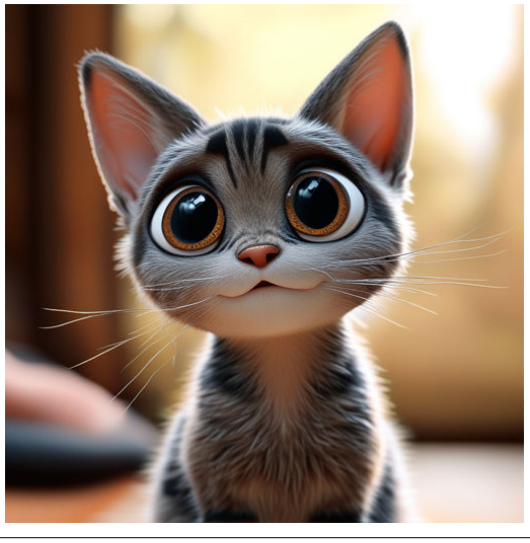

*Figure 4-7. Generated image with PixArt-Σ.*

However, if you want to generate several images or have a limited availability of GPU resources, you might want to optimize this, as explained in the next section.

#### Using Inference Engines for Image Generation

Another way to efficiently implement image generation in your application is the use of a cloud provider for inference. The benefit of this is that you don't have to set up the infrastructure yourself, and you pay by usage—that is, for images, this is usually per generated image. The prices also depend on the model you're using. In addition, the popular models are on so called *warm boots*, which just means that the model is already running, as opposed to *cold boots*, which means that when a model hasn't been used for a little while, it gets turned off by the provider. Some of these providers are Replicate and FireworksAi.

### Memory-efficient image generation

As models increase in size, their memory demands grow as well. This challenge becomes more significant because a diffusion pipeline generally involves several components: a text encoder, a diffusion backbone, and an image decoder. These substantial memory requirements can make it challenging to run these models on consumer GPUs, which in turn can slow adoption and make experimentation more difficult. In this section, you'll explore how to enhance the memory efficiency of transformer-based diffusion pipelines by using quantization.

First, enable memory history, which will add tracebacks and event history to snapshots:

```
torch.cuda.memory._record_memory_history()
```

Next, you set up your configurations, as shown in Example 4-11.

*Example 4-11. Configuration and model download*

```
quant_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)

text_encoder = T5EncoderModel.from_pretrained(
    "PixArt-alpha/PixArt-Sigma-XL-2-1024-MS",
    subfolder="text_encoder",
    quantization_config=quant_config,
    device_map="balanced",
)

pipe = PixArtSigmaPipeline.from_pretrained(
    "PixArt-alpha/PixArt-Sigma-XL-2-1024-MS",
    text_encoder=text_encoder,
```

```
    transformer=None,
    device_map="balanced"
)

with torch.no_grad():
    prompt = "Cute animated tabby with big eyes"
    prompt_embeds, prompt_attention_mask, negative_embeds,
    negative_prompt_attention_mask = pipe.encode_prompt(prompt)
```

Now that you've computed the embeddings, you can simply delete the text encoder model and the pipeline with:

```
    del text_encoder
    del pipe

    gc.collect()
    torch.cuda.empty_cache()
```

After that, you use the embeddings to compute the latents, as shown in Example 4-12.

*Example 4-12. Compute the latents*

```
pipe = PixArtSigmaPipeline.from_pretrained(
    "PixArt-alpha/PixArt-Sigma-XL-2-1024-MS",
    text_encoder=None,
    torch_dtype=torch.float16,
).to("cuda")

quantize(pipe.transformer, weights=qint8, exclude="proj_out")
freeze(pipe.transformer)

latents = pipe(
    negative_prompt=None,
    prompt_embeds=prompt_embeds,
    negative_prompt_embeds=negative_embeds,
    prompt_attention_mask=prompt_attention_mask,
    negative_prompt_attention_mask=negative_prompt_attention_mask,
    num_images_per_prompt=1,
    output_type="latent",
).images
```

Now, you can clean up your memory again with `del pipe.transformer` and `gc.collect()`, `torch.cuda.empty_cache()`. And then you can generate your image with the code from Example 4-13.

*Example 4-13. Optimized image generation*

```
with torch.no_grad():
    image = pipe.vae.decode(latents / pipe.vae.config.scaling_factor,
                            return_dict=False)[0]
image = pipe.image_processor.postprocess(image, output_type="pil")

image[0].save("tabby.png")
```

As a final step, to see the memory usage, you can run this command:

```
    torch.cuda.memory._dump_snapshot("PixArtSigma_quant.pickle")
```

This will create a pickle file that you now can upload to the PyTorch visualizer to visualize your memory usage history. This will create a plot similar to the one in Figure 4-8.

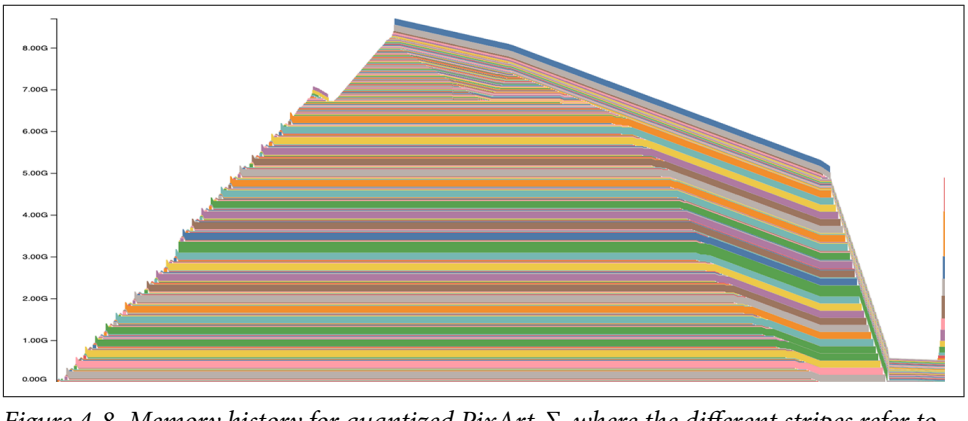


*Figure 4-8. Memory history for quantized PixArt-Σ, where the different stripes refer to different allocations from tensors; the smaller the stripes, the smaller the tensors allocated.*

If you now compare this to the memory usage without optimizing the model, as shown in Figure 4-9, you can clearly see how the optimization affects the overall memory allocation over the inference time.

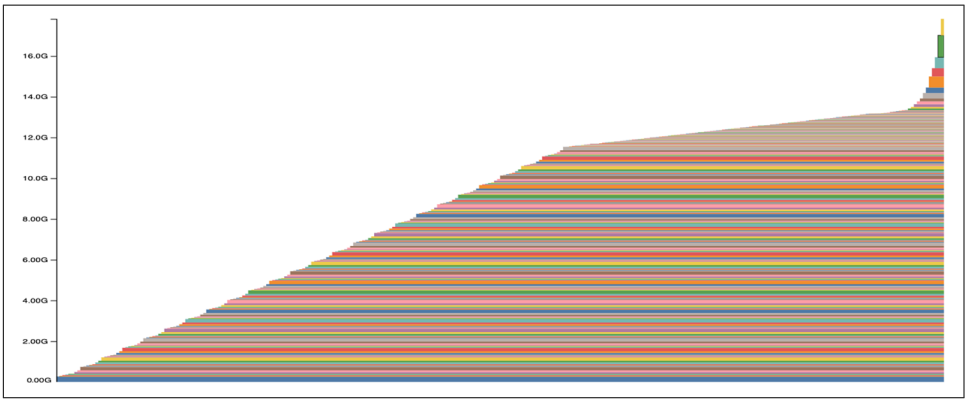


*Figure 4-9. Memory history for non-quantized PixArt-Σ. Again, the different stripes refer to different allocations from tensors; the smaller the stripes, the smaller the tensors allocated.*

The two plots clearly illustrate that using quantization saves more than half of the memory usage. To get an understanding of the memory consumption of your application, you can also run `torch.cuda.memory_summary()` to get a memory usage summary.

### Training a diffusion transformer

Fine-tuning a diffusion transformer is no easy task, often requiring significant resources and technical expertise. However, the SimpleTuner repository simplifies this complex process by offering a range of powerful features designed to make training more accessible and efficient.

SimpleTuner supports single- or multi-GPU training, allowing for faster and more efficient model development. It optimizes memory usage by caching image and caption features (embeds) to the hard drive in advance, enabling faster training with reduced memory consumption. The repository also includes *aspect bucketing*, which accommodates a variety of image sizes and aspect ratios, facilitating widescreen and portrait training.

SimpleTuner offers various training options for Stable Diffusion XL (SDXL) models. It supports Refiner LoRA or full training, enabling most models to be trained on a 24G GPU. LoRA/LyCORIS training is also available for PixArt, SDXL, Stable Diffusion 3, and Stable Diffusion 2.x models. For more resource-constrained environments, DeepSpeed integration allows full U-Net training for SDXL on just 12G of

video random access memory (VRAM). Additionally, SimpleTuner includes quantized LoRA training, which uses low-precision base model or text encoder weights to further reduce VRAM consumption. To enhance training stability, an optional exponential moving average (EMA) weight network is available, though it doesn't apply to LoRA training. In addition, SimpleTuner allows you to deploy the models directly to Hugging Face Hub for seamless model uploads and automatically generated model cards.

## Diffusion Vision Transformers for Image Generation

Diffusion Vision Transformers for Image Generation (DiffiT)[14] introduces *time-dependent multi-head self-attention* (TMSA), a methodology for more fine-grained control over spatial and temporal dependencies and their interaction during the denoising process. This addresses the limitations of AdaLN used in DiT, where, during the initial stages of denoising, the high-frequency content of the image is completely perturbed because the denoising network primarily focuses on predicting the low-frequency content. In TMSA, the key, query, and value weights are adjusted at each timestep during the denoising process. The following equation demonstrates how the time-dependent queries $q$, keys $k$, and values $v$ in the shared space are computed through a linear projection of the spatial and time embeddings, denoted as $x_s$ and $x_t$, respectively:

$$\mathbf{q_s} = \mathbf{x_s}\mathbf{W}_{qs} + \mathbf{x_t}\mathbf{W}_{qt}$$
$$\mathbf{k_s} = \mathbf{x_s}\mathbf{W}_{ks} + \mathbf{x_t}\mathbf{W}_{kt}$$
$$\mathbf{v_s} = \mathbf{x_s}\mathbf{W}_{vs} + \mathbf{x_t}\mathbf{W}_{vt}$$

Here, $\mathbf{W}qs$, $\mathbf{W}qt$, $\mathbf{W}ks$, $\mathbf{W}kt$, $\mathbf{W}vs$, and $\mathbf{W}vt$ represent the spatial and temporal linear projection weights for the corresponding queries, keys, and values. The time token is obtained by feeding positional time embeddings to a small MLP with *Swish activation*, which is simply:

$$f(x) = x \cdot \sigma(x)$$

Here, $\sigma(x) = (1 + \exp(-x))^{-1}$, which is the sigmoid function. Unlike ReLU, Swish is smooth and non-monotonic. This non-monotonicity property of Swish actually distinguishes it from most common activation functions.

14 Ali Hatamizadeh et al. "DiffiT: Diffusion Vision Transformers for Image Generation" (2024).

Key, query, and value are all linear functions of both time and spatial tokens, and they can adaptively modify the behavior of attention for different timesteps. The self-attention is then computed as follows:

$$\text{Attention}\,(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d}} + \mathbf{B}\right)\mathbf{V}$$

Here, query, key and value are defined as $\mathbf{Q}: = \{\mathbf{q}_s\}, \mathbf{K}: = \{\mathbf{k}_s\}$, and $\mathbf{V}: = \{\mathbf{v}_s\}$, respectively. $d$ is a scaling factor for the keys, and $B$ corresponds to a relative position bias. Note that directly incorporating relative position bias with time embedding can lead to suboptimal performance, as it must capture both spatial and temporal information. The transformer block is defined as:

$$\hat{\mathbf{x}}_s = \text{TMSA}\left(\text{LN}\left(\mathbf{x}_s\right), \mathbf{x}_t\right) + \mathbf{x}_s$$
$$\mathbf{x}_s = \text{MLP}\left(\text{LN}\left(\hat{\mathbf{x}}_s\right)\right) + \hat{\mathbf{x}}_s$$

Here, *LN* stands for layer norm. Figure 4-10 illustrates this architectural setup of DiffiT.

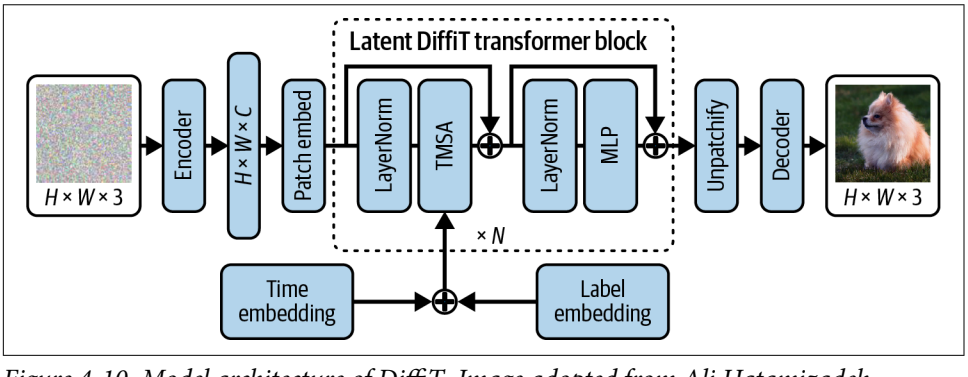


*Figure 4-10. Model architecture of DiffiT. Image adopted from Ali Hatamizadeh et al. (2024).*

DiffiT has an encoder-decoder architecture with contracting and expanding paths connected by skip connections at every resolution. Figure 4-11 depicts the image-space part of the DiffiT architecture.

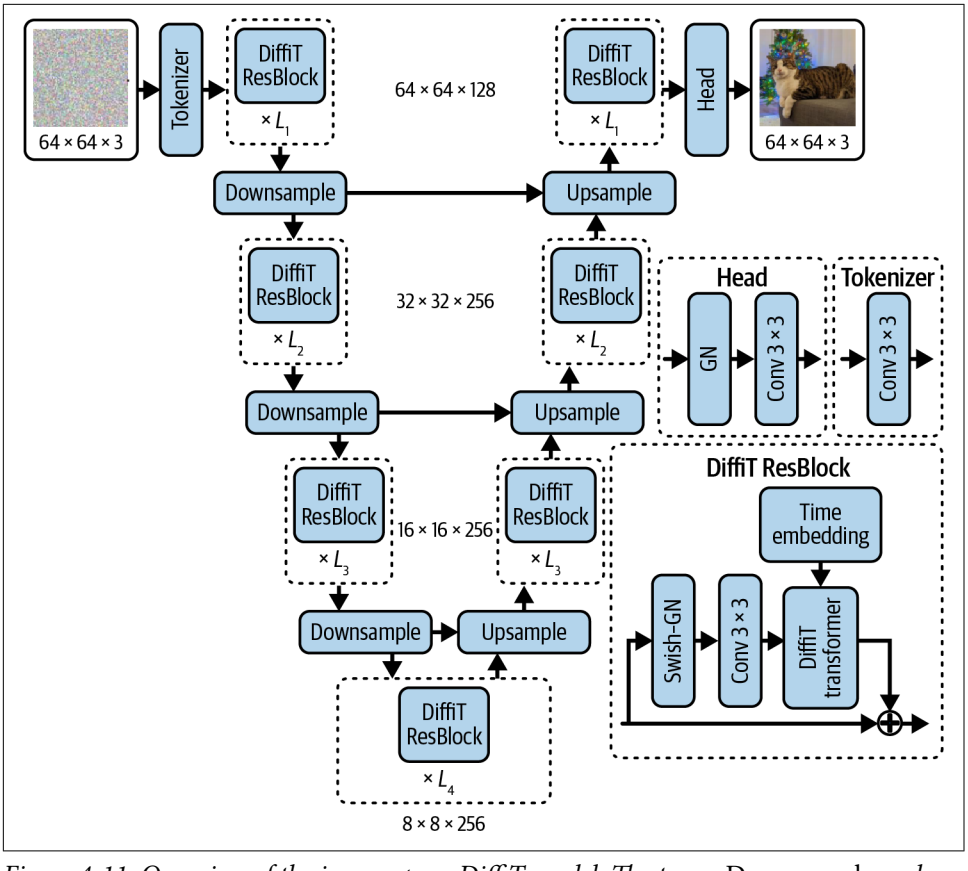


*Figure 4-11. Overview of the image-space DiffiT model. The terms* Downsample *and* Upsample *refer to the convolutional layers used for downsampling and upsampling, respectively. Image adopted from Ali Hatamizadeh et al. (2024).*

Each resolution in the encoder or decoder paths consists of $L$ DiffiT blocks that incorporate time-dependent self-attention modules. At the start of each path, a convolutional layer adjusts the number of feature maps, and upsampling or downsampling layers manage transitions between resolutions. These convolutional layers are embedded as inductive image biases, to enhance performance. The final residual cell for image space diffusion models is defined as:

$$\hat{\mathbf{x}}_s = \text{Conv}_{3\times 3}\left(\text{Swish}\left(\text{GN}\left(x_s\right)\right)\right)$$
$$\mathbf{x}_s = \text{DiffiT} - \text{Block}\left(\hat{\mathbf{x}}_s, x_t\right) + x_s$$

Here, *GN* denotes the *group normalization* operation. GN divides the channels into groups and calculates the mean and variance for normalization within each group.

Unlike other normalization methods, GN's computation does not depend on batch sizes, ensuring consistent accuracy across a wide range of batch sizes.

The DiffiT transformer block, used as residual cells, addresses the quadratic cost of attention, which scales poorly with large feature maps. By applying self-attention within non-overlapping partitioned windows, the U-Net structure enables information sharing between regions via bottleneck layers. The final residual cell combines the DiffiT transformer block with an additional convolutional layer, incorporating group normalization and the Swish activation function.

The DiffiT model achieves SOTA performance in terms of the *Fréchet Inception Distance* (FID) score on the ImageNet-256 dataset. The FID is a metric to assess the quality of generated images by calculating the difference between real images and images generated by the algorithm.

#### Diffusion Models for Image Captioning

While I won't cover image captioning models in this chapter, I still want to share with you that there is a diffusion-based image captioning model called Latent Diffusion-based Captioner (LaDiC).[15] It outperforms autoregressive-based models without the need for ancillary modules. The code for the model is openly available via the paper's GitHub repository.

## Interpretable Features with Diffusion Transformers

Even though the following section isn't about a specific model, I think you'll still find it insightful, as it will help you to better interpret and explore the dynamics of T2I diffusion transformers.

To achieve this interpretability, ConceptAttention[16] repurposes the parameters of DiT attention layers to generate contextualized text embeddings, each corresponding to visual concepts (e.g., "dragon," "sun"). For that, ConceptAttention requires no additional training. By simply applying linear projection of these concept embeddings and the image, it can produce high-quality saliency maps. *Saliency maps* visualize the areas of an image that most strongly influence a model's prediction, offering insight into its decision-making process. They highlight the regions that contribute most to the model's classification or other outputs.

15 Yuchi Wang et al. "LaDiC: Are Diffusion Models Really Inferior to Autoregressive Counterparts for Image-to-Text Generation?" (2024).

16 Alec Helbling et al. "ConceptAttention: Diffusion Transformers Learn Highly Interpretable Features" (2025).

To use ConceptAttention, you just specify a set of $r$ single token concepts, like "cat," "sky," and so on. These are then passed through a T5[17] encoder to produce an initial embedding $c^0$ for each concept.

PIXART-α and PixArt-Σ do fall under the category of multimodal DiTs that leverage *multimodal attention* (MMATTN) *layers* to jointly process both textual tokens and image patches. Meaning, MMATTN combines cross- and self-attention operations between the prompt and image tokens.

ConceptAttention normalizes the input concept embeddings $c^L$ at each MMATTN layer $L$ and repurposes the text prompt's projection matrices $K_p$, $Q_p$, and $V_p$ to generate the corresponding keys, queries, and values:

$$k_c = \left[K_p c_1, \ldots K_p c_k\right]$$
$$q_c = \left[Q_p c_1, \ldots Q_p c_k\right]$$
$$v_c = \left[V_p c_1, \ldots V_p c_k\right]$$

Each of these is in $\mathbb{R}^{r \times d}$.

To update the concept embeddings for compatibility with later layers, while preventing them from influencing the image tokens, it concatenates the image and concept keys and values. $k_x$ and $v_x$ are the keys and values for the image patches $x$, respectively, to get:

$$k_{\text{xc}} = \left[K_x x_1 \ldots,\ \text{K}_x x_n,\ \text{K}_p c_1 \ldots,\ \text{K}_p c_r\right]$$
$$v_{\text{xc}} = \left[V_x x_1 \ldots,\ \text{V}_x x_n,\ \text{V}_p c_1 \ldots,\ \text{V}_p c_r\right]$$

Then the concept output embeddings are computed via:

$$o_c = \text{Softmax}(q_c k_{xc}^T) v_{xc}$$

This attention operation combines both cross-attention from image patches to concepts and self-attention among the concepts. This setup allows the concept embeddings to repel one another, minimizing redundancy, while the image patches and prompt tokens attend only to each other as in the following equation:

17 Colin Raffel et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" (2023).

$$o_x, o_p = \text{Softmax}\left(q_{xp} k_{xp}^T\right) v_{xp}$$

$o_x$ and $o_p$ refer to the output embeddings of the image patches ($x$) and the prompt tokens ($p$), respectively.

A visualization of these operations appears in Figure 4-12.

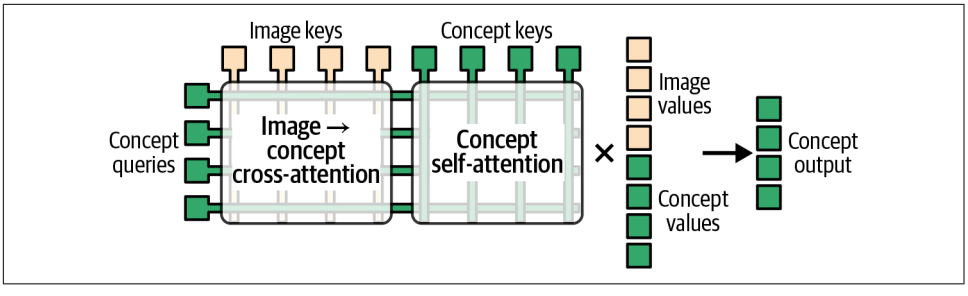


*Figure 4-12. ConceptAttention enables concept tokens to integrate information from both other concept tokens and image tokens, while preventing image tokens from accessing concept token information. Image adapted from Alec Helbling et al. (2025).*

The preceding operations form a residual stream of concept embeddings separate from the image and prompt token streams. In line with the pretrained transformer's architecture, we apply a projection matrix $P$ and an MLP after the MMATTN layer and then add the result back to $c^L$ residually.

An adaptive layer normalization is also applied, yielding a scale $\gamma$, shift $\beta$, and gating factors $\alpha_1$ and $\alpha_2$. The residual stream is updated as:

$$c^{L+1} \leftarrow c^L + \alpha_1 (P o_c)$$
$$c^{L+1} \leftarrow c^{L+1} + \alpha_2 \text{ MLP}\left((1+\gamma)\ \text{lnorm}\left(c^{L+1}\right) + \beta\right)$$

The projection, modulation, and MLP layers used here are shared with those used for processing the text prompt.

The concept embeddings, as I mentioned earlier, can be combined with the image patch embeddings to produce saliency maps at each layer $L$. Specifically, if you take the dot-product similarity between the image output vectors $o_x$ and the concept output vectors $o_c$, it results in high-quality saliency maps:

$$\phi(o_x, o_c) = \text{Softmax}(o_x o_c^T) \quad (13)$$

This contrasts with traditional cross-attention maps, which are based on image keys $k_x$ and prompt queries $q_p$.

To aggregate information across layers, you then average the saliency maps:

$$\frac{1}{|L|}\sum_{L=1}^{|L|}\phi(o_x^L, o_c^L)$$

where $|L|$ is the number of MMATTN layers. These attention-output-space maps are a distinct feature of MM-DiT architectures, as they directly leverage concept embeddings tied to textual semantics.

If you want to test this out, the repo is available on GitHub. The available code allows you to try ConceptAttention and generate saliency maps of text concepts for your image, as shown in Figure 4-13.

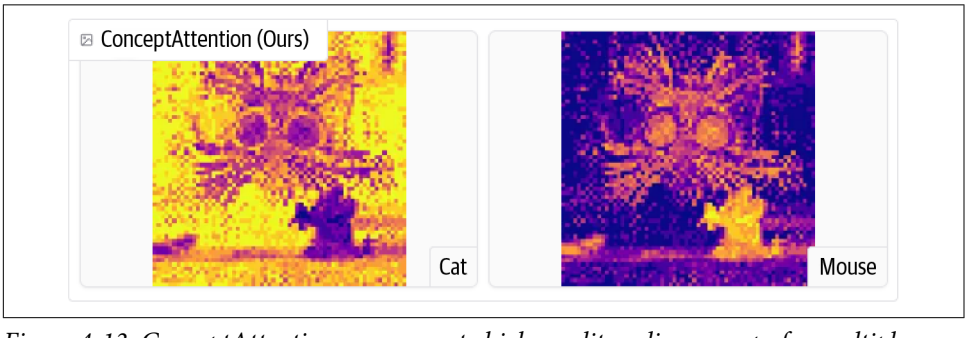


*Figure 4-13. ConceptAttention can generate high-quality saliency maps for multiple concepts simultaneously.*

ConceptAttention can also be easily integrated into open source models, especially those based on DiT architectures, because it operates entirely as a post hoc interpretability method without requiring retraining or modification of the core model weights.

# Conclusion

In this chapter, you explored the advancements in image generation made by diffusion models, particularly in the context of T2I applications. You learned how diffusion models have surpassed earlier methods such as GANs and autoregressive models by offering more stable training, better coverage of latent space, and superior image quality.

You gained an understanding of the mechanics of diffusion models and built your knowledge on how they progressively refine random noise into coherent images through an iterative denoising process. The introduction of classifier-free guidance and scalable diffusion models like DiTs highlighted significant enhancements in image generation, enabling higher accuracy and flexibility.

Furthermore, you learned about models such as PIXART-α, PixArt-Σ, and DiffiT, which push the boundaries of T2I models by incorporating innovations like cross-attention layers, adaptive layer normalization, and time-dependent multi-head self-attention. These advancements not only improve the quality and efficiency of image generation but also reduce the computational costs and environmental impact.

In addition, you were introduced to ConceptAttention, a method that enhances interpretability in diffusion transformers without requiring additional training. It leverages the existing attention layers in DiT architectures to generate high-quality saliency maps aligned with specific textual concepts. This allows for a deeper understanding of what parts of an image influence the model's output and how individual concept tokens interact with image patches.

The chapter also provided you with hands-on examples, from setting up diffusion transformers to generating high-quality images efficiently. It covered how you can train diffusion models on your own data, offering practical insights on how to use these models and how to visualize and interpret the underlying mechanisms.

In Chapter 5, you'll learn how you can use transformer models for audio tasks.

# CHAPTER 5
# Transformers for Video Generation

This chapter will be a fun ride, because a lot of what you've learned so far now comes together in SOTA text-to-video (T2V) and image-to-video (I2V) models. You've already seen how ViT restructures images as patches ("Embeddings and Tokenization for Vision Models" on page 61) and how DiT extends this into generative diffusion ("Scalable Diffusion Models with Transformers" on page 102). In fact, many T2V and I2V models build on DiT by adding a temporal dimension, stacking latent patches across time. And remember those rotary positional embeddings from "Longer Context Windows with Better Performance" on page 9? You'll see them here again too.

Since you already understand how T2I works, T2V and I2V become less of a leap and more of a natural generalization. You just move from generating a single frame to generating a coherent sequence. Most T2V and I2V models can also generate static images and even 3D views using the same core backbone. Most of the time, the architecture stays the same. Only the axis grows.

For me, that's the elegance of transformers. Once you understand how they work, you start to see how everything fits together, across image, video, audio, and more. That's exactly why I chose to write this book about transformers beyond language: because the deeper insight is not in treating each domain separately, as with other models in deep learning, but in realizing how naturally the architecture extends across different domains.

Hence, in this chapter I'll take you on a tour, from ViT to DiT and T2I, and we'll end up at video generation. Along the way, you'll understand how each innovation builds on the last and how these domains are more connected than they first appear. You'll explore different SOTA video generation models and how to apply them to your own projects and data. You'll also revisit previously introduced models like PIXART-$\alpha$ now in a new light. So, this chapter might be your turning point in the book, because the architecture you've started out to learn is about to grow dimensions, literally.

As before, I'll focus on open source models, and you'll find all code in the book's repository. Sora and Stable Diffusion 3, for example, can generate samples at arbitrary resolutions and demonstrate strong alignment with scaling laws. The *scaling law* states that model performance improves in a predictable power-law fashion as you increase the amount of data, the number of parameters, and compute used. However, these models offer only limited insight into their design choices and don't provide detailed implementation guidance or publicly accessible pretrained checkpoints. This significantly limits their usefulness for community adoption and replication.

## Hidden Effectiveness of Latent Diffusion

The word *latent* comes from the Latin word *latēns*, meaning "hidden" or "concealed." It's this hidden and compressed representation in the latent space that makes fast, high-resolution video generation possible. Because unlike traditional diffusion models that operate directly in pixel space, latent diffusion models shift the denoising process into the latent space. This means that instead of iteratively denoising raw image or video pixels, the model performs diffusion on a lower-dimensional, compressed representation of the data. Hence the name: latent diffusion.

What this means is that what you've learned about diffusion in Chapter 4, including noise injection, denoising steps, and sampling, now happens not on raw pixels but in latent space. So we're moving from diffusion models to latent diffusion models (LDMs).[1] For images, this might mean turning a $256 \times 256 \times 3$ image into something like $32 \times 32 \times 4$ in latent form. The result is a significant reduction in dimensionality, making it feasible to scale to higher resolutions and longer video sequences.

But when we move from images to videos, the latent representation becomes spatiotemporal. That is, instead of compressing a static frame, we compress a sequence of frames, represented as a 3D tensor, into a 4D latent representation using a 3D VAE. The model no longer just learns spatial patterns; it learns motion and dynamics across time. A video of shape `F × H × W × 3` becomes `f × h × w × c`, where both time and space are downsampled.

1 Robin Rombach et al. "High-Resolution Image Synthesis with Latent Diffusion Models" (2022).

### The Elegance of Latent Space

Einstein described how mass bends spacetime, creating curves like those in a stretched fabric. Latent space reveals a similar beauty: the more complex the data, the more elegant the compression. But here, spacetime isn't curved; it's discretized and tokenized.

This is what enables transformer-based architectures to operate in video diffusion: attention layers can now process temporally-aware tokens without exploding in memory cost.

The following SOTA models all build on this foundation to capture spatiotemporal information in videos efficiently. Each of these models extends diffusion transformers into the video domain, but each of them solves a different facet of the temporal generation challenge. In Table 5-1 you see an overview how these three models align and differ in their architecture.

*Table 5-1. Evolving DiTs for video*

| Feature | Latte | LTX-Video | Tora |
|---|---|---|---|
| ViT/DiT backbone | ✓ | ✓ | ✓ |
| VAE integration | Pretrained, external | Integrated, handles final denoise | Shared latent space with motion |
| RoPE | ✓ | ✓ | ✗ |
| Motion control | ✗ | ✗ | ✓ |
| Attention scheme | Spatiotemporal decomposition | Full spatiotemporal | Alternating Spatial/Temporal blocks |
| Speed focus | ✗ | ✓ | ✗ |
| Conditioning | S-AdaLN + timestep tokens | Cross-attn + timestep-based denoise | Adaptive norm + trajectory latents |

LTX-Video[2] solves for efficiency, Latte[3] solves fidelity, and Tora[4] aims at giving you more control of your motion trajectory. Their shared reliance on ViT/DiT-style architectures, positional encoding, and attention-based denoising shows how the image-based principles of ViT/DiT scale into more complex generative modalities.

2 Yoav HaCohen et al. "LTX-Video: Realtime Video Latent Diffusion" (2024).

3 Xin Ma et al. "Latte: Latent Diffusion Transformer for Video Generation" (2025).

4 Zhenghao Zhang et al. "Tora: Trajectory-Oriented Diffusion Transformer for Video Generation" (2025).

# LTX-Video: Video in Realtime

LTX-Video is a transformer-based LDM that prioritizes spatial and temporal dimensions equally. This design choice addresses a key limitation observed in earlier models, because having both dimensions in video introduces additional complexity. While models benefit from VAEs with higher spatial compression and latent spaces of up to 64 channels, applying this setup to video requires carefully balancing spatial detail with temporal consistency.

To solve this, instead of relying solely on a second-stage diffusion model or pixel-level loss within the latent space, LTX-Video delegates the final denoising step to the VAE decoder itself, simultaneously converting latents into pixels. Figure 5-1 illustrates this process, where $z_{n-1} \sim N(0, I)$ refers to the latent variable $z_{n-1}$, which is sampled from a standard multivariate normal distribution, where the mean $\mu = 0$ and covariance $\Sigma = I$, the identity matrix, respectively. This is exactly the latent space used in most VAEs and LDMs before any decoding or denoising starts. It's the compressed, learned representation in a low-dimensional, Gaussian, structured way to allow for efficient sampling and reconstruction.

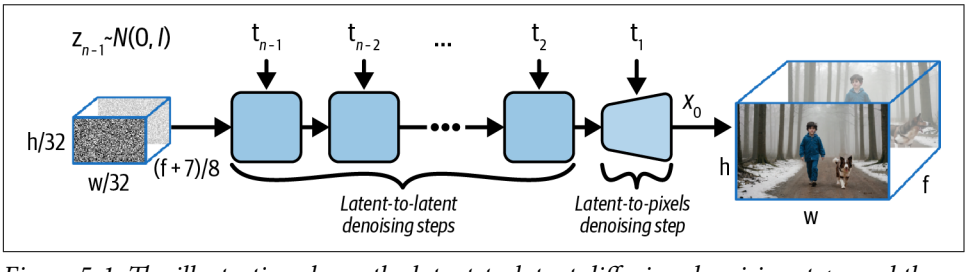


*Figure 5-1. The illustration shows the latent-to-latent diffusion denoising steps and the final latent-to-pixels denoising step. Image adapted from Yoav HaCohen et al. (2024).*

This modification is especially efficient with high latent compression rates, where certain high-frequency details can't be recovered and must be plausibly generated. In addition to T2V generation, the model also supports I2V by using a timestep-based conditioning mechanism. This allows it to be conditioned on any segment of the input, making it possible to continue from a specific frame of a video without the need for extra parameters or specialized tokens. Example 5-1 shows how to use the LTX-Video conditioning to generate a video from an image.

*Example 5-1. Video generation from image*

```
image = load_image("image.jpg") ❶

condition = LTXVideoCondition( ❷
    image=image,
    frame_index=0,
```

```
    strength=1.0  ❸
)

prompt = """A woman walking on a street with trees left and right. Above her, the
vibrant green leaves shimmer in the dappled sunlight, their color almost fluorescent
against the darker trunks. The air is alive with energy, sunlight flickering through
the canopy, painting shifting patterns across the trees."
negative_prompt = "worst quality, inconsistent motion, blurry, jittery, distorted"""


generator = torch.Generator("cuda").manual_seed(42) ❹

video = pipe(
    conditions=[condition],
    prompt=prompt,
    negative_prompt=negative_prompt,
    width=704,
    height=512,
    num_frames=48,  ❺
    frame_rate=24,
    num_inference_steps=40,
    guidance_scale=5.5,
    image_cond_noise_scale=0.15,
    generator=generator
).frames[0] ❻

export_to_video(video, "walking.mp4", fps=24) ❼
```

❶ Loads input image.

❷ Set conditioning.

❸ Apply full conditioning.

❹ Set reproducible generator.

❺ Set 2 seconds at 24 FPS.

❻ Run generation.

❼ Export to mp4.

After the model is downloaded from HuggingFace, it takes just a few seconds to generate the video from the image.

Operating in a compressed latent space is essential to the efficiency of transformer-based video generation, because, as you know, the attention mechanism scales quadratically with token count. Yet, most existing models compress space and time dimensions by factors such as $8 \times 8 \times 4$ or $8 \times 8 \times 8$, yielding compression ratios of

1:48 or 1:96 and token densities of 1:1024 or 1:2048 through patchifying. Here, *patchifying* refers to the process of grouping small spatiotemporal regions of the latent space into discrete tokens. However, this patchification and latent-space processing is conceptually similar to what you've already seen in the VAE + DiT pipeline in "Scalable Diffusion Models with Transformers" on page 102, where input images are compressed, patchified, and passed through a transformer backbone. LTX-Video builds on this, but now the Video-VAE of LTX-Video applies $32 \times 32 \times 8$ compression with 128 channels, resulting in a compression factor of 1:192 and a token density of 1:8192 for video generation. This higher compression directly enables much faster inference without degrading visual quality.

This pixels-to-latents compression is central to the model's performance and the key enabler of its speed. To support this high compression without quality loss, several architectural improvements over standard VAEs were introduced by the authors of LTX-Video. Unlike DiT-style models where patchification happens before the transformer, here it's moved into the VAE encoder. The VAE decoder is also enhanced to handle the final denoising step jointly with decoding the latents into the pixel space. This adjustment helps to bridge the gap between compressed representation and high-resolution outputs.

However, high compression introduces a challenge: imperfect convergence in rectified-flow denoising. In *rectified-flow* models, clean latents $z_0$ are predicted from noisy inputs $z_{t_i} = (1 - t_i)z_0 + t_i\epsilon$ by learning a direct mapping through a denoising function $f_\theta(z_{t_i}, t_i) = z_0$, enabling a non-iterative approximation of the denoising process. In practice, $z_0$ is initialized as pure noise $\epsilon$ and refined over a decreasing sequence of noise levels $\{t_N, t_{N-1}, \ldots, t_1\}$. Although this brings $z_0$ closer to the training distribution, the limited number of steps means residual uncertainty remains. This *residual uncertainty* refers to the mismatch between predicted and true latents, which can lead to small but noticeable deviations from the target distribution, particularly in compressed or high-frequency regions. In latent diffusion, where $z_0 = E(x_0)$ is a compressed representation of the sample $x_0$, this uncertainty can result in out-of-distribution latents and visible artifacts after decoding, especially in high-frequency regions that are poorly captured by aggressive compression.

While some models mitigate these artifacts using upsamplers operating either in pixel space or in less compressed latent spaces, these solutions are computationally expensive. Instead, LTX-Video introduces a decoder $\mathscr{D}$ that performs denoising and decoding jointly. It's trained to map noisy latents directly to clean pixels using the following equation:

$$x_0 = \mathscr{D}\left(z_{t_i}, t_i\right) = \mathscr{D}\left((1 - t_i)z_0 + t_i\epsilon, t_i\right)$$

In this equation, $z_{t_i}$ represents the noised latent, constructed as a linear interpolation between the clean latent code $z_0$ and Gaussian noise $\epsilon$, controlled by a diffusion timestep $t_i \in [0, 1]$. The decoder $\mathscr{D}$ is conditioned on this timestep and learns to recover the clean image $x_0$ directly from the noisy input. Unlike standard diffusion models that iteratively denoise latents, this setup enables a single-step denoising directly into pixel space. Since $\mathscr{D}$ maps across domains, latent to pixel, it bypasses the need for additional upsampling or post hoc refinement. This fusion of decoding and denoising ensures efficient inference while preserving visual fidelity, even under aggressive compression.

Furthermore, the model builds upon the PIXART-$\alpha$ architecture and adapts it for video generation by introducing several critical improvements. These changes include the replacement of LayerNorm with RMSNorm and the application of query-key normalization before attention to prevent low-entropy attention weights. Together, they make the model more robust and better suited for generating high-quality, temporally consistent video content. Figure 5-2 shows the transformer-block architecture of LTX-Video.

Unlike traditional absolute embeddings, RoPE is applied using *normalized fractional coordinates*, meaning they're computed in pixel and second units relative to predefined maximum resolution and duration. This helps to ensure consistency across variable-length sequences and frame rates, and allows for more natural motion generation. Additionally, the model uses exponentially increasing frequency bands in RoPE to improve stability, efficiency, and visual fidelity in both image and video generation.

#### Training and Fine-Tuning LTX-Video on Your Own Data

LTX-Video comes with a repository that offers tools and scripts for training and fine-tuning the model. It supports both LoRA-based training and full model fine-tuning on custom datasets. The trainer works with either video clips or individual images and includes additional utilities for dataset preprocessing, video captioning, and scene segmentation.

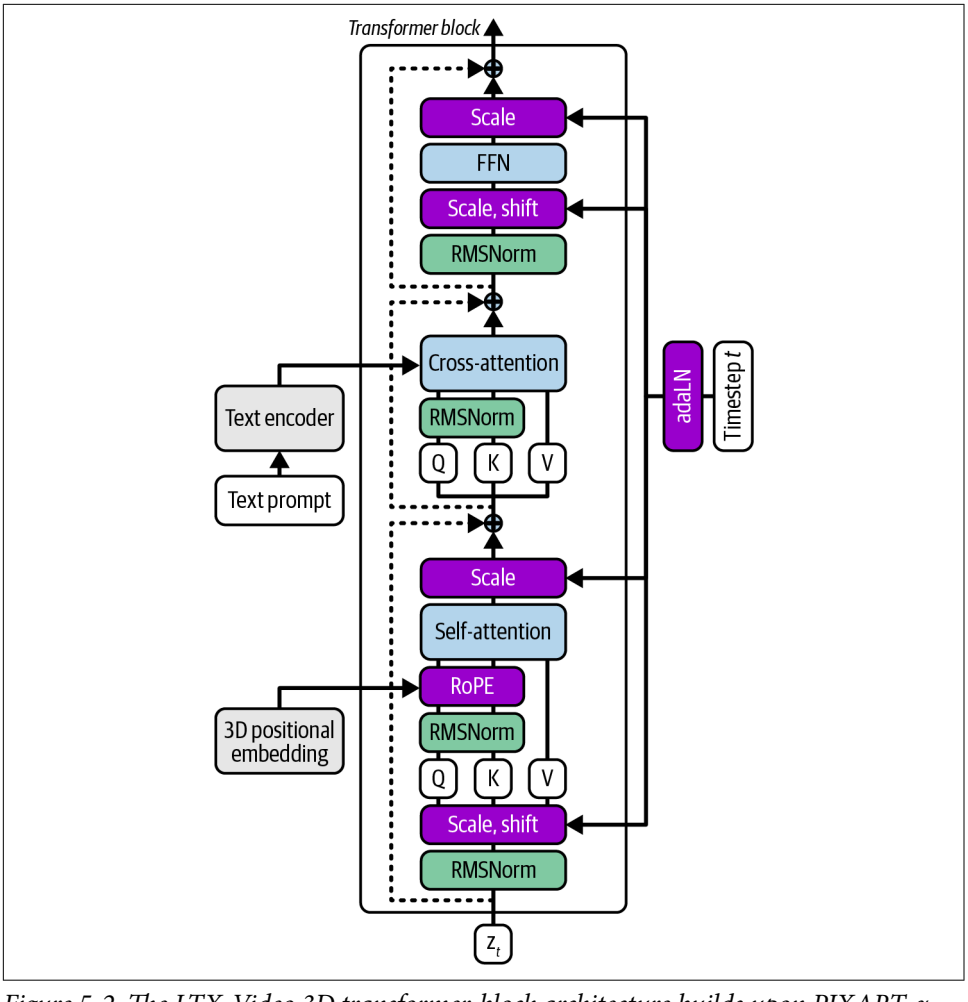


*Figure 5-2. The LTX-Video 3D transformer-block architecture builds upon PIXART-α, but replaces the LayerNorm with RMSNorm, while adding QK-normalization and RoPE. Image adapted from Yoav HaCohen et al. (2024).*

# Latte: Structured Detail, Poured into Every Video Frame

In the previous section, you learned about the key ingredients that go into building Latte, a latent diffusion transformer for video generation. This section focuses on the innovations and changes made in the specific architecture for Latte. Figure 5-3 shows an overview of Latte's four architecture variants.

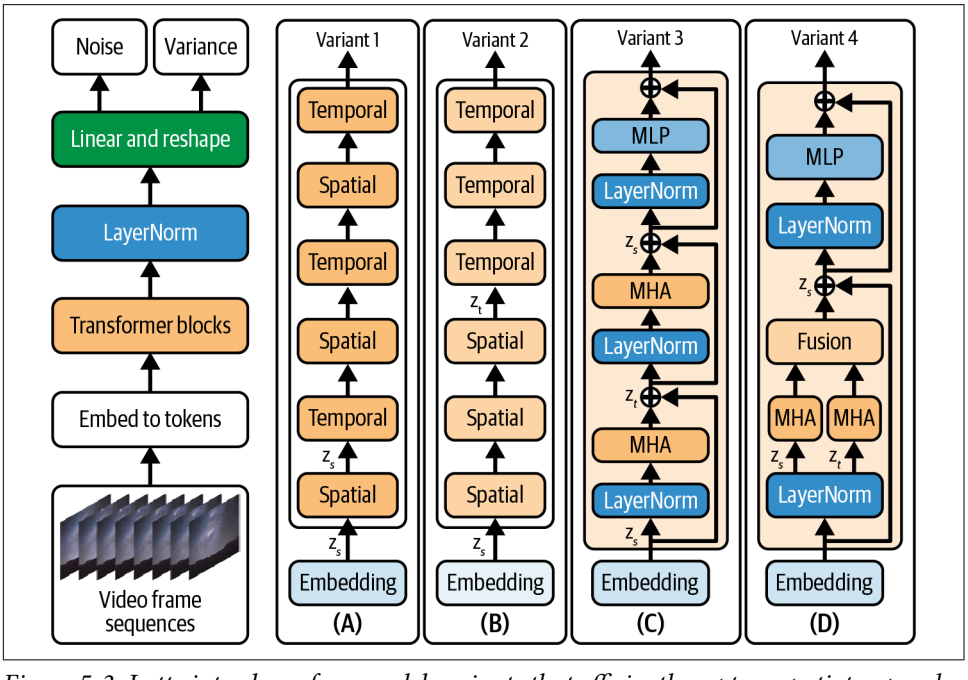


*Figure 5-3. Latte introduces four model variants that efficiently capture spatiotemporal information in videos. Each block in the diagram represents a transformer block. Variants (A) and (B) use standard blocks, while (C) and (D) include customized ones. The VAE's encoding and decoding are not shown. Image adapted from Xin Ma et al. (2025).*

Latte addresses the challenge of modeling the large number of spatiotemporal tokens extracted from videos in the latent space by introducing four transformer-based variants:

*Variant 1: Interleaved spatiotemporal attention*

This variant separates spatial and temporal attention into distinct transformer blocks. *Spatial blocks* process tokens that share the same temporal index, focusing on spatial relationships within individual frames. *Temporal blocks* capture dependencies across time by operating on spatially encoded tokens in an interleaved manner. A video clip in latent space is first tokenized into a sequence, with spatiotemporal positional embeddings added. The token sequence is reshaped to form the spatial input, then reshaped again to feed into the temporal transformer

block. This design enables the model to extract structured spatiotemporal features while maintaining factorized attention computation.

*Variant 2: Late fusion*
: This variant uses the same number of transformer blocks as variant 1 but applies a different temporal integration strategy. Instead of interleaving spatial and temporal attention, it applies them in succession. First, you apply spatial attention across tokens within each frame. Then, you apply temporal attention across frames. This late fusion design emphasizes temporal modeling as a second-stage refinement. The input shapes for both spatial and temporal blocks remain consistent with variant 1.

*Variant 3: Sequential attention decomposition*
: In contrast to the block-level separation in variants 1 and 2, this variant modifies the internal structure of each transformer block. Multi-head self-attention is decomposed sequentially, with attention first computed across spatial dimensions and then across temporal dimensions within the same block. This approach allows each transformer block to encode both spatial and temporal dependencies without requiring separate block types. The input sequences are reshaped accordingly before each attention computation.

*Variant 4: Parallel attention head splitting*
: This variant splits the attention heads within each multi-head attention layer into two groups. One group attends to spatial dimensions, and the other attends to temporal dimensions in parallel. After applying both types of attention, the outputs are reshaped and combined to serve as the input for the next module. This setup allows simultaneous modeling of spatial and temporal information while maintaining efficient attention operations. After the transformer backbone, the latent token sequence is decoded to produce both predicted noise and covariance. The output retains the same shape as the input latent video. This decoding is performed using a standard linear projection and reshaping operation, similar to the approach used in DiT.

As you can see, each variant reflects different ways of decoupling or coupling spatial and temporal dimensions. This exploration was done to understand the trade-off between spatial-temporal factorization and integration and to evaluate how the degree of decoupling impacts temporal coherence and model performance. The authors of Latte conclude that variant 1 offers the most effective balance between modeling expressiveness and architectural integrity, and adopt it as the default for both unconditional and T2V generation tasks. Table 5-2 compares, in short, the four variants.

*Table 5-2. Comparison of Latte variants*

| Variant | Design principle | Motivation | Key trade-offs |
|---|---|---|---|
| 1 | Interleaved spatial and temporal blocks | Separate but coordinated modeling of spatial and temporal dependencies | Best temporal coherence and performance but slightly more computation |
| 2 | Late fusion: all spatial first, then all temporal | Simpler to design and aligns with sequential modeling practices | Degrades temporal coherence over time, due to decoupled spatial focus |
| 3 | Sequential attention within blocks (spatial → temporal) | Mixes spatial and temporal modeling inside each block | Better than 2 and 4, but mismatched normalization reduces performance |
| 4 | Parallel attention with head splitting | Parallelism for efficiency; minimal FLOPs | Worst performance; fusion of mismatched features harms temporal consistency |

Example 5-2 explains how you can use Latte for your own video generation.

*Example 5-2. Video generation with Latte*

```
video_length = 16 ❶
pipe = LattePipeline.from_pretrained("maxin-cn/Latte-1",
torch_dtype=torch.float16).to(device) ❷

vae = AutoencoderKLTemporalDecoder.from_pretrained("maxin-cn/Latte-1",
subfolder="vae_temporal_decoder", torch_dtype=torch.float16).to(device)
pipe.vae = vae ❸

prompt = "A cat playing the guitar with a forest in the background"

video = pipe(prompt, video_length=video_length).frames[0] ❹
```

❶ Set 1 for text-to-image and 16 for text-to-video.

❷ Download the model.

❸ Use the VAE temporal decoder.

❹ Generate the video.

Now to export the generated video, you can use the utility functions of the HuggingFace diffusion models library, as done in Example 5-3.

*Example 5-3. Export generated video*

```
export_to_video(
    video_frames=video,   ❶
    output_video_path="latte.mp4", ❷
    fps=10, ❸
    quality=5.0, ❹
    bitrate=None, ❺
    macro_block_size=16  ❻
)
```

❶ Your list of frames (NumPy arrays or PIL images)

❷ Output path and filename

❸ Frames per second

❹ Variable bitrate quality (0–10)

❺ Optional fixed bitrate setting

❻ The size constraint for video, typically 16; can be 4, 8, or 1 to disable it

You could also export it to a *gif* file by using the function `export_to_gif`, if you're creating an animated video for your use case.

# Tora: From Trajectory to Storyline, One Frame at a Time

Tora builds upon the OpenSora[5] Spatial-Temporal DiT backbone and introduces two key innovations for motion control: a Trajectory Extractor (TE) and a Motion-guidance Fuser (MGF). Together, these modules give you precise control over object motion using trajectory inputs, directly integrated into the latent space of the video diffusion process.

5 Zangwei Zheng et al. "Open-Sora: Democratizing Efficient Video Production for All" (2024).

The TE transforms a user-defined trajectory into a format aligned with the video's latent tokens. Figure 5-4 shows two examples of how a user-drawn trajectory influences the motion in the generated video. Rather than working with simple frame-to-frame offsets, Tora first constructs a motion using horizontal and vertical displacements. To integrate motion with the transformer input, the compressed tensor is patchified and passed through convolutional layers with skip connections. This results in a set of hierarchical spacetime motion patches across both dimensions.

*Figure 5-4. Example of how Tora generates videos guided by user-defined trajectories. Image adapted from Zhenghao Zhang et al. (2024).*

The Spatial-Temporal DiT (ST-DiT) processes video latents by alternating between two transformer block types: spatial and temporal. A spatial block applies self-attention over the spatial dimension and includes cross-attention with the prompt. The temporal block replaces spatial self-attention with temporal self-attention, maintaining the same structure. The input latent sequence comes from patchifying the output of the video VAE, and conditioning is applied using a T5 encoder via cross-attention. Figure 5-5 illustrates the overall architectural setup of Tora.

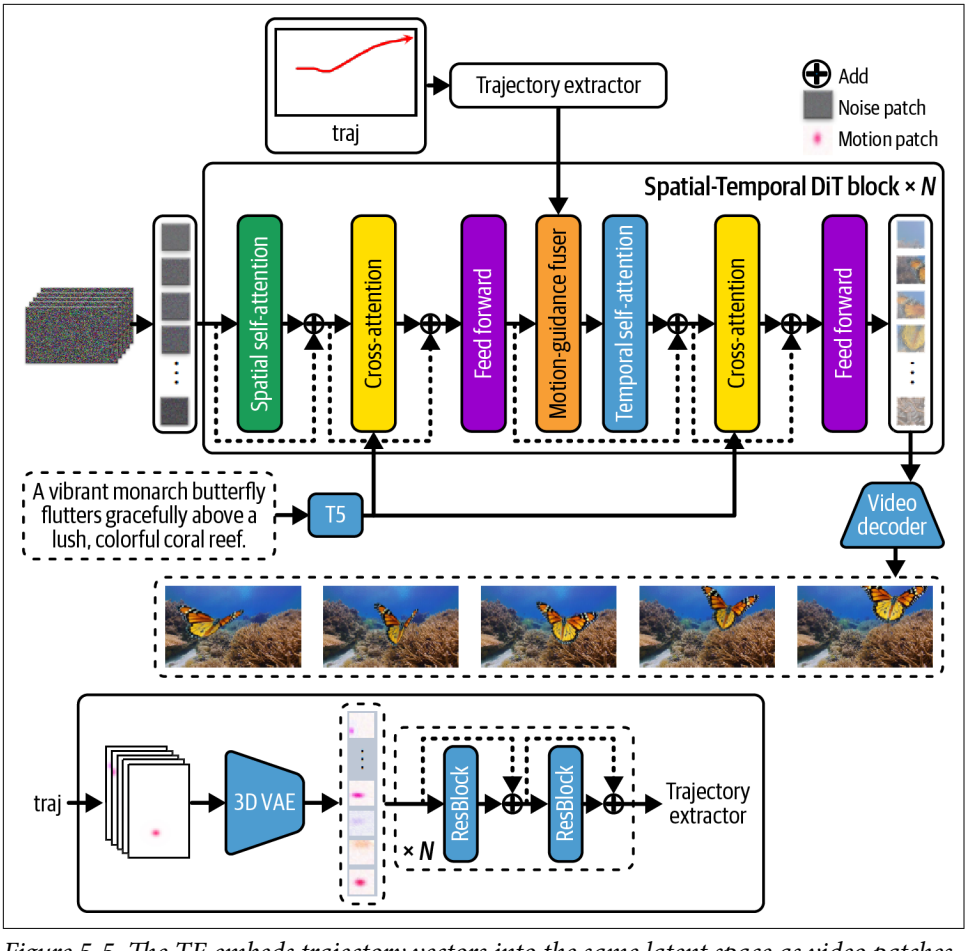


*Figure 5-5. The TE embeds trajectory vectors into the same latent space as video patches using a 3D motion VAE, preserving motion across frames. It applies stacked convolutions to extract hierarchical motion features. The MGF integrates these features into DiT blocks through adaptive normalization, guiding video generation along defined trajectories. Image adapted from Zhenghao Zhang et al. (2024).*

To inject motion control, the MGF injects trajectory-based motion information into the ST-DiT blocks using adaptive normalization. This mechanism modulates the transformer's hidden state with scale and shift parameters derived from motion embeddings. The MGF is shown in Figure 5-6.

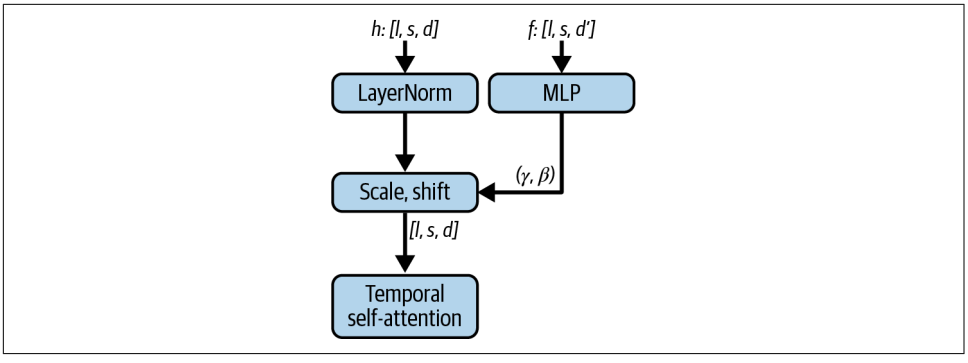


*Figure 5-6. Motion-guidance Fuser for incorporating trajectory conditioning with adaptive norm. Image adapted from Zhenghao Zhang et al. (2024).*

The following list explains each of the key variables involved:

$l$
: Temporal length of the latent video sequence after VAE downsampling

$s$
: Number of spatial patches per frame, typically $s = \frac{h \cdot w}{p^2}$

$d$
: Dimension of the DiT hidden state per token

$d'$
: Dimension of the motion embedding per token

$h_{i-1} \in \mathbb{R}^{l \times s \times d}$
: Hidden state input to DiT block $i$

$\gamma_i, \beta_i \in \mathbb{R}^{l \times s \times d}$
: Scales and shifts vectors derived from $f_i$

$f_i \in \mathbb{R}^{l \times s \times d'}$
: Motion condition for block $i$, extracted from the TE

The hidden state is updated using $h_i = \gamma_i \cdot h_{i-1} + \beta_i + h_{i-1}$.

This modulates token features per frame and patch, allowing motion control without disrupting the DiT's generative capacity. Both $\gamma_i$ and $\beta_i$ are learned via zero-initialized convolutions, ensuring smooth integration during training.

By injecting these motion features into the temporal blocks, Tora maintains realism in movement over long durations. This innovation makes Tora the first DiT-based model with native, trajectory-driven video generation.

The PyTorch code in Example 5-4 implements the simplified core logic of the TE, and Example 5-5 shows the original implementation of MGF, as described in Figures 5-5 and 5-6. These correspond directly to the motion conditioning pathway:

- Trajectory inputs $g_{\text{vis}} \rightarrow$ latents $g_m \rightarrow$ motion patches $f_i$
- Motion patches $f_i \rightarrow$ scale and shift $\rightarrow$ normalized transformer hidden states

*Example 5-4. Simplified version of Trajectory Extractor*

```
class TrajExtractor(nn.Module):
    def __init__(self, patch_size=2,
        channels=[320, 640, 1280, 1280], nums_rb=3, cin=2):
        super().__init__()
        self.patch_size = (1, patch_size, patch_size)
        self.downsize_patchify = nn.PixelUnshuffle(patch_size)

        cin_ = cin * patch_size**2 ❶
        self.conv_in = nn.Conv2d(cin_, channels[0], kernel_size=3, padding=1)

        body = []
        for i in range(len(channels)): ❷
            for j in range(nums_rb):
                in_c = channels[i - 1] if i > 0 and j == 0 else channels[i]
                out_c = channels[i]
                body.append(nn.Sequential(
                    nn.BatchNorm2d(in_c),
                    nn.ReLU(),
                    nn.Conv2d(in_c, out_c, kernel_size=3, padding=1)
                ))
        self.body = nn.ModuleList(body)

    def forward(self, x):
        # x: [B, C, T, H, W]
        T, H, W = x.shape[-3:]
        if W % self.patch_size[2] != 0:
            x = F.pad(x, (0, self.patch_size[2] - W % self.patch_size[2]))
        if H % self.patch_size[1] != 0:
            x = F.pad(x, (0, 0, 0, self.patch_size[1] - H % self.patch_size[1]))
        if T % self.patch_size[0] != 0:
            x = F.pad(x, (0, 0, 0, 0, 0, self.patch_size[0] - T % self.patch_size[0]))

        x = rearrange(x, "B C T H W -> (B T) C H W")
        x = self.downsize_patchify(x)
        x = self.conv_in(x)

        features = []
        for block in self.body: ❸
            x = block(x)
            features.append(x)
```

```
        return features ❹
```

❶ Initial projection from RGB flow image into DiT-compatible latent feature space.

❷ Stacked residual-style convolutional blocks extract multi-level motion features.

❸ Features are computed progressively for each hierarchy level, reflecting deeper spatial-temporal patterns.

❹ Returns a list of motion conditions $f_i$, aligned with DiT block depth.

*Example 5-5. Injecting motion features via adaptive normalization*

```
class MGF(nn.Module):
    def __init__(self, flow_in_channel=128, out_channels=1152):
        super().__init__()
        self.out_channels = out_channels
        self.flow_gamma_spatial = nn.Conv2d(flow_in_channel,
                                    self.out_channels // 4, 3, padding=1) ❶
        self.flow_gamma_temporal = zero_module( ❷
            nn.Conv1d(
                self.out_channels // 4,
                self.out_channels,
                kernel_size=3,
                stride=1,
                padding=1,
                padding_mode="replicate",
            )
        )
        self.flow_beta_spatial = nn.Conv2d(flow_in_channel,
                                   self.out_channels // 4, 3, padding=1)
        self.flow_beta_temporal = zero_module(
            nn.Conv1d(
                self.out_channels // 4,
                self.out_channels,
                kernel_size=3,
                stride=1,
                padding=1,
                padding_mode="replicate",
            )
        )
        self.flow_cond_norm = FloatGroupNorm(32, self.out_channels) ❸

    def forward(self, h, flow, T):
        if flow is not None:
            gamma_flow = self.flow_gamma_spatial(flow) ❹
            beta_flow = self.flow_beta_spatial(flow)
            _, _, hh, wh = beta_flow.shape
```

```
        gamma_flow = rearrange(gamma_flow, "(b f) c h w -> (b h w) c f", f=T)
        beta_flow = rearrange(beta_flow, "(b f) c h w -> (b h w) c f", f=T)
        gamma_flow = self.flow_gamma_temporal(gamma_flow) ❺
        beta_flow = self.flow_beta_temporal(beta_flow)
        gamma_flow = rearrange(gamma_flow,
            "(b h w) c f -> (b f) c h w", h=hh, w=wh)
        beta_flow = rearrange(beta_flow,
            "(b h w) c f -> (b f) c h w", h=hh, w=wh)
        h = h + self.flow_cond_norm(h) * gamma_flow + beta_flow ❻
    return h
```

❶ Spatial feature maps from flow input are projected to intermediate representations.

❷ Temporal convolution captures how gamma and beta evolve across frames; initialized to zero for smooth training.

❸ Group normalization ensures stability when modulating features.

❹ Motion embeddings are extracted frame-wise for gamma and beta.

❺ Temporal dynamics are added to gamma and beta across frames.

❻ Final fusion: latent state is modulated using learned scale and shift parameters.

#### Develop Your Own Transformer Variants

Tora was built with the help of the library SwissArmyTransformer. SwissArmyTransformer (sat) is a flexible library to bootstrap building your own custom transformer variants. It unifies models like BERT, GPT, T5, and ViT under one flexible backbone with lightweight mixins. Built on DeepSpeed-ZeRO and model parallelism, it supports efficient training and fine-tuning of large models with minimal code. You can access the open source library on GitHub.

I created a fork of the original repository of Tora to eliminate some bugs and to make it easier for you to run the model. As mentioned in the intro, the Jupyter notebook can be found in the book's repository. In the notebook you'll see a cell with:

```
!N_GPU=1 PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True torchrun --
standalone --nproc_per_node=1 sample_video.py --base configs/tora/model/
cogvideox_5b_tora.yaml configs/tora/inference_sparse.yaml --load ckpts/
tora/t2v --output-dir samples --point_path trajs/coaster.txt --input-
file assets/text/t2v/examples.txt
```

This replicates some of the examples used in the paper itself. However, you can also alter the *.txt* files in GitHub to create your own videos. The original repo comes also with a Gradio implementation. Gradio is a Python-based library to create a web interface for your machine learning applications. To start the application, you just have to run this line: `!python app.py --load ckpts/tora/t2v` in the Colab notebook. This will trigger Gradio to run. After all parts of the model are loaded, you'll see something similar to this line: *Running on public URL https://3f681e17cff82dabaf.gradio.live*. If you click on this link, a new browser tab will open, and you'll see the dashboard, as shown in Figure 5-7.

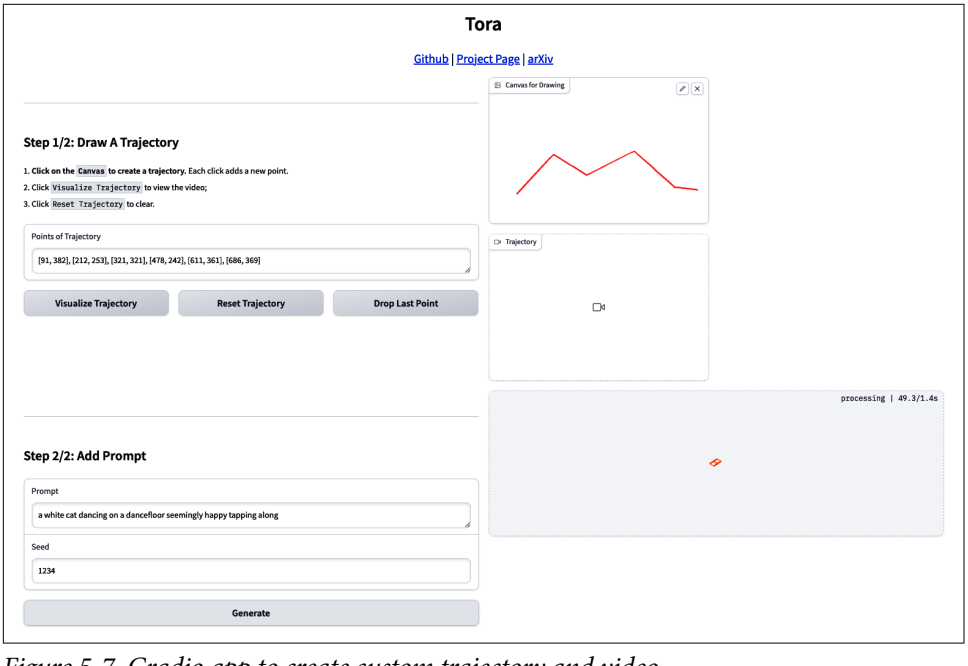


*Figure 5-7. Gradio app to create custom trajectory and video.*

Here you can create your own trajectories by clicking on the canvas and drawing the trajectory. After that, you can add a prompt and click `generate` to generate a video based on this. This process will take about 2–3 minutes. Alternatively, you can also adopt predefined trajectories and prompts. You can find them, if you scroll down to see the rest of the dashboard, as shown in Figure 5-8.

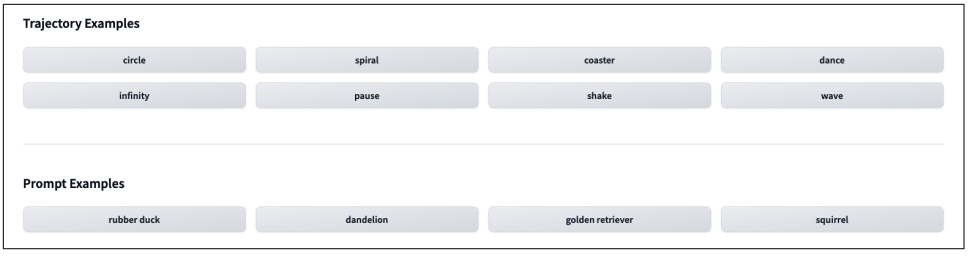


*Figure 5-8. Adapting predefined prompts and/or trajectories.*

Whether you choose to draw your own motion paths or start with the predefined examples, this interface gives you full creative control over how your videos unfold, one frame at a time. It's a powerful reminder that with the right tools and model, motion is no longer something you only observe. It becomes a trajectory you actively design for your own videos.

# Conclusion

In this chapter, you explored how the transformer architecture naturally extends from images to videos by building on core components such as ViT, DiT, and latent diffusion. You learned that compressing high-dimensional pixel data into structured latent spaces makes it feasible to generate coherent, high-resolution video sequences while preserving spatial and temporal consistency.

You saw how state-of-the-art models like LTX-Video, Latte, and Tora each tackle different challenges in video generation, ranging from real-time efficiency and high-fidelity output to motion control via user-defined trajectories. Each model extends the same architectural backbone in a unique direction: LTX-Video with tightly integrated VAEs and timestep conditioning, Latte with structured attention decomposition across spatiotemporal dimensions, and Tora with trajectory-guided generation using motion-conditioned normalization layers.

Throughout this chapter, you also deepened your understanding of how latent diffusion enables scalable video generation by shifting the denoising process to a lower-dimensional space. This, combined with transformer-based attention mechanisms and architectural innovations like adaptive normalization or RoPE with fractional coordinates, reveals how the transformer design evolves to handle increasingly complex generative tasks.

Most importantly, you've seen how the transformer's ability to model sequences generalizes from text to image to video, with no need to fundamentally redesign the architecture. This insight will serve as your foundation for Chapter 6. Just as motion unfolds frame-by-frame, sound unfolds waveform-by-waveform: structured, temporal, and tokenized.

CHAPTER 6

# From Sound to Token and Back: Transformers in the Audio Domain

This chapter takes you one step further along the dimensional axis. You started with the original transformer—that is, language—then moved to time series. After that, you explored vision and video. Now, you'll dive into the audio domain. And spoiler alert, once again, transformers prove to be surprisingly adaptable. As it turns out, it doesn't matter whether you're modeling discrete tokens, image patches, or spectrogram frames. The core logic remains intact. That is the elegance of the transformer. You change the axis but not the basic architecture. I'm sure, by now, you're seeing it for yourself: transformers are not just models; they're a framework for abstraction. This is why they aren't a passing trend but a new way to generalize and unify modeling across domains.

What makes audio particularly interesting is that it sits right at the intersection of temporal structure and spectral representation. At first glance, audio might look like a classic time series, just a waveform over time, but its richness lies in the frequency domain. That's why so many audio models convert raw waveforms into spectrograms or mel frequency features before tokenizing them.

A *spectrogram* shows how the energy of different frequency components in a signal changes over time, giving a time-frequency view of the sound. A *mel frequency* representation maps those frequencies onto a perceptual scale that better reflects how humans hear, emphasizing lower frequencies where our hearing is most sensitive. Figure 6-1 compares the spectrogram and mel spectrogram of electronic dance music (EDM) to those of classical music and shows how they exhibit distinct spectral patterns. Here, you can clearly see that the EDM signal shows dense, repetitive vertical bands indicating consistent high-energy beats across a wide frequency range. In contrast, the classical signal contains more subtle, temporally varied energy concentrated

in the lower-frequency range. This raw data form of sound is what enables transformer models to classify environmental sounds, recognize speech patterns, and even generate different musical styles with coherence and control.

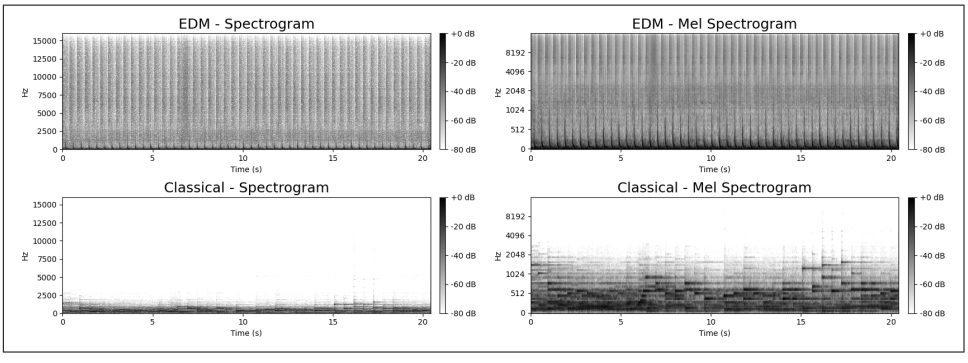


*Figure 6-1. Comparison of 20-second excerpts from EDM and classical music, each visualized as both a linear spectrogram (left) and a mel spectrogram (right).*

In this chapter, you'll learn about a range of core audio tasks, including text-to-speech (TTS), automatic speech recognition, speech-to-text (S2T), speech emotion recognition, and music generation. To keep the focus on modern architectures, classical models such as Whisper will be covered only briefly.[1]

While Whisper is still used as an encoder in many SOTA models today, I'll shift focus to newer audio foundation models that can handle multiple tasks within a single architecture. These large audio-language models (LALMs) include Qwen2-Audio[2] and Kimi-Audio,[3] which go beyond task-specific processing and demonstrate unified capabilities across recognition, generation, classification, and audio-language alignment. The chapter will also cover transformer-based models for music generation, showing how these models can be used to produce musically coherent sequences with long-range structure and stylistic control via text.

As with time series, audio models must reason across long contexts, adapt to different sampling rates, and handle variable-length inputs. But unlike time series, audio is denser, more perceptually nuanced, and often multi-channel. That adds complexity and opens up powerful opportunities for generative models to produce incredibly lifelike outputs.

1 Alec Radford et al. "Robust Speech Recognition via Large-Scale Weak Supervision" (2022).

2 Yunfei Chu et al. "Qwen2-Audio Technical Report" (2024).

3 KimiTeam et al. "Kimi-Audio Technical Report" (2025).

# From Waveforms to Spectrograms: Understanding the Structure of Audio Data

When working with audio data, especially in the context of transformer-based modeling, it's important to understand the structure and characteristics of audio signals. This section offers a conceptual and mathematical overview of digital audio, including its time-domain and frequency-domain properties, and the transformations commonly applied before feeding it into models.

## Audio as a Waveform

*Sound* is a continuous signal generated by changes in air pressure over time. In digital audio, this continuous signal is sampled at discrete time intervals to create a numerical sequence known as a *waveform*. The waveform represents amplitude (loudness) as a function of time, as shown in Figure 6-2 for various music genres.

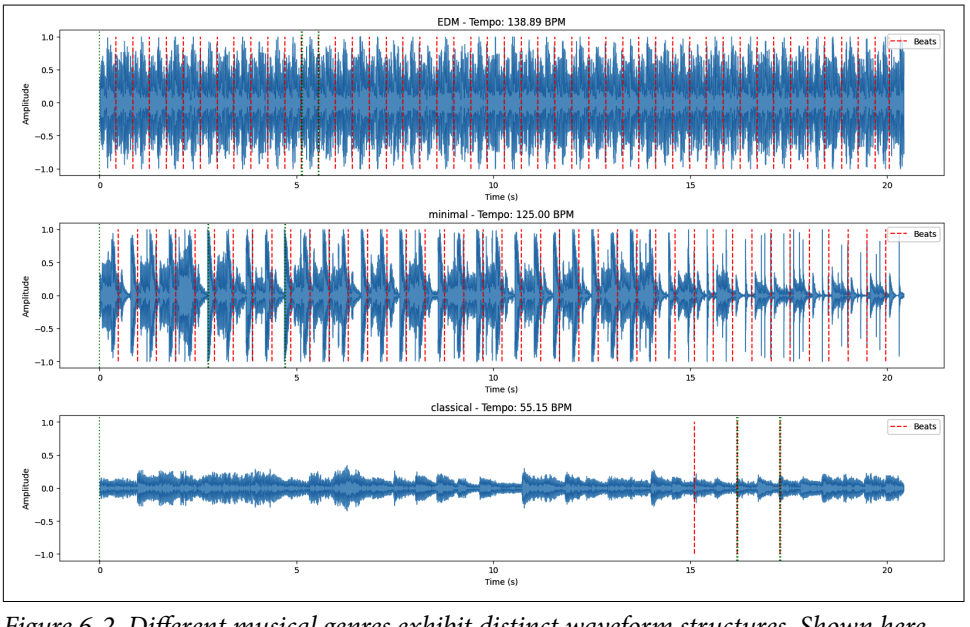


*Figure 6-2. Different musical genres exhibit distinct waveform structures. Shown here are 20-second excerpts from EDM, minimal techno, and classical music. The tempo and rhythmic patterns vary significantly: EDM has a fast, consistent pulse; minimal is more groove-oriented with dynamic amplitude shifts; and classical displays a fluid, low-tempo structure with more subtle dynamics. These differences are what transformer models can learn to recognize, classify, or even reproduce when trained on the right representations.*

Mathematically, a sampled audio signal can be described as a sequence of amplitude values:

$$x[n] = x(nT), \quad n = 0, 1, 2, \ldots, N-1$$

Here, $x(nT)$ represents the sampled amplitude at time $t = nT$, with $T = \frac{1}{f_s}$ being the sampling interval and $f_s$ the sampling rate in Hz (samples per second).

The duration $D$ of an audio signal in seconds is then:

$$D = \frac{N}{f_s}$$

For example, at a sampling rate of 16,000 Hz, a 5-second audio clip contains 80,000 discrete samples.

## Sampling Rate and the Nyquist Theorem

The sampling rate determines the highest frequency that can be accurately captured. This is governed by the *Nyquist theorem*, which states that the sampling rate must be at least twice the highest frequency present in the signal to avoid aliasing:

$$f_{\max} \leq \frac{f_s}{2}$$

This upper bound, $f_s/2$, is known as the Nyquist frequency. For speech signals, which typically fall under 8 kHz, a sampling rate of 16 kHz is usually sufficient. Using higher sampling rates (such as 44.1 kHz for music) can capture more detail but also increases memory and computational cost.

## Amplitude, Bit Depth, and Quantization

Each sampled value corresponds to the amplitude of the signal at a given moment, typically measured in decibels (dB). In digital audio, the amplitude is quantized into discrete levels, determined by the bit depth. Table 6-1 shows an overview.

*Table 6-1. Bit depth and amplitude resolution in digital audio*

| Bit depth | Amplitude values | Use case |
|---|---|---|
| 16-bit | 65,536 | Standard audio (e.g., CD quality) |
| 24-bit | 16,777,216 | Professional audio recording |
| 32-bit float | ~4.29 billion (theoretical) | High dynamic range, ML-based synthesis and training |

Quantization introduces a form of noise, which is more noticeable at lower bit depths. Formally, the quantization error $\varepsilon$ is:

$$\varepsilon[n] = x[n] - \hat{x}[n]$$

Here, $x[n]$ is the original signal, and $\hat{x}[n]$ is the quantized version. Higher bit depth reduces $|\varepsilon[n]|$ on average).

## The Frequency Domain and Fourier Transform

The *Fourier transform* decomposes an audio signal into its constituent frequencies. When applied to discrete signals, this process is referred to as the *discrete Fourier transform* (DFT). The DFT of a signal $x[n]$ of length $N$ is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \ldots, N-1$$

The magnitude $|X[k]|$ gives the amplitude of the $k$-th frequency component, and the phase is encoded in the complex angle.

Figure 6-3 shows the frequency spectrum of a trumpet note, with harmonics visible as peaks.

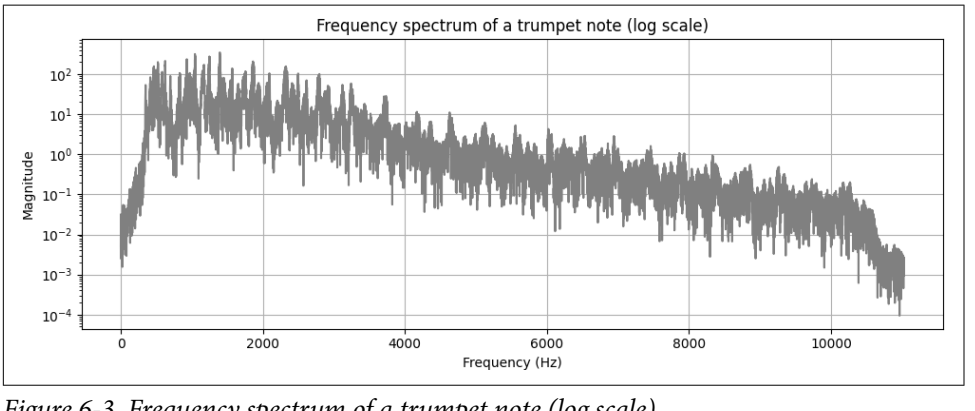


*Figure 6-3. Frequency spectrum of a trumpet note (log scale).*

### Audio and Music Signal Analysis in Python

Librosa is a Python package for music and audio analysis, designed to support the development of music information retrieval (MIR) systems. It offers a wide range of tools for signal processing, including loading and playing audio, computing spectrograms, extracting features, detecting onsets and beats, separating harmonic and percussive sources, and visualizing audio data.

## Spectrograms and the Short-Time Fourier Transform

To visualize how frequencies evolve over time, you compute the *short-time Fourier transform* (STFT), which segments the signal into overlapping windows and applies the DFT to each segment. This results in a spectrogram:

$$S(m,\omega) = \sum_{n=-\infty}^{\infty} x[n]w[n-m]e^{-j\omega n}$$

where $w[n]$ is the windowing function such as Hanning and $m$ indexes the window position. A *Hanning window* is a smooth, bell-shaped function applied to a signal segment to reduce edge discontinuities and minimize spectral leakage during Fourier analysis. The spectrogram is a 2D matrix with time on the $x$-axis, frequency on the $y$-axis, and magnitude represented by color intensity. Figure 6-4 illustrates the spectrogram of a trumpet signal, showing how its frequency content changes over time.

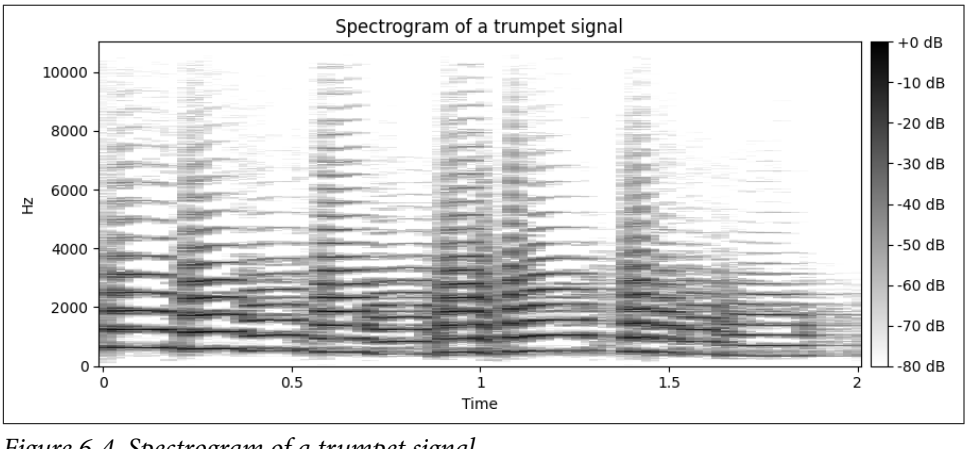


*Figure 6-4. Spectrogram of a trumpet signal.*

## The Mel Spectrogram and Perceptual Scaling

A *mel spectrogram* is a perceptually weighted spectrogram that maps linear frequency to the mel scale, which approximates human auditory sensitivity. The mel frequency $m$ is given by:

$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right)$$

This emphasizes lower frequencies, which humans are more sensitive to. Mel spectrograms are widely used in speech processing, automatic speech recognition, and audio classification tasks. Figure 6-5 shows an example.

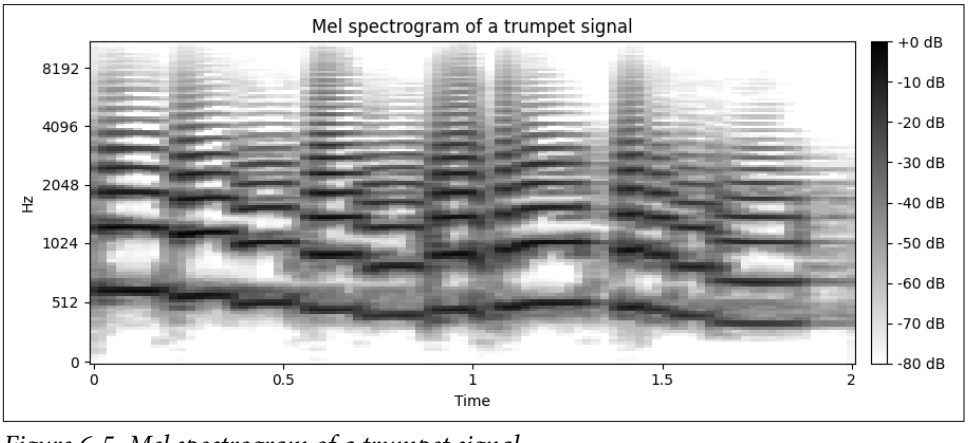


*Figure 6-5. Mel spectrogram of a trumpet signal.*

## Phase, Reconstruction, and Vocoders

Spectrogram-based representations typically discard phase information, which is essential for accurate waveform reconstruction. A *vocoder* is a model or algorithm that reconstructs audio waveforms from acoustic features like spectrograms, often handling both phase and amplitude to produce intelligible or natural-sounding speech. While the STFT fundamentally produces complex values that encode both magnitude and phase, many machine learning pipelines primarily operate on the magnitude spectrum. Traditional signal processing approaches, such as iterative phase estimation algorithms like Griffin-Lim, attempt to approximate the missing phase but often produce artifacts and lack perceptual quality. The *Griffin-Lim Algorithm* is a phase reconstruction method that iteratively enforces spectrogram consistency by leveraging the redundancy in the STFT. A spectrogram is considered *consistent* when inter-bin dependencies from the STFT are preserved. However, despite being widely used in early TTS systems, Griffin-Lim is often considered subpar in perceptual quality compared to modern learned vocoders. In modern deep

learning pipelines, this complex role of generating a plausible phase and amplitude jointly for high-fidelity synthesis is taken over by neural vocoders.

To understand how these spectrogram representations are derived (including magnitude and phase), consider the core components of an STFT in Example 6-1, which is abstracted from the original code of Mooncast.[4]

*Example 6-1. Simplified short-time Fourier transform*

```
class SimplifiedSTFT(nn.Module):
    def __init__(
        self,
        n_fft: int,
        hop_length: int,
        win_length: int,
        center: bool = True,
    ):
        super().__init__()
        self.n_fft = n_fft
        self.hop_length = hop_length
        self.win_length = win_length
        self.center = center
        window = torch.hann_window(win_length) ❶
        self.register_buffer("window", window) ❷

    def forward(self, x: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
        stft_spec = torch.stft( ❸
            x,
            self.n_fft,
            hop_length=self.hop_length,
            win_length=self.win_length,
            window=self.window,
            center=self.center,
            return_complex=False, ❹
        )

        real_part = stft_spec[:, :, :, 0] ❺
        imag_part = stft_spec[:, :, :, 1]

        magnitude = torch.sqrt(real_part.pow(2) + imag_part.pow(2) + 1e-5) ❻
        phase = torch.atan2(imag_part, real_part) ❼

        return magnitude, phase
```

❶ Create Hann window (commonly used in STFT).

4 Zeqian Ju et al. "MoonCast: High-Quality Zero-Shot Podcast Generation" (2025).

❷ Register window as buffer so it's saved with model.

❸ Perform STFT using PyTorch's built-in function.

❹ Separate real and imaginary parts instead of using complex tensors.

❺ Extract real and imaginary parts from last dimension.

❻ Compute magnitude (add epsilon for numerical stability).

❼ Compute phase using `atan2` to handle all quadrants.

The `SimplifiedSTFT` class, shown previously, demonstrates the analysis process: taking a raw waveform and converting it into its magnitude and phase spectrogram components. Internally, the short-time Fourier transform decomposes the signal into real and imaginary parts, which correspond to projections onto cosine and sine basis functions, respectively. These are not "imaginary" in the everyday sense. The imaginary part refers to a real-valued component from the mathematical concept of complex numbers, which is multiplied by the imaginary unit $j$, representing a 90-degree rotation in the complex plane. Together, they form a complex-valued representation that encodes both the amplitude and phase of each frequency component. While the phase information is explicitly captured, many downstream machine learning models often choose to process solely on the magnitude (or mel spectrogram derived from it) due to its interpretability and strong perceptual correlation. This decision, however, creates a need for robust reconstruction.

Waveform reconstruction involves taking the inverse short-time Fourier transform (ISTFT):

$$x[n] = \sum_{m} S(m, \omega) w[n - m] e^{j\omega n}$$

As seen in the `SimplifiedISTFT` class in Example 6-2, this inverse transformation (which can be a custom implementation or `torch.istft` when supported) produces realistic results only if both magnitude and phase are accurately provided in the complex spectrogram input. When only the magnitude (or a processed version like a mel spectrogram from a generative model) is available, neural vocoders step in.

*Example 6-2. Simplified inverse short-time Fourier transform*

```
class SimplifiedISTFT(nn.Module):
    def __init__(
        self, n_fft: int, hop_length: int, win_length: int, center: bool = True
    ):
```

```
        super().__init__()
        self.n_fft = n_fft
        self.hop_length = hop_length
        self.win_length = win_length
        self.center = center
        window = torch.hann_window(win_length)            ❶
        self.register_buffer("window", window)            ❷

    def forward(self, complex_spec: torch.Tensor) -> torch.Tensor:
        waveform = torch.istft(                           ❸
            complex_spec,
            self.n_fft,
            hop_length=self.hop_length,
            win_length=self.win_length,
            window=self.window,
            center=self.center,
        )
        return waveform                                   ❹
```

❶ Create Hann window for inverse transform.

❷ Register window as buffer (ensures persistence in model state).

❸ Apply inverse STFT to reconstruct the waveform from complex spectrogram.

❹ Output time-domain waveform (`shape: [Batch, Time]`).

*Vocoders* learn to infer a plausible phase signal that aligns with the spectral structure of the input, enabling the generation of high-fidelity audio by effectively supplying the missing phase and reconstructing the waveform through the ISTFT process. These vocoders can be autoregressive (like WaveNet),[5] GAN-based (like HiFi-GAN),[6] or based on other generative techniques such as diffusion. They have become standard components in TTS, music generation, and audio restoration tasks, learning to generate plausible phase and amplitude jointly.

Now that you understand how audio can be represented and reconstructed, the question becomes: what can you actually do with it? In other words, what are the key ML tasks in the audio domain, and how do transformers fit in?

Much like language-based tasks, audio tasks span a range of domains: classification, generation, recognition, translation, and more. The difference is in the input representation. The next section is a brief tour through the major categories.

5 Aaron van den Oord et al. "WaveNet: A Generative Model for Raw Audio" (2016).

6 Jungil Kong et al. "HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis" (2020).

# Audio Modeling in Various Application Domains

Transformer-based models are reshaping the landscape of audio tasks, much like they have in language and vision. Audio spans continuous and discrete forms, enabling a range of tasks across speech, music, and general sound processing. These include classification, generation, recognition, and translation. This section outlines the core types of audio tasks with brief application examples:

*Text-to-speech*
: Text-to-speech (TTS) converts written text into spoken audio. Applications include voice assistants, audiobooks, accessibility tools, and interactive systems. High-quality TTS requires control over emotion, speaker identity, and prosody, making it a rich testbed for conditional generation. *Prosody* is the melody of speech: the rhythm, stress, intonation, and emphasis patterns that shape how we perceive meaning, emotion, and intent of spoken language.

*Automatic Speech Recognition*
: Automatic Speech Recognition (ASR) transcribes spoken language into text. Applications include transcription tools, subtitles, meeting notes, and hands-free control. Whisper is a strong multilingual baseline that handles noise and accents well.

*Speech Translation*
: Speech Translation (ST) translates spoken audio in one language into text or speech in another. While traditional pipelines use ASR followed by machine translation, and optionally TTS, transformer models increasingly support end-to-end ST. This is key for real-time translation, global communication tools, and multilingual accessibility.

*Speech Emotion Recognition*
: Speech Emotion Recognition (SER) detects emotions such as joy, anger, sadness, or neutrality from vocal cues like pitch, tempo, and prosody. It's used in customer support, human–machine interaction, and adaptive learning systems.

*Speaker identification and diarization*
: Speaker identification determines who is speaking; diarization segments audio by speaker turns. These tasks use embedding-based methods like x-vectors or ECAPA, often refined using transformers. They're useful in meeting transcription, call center analysis, and forensic audio.

*Music generation*
: Music generation involves creating musical pieces from symbolic input or text prompts. Transformers like MusicGen produce coherent sequences with genre, rhythm, and structure. Applications include soundtrack generation, interactive tools, and digital composition.

*Audio classification*
: This task assigns labels to audio clips, such as genre, instrument, or environmental sound (e.g., siren, rain). It's widely used in smart devices, acoustic monitoring, sound tagging, and media organization.

*Audio segmentation and event detection*
: These tasks identify the timing of events such as speech onsets, gunshots, or musical transitions. Transformers model long-context sequences and event co-occurrence, enabling use in surveillance, podcast indexing, and scene analysis.

These tasks vary in output but share similar input formats, such as waveforms or spectrograms. The next step is to understand how transformer models process these representations. From focused systems like Whisper to foundation models like Qwen2 Audio and Kimia, architectures have rapidly evolved.

# Transformer Architectures for Audio: From Perception to Foundational Intelligence

This section connects your theoretical understanding of audio representations to their practical application in transformer-based models. You'll start off with task-specific models like Whisper, which set a new standard for robustness in speech recognition and translation. From there, you'll explore how newer audio foundation models move beyond narrow objectives and begin to unify capabilities across speech, music, and general acoustic understanding. These models don't just transcribe or generate; they listen, reason, and respond with contextual awareness, long-range memory, and multimodal alignment.

## The Rise of Speech Transformers: The Impact of Whisper

Whisper[7] marks a pivotal advancement in ASR and audio translation by leveraging a robust transformer-based architecture. Unlike earlier systems that depended heavily on supervised fine-tuning or unsupervised pretraining alone, Whisper demonstrates how large-scale weak supervision, coupled with a powerful encoder-decoder transformer model, can redefine the boundaries of generalization and robustness in speech processing.

At its core, Whisper processes audio inputs transformed into mel spectrograms. This is especially well-suited for transformers. These spectrograms serve as the model's input, passed through a convolutional part before being encoded by transformer blocks. The decoder, conditioned on audio and prior text, generates transcriptions or translations. This sequence-to-sequence approach simplifies the speech pipeline by

7 Alec Radford et al. (2022).

unifying tasks like transcription, translation, and voice activity detection under one model and enhances performance consistency across them. The overall architecture of Whisper is illustrated in Figure 6-6.

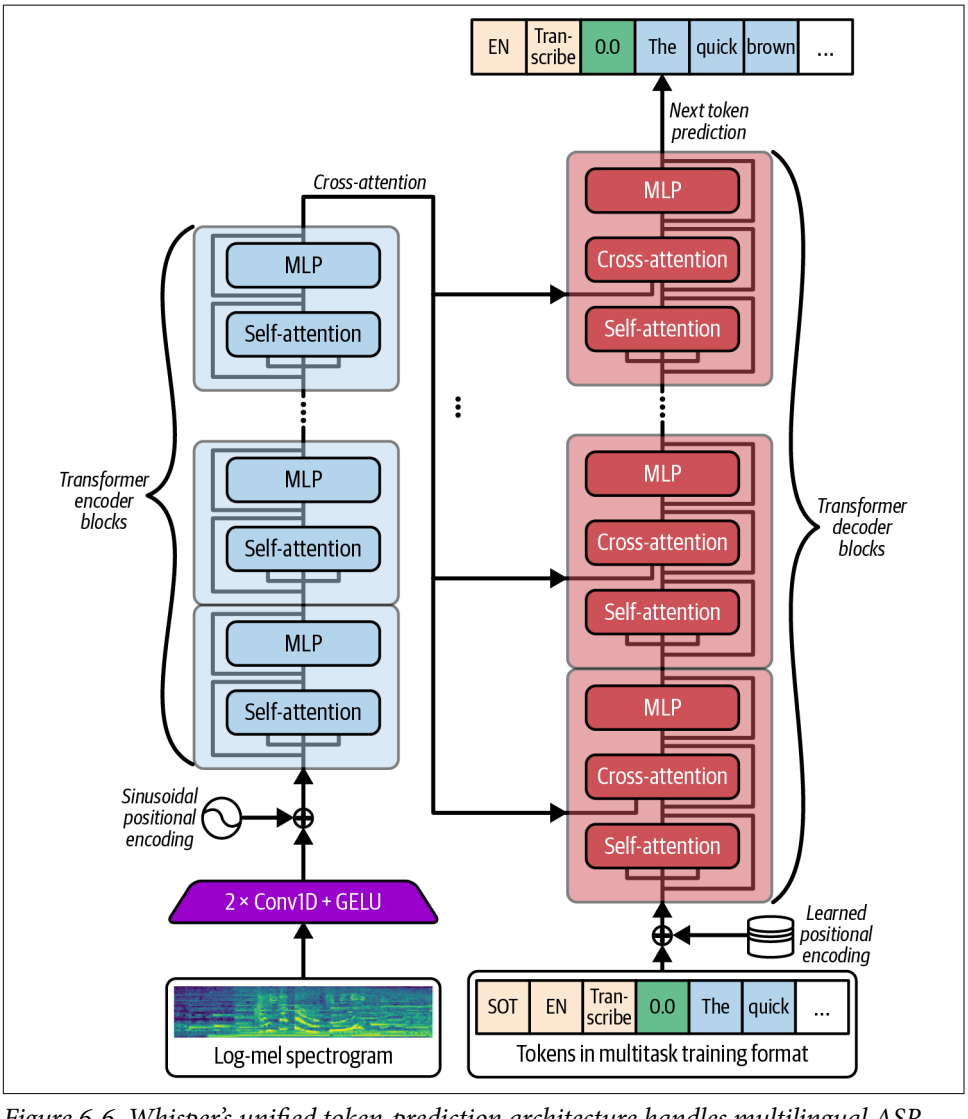


*Figure 6-6. Whisper's unified token-prediction architecture handles multilingual ASR, translation, language ID, and voice activity detection. Image adapted from Alec Radford et al. (2022).*

In practice, Whisper bridges signal and sequence through two core operations: transforming audio into mel spectrograms, then decoding these spectrograms into text. Example 6-3 distills audio into a transformer-ready format, while Example 6-4 executes the inference loop using learned multilingual and multitask priors. Both functions are simplified adoptions of the original Whisper code to illustrate the core functionality.

*Example 6-3. Simplified log-mel spectrogram computation*

```
def log_mel_spectrogram(audio: torch.Tensor, sample_rate:
                        int = 16000) -> torch.Tensor:
    mel_spec = torchaudio.transforms.MelSpectrogram( ❶
        sample_rate=sample_rate,
        n_fft=400,
        hop_length=160,
        n_mels=80
    )(audio)
    log_mel_spec = torch.log1p(mel_spec) ❷
    return log_mel_spec ❸
```

❶ Convert waveform to mel spectrogram using 25 ms windows and 10 ms stride.

❷ Apply logarithmic scaling to emphasize perceptual loudness differences.

❸ Output 80-channel log-mel spectrogram used as Whisper's input.

*Example 6-4. Simplified Whisper decoding for mel spectrogram input*

```
@torch.no_grad() ❶
def decode(
    model: "Whisper",
    mel: torch.Tensor,
    options: DecodingOptions = DecodingOptions(),
    **kwargs,
) -> Union[DecodingResult, List[DecodingResult]]:
    if single := mel.ndim == 2: ❷
        mel = mel.unsqueeze(0)

    if kwargs:
        options = replace(options, **kwargs) ❸

    result = DecodingTask(model, options).run(mel) ❹
    return result[0] if single else result ❺
```

❶ Disable gradient tracking to optimize inference (no backpropagation).

❷ Add batch dimension if input is a single mel spectrogram.

❸ Override default decoding options if extra arguments are provided.

❹ Run Whisper's decoding pipeline on the input.

❺ Return the decoded result, handling single or batch cases.

## Audio Foundation Models: Unifying Understanding, Generation, and Conversation

For years, AI models tackled audio tasks like speech recognition, emotion detection, or sound event classification as separate challenges, each requiring its own specialized approach. But just as the field of natural language processing saw the rise of LLMs that could handle diverse text tasks, audio processing is undergoing a similar revolution.

This shift is driven by the emergence of LALMs, often referred to as *audio foundation models*. These powerful transformer-based systems aim to unify a wide array of audio-related capabilities into a single, cohesive framework. Unlike their predecessors, these models are designed to process various audio signals, understand the different content, analyze the audio signals, and even generate direct textual responses or new audio, all based on natural language instructions.

Models like Qwen2-Audio and Kimi-Audio are examples of this new model paradigm. They leverage massive datasets, often scaling into millions of hours of audio data encompassing speech, environmental sounds, and music. By simplifying their pretraining through natural language prompts, these models gain exceptional instruction-following capabilities, whether it's analyzing an audio segment containing simultaneous sounds, multi-speaker conversations, a voice command, or free-form voice chat. These audio foundation models are built to intelligently comprehend the audio content and respond appropriately.

## Qwen2-Audio

At its core, Qwen2-Audio is an 8.2-billion parameter LALM engineered to process both audio and text inputs to produce textual outputs. Its architecture features an audio encoder paired with an LLM. The audio encoder's initialization is based on the Whisper-large-v3 model. For processing raw audio, Qwen2-Audio resamples the waveform to 16 kHz and converts it into a 128-channel mel spectrogram, followed by a pooling layer to efficiently reduce representation length. The LLM component is powered by Qwen-7B.[8]

Qwen2-Audio uses natural language prompts for different data and tasks. This strategy, combined with a scaled-up training dataset and further instruction tuning with direct preference optimization (DPO),[9] enhances the model's instruction-following ability and generalization. Figure 6-7 illustrates this process.

Qwen2 Audio provides two distinct interaction modes designed for flexible user engagement. In audio analysis mode, it can provide various audio types (speech, sound, music, or mixed audio) along with text or audio instructions for analysis. The model intelligently discerns commands within the audio itself, providing direct interpretations and responses. In voice chat mode, users can engage in free-form, multi-turn voice conversations with the model without requiring any text input or manual mode switching.

8 Jinze Bai et al. "Qwen Technical Report" (2023).

9 Rafael Rafailov et al. "Direct Preference Optimization: Your Language Model Is Secretly a Reward Model" (2023).

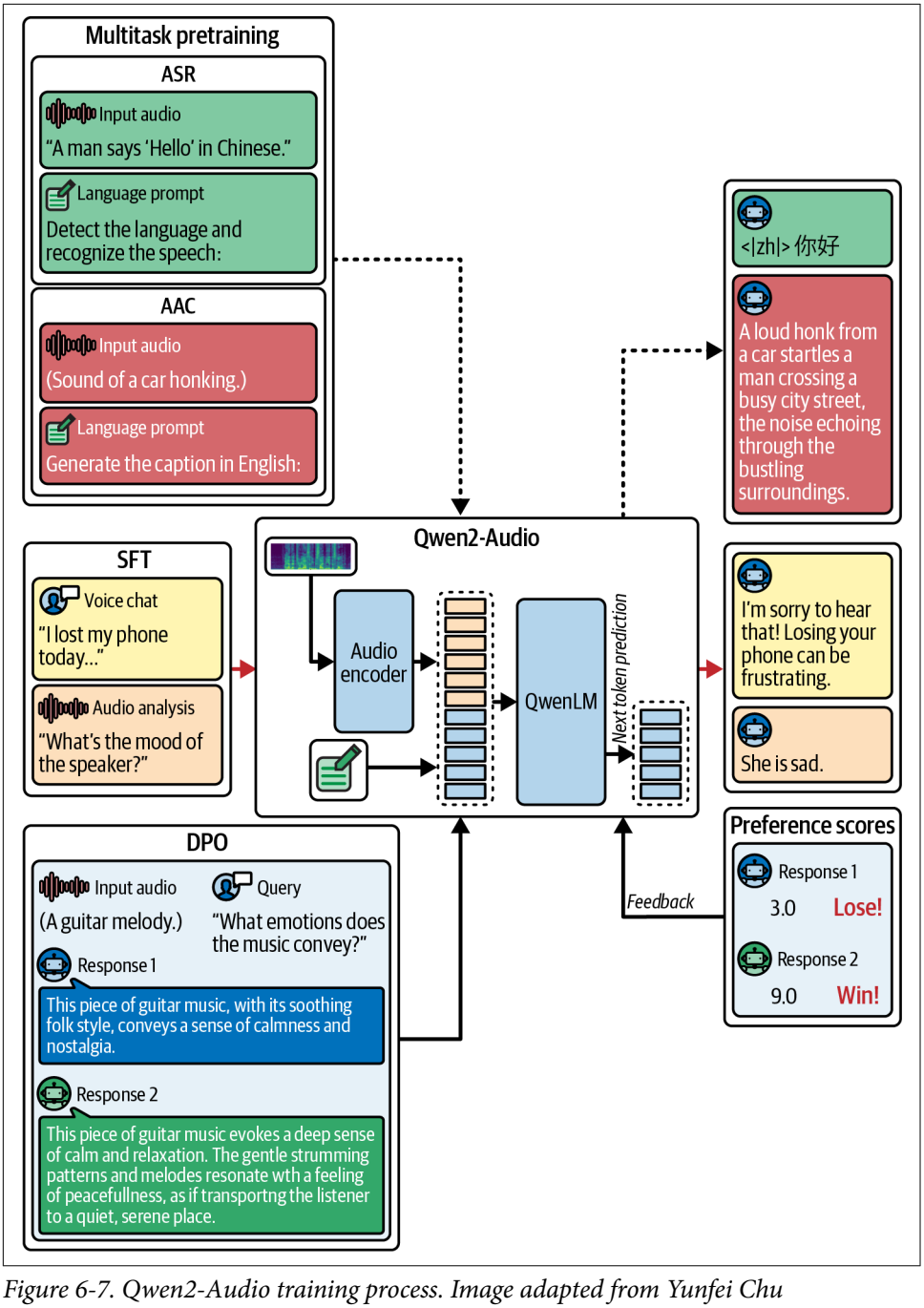


*Figure 6-7. Qwen2-Audio training process. Image adapted from Yunfei Chu et al. (2024).*

Example 6-5 shows different use cases of Qwen2-Audio.

*Example 6-5. Qwen2-Audio example use cases*

```
processor = AutoProcessor.from_pretrained("Qwen/Qwen2-Audio-7B-Instruct")
model = Qwen2AudioForConditionalGeneration.from_pretrained(
    "Qwen/Qwen2-Audio-7B-Instruct", device_map="auto"
)

conversation1 = [
    {"role": "user", "content": [
    {"type": "audio", "audio_url": "https://qianwen-res.oss-cn-beijing.aliyuncs
    .com/Qwen2-Audio/audio/glass-breaking-151256.mp3"},
    {"type": "text", "text": "What's that sound?"},
    ]},
]

conversation2 = [
    {"role": "user", "content": [
    {"type": "audio", "audio_url": "https://qianwen-res.oss-cn-beijing.aliyuncs
    .com/Qwen2-Audio/audio/1272-128104-0000.flac"},
    {"type": "text", "text": "What does the person say?"},
    ]},
]

conversation3 = [
    {"role": "user", "content": [
    {"type": "audio", "audio_url": "/content/gunshots.wav"},
    {"type": "text", "text": "What is that sound, and what does the person say?"},
    ]},
]

conversations = [conversation1, conversation2, conversation3]

text = [processor.apply_chat_template(conv, add_generation_prompt=True,
        tokenize=False) for conv in conversations]

sr = processor.feature_extractor.sampling_rate
audios = []

for conversation in conversations:
    for message in conversation:
        if isinstance(message["content"], list):
            for ele in message["content"]:
                if ele["type"] == "audio":
                    audio_path = ele['audio_url']
                    if audio_path.startswith("http"):
                        # Remote URL
                        audio, _ = librosa.load(BytesIO(urlopen(audio_path).
                                    read()), sr=sr)
                    else:
```

```
                        # Load local file
                        if not os.path.exists(audio_path):
                            raise FileNotFoundError(
                                    f"Local audio file not found: {audio_path}")
                        audio, _ = librosa.load(audio_path, sr=sr)
                    audios.append(audio)

inputs = processor(text=text, audio=audios, return_tensors="pt",
                   padding=True) ❶

device = model.device
inputs = {k: v.to(device) if isinstance(v, torch.Tensor) else v for
          k, v in inputs.items()}

generate_ids = model.generate(**inputs, max_new_tokens=512)
generate_ids = generate_ids[:, inputs["input_ids"].size(1):] ❷

responses = processor.batch_decode(generate_ids, skip_special_tokens=True,
                                   clean_up_tokenization_spaces=False) ❸

for i, r in enumerate(responses):
    print(f"Response {i+1}: {r}")
```

❶ Process batches.

❷ Generate response.

❸ Decode output.

The outputs of the model are as follows:

- Response 1: `It is the sound of glass breaking.`
- Response 2: `The original content of this audio is: "Mister Quiller is the apostle of the middle classes and we are glad to welcome his gospel."`
- Response 3: `In the audio, there is the sound of gunfire and artillery fire happening in the distance, and a male voice speaking English saying: "Can you guess where I am right now?", with a neutral mood.`

All the responses are correct, and both spoken texts (in conversation2 and conversation3) are accurately transcribed. In addition, in the third example, the model also predicted the speaker's mood correctly, meaning it performed SER. This can be helpful when analyzing distress in the speaker's situation especially given that the other sound were gunshots.

The training setup of Qwen2-Audio, illustrated in Figure 6-7, consists of an audio encoder alongside an LLM. For a given pair of audio and text sequences $(a, x)$, where

$a$ denotes the audio input and $x$ the corresponding text, the model is optimized to predict the next token in the text sequence. The training objective is to maximize the conditional probability of the next token, given both the audio representation and the preceding tokens in the text:

$$P_\theta(x_t \mid x_{<t}, \text{Encoder}_\phi(a))$$

Here, $\theta$ and $\phi$ represent the learnable parameters of the language model and audio encoder, respectively.

This unified approach enables Qwen2-Audio to comprehend intricate audio content, even in segments containing overlapping sounds, multi-speaker conversations, and voice commands, and to respond appropriately, as demonstrated in conversation3 in Example 6-5.

## Transcribing a Meeting with Kimi-Audio

Following the advancements seen in models like Qwen2-Audio, Kimi-Audio, released in April 2025, is another open source audio foundation model with capabilities in audio understanding, generation, and conversation. Kimi-Audio directly addresses limitations of prior works by emphasizing a truly universal scope, extensive multimodal pretraining, and full open source access.

Kimi-Audio uses a unified architecture with three core components:

*Audio tokenizer*
: This component transforms raw input audio into a dual representation. It produces discrete semantic audio tokens at a low 12.5 Hz frame rate, derived from a supervised speech tokenizer with vector quantization within a Whisper encoder architecture. It also produces complementary continuous acoustic vectors, derived from a pretrained Whisper model and downsampled from 50 Hz to 12.5 Hz. This hybrid tokenization strategy aims to balance semantic efficiency with rich acoustic detail for both perception and generation.

*Audio LLM*
: Serving as the central processing unit, this large language model is initialized from a pretrained text LLM (Qwen2.5 7B). It features shared transformer layers that process multimodal inputs, branching into two parallel heads: a text head for autoregressively predicting text tokens (such as transcripts or conversational responses) and an audio head for predicting discrete audio semantic tokens.

*Audio detokenizer*
: Responsible for synthesizing the final audio waveform, this component converts the discrete semantic audio tokens generated by the LLM back into coherent speech. It uses a flow-matching module to transform these tokens into mel

spectrograms and then leverages a high-fidelity BigVGAN vocoder[10] to generate the actual audio waveform. Kimi-Audio also incorporates a chunk-wise streaming detokenizer with a look-ahead mechanism to ensure low-latency speech generation in real-time conversational scenarios. The architectural overview is depicted in Figure 6-8.

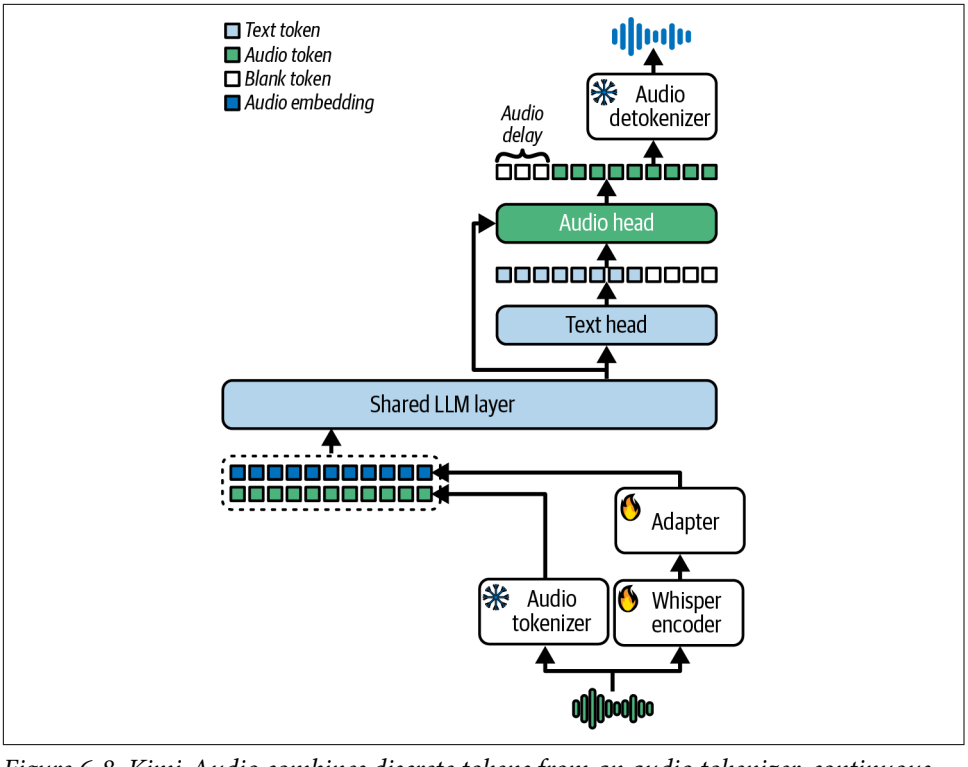


*Figure 6-8. Kimi-Audio combines discrete tokens from an audio tokenizer, continuous features from a Whisper encoder, and an audio language model to generate text or audio. An audio detokenizer reconstructs waveforms from audio tokens. Image adapted from KimiTeam et al. (2025).*

Kimi-Audio's pretraining corpus encompasses more than 13 million hours of raw audio data, including speech, diverse environmental sounds, and music. This massive dataset was curated through an efficient automatic audio processing pipeline that includes speech enhancement, speaker diarization, and sophisticated transcription using Whisper-large-v3. *Speaker diarization* is the process of dividing an audio stream with human speech into uniform segments based on speaker identity.

10 Sang-gil Lee et al. "BigVGAN: A Universal Neural Vocoder with Large-Scale Training" (2022).

The model's training process involves a multi-stage approach, initiating its Audio LLM from a pretrained text LLM and the continuous acoustic feature extractor from Whisper-large-v3. Pretraining tasks are designed as unimodal (audio-only, text-only), audio-to-text mapping (ASR, TTS), and audio-text interleaving tasks to align both modalities in a shared latent space. This is followed by extensive instruction-based supervised fine-tuning across a wide range of tasks, activating Kimi-Audio's ability to follow instructions and generate expressive speech in various conversational contexts.

This combination of architectural innovations, large-scale multimodal pretraining, and instruction fine-tuning positions Kimi-Audio as a versatile alternative to earlier models like Whisper and Qwen2-Audio. To better understand the functional differences across these models, Table 6-2 summarizes their capabilities across key dimensions of audio processing and interaction.

*Table 6-2. Model comparison: Whisper versus Qwen2-Audio versus Kimi-Audio*

| Feature | Whisper | Qwen2-Audio | Kimi-Audio |
|---|---|---|---|
| Diarization support | No | No | External only |
| ASR (basic transcription) | Yes | Yes (via prompt) | Yes (via message type) |
| Instruction-following | No | Yes (Instruct model) | Yes |
| Multi-turn audio QA | No | Partial | Yes |
| Audio-to-audio and text generation | No | No | Yes |
| Message-type distinction | No | No | Yes |
| Output audio generation | No | No | Yes |
| Multilingual and Chinese support | Limited | Limited | Yes |

With the architectural pieces in place, let's move on to build a a pipeline for speaker-aware audio transcription using a combination of pyannote-audio for diarization and Kimi-Audio-7B-Instruct for instruction-following transcription. The objective is to segment a multispeaker audio recording, identify speaker turns, and generate natural language transcriptions that identify when each speaker is speaking.

The process begins by loading a curated sample from the concatenated_librispeech dataset, which contains a dialogue constructed by concatenating a conversation from two speakers to produce a single audio file. This dataset format is used to test models on speaker segmentation. Example 6-6 shows how to load the audio file.

*Example 6-6. Loading concatenated_librispeech dataset*

```
dataset = load_dataset("sanchit-gandhi/concatenated_librispeech")
```

To segment the speakers in the conversation, you apply the `pyannote/speaker-diarization@2.1 pipeline`. This produces timestamped speaker segments for the input audio. The code for this process is shown in Example 6-7.

*Example 6-7. Speaker diarization with pyannote*

```
diarization_pipeline = Pipeline.from_pretrained("pyannote/speaker-diarization@2.1",
                                                use_auth_token=True)

sample = dataset[0]
audio_array = sample["audio"]["array"]
sr = sample["audio"]["sampling_rate"]

waveform = torch.tensor(audio_array[None, :]).float()
annotation = diarization_pipeline({"waveform": waveform, "sample_rate": sr})

segments = []
for turn, _, speaker in annotation.itertracks(yield_label=True):
    segments.append({
        "start": turn.start,
        "end": turn.end,
        "speaker": speaker
    })
```

After diarization, you slice the waveform into separate files (Example 6-8) for each speaker turn. These will later be passed to Kimi-Audio for transcription.

*Example 6-8. Slicing speaker segments into audio chunks*

```
speaker_segments = []
for seg in segments:
    start = int(seg["start"] * sr)
    end = int(seg["end"] * sr)
    audio_chunk = audio_array[start:end]
    speaker_segments.append({
        "speaker": seg["speaker"],
        "start": seg["start"],
        "end": seg["end"],
        "audio": audio_chunk
    })
```

After segmenting the waveform in memory (Example 6-8), the next step is to save each chunk as a *.wav* file for model input (Example 6-9). These audio files are paired with explicit user instructions and passed to Kimi-Audio, returning transcriptions accordingly.

*Example 6-9. Preparing data for Kimi-Audio*

```
os.makedirs("tmp_segments", exist_ok=True)

for i, seg in enumerate(speaker_segments):
    filename = f"tmp_segments/speaker_{i}.wav"
    sf.write(filename, seg["audio"], sr)
    speaker_segments[i]["file_path"] = filename
```

With the speaker segments saved, you can now transcribe each segment using Kimi-Audio by providing explicit prompts. Example 6-10 shows how to load Kimi-Audio and configure its generation parameters.

*Example 6-10. Loading Kimi-Audio and setting generation parameters*

```
model_path = "moonshotai/Kimi-Audio-7B-Instruct"
model = KimiAudio(model_path=model_path, load_detokenizer=True)

sampling_params = {
    "audio_temperature": 0.8,
    "audio_top_k": 10,
    "text_temperature": 0.0,
    "text_top_k": 5,
    "audio_repetition_penalty": 1.0,
    "audio_repetition_window_size": 64,
    "text_repetition_penalty": 1.0,
    "text_repetition_window_size": 16,
}
```

You can add an optional prompt to control the output, as illustrated in Example 6-11.

*Example 6-11. Add optional prompt to control output*

```
messages = []
for seg in speaker_segments:
    # Add optional prompt to control output
    messages.append({
        "role": "user",
        "message_type": "text",
        "content": f"What is {seg['speaker']}
        saying between {seg['start']:.1f}s and {seg['end']:.1f}s?"
    })
    messages.append({
        "role": "user",
        "message_type": "audio",
        "content": seg["file_path"]
    })

_, text_output = model.generate(messages, **sampling_params, output_type="text")
```

```
print(">>> Transcribed Multi-speaker Output:\n")
print(text_output)
```

Each audio segment is paired with an instruction prompt and passed to the model for transcription, as shown in Example 6-12. The result is a structured, speaker-attributed transcript that maintains temporal alignment with the original audio.

*Example 6-12. Transcribing each speaker segment with Kimi-Audio*

```
transcriptions = []

for seg in speaker_segments:
    msgs = [
        {"role": "user", "message_type":
        "text", "content": f"Transcribe what {seg['speaker']} says:"},
        {"role": "user", "message_type": "audio", "content": seg["file_path"]}
    ]
    _, output = model.generate(msgs, **sampling_params, output_type="text")
    transcriptions.append({
        "speaker": seg["speaker"],
        "start": seg["start"],
        "end": seg["end"],
        "text": output
    })
```

With the function from Example 6-13, you can print the speaker-tagged segments with aligned timestamps and corresponding transcriptions.

*Example 6-13. Simple text generation*

```
for t in transcriptions:
    print(f"{t['speaker']} ({t['start']:.1f}s - {t['end']:.1f}s): {t['text']}\n")
```

The output of the model is as follows:

```
SPEAKER_01 (0.0s - 14.5s): The second in importance is as follows sovereignty
may be defined to be the right of making laws in france the king really exercises
a portion of the sovereign power since the laws have no weight.

SPEAKER_00 (15.4s - 21.3s): He was in a fevered state of mind, owing to
 the blight his wife's action threatened to cast upon his entire future.
```

which is fully accurate, in terms of time and spoken text.

This kind of output highlights how instruction-tuned audio foundation models can not only transcribe speech but do so with granular control over speaker identity and temporal context, bridging classic diarization and modern generative modeling. You could even use Kimi-Audio to ask targeted questions such as, "What is speaker A saying between 4.2s and 11.7s?" enabling a more interactive and modular transcription

process. The output is a speaker-attributed, time-aligned transcript that reflects both the content and conversational structure of the input audio.

### Speech-Native Agents with Tool-Use Capabilities

If you're exploring voice-based agents that go beyond simple transcription, consider adopting a cascaded architecture that combines ASR, LLM reasoning, and TTS with integrated tool use. AURA is a strong open source candidate for such use cases: it supports multi-turn, speech-to-speech interaction with real-world APIs like email, calendar, and web search.

AURA employs ReAct-style reasoning and is modular enough to extend with your own tools using natural language prompts. For benchmarking, it performs competitively with GPT-4o on VoiceBench tasks. The agent is open source with a Gradio implementation available via its GitHub repo. This is a great start for anyone working on multimodal or task-oriented assistants.

## Segment Anything in Audio

You learned about diffusion transformers in Chapter 4, but in that chapter the focus was on image generation. This section gives you a different perspective. The components you saw in Chapter 4 are now repurposed for audio segmentation. This is one of the reasons I decided to write a book about transformers across domains. Once you understand the core components, you don't have to start from the ground up whenever a new model appears. You can recognize familiar building blocks and focus on how they're combined.

SAM Audio[11] is a good example. The model is built on a diffusion transformer architecture, the same model you encountered in "Scalable Diffusion Models with Transformers" on page 102. It also uses the similar VAE setup. A particularly interesting choice is that the authors of SAM Audio formulate the task as a *generative modeling problem*, instead of a *discriminative learning problem*, where a model predicts a mask over the audio spectrogram, which was usually the approach for audio segmentation.

The diffusion transformer is not the only familiar component. SAM 2 from "Segment Anything in Images and Videos" on page 83 also appears in SAM Audio. It provides the visual prompt for videos, since video is a common source of multimodal audio data, and a visual region is a natural way to isolate a sound source. To extract a sound from a video, you can indicate the region of interest with clicks or bounding boxes, as you saw in the SAM 2 section. Figure 6-9 illustrates the overall architecture.

11 Bowen Shi et al. "SAM Audio: Segment Anything in Audio" (2025).

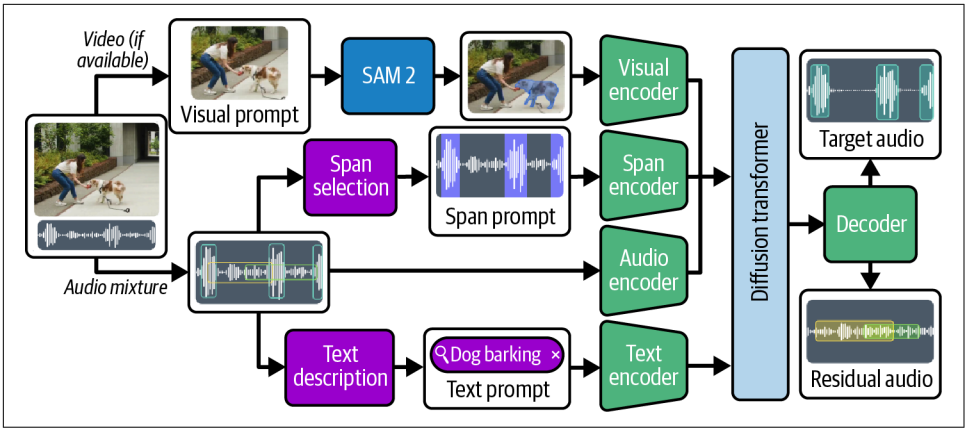


*Figure 6-9. Overview of SAM Audio. Image adapted from Bowen Shi et al. (2025).*

The following examples (Examples 6-14 through 6-16) show how to use SAM Audio to separate different sounds from the same audio clip using simple text descriptions. First, you load the model and its processor (Example 6-14). The processor prepares the inputs for the model and takes care of audio resampling and batching.

*Example 6-14. Initialize SAM Audio*

```
model = SAMAudio.from_pretrained("facebook/sam-audio-large").to(device).eval()
processor = SAMAudioProcessor.from_pretrained("facebook/sam-audio-large")
```

Example 6-15 initiates the separation and returns both the isolated sources and the residual audio for each (audio, description) pair.

*Example 6-15. Text-based source separation*

```
audio_file = "/content/gunshots.wav"
descriptions = ["gunshots", "male voice"] ❶
audios = [audio_file] * len(descriptions) ❷

batch = processor( ❸
    audios=audios,
    descriptions=descriptions,
).to("cuda")

with torch.inference_mode():
    result = model.separate(batch, predict_spans=True,
                            reranking_candidates=8)
```

❶ Describe the sounds you want to isolate.

❷ Use the same audio for each description.

❸ Process and separate.

Example 6-16 saves these tracks to separate files.

*Example 6-16. Saving separated audio tracks*

```
sample_rate = processor.audio_sampling_rate ❶

gunshots_target = ensure_2d(result.target[0]) ❷
male_voice_target = ensure_2d(result.target[1])

gunshots_residual = ensure_2d(result.residual[0])
male_voice_residual = ensure_2d(result.residual[1])

torchaudio.save("gunshots_target.wav", gunshots_target.cpu(), sample_rate) ❸
torchaudio.save("male_voice_target.wav", male_voice_target.cpu(), sample_rate)

torchaudio.save("gunshots_residual.wav", gunshots_residual.cpu(), sample_rate) ❹
torchaudio.save("male_voice_residual.wav", male_voice_residual.cpu(), sample_rate)
```

❶ Save separated audio.

❷ `result.target` and `result.residual` are lists of tensors, one per (audio, description) pair.

❸ The isolated sound.

❹ Everything else.

With only a few lines of code, SAM Audio segments a complex mixture of sounds into distinct, usable streams. It's important to note the relationship between the two outputs: the target is the specific sound you described, while the residual is everything else—that is, the "hole" left behind in the original audio.

The model doesn't just isolate the gunshots; it creates a version of the audio where the gunshots are removed, leaving the male voice intact in the residual track. This "target plus residual" approach is what makes SAM Audio more than just a filter. It's also an audio editor. Whether you're extracting a single instrument for a remix or cleaning a noisy interview by isolating the background hum, the model ensures that the different tracks, when combined, perfectly reconstruct the original signal.

## Beyond Text and Speech: Transformers as Music Composers

Generating music is a uniquely challenging task, demanding far more than simple sequence prediction. Unlike speech, music utilizes the full frequency spectrum, often requiring higher sampling rates (e.g., 44.1 kHz or 48 kHz). It weaves together intricate harmonies, melodies, and rhythms from diverse instruments, creating complex structures where human listeners are acutely sensitive to errors and disharmony. Furthermore, music creators demand precise control over genre, instrumentation, key, and melodic progression, adding layers of complexity to the generation process.

The MusicGen model stands as a pioneering example in the realm of conditional music generation. It introduces a paradigm-shifting approach that leverages a single-stage transformer language model (LM) to generate high-quality musical pieces. This single-stage design operates over several streams of compressed discrete music representation. It eliminates the need for cascading multiple models, a common strategy in prior music generation systems that relied on hierarchical or upsampling stages. This simplification leads to a more efficient and streamlined generation process.

The foundation of MusicGen lies in its audio tokenization. It employs EnCodec,[12] a convolutional autoencoder using *residual vector quantization* (RVQ) to convert raw audio into multiple parallel streams of discrete tokens. RVQ is a technique that encodes data by sequentially quantizing the residuals between the original input and the output of previous quantizers, allowing for more compact and accurate representations. Each timestep in the audio is represented by several quantized values from different learned codebooks, with a frame rate of 50 Hz. These discrete tokens form the sequential input that the transformer LM learns to model.

The core of MusicGen is an autoregressive transformer-based decoder that models these streams of acoustic tokens. Instead of simply flattening all token streams into a single, long sequence, MusicGen intelligently interleaves them (e.g., using "delay" or "parallel" patterns). This strategy allows the model to capture dependencies across parallel streams efficiently, leading to coherent music generation while significantly reducing the effective number of autoregressive steps compared to dense flattening.

MusicGen supports conditional generation. It can compose music based on textual descriptions (e.g., "90s rock song with a guitar riff"), using text encoders like T5.[13] To generate your own music from text, you need to first load the model and its processor, as shown in Example 6-17.

12 Alexandre Défossez et al. "High Fidelity Neural Audio Compression" (2022).

13 Colin Raffel et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" (2019).

*Example 6-17. Load model and processor*

```
model = MusicgenForConditionalGeneration.from_pretrained("facebook/musicgen-large",
        to_device="auto")
processor = AutoProcessor.from_pretrained("facebook/musicgen-large")
```

After that, you can simply define the prompts, as in Example 6-18.

*Example 6-18. Define genres and prompts*

```
genre_prompts = {
    "classical": "classical piano music",
    "minimal": "minimal house rather monoton in rhythm",
    "EDM": "EDM hardcore Techno sound",
}
```

Now you can easily generate and save each audio file, as implemented in Example 6-19.

*Example 6-19. Generate and save each audio file*

```
for genre, prompt in genre_prompts.items():
    inputs = processor(text=[prompt], padding=True,
                       return_tensors="pt").to(device) ❶

    audio_values = model.generate( ❷
        **inputs,
        do_sample=True,
        guidance_scale=3,
        max_new_tokens=1024
    )
    ❸
    audio_np = audio_values[0].cpu().numpy().squeeze()
    audio_np = np.clip(audio_np, -1.0, 1.0)
    audio_int16 = (audio_np * 32767).astype(np.int16)

    filename = f"{genre}.wav"
    write_wav(filename, sampling_rate, audio_int16) ❹

    display(Audio(audio_np, rate=sampling_rate)) ❺
```

❶ Prepare input.

❷ Generate audio.

❸ Convert to NumPy and clip.

❹ Save as WAV.

❺ Playback: this will display a play button, so you can play the audio directly.

It also supports unsupervised melody conditioning by leveraging chromagrams. *Chromagrams* are visual or numerical representations of the intensity of each of the 12 pitch classes (such as C, C#, D, etc.) over time, capturing the harmonic and tonal content of an audio signal regardless of octave. This feature allows you to provide a melodic structure (like a hummed tune or another audio track), and the model will generate music that adheres to that melodic input while still being guided by textual prompts. To use this functionality, you can start with any *.wav* or *.mp3* as the basis. To make the demonstration a bit easier, I'll use audio files from the Facebook Research library audiocraft. I start by cloning the repository and then display the contents of the folder to select one of the files, as shown in Example 6-20.

*Example 6-20. Clone repo and display audio files*

```
!git clone https://github.com/facebookresearch/audiocraft.git

asset_folder = "audiocraft/assets"
print(f"Listing contents of {asset_folder}:\n")
for file_name in os.listdir(asset_folder):
    print(file_name)
```

The provided notebook for this task also includes code that allows you to play all audio files.

The loading of the model and processor is the same as in Example 6-17. The audio file is sampled in another frequency, so you need to resample the audio after loading it. Example 6-21 implements this functionality.

*Example 6-21. Load and resample audio file*

```
file_path = "audiocraft/assets/CJ_Beatbox_Loop_05_90.wav"
waveform, orig_sr = torchaudio.load(file_path) ❶

if waveform.shape[0] > 1:
    waveform = waveform.mean(dim=0, keepdim=True) ❷

target_sr = 32000 ❸
if orig_sr != target_sr:
    resampler = Resample(orig_freq=orig_sr, new_freq=target_sr)
    waveform = resampler(waveform)

waveform_np = waveform.squeeze().numpy()
```

❶ Load audio file.

❷ Convert to mono.

❸ Resample to 32 kHz.

To prepare the inputs for the processor, follow the code in Example 6-22.

*Example 6-22. Prepare input*

```
inputs = processor(
    audio=waveform_np,
    sampling_rate=target_sr,
    text=["Hip hop with nuances of soul and funk"],
    padding=True,
    return_tensors="pt",
)
```

Now to generate and save your audio from the sample and extend it with additional seconds of generated music based on your textual input, follow the steps implemented in Example 6-23.

*Example 6-23. Generate music and save output*

```
audio_values = model.generate( ❶
    **inputs,
    do_sample=True,
    guidance_scale=3.0,
    max_new_tokens=512
)

output = audio_values[0].numpy()
output = output / np.max(np.abs(output) + 1e-8) ❷
output_int16 = (output * 32767).astype(np.int16) ❸

if output_int16.ndim > 1: ❹
    output_int16 = output_int16.squeeze()

write("hiphop_generated.wav", int(target_sr), output_int16) ❺

display(Audio("hiphop_generated.wav")) ❻
```

❶ Generate music.

❷ Normalize to [–1, 1].

❸ Convert to 16-bit pulse-code modulation (PCM).

❹ Make sure it's one-dimensional.

❺ Write with proper casting of sample rate.

❻ Display playback button.

Now that you've seen how MusicGen works, it's your turn to experiment. With just a prompt or a short audio sample, you can generate full musical pieces in your own style, be it classical, EDM, or something entirely new. The beauty of MusicGen lies not just in its technical elegance but in how easily it puts creative control into your hands. Try blending genres, guiding the model with your voice or a beatbox loop, or simply letting the model surprise you. Whether you're a musician, researcher, or just curious, this is a powerful playground for musical exploration. Let your ideas become sound.

# Conclusion

In this chapter, you extended your understanding of transformers into the audio domain, where time, frequency, and perception intersect. Building on earlier domains like text and time series, you saw how the same architectural foundation continues to adapt across modalities. From waveforms to spectrograms, from discrete tokens to learned representations, the transformer once again proves itself as a flexible and powerful modeling framework.

You explored how audio combines temporal progression with spectral richness. Unlike pure time series, sound reveals patterns in both dimensions, requiring careful preprocessing and domain-aware representation. Spectrograms, mel scales, and quantized audio tokens form the basis for how modern models perceive and generate sound. Models like Whisper, Qwen2-Audio, Kimi-Audio, and SAM Audio highlight this evolution. Each builds on the transformer backbone while solving different tasks: transcription, classification, segmentation, emotion recognition, and even audio generation, within a unified architecture.

These models reflect a larger shift. Rather than building separate systems for each task, audio foundation models now operate across recognition, generation, and understanding. Whether responding to user prompts, transcribing overlapping speech, or composing music from text, transformers enable scalable and generalizable solutions that match human expectations.

Throughout this chapter, you also learned how latent spaces, attention mechanisms, and neural vocoders come together to reconstruct and generate audio with high fidelity. From autoregressive decoding to instruction-tuned models, the transformer continues to push the limits of what machines can hear, understand, and create.

This adaptability will be critical as you move on to Chapter 7. Reinforcement learning introduces a new type of structure, not in data, but in decisions. Instead of modeling what is observed, the focus shifts to learning what to do. The transformer must now reason over states, actions, and long-term outcomes, often in uncertain and interactive environments.

Just as audio unfolds through time and frequency, behavior unfolds through trial and reward. Chapter 7 will show how transformers are increasingly being used to model agents that learn to act, explore, and improve over time.

CHAPTER 7

# Reinforcement Learning Transformers

*Reinforcement learning* (RL) is a paradigm based on making sequential decisions through feedback from an environment. If you think about it, learning through interaction is not a new concept. In fact, you've been familiar with the basic principles of RL all your life. This is because you often learn by interacting with your surroundings without needing an explicit teacher. For example, as a child, you might have touched things or put objects in your mouth to see how your environment reacted. Based on the outcome, such as crying after being scratched by the family cat, you learned what to do next. From such experiences, you developed a "policy" for future actions, like not pulling the cat's tail again.

This learning process mirrors how an RL agent operates. Consequently, this chapter will leverage your natural understanding of learning from an environment and build on it by discussing key RL concepts. Specifically, I'll provide an introductory overview of traditional RL algorithms, including distinctions between online and offline learning, model-based and model-free approaches, and on-policy versus off-policy RL.

Once you have a basic understanding of these foundational concepts, you'll explore recent architectures in RL transformers, such as decision transformer, online decision transformer, and efficient stochastic transformer-based world models.

## Getting Started with Reinforcement Learning

Typically, the interaction with the environment is framed within *Markov decision processes* (MDPs) MDPs are mathematical frameworks used to model decision making in environments where outcomes are partly random and partly under the control of a decision maker, characterized by states, actions, transition probabilities, and rewards. Here's how it works: an agent chooses an action $a \in A$ based on the current state

$s \in S$ of its environment. The agent then receives feedback in the form of rewards $r \in R$. Following this feedback, the agent updates its situation to a new state ($s' \in S$). Figure 7-1 illustrates this interaction.

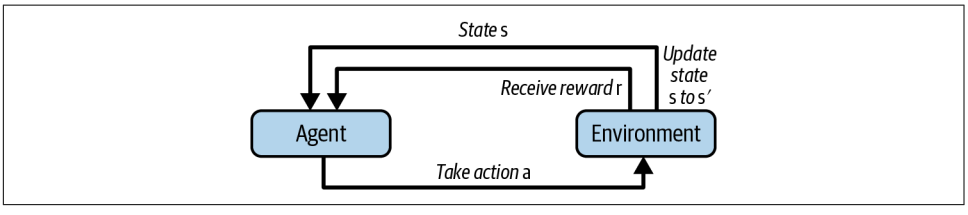


*Figure 7-1. Flowchart showing how the agent interacts with its environment.*

Mathematically, this can be expressed as follows:

$$R^{a}_{ss'} = E\{r_{t+1} \mid s_t = s, a_t = a, s_{t+1} = s'\}$$

**Markovian Processes**

A process is Markovian if the future state depends only on the current state and action, not on the sequence of events that preceded it. This property is known as the Markov property.

The policy $\pi$ maps each state $s \in S$ and action $a \in A$ to the probability $\pi(s, a)$ of taking an action $a$ while the agent is in a certain state $s$. A *policy* is a rule that guides an agent by specifying the probability of choosing a particular action while in a given state. Essentially, it's the strategy that the agent uses to determine its actions based on its current situation.

Most RL algorithms operate on the principle of *value functions*. These are functions that evaluate the potential benefits of being in a certain state or performing a certain action within that state for the agent. By using value functions, the agent quantifies the anticipated future rewards or, more accurately, the expected return. Of course, future rewards that an agent might receive are based on the actions it chooses to take. Consequently, the calculation of value functions is always tied to specific policies that the agent follows.

More technically, we have two value functions: $V^{\pi}(s)$ (the value of a state under a policy) and $Q^{\pi}(s, a)$ (the value of taking action in a state under a policy). The first is called *value-function* and the second *action-value function*. These interactions between policy, state, and action are fundamental to the agent's learning process.

#### The Value Function Is the Unique Solution to Its Bellman Equation

Value functions have a key feature: they follow specific recursive relationships. The *Bellman equation* helps illustrate this by defining the value function of a policy in terms of expected future rewards. Essentially, it links the value of a current state to the values of possible future states, almost like peeking ahead from one state to the next. The Bellman equation takes all potential outcomes into account, weighting each according to how likely it is to happen, and thereby simplifies your state value or state-action value calculation.

The goal of reinforcement learning is to identify the optimal policy $\pi^{\star}$ that maximizes the expected total reward over time $T$ for each state $s$ in the sequence of timesteps $t \in T$. Basically, within RL, you want to tackle two different learning objectives: learning to represent a state and learning how to take an action. Learning to represent a state involves understanding and encoding the environment's current situation in a way that's useful for decision making. Learning how to take an action refers to the process of deciding which action to take based on the current state representation, typically in a way that's expected to maximize some notion of cumulative reward. In the next section, you'll learn how you can achieve this.

# Foundational Concepts in Reinforcement Learning

In this section, you'll explore different methods for teaching your RL algorithm how to encode the environment and, based on that, determining which actions it should take.

## Online and Offline Reinforcement Learning

Reinforcement learning involves repeatedly interacting with a specific environment to gather experiences using the most recently updated policy and then use those experiences directly to enhance the policy. This approach is considered an *online learning paradigm*, and for some applications, such as healthcare or autonomous driving, it's impractical.

In *offline*, or *batch*, *reinforcement learning*, the agent relies on a *static* dataset without interacting with the environment during training. This means the dataset doesn't change while the agent is learning. Once the policy has been fully trained using this dataset, it's then deployed. The drawback here is that the agent can't adapt to new information. The difference between the two approaches is illustrated in Figure 7-2.

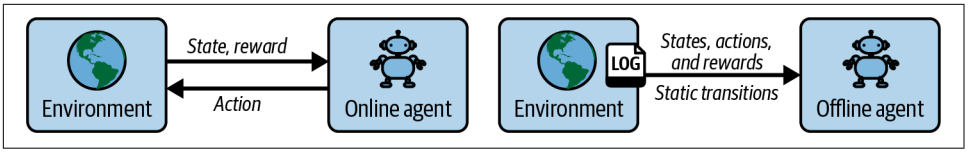


*Figure 7-2. Online (right) versus offline (left) reinforcement learning.*

## Model-Based and Model-Free Approaches

In *model-based RL*, the agent learns a *transition function* from transitions $(s, a, r, s')$, which are derived through interactions with the environment. This function models the probability distribution $p(s', r \mid s, a)$ of the next state $s'$ and reward $r$, given the current state $s$ and action $a$. The agent plans and selects actions aimed at maximizing the expected return. Even though this method can be more computationally demanding, it can achieve good performance with fewer environmental interactions than other methods.

In *model-free RL*, the agent learns optimal actions directly, through interactions with the environment, without modeling the state-transition dynamics. This can lead to slower learning; however, model-free RL excels in adapting to changes in the environment, enhancing its robustness in complex or noisy settings. It's also less computationally demanding because it doesn't require the agent to learn an environmental model. An *environmental model* represents the various dynamic states of an environment and the pathways through which these states result in rewards.

## On-Policy Versus Off-Policy Reinforcement Learning

There are two strategies to ensure that the agent selects all actions infinitely often and persistently. These two strategies are known as *on-policy* and *off-policy* methods. *On-policy* focuses on evaluating or enhancing the policy that the agent is currently using to make decisions. Essentially, the policy that the agent follows during learning is the same policy it's attempting to improve. In contrast, *off-policy* works on evaluating or improving a policy that's different from the one used to collect the data. The agent learns the value function separately from the policy it follows, allowing it to use data generated by a different policy. This flexibility means the agent can learn from previous experiences or those of other agents. Additionally, the agent can discover the optimal policy while engaging in exploratory or random actions.

For example, in the SARSA algorithm, which is an on-policy method, transitions are collected as a quintuple ($s$, $a$, $r$, $s'$, $a'$), which includes the current state-action pair ($s$, $a$), the immediate reward $r$, and the subsequent state-action pair ($s'$, $a'$). This quintuple is the reason the algorithm is named SARSA. These values are employed to estimate the returns for both the current and next state-action pairs.

Despite being relatively straightforward to implement, on-policy methods have several limitations. They're often *sample inefficient*, meaning they need extensive environmental interaction to accomplish a good performance. Moreover, they're prone to issues such as *policy oscillation* (also known as *policy chattering*),[1] where the policy frequently changes direction without converging to an optimal strategy, and *alternating* policies, where the policy switches back and forth between different strategies. Instability refers to the policy's failure to consistently improve or stabilize. These issues can hinder exploration and slow the learning process, often leading to suboptimal policies. This poses significant challenges in dynamic environments like financial markets or complex simulations.

Off-policy learning uses a target policy that's evaluated based on experiences gathered by following a different behavior policy, rather than adhering directly to the target policy itself. *Q-learning*[2] is an off-policy method and an important breakthrough in RL. Here, the learned action-value function, $Q$, directly approximates the optimal action-value function, independent of the policy being followed. However, the policy still has an effect in that it determines which state-action pairs are visited and updated.

Therefore, off-policy learning involves two policies: the *behavior policy*, which is employed for data collection and allows exploration of various states and actions, and the *target policy*, typically a deterministic greedy policy optimized based on current action-value function estimates. This setup allows off-policy methods to assess the value of specific actions and states without the target policy being directly followed during data collection.

The primary benefit of off-policy methods is their capacity to learn from a diverse set of experiences, potentially speeding up the learning process. Yet, they can be more complex to implement than on-policy learning methods. In addition, the agent might encounter difficulties due to how experience data is distributed, like a tendency to overfit to particular experiences.

1 Paul Wagner. "A Reinterpretation of the Policy Oscillation Phenomenon in Approximate Policy Iteration". In NIPS'11: Proceedings of the 25th International Conference on Neural Information Processing Systems. Curran Associates (2011).

2 Christopher J. C. H. Watkins et al. "Q-Learning" (1989).

## Temporal Difference Learning

*Temporal difference* (TD) learning is an important method in RL that enables an agent to learn directly from raw experience without needing a model of the environment's dynamics. TD learning integrates principles from the Monte Carlo and dynamic programming (DP) methods. Unlike Monte Carlo methods, which require waiting until the end of an episode to update value estimates, TD learning allows for incremental updates at each step. Similarly, while DP methods require a complete model of the environment, TD learning *bootstraps*: it updates estimates based on other learned estimates without needing full environmental knowledge.

TD learning is particularly useful because it allows for continuous updates and learning in complex environments. It serves as the foundation for many practical RL algorithms, such as SARSA and Q-learning, which implement TD updates to refine their strategies over time.

## World Models in Reinforcement Learning

As humans, we create an internal mental model of the world from the sensory information we gather. Our decisions and actions are guided by this internal representation. However, in our daily lives, we're overwhelmed with information, so we abstract key aspects of the gathered information. This is similar to how an RL algorithm learns its world model.[3] That is, with RL, it's not only about having perceptual knowledge but about using that perceptual knowledge to make decisions.

A *world model* in RL is an internal representation of the environment that the agent learns. It's used to predict future states and rewards based on the current state and actions taken. This concept is especially relevant in model-based RL, where the agent uses the world model to plan and make decisions without the need to constantly interact with the real environment. World models are beneficial because they allow the agent to simulate various scenarios internally and evaluate potential outcomes before taking any real actions. This simulation can lead to more efficient learning since the agent can explore a wide range of possibilities without the cost or risk associated with real-world interactions. The agent can generate hypothetical scenarios and optimize its policy based on these simulations. World models have been successfully applied in various applications, including robotics and autonomous driving, where they assist in planning and decision-making processes.

3 David Ha et al. "World Models" (2018).

**Deeper Understanding of Classical Reinforcement Learning**

If you want to get a deeper understanding of reinforcement learning, the book *Reinforcement Learning: An Introduction* by Richard S. Sutton and Andrew G. Barto (MIT Press) is a highly regarded resource. It provides a thorough foundation in the theory and practice of reinforcement learning.

## Transformers in Reinforcement Learning

Transformers, which are widely successful in fields like NLP, have been adopted more slowly in RL due to specific challenges. Initially, most transformer-based RL applications focused on state representation learning and memory enhancement. These approaches continued to face issues associated with traditional RL frameworks, such as temporal difference learning and policy optimization. Another significant challenge is the *non-stationarity* of the training process, which refers to the changing dynamics of the environment over time, making it difficult to maintain consistent learning and optimization. Nonetheless, one of the key areas where transformers are beneficial is in modeling temporal dependencies. In many RL scenarios, an agent must consider a sequence of past observations or actions to make informed decisions in the present. Thanks to their self-attention mechanism, transformers excel at modeling these long-range dependencies in the data.

Transformers can also be beneficial in multi-agent settings within reinforcement learning, where understanding the actions and impacts of multiple agents on the environment is crucial. They can process and contextualize the actions and states of all agents, providing a comprehensive representation of the environment. This capability extends to *hierarchical reinforcement learning*, where transformers help agents decompose policies into high-level strategies and specific actions, effectively learning at different levels of abstraction.

In *model-based reinforcement learning*, transformers assist agents in learning models of their environments to enhance planning. They take sequences of past states and actions as input to predict future states. Moreover, transformers are instrumental in learning from demonstrations, such as in imitation learning or inverse reinforcement learning, where they map sequences of states and actions onto expert policies.

Transformers' ability to manage diverse data types also makes them suitable as generalist agents in AI systems, performing a wide range of tasks with minimal task-specific adjustments. Despite their promise, transformers face challenges in real-time RL scenarios, primarily due to their extensive parameter count and substantial computational demands, which are less than ideal for real-time applications.

However, a significant shift occurs when transformers leverage offline RL methods. By training transformers on previously collected, static data, these methods have enabled state-of-the-art achievements. The decision transformer, to which we now turn, was one of the pioneers in this approach, modeling RL as an *autoregressive* sequence generation task, where each element in the sequence is predicted based on the preceding elements, to create desired trajectories.

## Decision Transformer

The *decision transformer* (DT)[4] reinterprets RL as a conditional sequence modeling task. It's trained using an autoregressive model that integrates desired rewards with past states and actions, stepping away from traditional RL methods like temporal difference learning. This model employs a fixed, limited dataset consisting of trajectory rollouts from various policies. As you learned in the previous section, this approach aligns with offline and model-free RL principles. That is, this strategic shift uses preexisting data to bypass common challenges in traditional RL, allowing for more stable and effective learning environments. One such challenge is *bootstrapping errors*, which arise when the agent updates policy estimates based on other estimated values rather than on complete, actual outcomes. TD estimates (bootstraps) the values rather than relying on Monte Carlo methods.

The DT uses an *autoregressive (left-to-right)* approach as in the GPT architecture, building on the principles of probabilistic inference[5] and sequence modeling, meaning each state leads to an action, and each action leads to a subsequent state and observation. In addition, the DT processes trajectories in a sequential manner, predicting the next element in the sequence given the past elements. This autoregressive modeling approach results in a non-*Markovian* process. This allows the model to leverage richer information for decision making, going beyond the limitations of Markovian processes where only the current state is considered. The trajectory representation, $\tau$, based on the input sequence of states $s$, actions $a$, returns-to-go $\widehat{R}_t = \sum_{t'=t}^{T} r_{t'}$, is represented as follows:

$$\tau = \left(\widehat{R}_1, s_1, a_1, \widehat{R}_2, s_2, a_2, \ldots, \widehat{R}_T, s_T, a_T\right)$$

4 Lili Chen et al. "Decision Transformer: Reinforcement Learning via Sequence Modeling" (2021).

5 Sergey Levine. "Reinforcement Learning and Control as Probabilistic Inference: Tutorial and Review" (2018).

*Returns-to-go* are the future desired returns, rather than the past rewards. The output of the model is a sequence of predicted actions:

$$\hat{a}_1, \hat{a}_2, \cdots, \hat{a}_T$$

This exact autoregressive sequence-modeling approach allows the model to use richer historical information for decision making.

Since returns, states, and actions are processed through modality-specific linear embeddings, and a positional timestep encoding is applied, you now have three distinct types of tokens. Contrast this to using an LLM, which has only text tokens. Example 7-1 shows how to generate the embeddings.

*Example 7-1. Decision transformer embeddings*

```
embed_timestep = nn.Embedding(max_ep_len, hidden_size) ❶
embed_return = torch.nn.Linear(1, hidden_size)
embed_state = torch.nn.Linear(self.state_dim, hidden_size)
embed_action = torch.nn.Linear(self.act_dim, hidden_size)

state_embeddings = embed_state(states) ❷
action_embeddings = embed_action(actions)
returns_embeddings = embed_return(returns_to_go)
time_embeddings = embed_timestep(timesteps)

state_embeddings = state_embeddings + time_embeddings ❸
action_embeddings = action_embeddings + time_embeddings
returns_embeddings = returns_embeddings + time_embeddings
```

❶ Embeddings are per token.

❷ Each modality is embedded with a different head.

❸ Time embeddings are treated similarly to positional embeddings.

Figure 7-3 illustrates the overall architecture of the Decision Transformer.

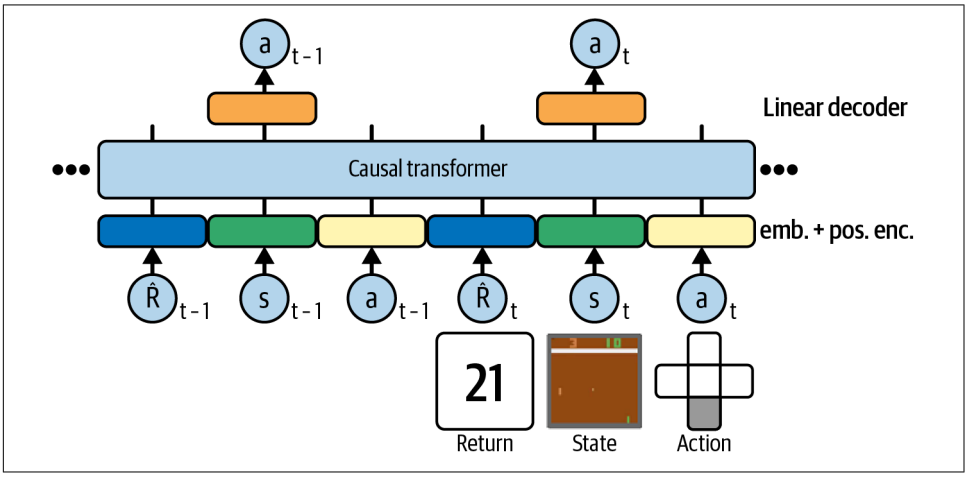


*Figure 7-3. Returns, states, and actions are processed through modality-specific linear embeddings, with an additional positional timestep encoding applied. These tokens are then input into a GPT architecture, which predicts actions in an autoregressive manner using masked self-attention. Image adapted from Lili Chen et al. (2021).*

#### Positional Embeddings in the Decision Transformer

You might wonder why the positional embeddings are added and not concatenated to the embeddings for the tokens. Adding them integrates positional information directly into the original embeddings. The resulting vector has the same dimensionality as the input embeddings, preserving the model architecture. This method embeds the positional information in a way that the subsequent layers of the neural network can naturally process. In this book's GitHub repository, the notebook *ch_03_add_vs_cat_embeddings.ipynb* illustrates this further in code.

To try the DT out, you'll use two famous RL environments in *Gymnasium*: HalfCheetah and Walker2D. Gymnasium is a project that offers an API for various single-agent reinforcement learning environments. The HalfCheetah (Figure 7-4) is a 2D robot with nine body parts and eight joints, including two paws. The goal is to apply torque to the joints to make the cheetah run forward as fast as possible, earning positive rewards for forward movement and negative rewards for backward movement. Torque can be applied to the six joints in the thighs, shins, and feet, while the torso and head remain fixed.

Walker2D builds on the Hopper environment by adding another set of legs, enabling the robot to walk forward instead of hopping, adding more complexity to the task.

The Hopper is a two-dimensional, one-legged figure with four main body parts: a torso, a thigh, a leg, and a single foot. The goal in the Hopper environment is to make forward hops by applying torque to the three hinges connecting these parts. In contrast, the Walker is a two-dimensional, two-legged figure with seven main body parts: a single torso, two thighs, two legs, and two feet. The goal is to make it walk forward by applying torque to the six hinges connecting these body parts. Figure 7-4 shows the different environments.

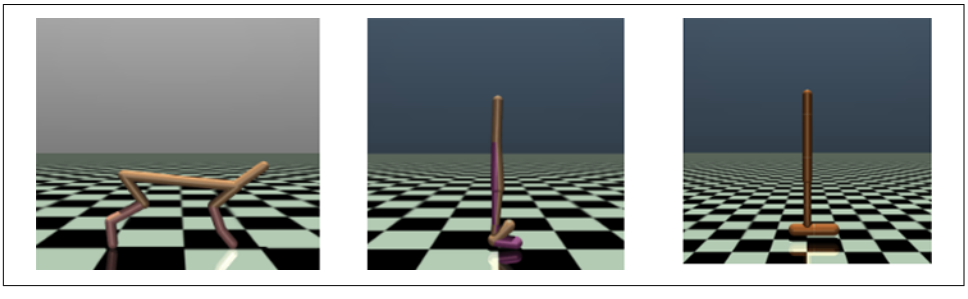

*Figure 7-4. Left is the HalfCheetah, in the middle the Walker2D, and on the right the Hopper environment.*

Next, to use the environments with the decision transformer, you need to compute the actions. This can be done as follows in Example 7-2.

*Example 7-2. Compute actions*

```
def get_action(model, states, actions, rewards, returns_to_go, timesteps):
    max_length = model.config.max_length
    state_dim = model.config.state_dim
    action_dim = model.config.act_dim

    states = states.reshape(1, -1, state_dim)[:, -max_length:]
    actions = actions.reshape(1, -1, action_dim)[:, -max_length:]
    returns_to_go = returns_to_go.reshape(1, -1, 1)[:, -max_length:]
    timesteps = timesteps.reshape(1, -1)[:, -max_length:]
    padding = max_length - states.shape[1]

    attention_mask = torch.cat([torch.zeros(padding),
                    torch.ones(states.shape[1])],
                    dim=0).to(dtype=torch.long).reshape(1, -1) ❶
    pad_tensor = lambda x, dim: torch.cat([torch.zeros((1,
                 padding, dim)), x], dim=1).float()

    states = pad_tensor(states, state_dim)
    actions = pad_tensor(actions, action_dim)
    returns_to_go = pad_tensor(returns_to_go, 1)
    timesteps = torch.cat([torch.zeros((1, padding),
                           dtype=torch.long), timesteps], dim=1)
```

```
    state_preds, action_preds, return_preds = model(
        states=states,
        actions=actions,
        rewards=rewards,
        returns_to_go=returns_to_go,
        timesteps=timesteps,
        attention_mask=attention_mask,
        return_dict=False,
    )

    return action_preds[0, -1]
```

❶ Pad all tokens to sequence length.

Now you need a function to interact with the chosen environment, which you'll add in Example 7-3.

*Example 7-3. Interact with the environment*

```
def interact_with_environment(env, model, state_mean, state_std, max_ep_len,
                              target_return_value, state_dim, act_dim, scale, device):
    episode_return, episode_length = 0, 0
    state = env.reset()
    target_return = torch.tensor([target_return_value], device=device,
                    dtype=torch.float32).reshape(1, 1)
    states = torch.from_numpy(state).reshape(1, state_dim).to(
                device=device, dtype=torch.float32)
    actions = torch.zeros((0, act_dim), device=device, dtype=torch.float32)
    rewards = torch.zeros(0, device=device, dtype=torch.float32)
    timesteps = torch.tensor([[0]], device=device, dtype=torch.long)

    for t in range(max_ep_len):
        actions = torch.cat([actions,
                    torch.zeros((1, act_dim), device=device)], dim=0)
        rewards = torch.cat([rewards, torch.zeros(1, device=device)], dim=0)

        normalized_states = (states - state_mean) / state_std
        action = get_action(model, normalized_states, actions,
                            rewards, target_return, timesteps)
        actions[-1] = action

        state, reward, done, _ = env.step(action.detach().cpu().numpy())
        cur_state = torch.from_numpy(state).reshape(1, state_dim).to(device)
        states = torch.cat([states, cur_state.to(device)], dim=0)
        rewards[-1] = reward

        pred_return = target_return[0, -1] - (reward / scale)
        target_return = torch.cat([target_return,
                        pred_return.reshape(1, 1).to(device)], dim=1)
        timesteps = torch.cat([timesteps, torch.tensor([[t + 1]],
                    device=device, dtype=torch.long)], dim=1)
```

```
        episode_return += reward
        episode_length += 1

        if done:
            break

    return episode_return, episode_length
```

To pass in your chosen environment into the preceding function, you just have to run the function in Example 7-4 in the accompanying notebook.

*Example 7-4. Choose environment*

```
user_input = input("""Enter the environment name
                   (e.g., 1 for 'HalfCheetah-v3' or
                   2 for 'Walker2d-v3'): """)
```

The code in Example 7-5 will use your chosen environment in the Gymnasium's setup.

*Example 7-5. Create environment*

```
if user_input in env_options:
    env_name = env_options[user_input]
else:
    print("Invalid input. Please enter 1 or 2.")


env = gym.make(env_name)
env = Recorder(env, directory, fps=30)

env, model, state_mean, state_std, state_dim, act_dim = setup_environment(env_name)
```

Now you can call the environment with the play function (see Example 7-6).

*Example 7-6. Create video from interaction with environment*

```
episode_return, episode_length = interact_with_environment(env,
                                 model, state_mean, state_std,
                                 max_ep_len, TARGET_RETURN,
                                 state_dim, act_dim, scale, device)

env.play() ❶
```

❶ Plays the environment after interaction.

This will output a short video, where you can see how the decision transformer performs in your chosen environment.

Offline RL, as with the DT, allows training policies to use precollected data without the need to explore in the real environment. This approach mitigates the risk of accidents or unsafe behaviors during the learning process, which is crucial for maintaining safety in real-world applications such as robotics and autonomous driving. However, this method restricts the agent's training to the quality of the offline dataset. In the following section, you'll discover how to fine-tune the agent for the task at hand through online interactions with the environment.

## Going Live: Online Decision Transformer

The online decision transformer (ODT)[6] blends offline policy training on a precollected dataset with online fine-tuning.

In certain situations, collecting data for online reinforcement learning is either too expensive or too dangerous. However, online RL is crucial for adapting to changing conditions and unexpected scenarios not captured in the offline dataset. The ODT enhances the DT training approach, much like approaches used in language and vision models, where the model's performance can be significantly improved with minimal online data.

To achieve this, ODT extends the decision transformer's architecture by transitioning from a deterministic to a stochastic policy, where actions are based on the recent $K$ (input sequence length) states and returns-to-go. Additionally, the approach introduces *max-entropy sequence modeling*, which is a method that aims to maximize the entropy of the predicted sequences to ensure more exploration and diversity in the generated outcomes. In this setting, the policy entropy depends on the data distribution $\tau$, which is static during the offline pretraining phase but dynamic during fine-tuning as it adapts to the online data acquired through exploration. The key difference from other classical max-entropy RL algorithms,[7] which focus on maximizing returns through expected per-action entropy, is that ODT relies solely on supervised learning of action sequences. Additionally, the entropy is defined at the sequence level rather than the transition level.

The training pipeline also consists of a *replay buffer*, which stores past transitions to be randomly sampled and reused during training. This improves learning efficiency and stability, helps the agent learn from diverse data, and enhances the generalizability of the learned policy.

6 Qinqing Zheng et al. "Online Decision Transformer" (2022).

7 Tuomas Haarnoja et al. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor" (2018).

For ODT, the replay buffer consists of trajectories rather than transitions. After offline pretraining, the replay buffer $\mathcal{T}_{\text{replay}}$ is initialized with the top $N$ trajectories from the offline data $\mathcal{T}_{\text{offline}}$. The algorithm iterates for $R$ rounds or until convergence. In each round, trajectory $\tau$ is collected using the current policy $\pi_\theta$ and exploration returns-to-go. The oldest trajectory in the replay buffer is replaced with the new trajectory $\tau$, in a first-in-first-out manner. The policy $\pi_\theta$ is fine-tuned on the updated replay buffer $\mathcal{T}_{\text{replay}}$ $I$ iterations using the ODT training algorithm. Example 7-7 shows the original implementation from the paper.

*Example 7-7. Replay buffer in online decision transformer*

```
class ReplayBuffer(object):
    def __init__(self, capacity, trajectories=[]):
        self.capacity = capacity
        if len(trajectories) <= self.capacity:
            self.trajectories = trajectories
        else:
            returns = [traj["rewards"].sum() for traj in trajectories]
            sorted_inds = np.argsort(returns)  # lowest to highest
            self.trajectories = [
                trajectories[ii] for ii in sorted_inds[-self.capacity :]
            ]

        self.start_idx = 0

    def __len__(self):
        return len(self.trajectories)

    def add_new_trajs(self, new_trajs):
        if len(self.trajectories) < self.capacity:
            self.trajectories.extend(new_trajs)
            self.trajectories = self.trajectories[-self.capacity :]
        else:
            self.trajectories[
                self.start_idx : self.start_idx + len(new_trajs)
            ] = new_trajs
            self.start_idx = (self.start_idx + len(new_trajs)) % self.capacity

        assert len(self.trajectories) <= self.capacity
```

To better understand how this replay buffer works, you can use the code in Example 7-8.

*Example 7-8. Inner workings of replay buffer*

```
trajectories = [
    {"rewards": np.array([1, 2, 3]),
    "states": np.array([0, 1, 2]), "actions": np.array([0, 1, 2])},
```

```
    {"rewards": np.array([2, 3, 4]),
    "states": np.array([1, 2, 3]), "actions": np.array([1, 2, 3])},
    {"rewards": np.array([3, 4, 5]),
    "states": np.array([2, 3, 4]), "actions": np.array([2, 3, 4])},
    {"rewards": np.array([4, 5, 6]),
    "states": np.array([3, 4, 5]), "actions": np.array([3, 4, 5])}
] ❶

replay_buffer = ReplayBuffer(capacity=3, trajectories=trajectories[:2]) ❷

print("Initial replay buffer:")
for traj in replay_buffer.trajectories:
    print(traj)

new_trajectories = [
    {"rewards": np.array([5, 6, 7]),
    "states": np.array([4, 5, 6]), "actions": np.array([4, 5, 6])},
    {"rewards": np.array([6, 7, 8]),
    "states": np.array([5, 6, 7]), "actions": np.array([5, 6, 7])}
] ❸

replay_buffer.add_new_trajs(new_trajectories) ❹

print("\nReplay buffer after adding new trajectories:")
for traj in replay_buffer.trajectories:
    print(traj)
```

❶ Create example trajectory data.

❷ Initialize the replay buffer with a capacity of 3.

❸ Create new trajectories for the buffer.

❹ Add new trajectories to the buffer.

This results in the following output:

```
    Initial replay buffer:
    {'rewards': array([1, 2, 3]), 'states': array([0, 1, 2]), 'act
    ions': array([0, 1, 2])}
    {'rewards': array([2, 3, 4]), 'states': array([1, 2, 3]),
    'actions': array([1, 2, 3])}

    Replay buffer after adding new trajectories:
    {'rewards': array([2, 3, 4]), 'states': array([1, 2, 3]),
    'actions': array([1, 2, 3])}
    {'rewards': array([5, 6, 7]), 'states': array([4, 5, 6]),
    'actions': array([4, 5, 6])}
    {'rewards': array([6, 7, 8]), 'states': array([5, 6, 7]),
    'actions': array([5, 6, 7])}
```

Another significant addition to the training pipeline of ODT is *hindsight return relabeling* (HER),[8] a technique to enhance sample efficiency for goal-conditioned agents. The key idea is to relabel the agent's trajectories with the goals actually achieved, rather than the originally intended goals. In the case of ODT, policies are learned based on initial returns-to-go. However, the return achieved during a policy rollout and the induced returns-to-go can differ from the intended returns-to-go. Inspired by HER, the returns-to-go token for the rolled-out trajectory is relabeled with the achieved returns, ensuring that the returns-to-go token at the last timestep matches the reward obtained by the agent. This return relabeling strategy is applicable to environments with both sparse and dense rewards.

## A Brave New World: Stochastic Transformer-Based World Model

Efficient stochastic transformer-based world models for reinforcement learning (STORM),[9] is a model-based (incorporating a world model) reinforcement learning algorithm. However, it's more efficient in real-time interactions than other model-based RL algorithms and sets a new benchmark among state-of-the-art algorithms.

The imagining process with a world model involves an autoregressive process that can accumulate prediction errors over time. The reason for this is that there may be discrepancies between the imagined trajectory and the real trajectory. That can cause the agent to inadvertently pursue virtual goals, leading to poor performance in the real environment. To address this issue, STORM uses a *variational autoencoder* (VAE), which can automatically learn low-dimensional latent representations of high-dimensional data while incorporating random noise into the latent space. Figure 7-5 shows an overview of a VAE.

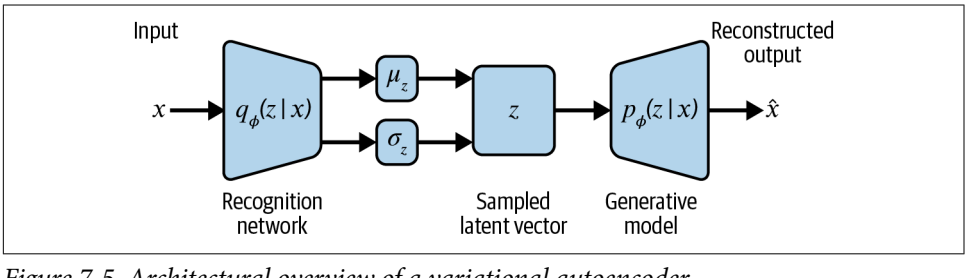


*Figure 7-5. Architectural overview of a variational autoencoder.*

8 Dibya Ghosh et al. "Learning to Reach Goals via Iterated Supervised Learning" (2019).

9 Weipu Zhang et al. "STORM: Efficient Stochastic Transformer Based World Models for Reinforcement Learning" (2023).

This added noise into the world model can help the agent generalize better and be more robust to variations, thereby improving its performance in real-world scenarios.

Most past model-based *deep reinforcement learning* (DRL) has leveraged recurrent neural networks (RNNs) as the sequence model. DRL combines deep learning with reinforcement learning principles to create algorithms that can learn from high-dimensional inputs. But, as you know, the recurrent nature of RNNs hinders parallelized computing, which results in slower training speed. In addition, RNN-based models face challenges with long-term dependencies. Therefore, leveraging a transformer as a sequence model comes as the natural choice to tackle both problems.

STORM's world model generates image observations $o_t$ from the environment. However, directly modeling dynamics on raw images is computationally expensive and prone to errors. To address this, the VAE converts the images into latent stochastic categorical distributions $Z_t$. This distribution comprises 32 categories, each with 32 classes. The encoder ($q_\varphi$) and decoder ($p_\varphi$) are implemented as convolutional neural networks (CNNs). A latent variable $z_t$ is sampled from $Z_t$ to represent $o_t$. To preserve gradients during sampling, STORM applies the straight-through gradients trick, a method that allows gradients to be backpropagated through discrete variables by treating them as continuous during the backward pass, thereby enabling efficient training of models involving stochastic sampling.

Next, an image encoder $z_t \sim q_\varphi(z_t \mid o_t) = \mathcal{Z}_t$ and decoder $\hat{o}_t = p_\varphi(z_t)$ process the observations. Before entering the sequence model, the latent sample and the action are combined into a single token using multilayer perceptrons (MLPs) and concatenation. This process is shown in Example 7-9.

*Example 7-9. Create single token*

```
stem = nn.Sequential(
    nn.Linear(stoch_dim + action_dim, feat_dim, bias=False), ❶
    nn.LayerNorm(feat_dim),
    nn.ReLU(inplace=True),
    nn.Linear(feat_dim, feat_dim, bias=False),
    nn.LayerNorm(feat_dim)
)
```

❶ Combine image and action embeddings.

The sequence model takes the sequence as input and produces hidden states. It uses a GPT-like transformer structure, with self-attention blocks masked to allow it to attend to the sequence. It uses MLPs to predict the current reward, the continuation flag, and the next distribution.

The model is trained in a self-supervised manner. The total loss function includes reconstruction loss, prediction loss of the reward, prediction loss of the continuation flag, dynamics loss, and representation loss:

$$\mathscr{L}(\varphi) = \frac{1}{BT}\sum_{n=1}^{B}\sum_{t=1}^{T}\left[\mathscr{L}_t^{\text{rec}}(\varphi) + \mathscr{L}_t^{\text{rew}}(\varphi) + \mathscr{L}_t^{\text{con}}(\varphi) + \beta_1\mathscr{L}_t^{\text{dyn}}(\varphi) + \beta_2\mathscr{L}_t^{\text{rep}}(\varphi)\right]$$

The hyperparameters $\beta_1$ and $\beta_2$ are fixed with values 0.5 and 0.1, respectively. $B$ is the batch size, $T$ is the batch length, and $\varphi$ represents the parameters of the various components of the model used to compute the respective losses.

The reconstruction loss, $\mathscr{L}_t^{\text{rec}}(\varphi)$, is the loss of the original image and is formulated as follows:

$$\mathscr{L}_t^{\text{rec}}(\varphi) = \left\|\hat{o}_t - o_t\right\|_2$$

The prediction loss, $\mathscr{L}_t^{\text{rew}}(\varphi) = \mathscr{L}^{\text{sym}}\left(\hat{r}_t, r_t\right)$, of the reward includes the symlog two-hot loss, $\mathscr{L}^{\text{sym}}$, which transforms the regression problem into a classification problem to ensure that the loss is consistently scaled across the various environments, whereas the prediction loss of the continuation flag is mathematically represented as:

$$\mathscr{L}_t^{\text{con}}(\varphi) = c_t\log\hat{c}_t + (1 - c_t)\log\left(1 - \hat{c}_t\right)$$

Here, $g_\varphi^C$ is an MLP that takes the hidden state $h_t$ produced by the sequence model and predicts the continuation flag $\hat{c}_t$.

The dynamics loss, expressed as Kullback–Leibler divergence but with different gradient backward and weighting, guides the sequence model in predicting the next distribution:

$$\mathscr{L}_t^{\text{dyn}}(\varphi) = \max\left(1, \text{KL}\left[\text{sg}\left(q_\varphi\left(z_{t+1} \mid o_{t+1}\right)\right) \| g_\varphi^D\left(\hat{z}_{t+1} \mid h_t\right)\right]\right)$$

Here sg ( · ) stands for the stop-gradients.

The representation loss, also expressed as Kullback–Leibler divergence with different gradient backward and weighting processes, allows the encoder's output to be weakly influenced by the sequence model's prediction, ensuring that the learning of distributional dynamics is not excessively challenging.

This equation is formulated as follows:

$$\mathscr{L}_t^{\text{rep}}(\varphi) = \max\left(1,\ \text{KL}\left[q_\varphi(z_{t+1} \mid o_{t+1}) \| \ \text{sg}\left(g_\varphi^D\left(\hat{z}_{t+1} \mid h_t\right)\right)\right]\right)$$

sg ( · ) again denotes the stop-gradients.

During the imagination process, a brief contextual trajectory is randomly selected from the replay buffer, and the initial posterior distribution is computed. During inference, sampling is done from the prior distribution. The agent's state is formed by concatenating $z_t$ and $h_t$, defined as $s_t = [z_t, h_t]$. The critic approximates the value $V_\psi(s_t)$, and the actor samples actions $a_t \sim \pi_\theta(a_t \mid s_t)$. Figure 7-6 illustrates STORM's overall architecture.

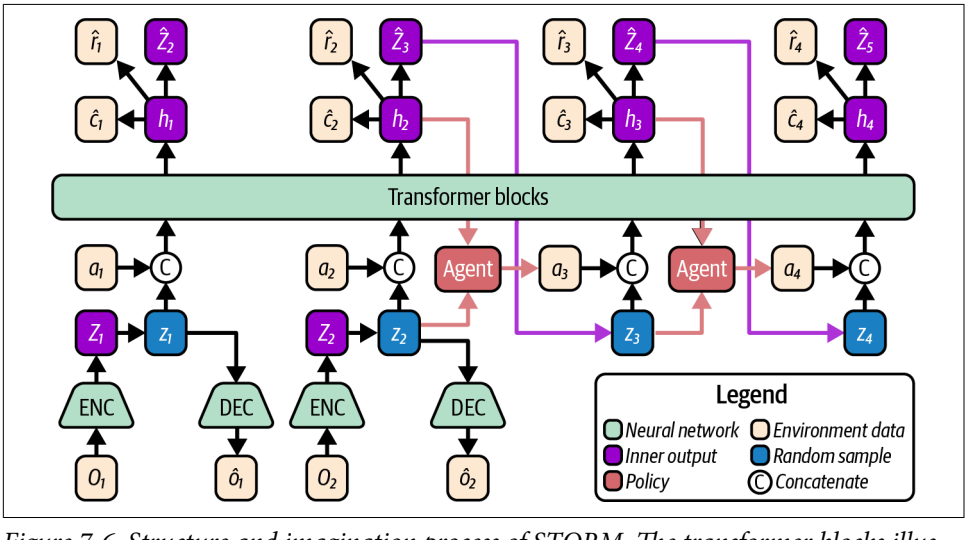


*Figure 7-6. Structure and imagination process of STORM. The transformer blocks illustrate the sequence model. The agent block, shown as a neural network, represents the policy function for action selection given the state. Image adopted from Weipu Zhang et al. (2023).*

Actor learning settings are adopted from DreamerV3,[10] a reinforcement learning algorithm that leverages an RNN-based world model to predict future states and rewards. The complete loss of the actor-critic algorithm includes terms for the predicted reward and continuation flag. The actor loss involves the $\lambda$-return, with normalization across the batch. The critic, which evaluates the actions by estimating the value function, uses an exponential moving average (EMA) of its parameters, $\psi$,

10 Danijar Hafner et al. "Mastering Diverse Domains Through World Models" (2023).

to regularize the value function. This approach stabilizes training and prevents overfitting.

### Testing STORM with Road Runner

Now that you're familiar with the details of STORM, it's time to test the model. For this, you'll use the Road Runner environment in the Arcade Learning Environment (ALE), a framework designed for researchers and developers to create AI agents for Atari 2600 games. Built on the Atari 2600 emulator Stella, ALE abstracts the complexities of emulation, allowing users to focus on designing their agents. *Road Runner* is a popular benchmark game in ALE.

I trained STORM as per information in the paper repository and saved the resulting checkpoints. To run the evaluation of the model in Google Colab, you have to create an evaluation script. Example 7-10 shows a sample of the code. You can find the rest in the book's repository.

*Example 7-10. Evaluation script*

```
def build_single_env(env_name, image_size, seed):
    env = gymnasium.make(env_name, full_action_space=False,
    render_mode="rgb_array", frameskip=1) ❶
    env = env_wrapper.SeedEnvWrapper(env, seed=seed) ❷
    env = env_wrapper.MaxLast2FrameSkipWrapper(env, skip=4) ❸
    env = gymnasium.wrappers.ResizeObservation(env, shape=image_size)
    env = env_wrapper.LifeLossInfo(env) ❹
    return env

def evaluate(env, world_model, agent, video_path):
    obs, _ = env.reset()
    total_reward = 0
    done = False
    frames = []

    while not done:
        frames.append(env.render())
        action = agent.select_action(world_model.encode_obs(obs)) ❺
        obs, reward, done, _, _ = env.step(action)
        total_reward += reward

    env.close()
    imageio.mimsave(video_path, frames, fps=30)
    return total_reward
```

❶ Create the environment using Gymnasium with specified settings.

❷ Wrap the environment to ensure reproducible results by setting the seed.

❸ Apply a wrapper to skip frames to speed up the environment's execution.

❹ Capture life loss information.

❺ Select an action using the agent based on the encoded observation from the world model.

If you run the code in the ch03_STORM.ipynb notebook, you'll get the mean reward. I got the following output for STORM:

```
Mean reward: 16960.0
```

whereas a random agent would achieve around 200–300 points and DreamverV3 would achieve a score of 15,565 on the same benchmark. The highest human score, in the Atari 100k benchmark is 7,845. The "100k" in the Atari 100k benchmark refers to the limit of 100,000 interactions (frames) that the reinforcement learning agent is allowed to have with the environment during training. High performance on the Atari 100k benchmark indicates that the algorithm can learn quickly and effectively from limited data.

## TWISTER: Transformer-Based World Models with Contrastive Predictive Coding

While STORM demonstrated the efficacy of transformer-based world models for reinforcement learning, an inherent limitation of many prior approaches, including STORM, is their reliance on a next-state prediction objective. This objective, while seemingly intuitive, can be insufficient to fully leverage the powerful representation capabilities of transformers, especially when dealing with subtle changes between consecutive video frames in environments like Atari games. A high cosine similarity (see Figure 7-7) between adjacent latent states suggests that predicting the immediate next state might not result in a deep understanding of the past context, unlike in domains such as neural language modeling.

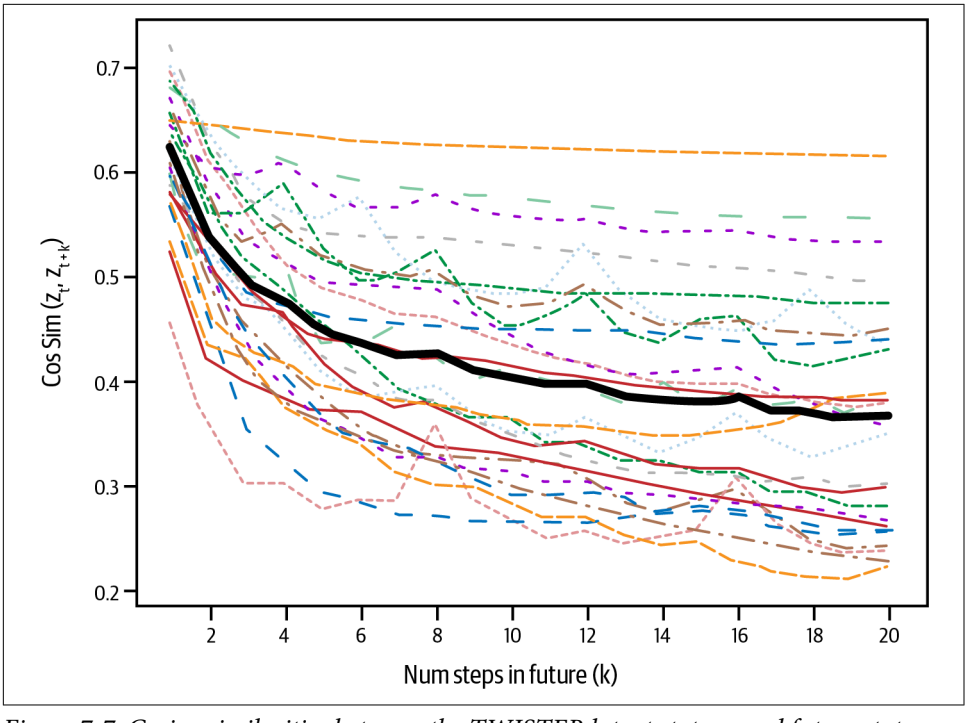


*Figure 7-7. Cosine similarities between the TWISTER latent state $z_t$ and future states $z_{t+k}$ aggregated across all 26 games in the Atari 100k benchmark. The results represent average similarities computed over 5 different seeds. Image adapted from Maxime Burchi et al.*

TWISTER (Transformer-based World model wIth contraSTivE Representations)[11] addresses this limitation by extending world model predictions to longer time horizons and introducing an *action-conditioned contrastive predictive coding* (AC-CPC) objective. This approach allows the world model to learn high-level temporal feature representations, improving agent performance without using lookahead search.

TWISTER, similar to other model-based RL algorithms you've explored, has three main neural networks: a world model, an actor network, and a critic network. These components are trained concurrently, leveraging a replay buffer filled with past experiences. The core of TWISTER's innovation lies in its world model and the AC-CPC objective. Analogous to STORM, TWISTER uses a transformer-based architecture, specifically a transformer state-space model (TSSM), as its world model. Figure 7-8 shows the transformer-based world model with contrastive representations.

11 Maxime Burchi et al. "Learning Transformer-Based World Models with Contrastive Predictive Coding" (2025).

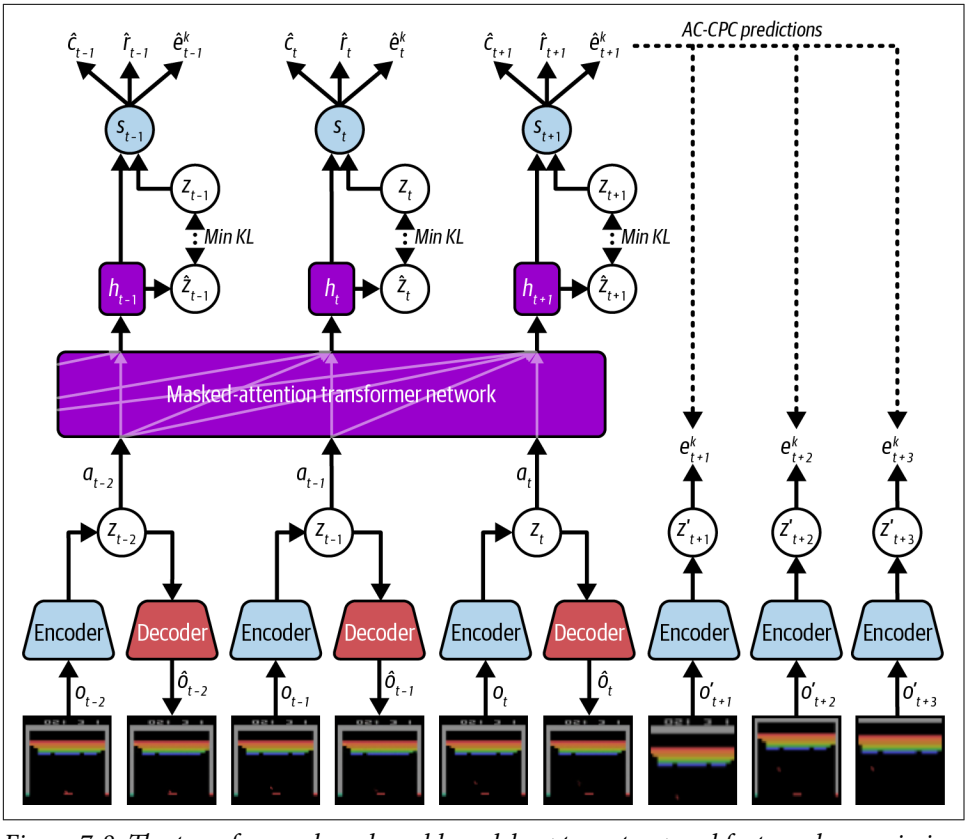


*Figure 7-8. The transformer-based world model captures temporal features by maximizing the mutual information between current model states $s_t$ and future stochastic states $z'_{t:t+K}$, derived from augmented image observations. Image adapted from Maxime Burchi et al. (2025).*

The TSSM uses masked self-attention with relative positional encodings to predict future stochastic states. A key feature is the caching mechanism for hidden states across autoregressive segments, functionally similar to KV caching in autoregressive LLMs, which allows TWISTER to efficiently simulate long-horizon trajectories without recomputing attention over prior context. The world model operates in a latent space, like STORM, encoding high-dimensional image observations $o_t$ into discrete stochastic states $z_t$ using a convolutional VAE with categorical latents. The decoder reconstructs images from these stochastic states. The transformer network outputs hidden states $h_t$, which are concatenated with the stochastic states $z_t$ to form the model states $s_t = z_t, h_t$ (similar to STORM). These model states are important for predicting environmental rewards, episode continuation flags, and the AC-CPC features.

Example 7-11 shows the `forward_img` function from the `TSSM` class (taken from the original paper), which encodes temporal context using masked attention and predicts the next latent.

*Example 7-11. Encoding temporal context*

```
def forward_img(self, prev_states, prev_actions, mask,
                return_att_w=False, return_blocks_deter=False): ❶

        if self.action_clip > 0.0:
            prev_actions = prev_actions * (self.action_clip / torch.clip(
                           torch.abs(prev_actions),
                           min=self.action_clip)).detach() ❷

        if self.discrete:
            stoch = prev_states["stoch"].flatten(start_dim=-2, end_dim=-1) ❸
        else:
            stoch = prev_states["stoch"]

        x = self.action_mixer(torch.concat([stoch, prev_actions], dim=-1)) ❹

        outputs = self.transformer(x, hidden=prev_states["hidden"], mask=mask,
                                   return_hidden=True, return_att_w=return_att_w,
                                   return_blocks_x=return_blocks_deter) ❺
        deter, hidden = outputs.x, outputs.hidden

        add_out_dict = {}
        if return_att_w:
            add_out_dict["att_w"] = outputs.att_w
        if return_blocks_deter:
            add_out_dict["blocks_deter"] = outputs.blocks_x

        logits = self.dynamics_predictor(deter).reshape(
                deter.shape[:-1] + (self.stoch_size, self.discrete)) ❻
        dist_params = {'logits': logits}

        stoch = self.get_dist(dist_params).rsample() ❼

        return {"stoch": stoch, "deter": deter, "hidden": hidden,
                **dist_params, **add_out_dict} ❽
```

❶ Defines the main transition function in latent space, predicting the next state.

❷ Clips the action magnitude to remain within a bounded range for stable training.

❸ Flattens the discrete latent variables across categories for transformer input.

❹ Combines stochastic latents and actions via an MLP to form the transformer input embedding.

❺ Applies the masked transformer network with causal attention for temporal modeling.

❻ Predicts logits over the next latent distribution (discrete categorical).

❼ Samples the next latent state from the predicted distribution.

❽ Returns the updated stochastic and deterministic state components.

This function forms the basis of both training-time rollout and the agent's imagination during planning, enabling TWISTER to simulate sequences entirely in latent space.

The central innovation of TWISTER is the integration of action-conditioned contrastive predictive coding into the world model's learning objective. *Contrastive predictive coding* (CPC), originally applied in domains like speech and image processing, is a self-supervised learning technique that aims to learn representations by maximizing the mutual information between an autoregressive model's output features and future encoded representations. In TWISTER, this concept is adapted for model-based reinforcement learning. Unlike previous transformer-based world models that typically focus on predicting only the next latent state, TWISTER's AC-CPC objective extends predictions to $K = 10$ future stochastic states. This forces the model to learn more abstract and temporally extended representations. With this technique, the world model learns to distinguish between positive samples (actual future stochastic states corresponding to the current model state and sequence of future actions) and negative samples (other augmented stochastic states within the batch). The AC-CPC loss trains the world model to distinguish true future states from negative samples using InfoNCE loss.[12] Each pair consists of a projected latent state and a prediction conditioned on the model state and future actions. The similarity is computed as a dot product of the two projections. The CPC loss encourages similarity of true pairs while repelling negatives.

The predictions are conditioned on the action sequence $a_{t:t+k}$, which helps reduce future uncertainty. For each prediction step $k$, two MLPs are learned:

- $q_\phi^k$, which projects the discretized stochastic future state $z'_{t+k}$ into the contrastive space.
- $p_\phi^k$, which predicts that same space from the current model state $s_t$ and the future actions.

12 Evgenia Rusak et al. "InfoNCE: Identifying the Gap Between Theory and Practice" (2025).

The InfoNCE loss is computed over all steps $k \in 1, \ldots, K$ using the following formulation:

$$L_{cpc}(\phi) = -\frac{1}{K}\sum_{k=0}^{K-1}\log\frac{\exp\left(\text{sim}\left(z'_{t+k}, s_t\right)\right)}{\Sigma_{z'_j \in Z'}\exp\left(\text{sim}\left(z'_j, s_t\right)\right)}$$

The world model learns to predict $K = 10$ future stochastic states among the batch of augmented samples. The similarity function `sim` is implemented as:

$$\text{sim}\left(z'_j, s_t\right) = q_\phi^k\left(z'_j\right)^T p_\phi^k\left(s_t, a_{t:t+k}\right)$$

learning two MLP networks $q_\phi^k$ and $p_\phi^k$ for each step. This is contrary to the original CPC paper, which experiments with continuous feature states, whereas TWISTER uses discrete latent states for the world model. Example 7-12 shows the implementation of the two MLPs from the original code, computing the feature pair for similarity computation in the InfoNCE loss.

*Example 7-12. MLPs for contrastive computation*

```
class ContrastiveNetwork(nn.Module): ❶

    def __init__(
        self,
        hidden_size=512,
        out_size=512,
        feat_size=32*32+512,
        embed_size=4*4*8*32,
        act_fun="ELU",
        num_layers=2
    ):
        super(ContrastiveNetwork, self).__init__()

        self.mlp_feats = modules.MultiLayerPerceptron( ❷
            dim_input=feat_size,
            dim_layers=[hidden_size for _ in range(num_layers-1)] + [out_size],
            act_fun=act_fun
        )
        self.mlp_embed = modules.MultiLayerPerceptron( ❸
            dim_input=embed_size,
            dim_layers=[hidden_size for _ in range(num_layers-1)] + [out_size],
            act_fun=act_fun
        )

    def forward(self, feats, embed): ❹
        x = self.mlp_feats(feats) ❺
```

```
        y = self.mlp_embed(embed) ❻

        return x, y ❼
```

❶ Defines the `ContrastiveNetwork` class that encapsulates both projection and prediction heads.

❷ Builds the projector MLP $q_\phi^k$ for contrastive features from future latent states.

❸ Builds the predictor MLP $p_\phi^k$ conditioned on current model state and action sequence.

❹ The forward method receives the future latent and current model-action embedding.

❺ Projects the latent state to the contrastive space.

❻ Predicts the contrastive feature representation from model state and action context.

❼ Returns the feature pair for similarity computation in the InfoNCE loss.

The actor and critic networks in TWISTER are trained using imaginary trajectories generated by the learned world model, similar to the approach in DreamerV3. Learning occurs entirely in the latent space, which benefits from larger batch sizes and improved generalization. The cached self-attention keys and values from the world model training phase are reused during behavior learning to maintain context during imagination. The actor learns to select actions that maximize predicted returns, while the critic evaluates the value of states.

Compared to other leading model-based agents, TWISTER achieves improvements in both sample efficiency and overall performance. The model achieves a human-normalized mean score of 162% on the Atari 100k benchmark, setting a new record among SOTA methods in this category.

# Conclusion

This chapter introduced you to key concepts in reinforcement learning, including online and offline learning, model-based and model-free approaches, and on-policy versus off-policy methods. You learned about traditional RL algorithms and foundational principles, providing a solid grounding for understanding recent advances in RL transformers.

You explored the decision transformer, which reinterprets RL as a conditional sequence modeling task, using an autoregressive model to integrate desired rewards with past states and actions. This approach aligns with offline and model-free RL principles, enabling more stable and effective learning environments.

The online decision transformer combines offline pretraining with online fine-tuning, enhancing adaptability to changing conditions. This method employs a replay buffer and hindsight return relabeling to optimize the learning process.

You then explored STORM, a stochastic transformer-based world model that incorporates variational autoencoders to enhance robustness in noisy, real-world settings. STORM's success on the Road Runner Atari benchmark highlighted its performance, surpassing both random baselines and earlier model-based methods.

Finally, you encountered TWISTER, a transformer-based world model trained using contrastive predictive coding. TWISTER advances the field by encouraging deeper temporal abstraction and action-conditioned, long-horizon planning. It achieves state-of-the-art results on the Atari 100k benchmark and demonstrates how contrastive objectives can improve representation learning in reinforcement settings.

The knowledge you've gained in this chapter will be particularly valuable for the next one, where we move beyond reward-based training into transformers for planning, reasoning, and coding. Some of the RL foundations you've just learned will be helpful as you explore how transformers power agents that plan, reason, and code across diverse tasks.

CHAPTER 8

# Embracing the Era of Experience: Transformers for Planning, Reasoning, and Coding

This chapter marks not only a pivotal shift in your exploration of transformers but also a shift in AI from human-data-centric approaches to experience-driven learning, which is highly relevant to how transformers can be used and advanced.

So far, you've seen how transformers have adapted to modeling data across diverse domains: from language and time series to images, video, audio, and, as explored in Chapter 7, reinforcement learning (RL). That chapter laid the groundwork for your understanding of this new era. The next frontier in transformers is not just about comprehending what is but about learning what to do: to plan, reason, and create new knowledge through active interaction with the world.

In this chapter, you'll learn how transformer-based models are engineered to embody these capabilities. You'll look into various approaches and architectures that enable transformers to excel in planning, reasoning, and coding by learning from experience. This includes exploring how they process streams of continuous interaction; execute actions and observe their grounded consequences, using objectively verifiable rewards to drive self-improvement; and develop new non-human reasoning strategies. You'll understand how LLMs are incentivized to develop remarkable reasoning capabilities via RL, showcasing their "aha moments" and impressive performance in complex domains like mathematics and competitive programming. You'll see how reasoning can be encouraged, shaped, or even disabled and budgeted at inference time, and how to strategically give smaller models more time to think and reason better—and thereby even outsmart bigger models.

## From Human Data to Lived Experience

For years, we've been witnessing the era of data. Now we're entering what David Silver and Richard S. Sutton, authors of *Designing an Intelligence* (MIT Press) and titans in the field of RL, describe as the "era of experience." Sutton's foundational work on RL, particularly concepts like temporal difference learning and the Dyna architecture,[1] has shaped our understanding of how intelligent systems can learn through trial and error.

To progress significantly further, models require a new, continually improving source of data: their own lived experience. This means learning autonomously by interacting with an environment, generating data that adapts and evolves as the model itself becomes stronger. This is where RL reclaims its central role, providing the framework for models to learn by doing, receiving feedback directly from their actions in the world.

For complex cognitive tasks like reasoning, this shift involves a powerful synergy: the combination of the transformer's inherent ability to generate chain-of-thought (CoT)[2] reasoning with the iterative refinement provided by RL. While initial CoT capabilities might be bootstrapped from human examples, the "era of experience" pushes this further by using RL to autonomously discover and improve reasoning paths. This includes paths that may diverge from human-like thinking but lead to objectively correct outcomes. This important feedback loop allows the transformer to test its internal "thoughts" against the reality of the environment (such as a mathematical proof checker or a code compiler) and refine them accordingly.

A key example is the DeepSeek-R1-Zero model,[3] which demonstrates that high-level reasoning can emerge purely through large-scale reinforcement learning, without supervised fine-tuning. Trained directly on DeepSeek-V3-Base using the Group Relative Policy Optimization (GRPO) framework, the model learned complex CoT reasoning strategies. Over thousands of RL steps, it showed consistent self-evolution, increasing its pass@1 score (see sidebar) on AIME 2024 from 15.6% to 71.0%. The AIME benchmark consists of mathematical problems modeled after a high school–level math competition.

1 Richard S. Sutton et al. "Dyna-Style Planning with Linear Function Approximation and Prioritized Sweeping" (2012).

2 Jason Wei et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models" (2023).

3 Daya Guo et al. "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning" (2025).

## Understanding the Metric Pass@k

Pass@k is a standard metric used to evaluate the performance of code generation models. It estimates the probability that at least one of k independently sampled code completions is correct. This is particularly useful in settings where models generate multiple candidate programs, and the best one is selected either manually or via some scoring function. The metric captures the practical success rate under multiple attempts, offering a more forgiving view of performance than strict one-shot accuracy. The higher the k, the more chances the model gets to succeed.

Formally, pass@k is computed as:

$$\text{pass@k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

For example, if 70 out of 100 completions are correct, then:

$$\text{pass@3} \approx 1 - \frac{\binom{30}{3}}{\binom{100}{3}} \approx 0.97$$

By contrast, pass$^k$ measures consistency, estimating the probability that all k outputs are correct:

$$\text{pass}^3 = \left(\frac{c}{n}\right)^3 = (0.7)^3 = 0.343$$

This means there is a 97% chance that at least one of three completions is correct.

All three completions would succeed only 34.3% of the time. While pass@k emphasizes coverage, pass$^k$ reflects robustness. Both are informative in code generation tasks depending on whether the goal is to find a good solution or to rely on consistent correctness across multiple samples.

DeepSeek-R1-Zero exhibited emergent behaviors like reflection (revisiting and reevaluating previous steps) and spontaneous exploration of alternative strategies. These were not hardcoded but arose from interaction with the RL environment. Researchers even observed an "aha moment" when the model anthropomorphically shifted its strategy midway, reallocating computational resources to improve its initial thinking.

Models in this new era move beyond mere memorization and pattern matching, instead autonomously discovering unseen solutions and strategies through their own interactions. They can now excel in complex cognitive tasks. This new synergy, with CoT reasoning refined through RL, enables transformers to surpass the limitations of static pattern recognition.

## Learning to Reason: From Pretraining to Reinforcement Learning

After the pretraining stage, LLMs typically undergo *supervised fine-tuning* (SFT), a phase in which the model is further trained on labeled examples to improve its performance on specific tasks. A common form of SFT is instruction tuning, where the model learns to follow human-written instructions and produce helpful, contextually appropriate responses. This is achieved using diverse datasets that capture real-world interaction patterns, often developed through careful manual annotation and refinement to reflect everyday human communication needs.

In contrast to pretraining, *fine-tuning* focuses specifically on computing the loss for the model's answers, typically disregarding the loss associated with the input questions. Moreover, incorporating datasets that feature CoT reasoning—that is explicitly showing intermediate reasoning steps—has been shown to significantly improve the reasoning abilities of LLMs. Even a simple sentence at the beginning of the prompt like "Let's think step-by-step" can improve the model's reasoning process.

This produces a shift, moving LLMs from passively consuming data to actively generating and refining their own prompts. Unlike domains such as image classification or next-word prediction, planning and reasoning require the model to consider sequences of actions, simulate outcomes, and weigh alternatives. This shift brings language models into closer alignment with RL, where policies and reward models provide the structure for iterative improvement.

However, traditional reinforcement learning methods in LLMs have often focused on *outcome reward models* (ORMs), where the model receives feedback only after producing a final solution. ORM frameworks such as *proximal policy optimization* (PPO),[4] which is a policy gradient method that balances exploration with stability by limiting how much the policy updates at each step, have been used to guide models toward better final answers. However, ORM approaches like PPO struggle with a known challenge: the *credit assignment problem*—that is, identifying which step in a reasoning chain contributed most to success or failure.

4 John Schulman et al. "Proximal Policy Optimization Algorithms" (2017).

In contrast, *process reward models* (PRMs) represent a major shift in this landscape. Rather than assigning feedback only at the end, PRMs distribute rewards across the reasoning trajectory. This enables more granular optimization and aligns better with how humans evaluate logical reasoning: step-by-step. PRMs can dramatically improve performance in tasks like coding and math, especially when paired with tree-search methods. PRMs also provide value beyond mathematics and coding. In tasks involving dialogue or decision making, PRMs enable LLMs to receive fine-grained, interpretable supervision. This step-level feedback improves reasoning, supports error correction, and allows for more reliable learning from interaction. Figure 8-1 illustrates the distinction between outcome and process reward models, highlighting how feedback is assigned either at the end of a reasoning episode or along the trajectory. This comparison underscores the growing emphasis on fine-grained, interpretable reward structures that help LLMs reason more reliably and scalably.

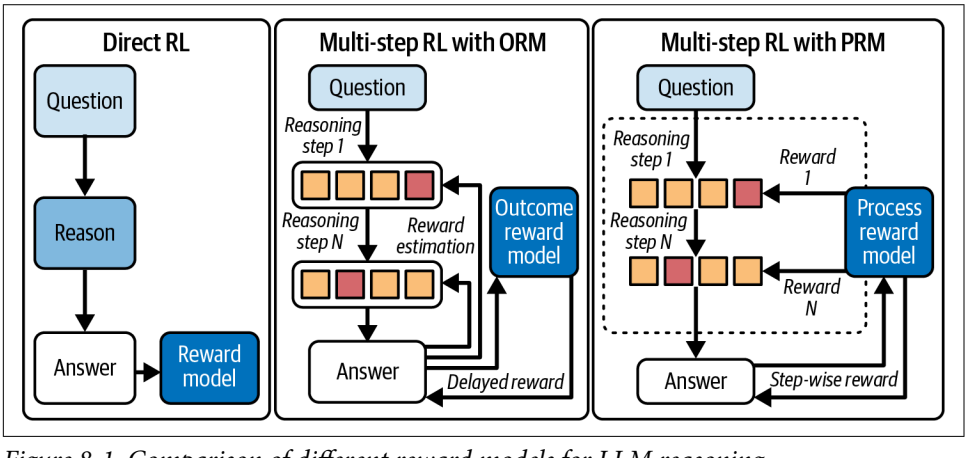


*Figure 8-1. Comparison of different reward models for LLM reasoning.*

This marks a fundamental transition: from learning to imitate human responses with SFT toward learning to reason. When researchers integrate reward-based optimization, especially via process-level supervision, LLMs begin to exhibit more robust planning capabilities, laying the groundwork for agentic behavior in complex environments.

# DeepSeek-R1: Reinforcing Reasoning Capabilities

DeepSeek-R1 follows a multi-stage training pipeline that begins with a cold-start phase, using SFT on curated long CoT examples to establish structural clarity and coherent reasoning patterns. This initial stage, known as *cold-start SFT*, uses human-aligned data to bootstrap the model's ability to perform structured reasoning.

Cold-start SFT provides initial structure and readability that pure RL lacks. Following this, the model undergoes reasoning-oriented RL using GRPO.[5]

The reward design avoids a common failure mode in RL systems known as *reward hacking*, where a model exploits weaknesses in the reward function. For example, it may produce outputs that appear correct to the reward model but are logically invalid or misleading. To prevent this, the design relies on rule-based signals, including accuracy feedback based on whether mathematical answers or code outputs are correct, as well as formatting constraints that promote structured reasoning within `<think>` tags. GRPO removes the need for a separate critic model, which is typically as large as the policy model. Instead, it estimates the baseline directly from group-level scores. For each question $q$, GRPO samples a set of outputs $o_1, o_2, \ldots, o_G$ from the old policy $\pi_{\theta_{\text{old}}}$. The current policy $\pi_\theta$ is then optimized by maximizing the following objective:

$$\mathcal{J}\text{GRPO}(\theta) = \mathbb{E}q, o_i\left[\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{\text{old}}}(o_i \mid q)}A_i,\ \text{clip}\left(\frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{\text{old}}}(o_i \mid q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right)\right.\right.$$
$$\left.\left. - \beta, \mathbb{D}KL(\pi\theta \mid \pi_{\text{ref}})\right)\right]$$

This objective mirrors PPO-style clipping, where the policy ratio $\frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{\text{old}}}(o_i \mid q)}$ is constrained to avoid excessively large updates. The term $A_i$ is the advantage estimate for output $o_i$, computed using the normalized difference between the reward for $o_i$ and the average reward within the sampled group:

$$A_i = \frac{r_i - \text{mean}\,(r_1, r_2, \ldots, r_G)}{\text{std}\,(r_1, r_2, \ldots, r_G)}$$

Here, the advantage is scaled by the group's standard deviation to stabilize updates across samples. The final term in the GRPO objective, $\mathbb{D}KL(\pi\theta \mid \pi_{\text{ref}})$, acts as a KL divergence regularizer, penalizing deviations from a reference policy. As explained in Chapter 4, KL divergence quantifies how much one probability distribution diverges from another. In this setting, it encourages the updated policy to stay close to the reference, and it's computed in closed form as:

5 Zhihong Shao et al. "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models" (2024).

$$\mathbb{D}KL(\pi\theta|\pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(o_i \mid q)}{\pi_\theta(o_i \mid q)} - \log\frac{\pi_{\text{ref}}(o_i \mid q)}{\pi_\theta(o_i \mid q)} - 1$$

The KL divergence term is often shown as an explicit regularizer subtracted from the total objective:

$$\mathcal{J}\text{GRPO}(\theta) = \mathbb{E}[\ldots] - \beta, \mathbb{D}KL(\pi_\theta|\pi_{\text{ref}})$$

In practice, the implementation adopts a finer-grained, token-level formulation. Rather than computing KL after full sequence generation, the penalty is integrated directly into the per-token loss, improving training stability and efficiency.

An example of this token-level KL integration in the GRPO training loop is shown in Example 8-1, adapted from a reference implementation from a library for RL teachers.[6]

*Example 8-1. GRPO token-level KL loss*

```
def _compute_backprop_step_losses(
    self, model, prompt_ids, prompt_mask, completion_ids,
    completion_mask, ref_per_token_logps, advantages
):

    input_ids = torch.cat([prompt_ids, completion_ids], dim=1) ❶
    attention_mask = torch.cat([prompt_mask, completion_mask], dim=1)
    logits_to_keep = completion_ids.size(1)

    per_token_logps = self._get_per_token_logps( ❷
        model, input_ids, attention_mask, logits_to_keep
    )

    per_token_kl = torch.exp(ref_per_token_logps - per_token_logps) \
                   - (ref_per_token_logps - per_token_logps) - 1 ❸

    per_token_loss = torch.exp(
        per_token_logps - per_token_logps.detach()
    ) * advantages.unsqueeze(1) ❹

    per_token_loss = -(per_token_loss - self.beta * per_token_kl) ❺


    with torch.no_grad(): ❻
        mean_kl = ((per_token_kl * completion_mask).sum(dim=1) /
                   completion_mask.sum(dim=1))
```

6 Edoardo Cetin et al. "Reinforcement Learning Teachers of Test Time Scaling" (2025).

```
    loss = ((per_token_loss * completion_mask).sum(dim=1) /
            completion_mask.sum(dim=1))
    return loss, mean_kl
```

❶ Combine the prompt and generated completions.

❷ Get log probabilities from the current policy model.

❸ Compute KL divergence term (token-wise and closed form).

❹ Compute the policy gradient term as likelihood ratio * advantage, using `per_token_logps.detach()` as the baseline.

❺ Compute the final GRPO loss (negative reward-weighted log-prob, plus KL penalty).

❻ Mask and average over valid tokens only.

This implementation enables per-token control over policy updates and ensures that KL regularization is smoothly applied across the full sequence rather than as a scalar penalty. The token-level formulation corresponds to the following expression:

$$\mathcal{L}\text{GRPO} = -\sum t = 1^{T}\left[\exp(\Delta\log\pi_t)\cdot A - \beta\cdot\mathbb{D}_{KL}^{(t)}\right]$$

Here, $\Delta\log\pi_t = \log\pi_\theta(o_t) - \log\pi_\theta^{\text{stopgrad}}(o_t)$. The KL divergence $\mathbb{D}_{KL}^{(t)}$ is computed for each token and aggregated over the sequence, providing stability and precision without the overhead of a separate critic network.

GRPO is highly scalable and robust for reasoning tasks in LLMs. By combining structured initialization through cold-start SFT with critic-free, token-level policy optimization, DeepSeek-R1 establishes a stable training recipe. This approach promotes reliable multi-step reasoning, reduces reward hacking, and avoids the instability often associated with traditional actor-critic methods. This architecture initiated a broader trend in alignment research: moving away from monolithic reward models toward interpretable, modular, and structure-aware learning pipelines.

## Qwen3: Unified Reasoning with Dynamic Control

While DeepSeek emphasizes stable reward optimization, Qwen3 introduces several key innovations that enable flexible and efficient reasoning. One such feature is *dynamic mode switching*, allowing the model to alternate between reasoning-intensive and rapid-response modes based on user instructions. Example 8-2 shows how to do

this if you're using the Transformers library from Hugging Face. Note that the full code of all examples in this chapter is accessible in the book's repo.

*Example 8-2. Switching between thinking and non-thinking mode with Transformers*

```
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
    enable_thinking=True ❶
)
```

❶ Enable thinking, which is the default mode.

If you're using, for instance, vLLM for your model deployment, you need to add extra parameters to your chat completion in your OpenAI Python software development kit (SDK). How to achieve this is shown in Example 8-3.

*Example 8-3. Switching between thinking and non-thinking mode with vLLM*

```
chat_response = client.chat.completions.create(
    model="Qwen/Qwen3-8B",
    messages=[
        {"role": "user", "content":
        "What are reasoning models, in terms of Large Language Models?"},
    ],
    max_tokens=8192,
    temperature=0.7,
    top_p=0.8,
    presence_penalty=1.5,
    extra_body={
        "top_k": 20,
        "chat_template_kwargs": {"enable_thinking": False}, ❶
    },
)
```

❶ Disables thinking.

However, you can also "just tell" the model to not use thinking in the prompt.

Even when reasoning is disabled, the model preserves an internal `<think>` block structure to maintain format consistency. Another important feature is the *thinking budget*, which lets you specify a cap on the number of tokens allocated for the reasoning process. If this budget is exhausted mid-inference, the model gracefully truncates its thinking and continues with fallback responses, sometimes even stating, "Considering the limited time by the user, I have to give the solution based on the thinking directly now." Example 8-4 illustrates how to enforce the thinking budget.

*Example 8-4. Set thinking budget*

```
thinking_budget = 1024
max_new_tokens = 32768

# Rest of code is omitted for brevity

if 151645 not in output_ids:

    if 151668 not in output_ids: ❶
        print("thinking budget is reached")
        early_stopping_text = "\n\nConsidering the limited time by the user,
        I have to give the solution based on the thinking directly now.\n</think>\n\n"
        early_stopping_ids = tokenizer([early_stopping_text], return_tensors="pt",
        return_attention_mask=False).input_ids.to(model.device)
        input_ids = torch.cat([generated_ids, early_stopping_ids], dim=-1)
    else:
        input_ids = generated_ids
    attention_mask = torch.ones_like(input_ids, dtype=torch.int64)

    generated_ids = model.generate( ❷
        input_ids=input_ids,
        attention_mask=attention_mask,
        max_new_tokens=input_length + max_new_tokens - input_ids.size(-1) ❸
    )
    output_ids = generated_ids[0][input_length:].tolist()
```

❶ Check whether the thinking process has finished (`151668 is </think>`), then prepare the second model input.

❷ Run the generation.

❸ Handle potential negative results when `max_new_tokens` is too small (the early stopping text is 24 tokens).

**Right Amount of Thinking Budget**

In practice, you shouldn't set the `thinking_budget` too low, as this can significantly limit the model's ability to perform meaningful reasoning. I recommend adjusting it based on the level of latency that you or your end users are willing to tolerate. In general, setting it to a value higher than 1,024 tends to yield more substantial improvements in performance across a variety of tasks, particularly those requiring multi-step or complex reasoning.

Qwen3's training pipeline also includes a sophisticated multi-stage post-training process. This begins with a Long-CoT cold start, using a curated dataset of math, code, and logical reasoning problems to instill fundamental reasoning behavior. It's followed by a GRPO-based reinforcement learning phase focused on enhancing reasoning performance through large-scale, off-policy updates. The third stage involves SFT that fuses reasoning and non-reasoning capabilities into a unified model using structured chat templates. Finally, a fourth stage of general-purpose RL is applied to align the model with human preferences, formatting conventions, and agent behavior across a wide range of real-world scenarios.

To support deployment at different scales, Qwen3 also incorporates an advanced distillation framework. Smaller models, such as Qwen3-8B, are trained using both *off-policy distillation*, which transfers teacher-generated outputs, and *on-policy distillation*, where student models align their logits with those of a stronger teacher by minimizing divergence. This combination allows the distilled models to retain reasoning capabilities and mode-switching flexibility while reducing training cost and inference latency.

Qwen3-235B-A22B,[7] a *mixture of expert* (MoE) model with a total of 128 experts, 8 of which experts are activated during inference, demonstrates SOTA results across reasoning and general benchmarks. In "thinking mode," it surpasses DeepSeek-R1 in 17 out of 23 reasoning-heavy tasks.

7 An Yang et al. "Qwen3 Technical Report" (2025).

## Mixture of Experts for Efficient Reasoning

MoE models introduce conditional computation by activating only a small subset of specialized subnetworks (called *experts*) for each input. This allows models to scale to trillions of parameters while keeping inference costs comparable to much smaller dense models.

In transformer architectures, MoE layers typically replace the feed-forward block with a set of expert networks. Figure 8-2 illustrates a simplified overview of this.

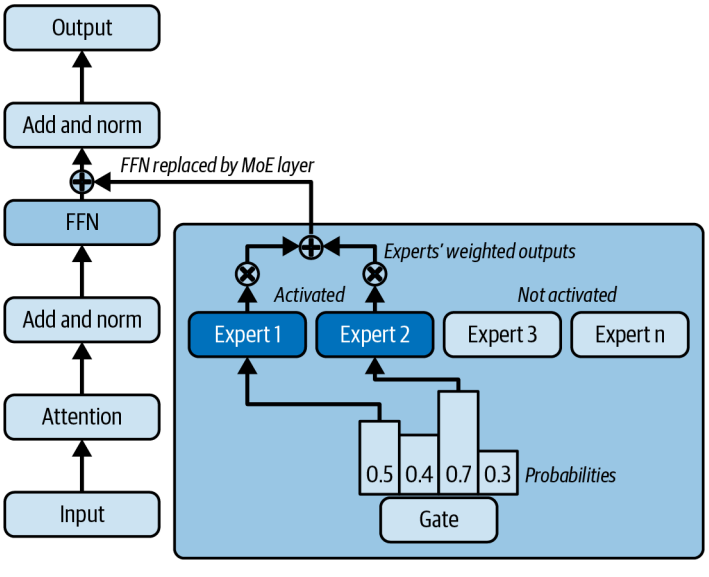


*Figure 8-2. High-level overview of the MoE architecture and where the experts are usually placed.*

A gating function, often a simple linear layer followed by top-k selection, determines which experts are used for each token. It assigns a probability distribution over the available experts and activates only the top-k with the highest scores. The outputs of these selected experts are then combined, often as a weighted sum based on their gating probabilities.

This mechanism enables specialization: different experts can learn to handle different types of reasoning patterns or task structures. It also facilitates multi-task learning, where diverse capabilities are needed but not all at once. By routing tokens to the most suitable experts, the model improves both efficiency and generalization.

These results illustrate how transformers, empowered by experience and interaction, can reason, plan, and generate new knowledge, moving beyond static data into a new era of intelligence.

## Qwen3-Coder: Agentic Reasoning for Open-Ended Coding

Qwen3-Coder is designed for advanced, open-ended programming tasks and multi-step agentic workflows. It's capable of software engineering and autonomous tool use. Qwen3-Coder-480B-A35B-Instruct is a 480B-parameter MoE model with 35B active parameters. Qwen3-Coder sets new benchmarks among open models across agentic coding, browser, and tool use, rivaling proprietary systems like Claude Sonnet 4 in its reasoning and interaction capabilities.

Alongside the model, the team open-sourced a command-line tool named Qwen Code, adapted from Gemini Code, with customized prompts and tool-calling protocols. This command-line interface (CLI) integrates seamlessly with modern developer tools and supports OpenAI-compatible APIs, enabling you to deploy agentic coding workflows using Qwen3-Coder with just a few configuration steps.

Qwen3-Coder's pretraining strategy is to build on a foundation for agentic reasoning. It's trained on 7.5 trillion tokens with a 70% code ratio, to ensure proficiency across coding, mathematics, and general language understanding. To accommodate increasingly complex development contexts, the model natively supports a context length of 256,000 tokens and can be extended up to 1 million using YaRN-based extrapolation methods (see "Longer Context Windows with Better Performance" on page 9). This makes it well suited for large-scale codebases and dynamic inputs such as pull requests.

Departing from the conventional focus on competitive programming benchmarks, Qwen3-Coder enables execution-driven reinforcement learning (Code RL)[8] to address real-world coding challenges. By automatically generating diverse test cases, the Qwen3 team expanded task coverage and created high-quality learning signals, which led to improvements in execution success rates. To further support long-horizon reasoning in complex environments like SWE-Bench,[9] Qwen3-Coder was post-trained using Agent RL.[10] This approach requires simulating multi-turn tool use, planning, and decision making. To make this feasible at scale, the team deployed a custom infrastructure capable of running 20,000 parallel environments, providing

8 Hung Le et al. "CodeRL: Mastering Code Generation Through Pretrained Models and Deep Reinforcement Learning" (2022).

9 Carlos E. Jimenez et al. "SWE-Bench: Can Language Models Resolve Real-World GitHub Issues?" (2024).

10 Xinji Mai et al. "Agent RL Scaling Law: Agent RL with Spontaneous Code Execution for Mathematical Problem Solving" (2025).

rich interaction feedback. This enabled Qwen3-Coder to achieve SOTA performance among open models on SWE-Bench Verified.

There are several ways to use Qwen3-Coder in your own projects. While cloud providers like Google Cloud (GCP) offer deployment options, this can add deployment complexity and can be costly as well as introduce latency challenges. For example, deploying the 480B Instruct model with vLLM on GCP using a 64k token context length requires 8 H200 GPUs. Running this setup 24/7 can easily exceed $60,000 per month, unless you implement autoscaling to reduce usage when idle. However, scaling to zero comes with trade-offs: reloading and redeploying the model can introduce startup delays.

One workaround is to keep the model weights stored in a bucket (such as GCS), so only the GPU runtime needs to spin up. Still, even this approach adds noticeable latency during startup. Because of these practical considerations, I recommend experimenting with the model first via hosted API providers such as Together AI or Novita AI, which abstract away the infrastructure complexity and allow you to evaluate performance before committing to full-scale deployment.

Example 8-5 shows how to use the model with an Open AI–compatible API and Novita AI.

*Example 8-5. Run Qwen3-Coder via Novita AI*

```
client = OpenAI(
    base_url="https://api.novita.ai/v3/openai",
    api_key = "<Your API Key>",
)

model = "qwen/qwen3-coder-480b-a35b-instruct"
stream = True # or False
max_tokens = 65536
system_content = "You are a helpful coding assistant"
temperature = 0.7 ❶
top_p = 0.8
min_p = 0
top_k = 20
presence_penalty = 0
frequency_penalty = 0
repetition_penalty = 1.05
response_format = { "type": "text" }

chat_completion_res = client.chat.completions.create(
    model=model,
    messages=[
        {
            "role": "system",
            "content": system_content,
        },
```

```
        {
            "role": "user",
            "content": "Write a quick sort algorithm.",
        }
    ],
    stream=stream,
    max_tokens=max_tokens,
    temperature=temperature,
    top_p=top_p,
    presence_penalty=presence_penalty,
    frequency_penalty=frequency_penalty,
    response_format=response_format,
    extra_body={
      "top_k": top_k,
      "repetition_penalty": repetition_penalty,
      "min_p": min_p
    }
  )

if stream:
    for chunk in chat_completion_res:
        print(chunk.choices[0].delta.content or "", end="")
else:
    print(chat_completion_res.choices[0].message.content)
```

❶ Recommended parameter setup for temperature, top k, and repetition penalty.

When I ran this code, the model successfully wrote, within seconds, a quick sort algorithm and explained its key features, including the in-place implementation, which is more memory efficient as it sorts the array without creating new lists. It also covered time and space complexity, which are important to consider. I'm sure you, like me, remember having to work through those calculations by hand during computer algorithm exams.

## Kimi K2: Open Agentic Intelligence at Scale

As a 1 trillion-parameter MoE model with 32 billion active parameters, Kimi K2 is designed to do more than answer; it's designed to act. It's optimized for knowledge-intensive tasks in math, coding, and reasoning as well as for dynamic and tool-augmented workflows. Kimi K2 achieves SOTA performance among non-thinking models across a wide spectrum of agentic benchmarks. From SWE-Bench and LiveCodeBench to Tau2 and AIME, which target different capabilities and levels of difficulty, Kimi K2 consistently delivers results that rival or surpass proprietary models, without relying on extended thinking or hidden compute boosts. That said, K2 isn't an explicit reasoning model with a dedicated thinking mode.

The Kimi K2 model family is open sourced in two forms. The Kimi-K2-Base variant provides a foundation model for researchers and developers interested in fine-tuning

and customization. In contrast, Kimi-K2-Instruct is a ready-to-deploy model, suitable for general-purpose conversational and agentic applications.

Kimi K2 is engineered to operate in complex environments with minimal human intervention. For instance, in agentic data analysis tasks, it can autonomously filter datasets, visualize distributions, perform statistical tests, and generate polished HTML dashboards, all using natural language commands and multi-step tool execution.

This kind of automation is made possible through large-scale agentic data synthesis, modeled after real-world scenarios. Inspired by ACEBench,[11] a tool-usage benchmark, the developers of Kimi K2 created a vast simulation pipeline that involves hundreds of domains populated with thousands of tools. These environments are designed to support diverse multi-turn interaction between Kimi K2 and simulated user agents, resulting in realistic tool-use dialogues. A rubric-guided LLM judge filters the resulting interactions to curate high-quality data, which then fuels training via rejection sampling and RL.

Kimi K2's training pipeline goes beyond verifiable environments like math or programming competitions. It incorporates a general RL mechanism capable of scaling to tasks where success criteria may be ambiguous or open-ended, such as writing reports or strategic planning. In these cases, Kimi K2 serves as its own critic, using rubric-based evaluation to assess its outputs. To refine this self-judging mechanism, the system simultaneously runs on-policy rollouts in verifiable settings, ensuring that even subjective evaluations remain anchored in measurable standards.

## Muon: Scaling Optimization for the Agentic Era

The performance of Kimi K2 isn't based only on training data but also on an optimized foundation: the Muon optimizer. Muon is a norm-aware optimizer that interprets training as steepest descent under a spectral norm constraint.[12] *Steepest descent under the spectral norm* refers to an optimization strategy where the update direction for a weight matrix is determined by minimizing a local quadratic model of the loss function, and the "size" of the update is measured using the spectral norm. This means the optimizer prioritizes updates that are "aligned" with the principal components of the gradient, effectively making semi-orthogonal updates to the weight matrices. The weight matrices are updated by applying orthogonalized gradients via Newton-Schulz iterations. *Newton-Schulz iteration* is a matrix adaptation of Newton's method that iteratively improves estimates of a matrix inverse using first-order approximations, much like Newton's method is an iterative algorithm that refines

11 Chen Chen et al. "ACEBench: Who Wins the Match Point in Tool Usage?" (2025).

12 Jeremy Bernstein et al. "Old Optimizer, New Norm: An Anthology" (2024).

estimates of a function's root by using the local slope (its derivative). This effectively suppresses update collapse and encourages broader exploration of parameter space. In addition, this directly counters one of the persistent inefficiencies in AdamW as an optimizer: overfitting to dominant singular directions in the gradient space.

The Kimi K2 team scaled Muon to multi-trillion-token training regimes across multi-billion-parameter LLMs, including MoE architectures. Kimi K2 was trained on 15.5 trillion tokens using an evolved variant of Muon called Distributed Muon.[13] This optimizer builds on the original Muon by addressing a critical challenge: exploding attention logits during pretraining. Two critical techniques underpin this scalability:

- Standard AdamW-style weight decay ($W_t = W_{t-1} - \eta_t(O_t + \lambda W_{t-1})$) prevents excessive growth of model weights and the root mean square (RMS) of layer outputs, which would otherwise lead to performance degradation and numerical instability, especially under bf16 precision.
- Update scale adjustment ensures consistent update magnitude across layers of different matrix shapes. Muon's inherent update RMS varies with shape ($\sqrt{1/\max(A,B)}$), which previously led to suboptimal updates: too small for large matrices and too large for small ones. To normalize this, updates are proportionally scaled by $\sqrt{\max(A,B)}$. Additionally, Muon's update RMS is matched to AdamW's empirical range (0.2–0.4) and fixed at 0.2 for compatibility (e.g., $W_t = W_{t-1} - \eta_t(0.2 \cdot O_t \cdot \sqrt{\max(A,B)} + \lambda W_{t-1})$).

This stabilizes attention dynamics at the source without introducing additional layers or inference-time overhead. In addition, Distributed Muon preserves token efficiency throughout training. Scaling law analyses reveal that Muon can achieve the same loss with nearly 50% fewer FLOPs compared to AdamW.

## Inference with Kimi K2

Although Kimi K2 is a MoE model, the unquantized version still requires approximately 1.09 TB of disk space. While you can try the model at Kimi.com, this option doesn't support use in your own projects. Similar to Qwen3-Coder, deploying the model yourself requires a substantial number of GPUs.

As before, I recommend starting with a hosted API provider. To do this, you simply need to change the model name from Example 8-5 to `moonshotai/kimi-k2-instruct`, and everything else remains the same, though you might want to test different parameters for the sampling and decoding methods.

13 Jingyuan Liu et al. "Muon Is Scalable for LLM Training" (2025).

**Control Language Diversity**

*Sampling* and *decoding strategies* shape the trade-off between determinism and diversity in language or code generation. *Temperature* controls randomness by scaling logits before applying Softmax, with higher values leading to more diverse outputs. *Top-k sampling* selects from the k most probable tokens, while *top-p (nucleus) sampling* considers the smallest set of tokens whose cumulative probability exceeds a threshold p. Each method offers a different way to balance fluency and creativity.

Like Qwen3-Coder, Kimi K2 also provides an in-place sorting implementation. However, it outputs only the time complexity, which matches the algorithm suggested by Qwen3-Coder. To find out which hosted API provider supports the model you're looking for, check Hugging Face's model support matrix, which provides a convenient overview of inference providers for different open source models. Alternatively, Unsloth, an AI startup focusing on making LLM training and inferences faster, provides a compressed, 1.8-bit version, which reduces the model to just 245 GB (–80% size), and instructions for using Kimi K2 locally.

## Scaling Reasoning at Test-Time: Smarter, Not Just Bigger

Training ever-larger language models is not only expensive but also unsustainable for many practical applications. Yet, reasoning quality doesn't solely depend on model size. Recent advances show that even smaller models can deliver strong performance, if we give them more room to think at test time. Rather than increasing parameter count, we can increase compute during inference to stipulate deeper reasoning. This includes sampling multiple reasoning paths, reflecting on intermediate steps, or refining outputs post hoc. These methods offer a powerful alternative: scaling thinking time instead of model size while still getting advanced reasoning capabilities.

To support this shift, PRMs play an increasingly important role, now not just during training but as implicit evaluators at inference time. Instead of optimizing for a single best final output, you evaluate reasoning trajectories step by step, much like how a human might review a line of argument or lines of code. With PRMs guiding the evaluation of intermediate steps, these strategies allow models to explore, compare, and refine candidate solutions at inference, bringing structured decision making into the decoding loop. In this way, smaller models, equipped with smart inference-time control, can rival or even surpass larger counterparts on reasoning-heavy tasks. This is helpful for deploying LLMs on resource-constrained devices or reducing inference costs.

To scale test-time compute, tree search algorithms are mostly used during the inference phase of LLMs. With that, language models can enhance their problem-solving

capabilities by systematically exploring a vast space of possible answers. This process involves the LLM generating candidate outputs, which are then evaluated, and the search is guided by these evaluations. The primary goal is to find the highest-scoring solution within a given computational budget, typically measured by the number of LLM calls.

Monte Carlo Tree Search (MCTS) is a prominent algorithm for decision making in complex domains, including games and, more recently, LLM-based problem solving. At its core, MCTS builds a search tree through four iterative steps. It begins with selection, where the algorithm traverses the tree from the root, choosing nodes according to a policy that balances exploration and exploitation. A common algorithm is the Upper Confidence Bound for Trees (UCT), which scores each node based on both its estimated value and how often it has been visited. This leads to the expansion step, where new child nodes are created from a promising leaf, representing candidate answers or refinements. From there, a simulation or rollout is performed, in which the model generates a complete solution from the newly expanded node and evaluates it, often with the help of an external feedback mechanism such as a preference or reward model. Finally, the backpropagation step updates all ancestor nodes with the score obtained, allowing the tree to accumulate better estimates over time. Figure 8-3 illustrates this process.

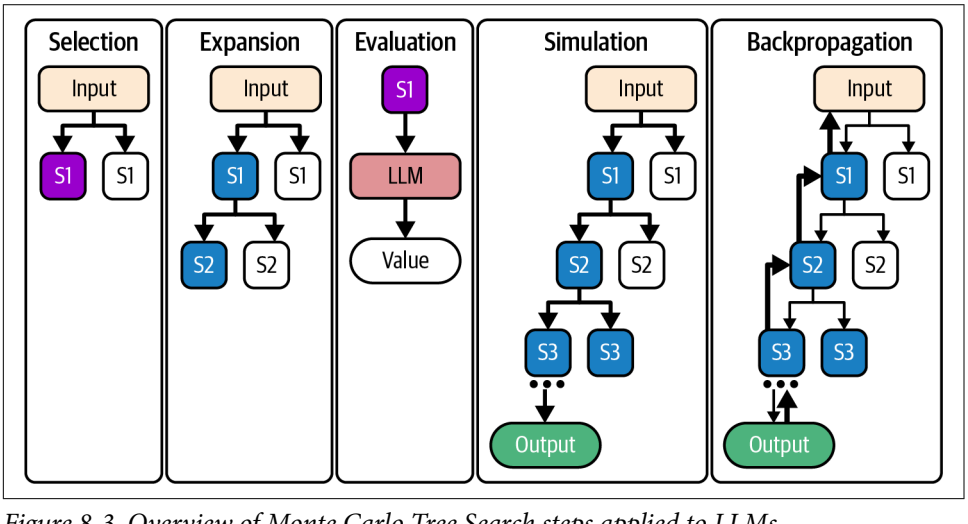


*Figure 8-3. Overview of Monte Carlo Tree Search steps applied to LLMs.*

The node selected for expansion in the next iteration is typically the one with the highest UCT score, defined as:

$$\text{UCT}(s) = V(s) + c\sqrt{\frac{\ln N(p)}{N(s)}}$$

Here $V(s)$ is the value estimate of node $s$, $N(s)$ is the visit count of node $s$, $N(p)$ is the visit count of the parent node, and $c$ is a constant controlling the exploration-exploitation trade-off.

At test time, the compute allocated for tree search is spent iteratively performing these steps. The language model is called to generate or refine answers, and a scoring function evaluates their quality. This setup enables the system to explore a broader space of solutions and converge on higher-quality outputs than single-pass decoding alone does.

## Adaptive Branching Monte Carlo Tree Search (AB-MCTS)

Before looking into specialized tree-based algorithms, it's helpful to understand two simpler strategies. As illustrated in Figure 8-4, in repeated sampling you draw multiple responses from a model given the same task prompt. This approach only goes wide—that is, sampling across different possibilities without revisiting or improving any of them—making it a case of pure exploration. In contrast, in sequential refinement you start with an initial response and improve it iteratively. This algorithm goes deep, focusing all computation on refining a single path, and thus is just pure exploitation.

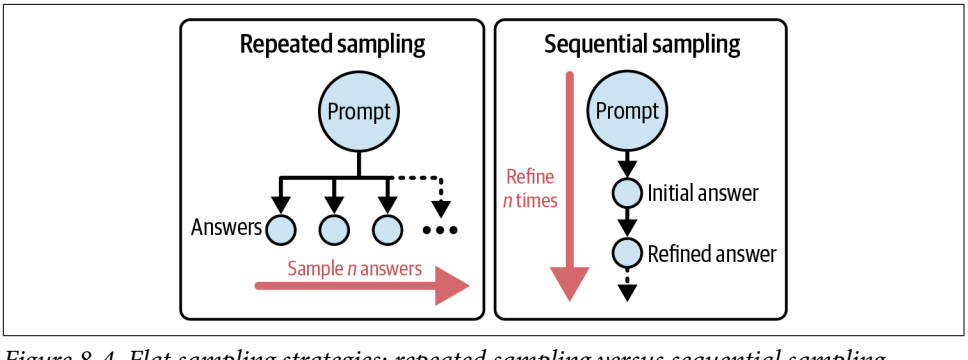


*Figure 8-4. Flat sampling strategies: repeated sampling versus sequential sampling.*

Adaptive Branching Monte Carlo Tree Search (AB-MCTS)[14] is a framework that extends traditional MCTS to better suit the characteristics of LLM inference. Unlike standard MCTS, which typically uses a *fixed branching factor* (a predetermined number of child nodes generated at each expansion), AB-MCTS dynamically decides whether to "go wider" (explore new candidate responses) or "go deeper" (revisit and refine existing responses) based on external feedback signals. This paradigm is also called the *exploration-exploitation trade-off* in RL-based algorithms.

This adaptive nature is especially important because LLMs, when prompted with a nonzero temperature, can produce an unbounded number of diverse outputs from the same prompt. Therefore, a fixed branching factor, as used in standard MCTS, can limit the potential for discovering high-quality solutions. Figure 8-5 compares standard MCTS and AB-MCTS. AB-MCTS aims to overcome the limitations of fixed-width trees by enabling dynamic branching decisions during inference.

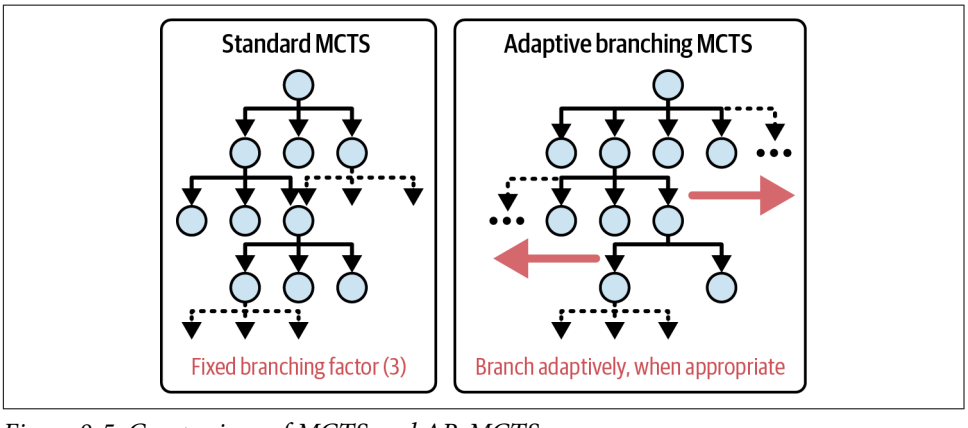


*Figure 8-5. Comparison of MCTS and AB-MCTS.*

AB-MCTS addresses this by introducing a GEN node. Every node in the search tree has a GEN node as a child. When this GEN node is selected during the search, it signals the generation of a new child node, effectively allowing for unbounded branching. The decision of whether to select a GEN node or an existing child node for further exploration is guided by *Thompson sampling*, a Bayesian approach that balances exploration and exploitation.

14 Yuichi Inoue et al. "Wider or Deeper? Scaling LLM Inference-Time Compute with Adaptive Branching Tree Search" (2025).

The framework enables two main variants. The first, AB-MCTS-M (mixed model), uses a node-specific Bayesian model to estimate scores for both GEN nodes and existing children. It uses observed scores from subtrees to form posterior predictive distributions and shares statistical strength across different parts of the tree. This allows the model to infer a likely reward distribution even when limited or no observations exist for a given node. As a result, decisions about when to explore or exploit are guided by a principled estimate of uncertainty. An example tree structure of this method is illustrated in Figure 8-6.

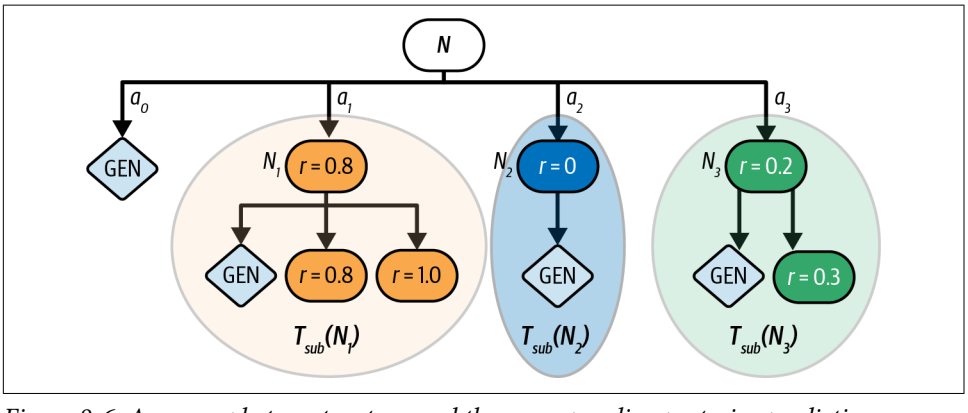


*Figure 8-6. An example tree structure and the corresponding posterior predictive score distributions for AB-MCTS using mixed models (AB-MCTS-M). In this case, action* $a_1$ *expands into child nodes that exhibit higher predicted scores. Image adapted from Yuichi Inoue (2025).*

The second variant, AB-MCTS-A (node aggregation), is computationally more lightweight and closer in form to standard UCT-based MCTS. It adds a CONT node under each answer node, representing the option to continue refining an existing answer, alongside the GEN node for generating new candidates. Score propagation is simplified: updates flow back through the GEN node and its ancestors but not to other GEN nodes, which limits computational complexity. AB-MCTS-A also includes two subtypes: one using a Gaussian prior with a normal-inverse-$\chi^2$ distribution for unbounded score settings, and one using a Beta prior when scores are normalized to the [0, 1] range. An example of how this tree could unfold is shown in Figure 8-7.

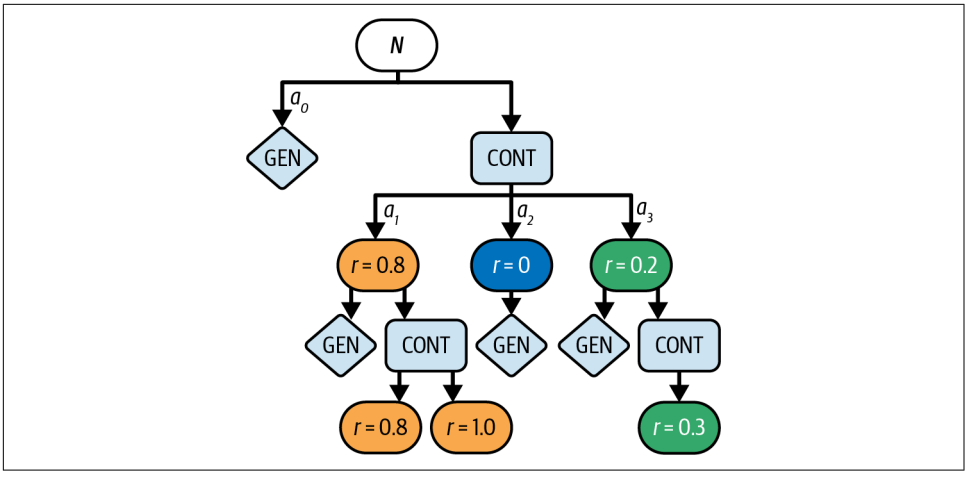


*Figure 8-7. Illustrative tree structure for AB-MCTS-A. All child nodes are grouped under a CONT node, while a GEN node doesn't have any children. Image adapted from Yuichi Inoue (2025).*

Both AB-MCTS-M and AB-MCTS-A variants use Bayesian updates to guide search decisions and are deployed at inference time by repeatedly querying the LLM, evaluating the responses, and updating the search tree accordingly. This allows the algorithm to allocate computational resources more intelligently, broadening exploration when necessary while also deepening promising solution paths. This makes AB-MCTS well suited for reasoning-intensive tasks, where the solution space is both vast and unevenly structured.

Let me walk you through a simple example using the TreeQuest library, a tree search library featuring AB-MCTS. You begin by defining a simple data class (Example 8-6) to hold each state of the search.

*Example 8-6. MCTS-based answer refinement with TreeQuest*

```
@dataclass
class State: ❶
    llm_answer: str
    score: float
```

❶ Define the state; the state captures each LLM-generated answer and its evaluation score.

Next, you define the initial generation step (Example 8-7). This produces the first candidate answer from the LLM.

*Example 8-7. Initial generation from the LLM*

```
def initial_generation() -> State:
    prompt = "Q: Write code in Python for the Fibonacci sequence. \nA:"
    response = client.chat.completions.create(
        model="gpt-4o",  # Must be a JSON-mode-supported model
        messages=[{"role": "user", "content": prompt}],
        temperature=0.7,
    )
    answer = response.choices[0].message.content.strip()
    score = evaluate_answer(answer)
    return State(llm_answer=answer, score=score)
```

After generating an initial answer, you define a way to improve upon it through refinement. The implementation is shown in Example 8-8.

*Example 8-8. Refine an existing answer*

```
def refine_answer(llm_answer: str, score: float) -> State:
    prompt = f"""The current answer is:
    \n\n{llm_answer}\n\nPlease improve this answer to be more informative,
    accurate, and clear."""
    response = client.chat.completions.create(
        model="gpt-4o",  # Must be a JSON-mode-supported model
        messages=[{"role": "user", "content": prompt}],
        temperature=0.7,
    )
    refined = response.choices[0].message.content.strip()
    score = evaluate_answer(refined)
    return State(llm_answer=refined, score=score)
```

Both generation steps require a scoring function. You can use the LLM itself to self-evaluate its output, as shown in Example 8-9.

*Example 8-9. Score each generated answer*

```
class ScoreResponse(BaseModel):
    score: float = Field(..., ge=0.0, le=1.0)

def evaluate_answer(answer: str) -> float:
    prompt = (
        f"Evaluate the quality of this answer on a scale from 0 to 1.\n"
        f"Return a JSON object like .\n\n"
        f"Answer:\n{answer}"
    )

    try:
        completion = client.chat.completions.parse(
            model="gpt-4o",
            messages=[{"role": "user", "content": prompt}],
```

```
            response_format=ScoreResponse,
        )
        return completion.choices[0].message.parsed.score
    except Exception as e:
        print(f"[Evaluation error] {e}")
        return 0.5
```

This scoring function ensures that the model can compare outputs objectively during tree search. Now you define the interface used by the tree search algorithm to generate child nodes, as implemented in Example 8-10.

*Example 8-10. Generation logic for TreeQuest*

```
def generate(parent_state: State | None) -> tuple[State, float]:
    if parent_state is None:
        return initial_generation(), initial_generation().score
    return refine_answer(parent_state.llm_answer,
                         parent_state.score), parent_state.score
```

This function determines what new state to generate next, either from scratch or by refining an existing one. Now you run the actual MCTS loop, as shown in Example 8-11.

*Example 8-11. Run MCTS for LLM refinement*

```
# TreeQuest loop
algo = tq.StandardMCTS()
search_tree = algo.init_tree()

for i in range(5):
    search_tree = algo.step(search_tree, {'LLM-Refine': generate})
    if (i + 1) % 5 == 0:
        best, _ = tq.top_k(search_tree, algo, k=1)[0]
        print(f"[Step {i+1}] Best so far:
        {best.llm_answer} (score={best.score:.2f})")

best_state, _ = tq.top_k(search_tree, algo, k=1)[0]
```

Finally, you can retrieve the best answer found during the search and display it, as shown in Example 8-12.

*Example 8-12. Print final best answer*

```
print(f"\n Best Answer: {best_state.llm_answer} (score={best_state.score:.2f})")
```

This full loop demonstrates how tree search can be applied to navigate and refine LLM outputs, balancing between new generations and focused improvement of promising candidates. LLMs are increasingly being used for code generation, a

complex domain that presents unique challenges compared to other tasks like mathematical or textual reasonings. These challenges are the focus of the following sections and methods.

## The RethinkMCTS Framework for Code Generation

RethinkMCTS is a thought-level MCTS framework that improves code generation by integrating execution feedback into the reasoning process. Unlike prior approaches that operate at the token or code level, RethinkMCTS performs MCTS over chains of thought, allowing it to explore diverse reasoning strategies before generating code. A central feature is its rethink operation, which uses fine-grained feedback from block-level code analysis to refine erroneous thoughts during the search.

The framework approaches code generation in two stages. First, it performs a thought search phase guided by P-UCB (Policy Upper Confidence Bound) scores, which extend the Upper Confidence Bound (UCB) strategy by incorporating both model likelihood and execution-based value estimates to balance exploitation and exploration. Then, it enters a code generation phase where accumulated thoughts are translated into executable programs. During execution, scalar rewards and verbal feedback are collected, with the feedback derived from variable traces and control flow analysis. If the generated code fails, the rethink mechanism applies this feedback to revise the current thought, allowing the search to continue along a corrected path without restarting the full reasoning trace.

To improve selection quality, RethinkMCTS combines results from public test cases with LLM-based self-evaluation. This approach reduces the risk of overfitting to known tests and supports more robust generalization.

Results on HumanEval and APPS[15] benchmarks show significant improvements over both search-based (such as Tree of Thoughts)[16] and feedback-based methods (like Reflexion).[17] For instance, GPT-3.5-turbo's pass@1 on HumanEval rises from 70.12 to 89.02 with RethinkMCTS. Ablation studies, which measure the effect of removing or altering specific components of a system, confirm that verbal feedback and the rethink operations are key drivers of this performance gain.

By explicitly linking reasoning and execution, RethinkMCTS increases the quality of thought-level search, offering a general mechanism to correct logical errors mid-search. This helps you to bridge the gap between symbolic planning and grounded code synthesis. To try RethinkMCTS on a benchmark like HumanEval, you can simply run the command:

15 Dan Hendrycks et al. "Measuring Coding Challenge Competence With APPS" (2021).

16 Shunyu Yao et al. "Tree of Thoughts: Deliberate Problem Solving with Large Language Models" (2023).

17 Noah Shinn et al. "Reflexion: Language Agents with Verbal Reinforcement Learning" (2023).

```
python run.py --rollout 16 --start 4000 --end 4001 --arch gpt4o-mini --
model rethinkmcts --dataset humaneval
```

This command launches an evaluation run. The `--rollout 16` argument sets the number of MCTS rollouts per task, allowing deeper exploration. `--start 4000 --end 4001` specifies the task ID range (in this case, a single task). The `--arch gpt4o-mini` selects the backbone model architecture, and `--model rethinkmcts` activates the full feedback-based thought refinement system. Finally, `--dataset humaneval` indicates that the evaluation should be performed on the HumanEval benchmark.

However, make sure to use my corrected repository instead of the original paper's version, as I encountered some bugs in their implementation. You can also find a notebook with all the necessary dependencies and instructions for running the algorithm in the book's repo.

## The *S** Framework for Code Generation

The *S**[18] framework uses a hybrid test-time scaling framework designed to enhance both the coverage and selection accuracy of generated code. *S** operates as a two-stage hybrid framework: a generation stage and a selection stage. This design extends existing parallel scaling paradigms by incorporating sequential scaling and introduces a selection mechanism grounded in execution feedback. The core idea is to move beyond simple repeated sampling by integrating iterative refinement and intelligent selection.

The generation stage focuses on improving the "coverage" of solutions, meaning the fraction of problems for which at least one correct sample is generated. *S** achieves this by augmenting traditional parallel sampling with sequential scaling through iterative debugging. Initially, multiple code solutions are generated in parallel. Each of these initial samples then undergoes a refinement process involving multiple rounds of "iterative debugging." In this process, the generated code is executed against public test cases. The outputs or error messages from these executions are then fed back to the LLM to guide further refinement of the code. This iterative debugging continues until the code passes all public tests or a maximum number of revision attempts is reached. This approach uses the programmatic nature of code to provide concrete, execution-grounded feedback, a distinct advantage over other domains where such direct verification might be difficult.

Following the generation of candidate solutions, the selection stage is tasked with identifying the single best solution. This is a critical step, as public tests alone, already

18 Dacheng Li et al. "*S**: Test Time Scaling for Code Generation" (2025).

used during the generation phase, may not be sufficient for robust selection. The pseudocode of the *S** framework is shown in Example 8-13.

*Example 8-13. Best sample selection in S**

```
# Algorithm 1: Best Sample Selection in S*
# Input:  Problem description: P
# Input:  Candidate samples: X
# Output: The best selected sample: x*

T = llm_test_input_gen(P)
O = sample_execution(X, T)
C = sample_clustering(O)
Scores = defaultdict(int)

for Ci, Cj in all_cluster_pairs(C):
    xi, xj = sample_pair(Ci, Cj)
    Ta = adaptive_input_gen(xi, xj)
    better_idx = exec_and_llm_select(xi, xj, Ta)
    Scores[better_idx] += 1

C_star = max(Scores, key=Scores.get)
x_star = random_pick(C_star)

return x_star
```

Traditional methods, such as relying on an LLM as a judge or generating indiscriminate test cases, often fall short because LLMs can struggle to accurately predict program behavior or generate high-quality distinguishing inputs. To overcome these limitations, *S** introduces *adaptive input synthesis*. This mechanism prompts an LLM to specifically generate "distinguishing inputs" for pairwise comparisons between candidate code samples. These generated inputs are then executed, and the actual execution results are used to inform the LLM's decision on which sample is superior. This adaptive, execution-grounded approach ensures a more reliable and accurate selection of the correct solution by directly observing how candidate programs behave on inputs designed to highlight their differences.

*S** improves models of different sizes, types (instruction based versus reasoning), and access (open versus closed). It can enable smaller models to surpass the performance of larger models within the same family. For example, a 3B model augmented with *S** has been observed to outperform GPT-4o-mini. Furthermore, *S** can boost non-reasoning models to outperform reasoning models. Even SOTA reasoning models experience performance gains when integrated with *S**, achieving results comparable to top-tier closed models. Code is available via GitHub.

# Conclusion

This chapter marked a turning point in your journey through transformers, moving from passive pattern recognition toward active, experience-driven intelligence. You explored how large language models are being reimagined, not just to understand and respond but to act, plan, reason, and evolve through interaction.

You began by understanding that rather than relying solely on static human-curated inputs, transformers now learn from their own lived experience. Reinforcement learning is the key mechanism enabling this shift, empowering models to explore, reflect, and refine their reasoning strategies. Through examples like DeepSeek-R1 and Qwen3, you saw how structured training pipelines and PRMs help develop multi-step reasoning, CoT capabilities, and alignment with verifiable objectives.

You also discovered how inference itself is being transformed. Rather than relying solely on larger models, recent work emphasizes smarter decoding strategies, such as thinking budgets, mode switching, and tree search methods like MCTS and AB-MCTS. These innovations allow models to scale their cognitive effort at runtime, making deeper reasoning accessible without always increasing model size and inference costs.

Adding to this, the RethinkMCTS framework brought the idea of thought-level planning and correction in code generation. By separating reasoning from code emission and integrating verbal feedback based on execution traces, RethinkMCTS shows how agents can reflect on failed reasoning and revise mid-search—mirroring human trial-and-error problem solving. RethinkMCTS's performance gains on benchmarks like HumanEval demonstrate that it's not just reasoning ability that matters but the capacity to learn from one's own mistakes in context. This aligns perfectly with the broader shift toward adaptive, inference-time intelligence.

Finally, you explored how these principles come together in models specialized for coding. Whether through execution-driven feedback or test-time optimization with frameworks like $S^*$, you now know that reasoning is no longer just a training artifact; it's an active, adaptive process.

This sets the stage for Chapter 9, where you'll learn about AI agents: systems that combine memory, tools, planning, and communication to operate autonomously in complex environments. Everything you've seen here—interaction, reflection, and learning through doing—forms the foundation of what makes these agents possible.

CHAPTER 9

# From Scripts to Thinking: AI Agents for Complex Tasks

Traditional workflows are like well-rehearsed plays: the sequence of actions is fixed, the actors stick to their lines, and the ending is predetermined. Agentic systems break this pattern. Agents, by contrast, are improvisers. They can decide which scene to play next, change their lines on the fly, and even invite new actors onto the stage. In improv, there is the "Yes, and…" rule: accept what is given and add to it so the story moves forward. In AI terms, it becomes: Yes → accept the current state, context, or contribution from another agent or tool without discarding it; and → build on it by adding reasoning, actions, or new information that moves the task forward. Instead of marching in a straight and inflexible line, agentic systems loop through cycles of planning, action, and self-correction. This shift from predefined scripts to adaptive, collaborative decision making lies at the heart of autonomy in AI agents. Table 9-1 compares both workflows.

*Table 9-1. Comparison of traditional and agentic workflow*

| Traditional workflow | Agentic system |
|---|---|
| Predefined sequence | Dynamic decision loop |
| Limited flexibility | Context-aware, tool-using |
| Linear execution | Iterative, self-correcting |
| No adaptation to feedback | Adapts plans based on feedback |
| Fixed toolset | Selects and combines tools as needed |
| Single-pass completion | Can revisit and refine earlier steps |
| Static goal interpretation | Can reinterpret goals based on evolving context |

In the following sections, you'll explore what autonomy means today, where its limits lie, and how to use frameworks like LangGraph and SWE-agent to design agents that plan, act, and improve their performance in real time. You'll also see why human-in-the-loop oversight is an essential safeguard in agentic workflows, ensuring alignment and trust, and how context engineering can dynamically provide agents with the right information at the right moment.

## Autonomy: What's Possible at the Moment?

Today's agentic systems are far from the fully general, self-directed intelligence of science fiction, but they already demonstrate practical forms of autonomy that can be deployed in real-world workflows. Agentic workflows combine LLMs with orchestration logic and tools, enabling agents to operate in modular, state-driven workflows rather than rigid scripts. They can select tools, route tasks conditionally ("if reflection fails, call retry agent"), and adapt their next step based on intermediate results. While still dependent on their prompts, available tools, and framework constraints, such agents exhibit a growing ability to make context-aware choices, self-correct, and collaborate with other agents.

However, this autonomy is "orchestrated" rather than free-form: the control flow is defined in advance, but within that structure the agent can choose among multiple paths, invoke different tools, revise its own prompt or toolset, or even write and run code to determine its next steps. For example, Retrieval-Augmented Generation (RAG) always retrieves documents before calling the LLM, but agentic architectures allow the LLM to decide the control flow itself. This means an agent can:

- Route between potential execution paths.
- Decide which of many tools to call.
- Determine whether its output is sufficient or more work is needed.

Moreover, different architectures give the LLM different levels of control. At the simplest end are routers, which make a single decision from predefined options. More advanced designs, like tool-calling agents, support multi-step decision making, memory, reflection, and planning. Tool calling allows the agent to interact with external systems via defined schemas, memory lets it retain and reuse information across steps or sessions, and planning enables the LLM to iteratively decide the next best action until a goal is achieved. Frameworks like LangGraph extend this further with support for subgraphs, human-in-the-loop intervention, reflection, and parallelization, enabling modular, specialized, and collaborative workflows.

Today's systems typically fall closer to the router end of the spectrum, where the LLM makes limited decisions such as which tool to call or which branch of the workflow to follow within a narrow set of predefined options. Fully autonomous agents, capable of generating, executing, and adapting multi-step plans without fixed orchestration, remain a research goal rather than a production reality. Figure 9-1 illustrates this range of control, from minimal routing autonomy to complete self-directed execution. Most real-world implementations therefore fall under the category of agentic systems, not fully autonomous agents. The autonomy is orchestrated, not self-evolving, with decisions made inside a framework of predefined states, transitions, and toolsets, even if the agent can adaptively choose between them at runtime.

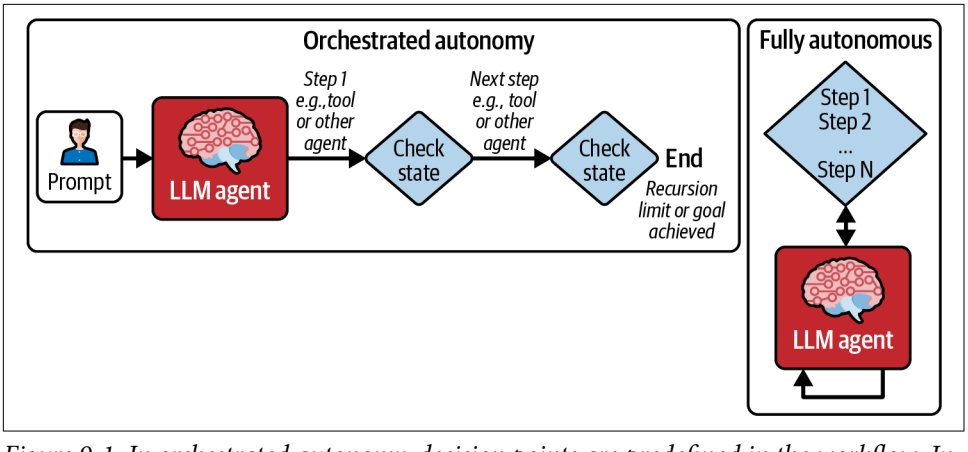


*Figure 9-1. In orchestrated autonomy, decision points are predefined in the workflow. In full autonomy, the LLM can create new tools, actions, or decisions to achieve its goal.*

The expression of autonomy depends heavily on the architecture used. Some systems center on a single LLM acting as an agent, while others adopt a *multi-agent system* (MAS) approach, where multiple specialized agents collaborate toward a shared goal. These architectures differ not just in structure but also in decision-making style, adaptability, and depth of expertise. Table 9-2 compares both architecture types, highlighting their differences in these areas.

*Table 9-2. Comparison of LLM-based single-agent systems and multi-agent systems*

| LLM-based single-agent system | Multi-agent system (MAS) |
|---|---|
| Centralized decision making | Decentralized, distributed decision making |
| General-purpose language capabilities | Specialized agents with defined roles |
| Reactive to prompts | Proactive, predictive, and anticipatory actions |
| Processes tasks in isolation | Collaborates and coordinates with other agents |
| Static unless externally adapted | Real-time adaptation to feedback and environment |
| Single reasoning process | Multiple reasoning processes running in parallel |
| Broad but shallow domain coverage | Deep expertise within specific domains |
| Single shared context window | Separate context windows and even distinct memory systems for each agent, allowing tailored knowledge and recall |

Whether orchestrated or collaborative, these agentic systems ultimately rely on their underlying architecture to define how decisions are made, how information flows, and how tasks are coordinated. The choice of architecture shapes not only the agent's capabilities but also its flexibility, fault tolerance, and ability to scale.

# Designing Agent Workflows

Designing an agent workflow is fundamentally about defining the environment in which an agent can make context-aware choices while maintaining control, reliability, and safety. In the scripted pipelines of traditional automation, this meant specifying a fixed series of steps from input to output. In an agentic system, the task is more about shaping a framework in which the agent can plan, act, and adapt, while the orchestration logic constrains its behavior to remain aligned with the overall goal.

The simplest form of such a workflow centers on a single LLM acting as the decision engine, supported by a memory module, a planning module, and access to tools for specific tasks. The baseline architecture in Figure 9-2 shows how the agent core coordinates these elements: the user request is received, planning and memory modules contribute context, and the appropriate tools are called to produce the final output.

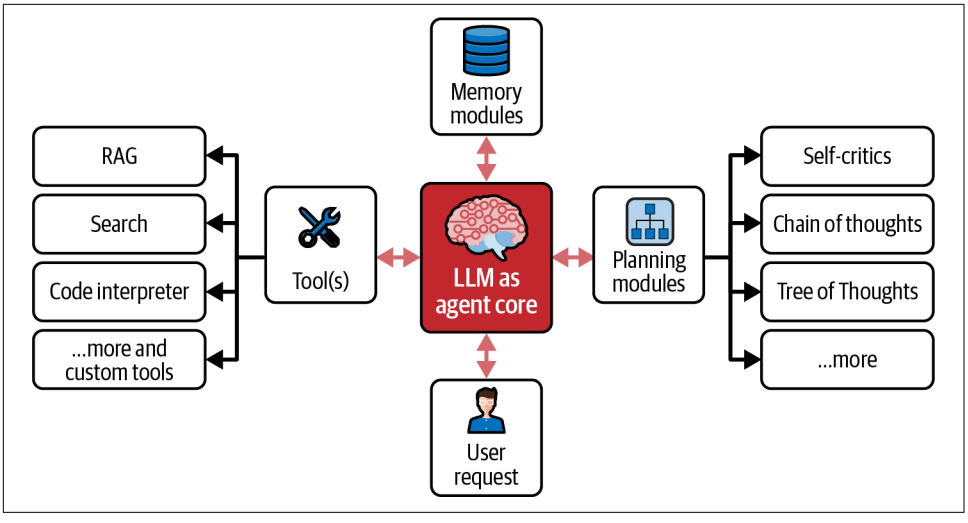


*Figure 9-2. In an LLM-powered agent, the LLM acts as the brain, supported by components that process information and provide explanations.*

Even within this basic structure, the way the agent sequences its actions can vary. A single-step agent takes the user's request, decides on all required actions at once, executes them, and returns the result. A multi-step agent, in contrast, follows an iterative loop such as the ReAct[1] pattern: it alternates between generating a reasoning trace, deciding on the next action, executing it, and updating its memory with new observations. This allows the agent to refine its plan dynamically, responding to partial results or new information as the task unfolds. The difference is illustrated in Figure 9-3.

1 Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models" (2023).

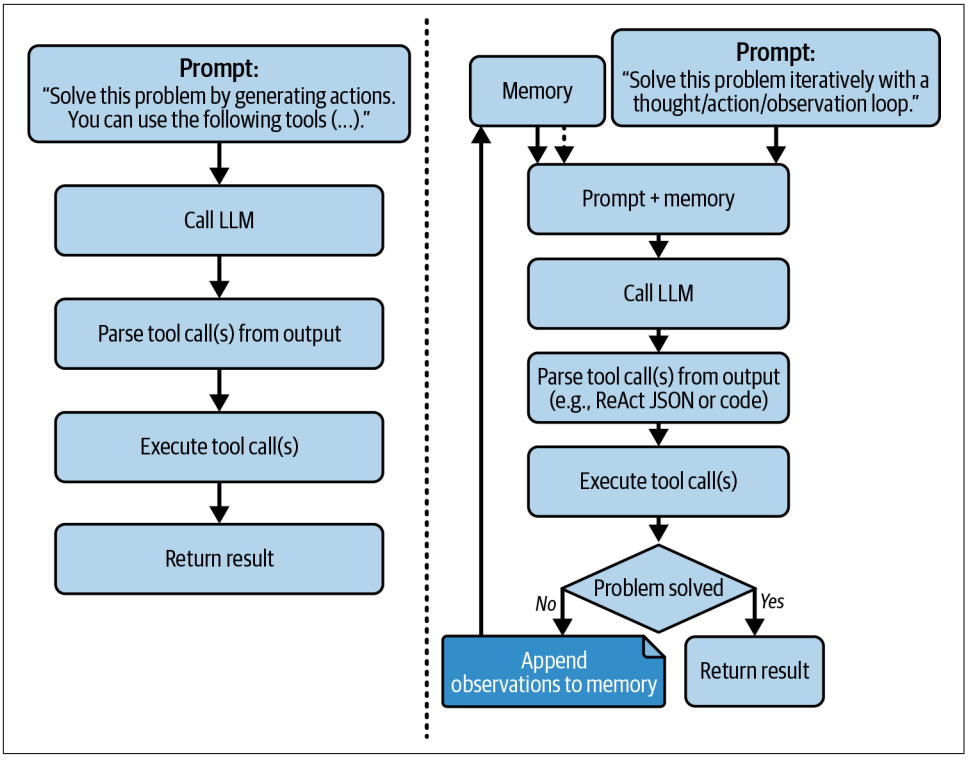


*Figure 9-3. Single-step (left) versus ReAct (right) agent workflows.*

While Figure 9-3 shows the general flow, a concrete example makes the contrast clearer. Table 9-3 compares how each approach would handle the same task step by step.

*Table 9-3. Single-step versus multi-step (ReAct) execution for the task "Summarize the document and create a bar chart of key statistics"*

| Step | Single-step agent | Multi-step (ReAct) agent |
|---|---|---|
| 1 | Reads the request once | Reads the request and forms an initial **Thought** on the first action (extract statistics) |
| 2 | Plans the entire sequence internally | **Action**: execute extraction; **Observation**: store extracted data in memory |
| 3 | Generates summary and chart in a single pass | **Thought**: plan next action to generate the summary from extracted data |
| 4 | Returns outputs without revisiting intermediate results | **Action**: generate summary; **Observation**: store output in memory |
| 5 | No mid-execution adjustments | **Thought**: plan final action to create the bar chart; **Action**: create chart; **Observation**: review output |
| 6 | — | If intermediate results are incomplete, repeat the loop until satisfied, then return final outputs |

From here, more advanced planning strategies can be integrated. Reflection-based methods introduce self-critique and revision stages. Language Agent Tree Search (LATS)[2] combines these with Monte Carlo Tree Search to search over decision trajectories.

## Multi-Agent Architectures

The same orchestration principles for a single LLM decision maker can be extended to MAS. These distribute responsibilities and decision making across multiple agents, each specialized in a different domain. Figure 9-4 compares the single-agent approach with these two common agentic patterns.

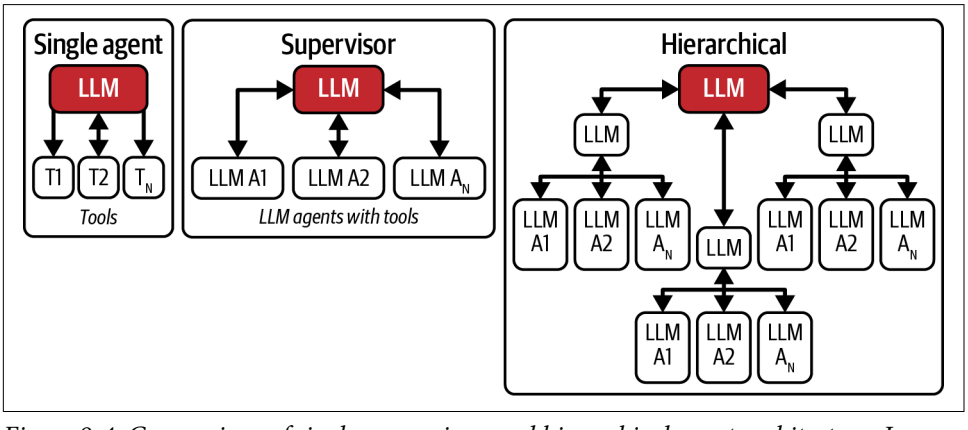


*Figure 9-4. Comparison of single, supervisor, and hierarchical agent architecture. In a supervisor architecture, agents report to one supervisor that decides the next call. A hierarchical setup has supervisors of supervisors, enabling more complex control flows.*

Depending on the design, a multi-agent setup can be as simple as a single supervisor delegating to a few specialized agents or as complex as a hierarchical structure where supervisors manage other supervisors. You can think of it as the difference between a single team working under a tech lead (supervisor pattern) and an organization where a VP of engineering (hierarchical pattern) oversees several tech leads, each with their own team, all contributing to the final product. As orientation, you might consider a multi-agent design if you encounter challenges such as:

- The agent has too many tools at its disposal and struggles to choose the most effective one for the task.
- The context becomes too complex for a single agent to manage effectively.

2 Andy Zhou et al. "Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models" (2024).

- The system requires expertise across several areas, such as planning, research, or mathematical reasoning.

By breaking the application into multiple smaller, independent agents and composing them into a coordinated system, you gain modularity, since independent agents are easier to develop, test, and maintain. You also enable specialization, as each agent can be tailored to excel in a specific domain, improving overall performance and accuracy. Finally, you gain greater control by explicitly managing how agents communicate and coordinate, rather than relying solely on implicit function calling.

Humanizing agents through anthropomorphism can be more than just a user experience choice: when done with precision, it can improve the quality of results. Assigning each agent a personality or role that aligns closely with its task, such as a precise financial analyst for quantitative research or a creative strategist for ideation, helps focus its reasoning and outputs. In MAS, defining these different, purposeful personas and aligning them with domain expertise often leads to more coherent, accurate, and contextually relevant results. Example 9-1 shows example agent personas for a market research agent. Note the full market research agents implementation with LangGraph can be found in the book's repository.

*Example 9-1. Example agent personas*

```
exa_search_agent = create_agent(
    llm,
    [exa_search],
    """You are a research assistant who can search for all recent info on Exa Search
    your response should clearly articulate the key points you found.""",
)
exa_search_node = functools.partial(agent_node,
                                    agent=web_search_agent, name="ExaSearch")


patent_search_agent = create_agent(
    llm,
    [patent_search],
    """You are a market researcher with 20 years of experience, and very
    knowledgeable in patent research and in finding up-to-date info about
    patents using the Google patents API.""",
)
patent_search_node = functools.partial(agent_node,
                                       agent=web_search_agent, name="PatentSearch")
```

To create the previously mentioned supervisor architecture from here, you need to create a team supervisor, as shown in Example 9-2, which basically acts as a router for the tasks.

*Example 9-2. Simple text generation*

```
def create_team_supervisor(llm: ChatOpenAI, system_prompt, members) -> str:
    options = ["FINISH"] + members
    function_def = {
        "name": "route",
        "description": "Select the next role.",
        "parameters": {
            "title": "routeSchema",
            "type": "object",
            "properties": {
                "next": {
                    "title": "Next",
                    "anyOf": [
                        {"enum": options},
                    ],
                },
            },
            "required": ["next"],
        },
    }
    prompt = ChatPromptTemplate.from_messages(
        [
            ("system", system_prompt),
            MessagesPlaceholder(variable_name="messages"),
            (
                "system",
                "Given the conversation above, who should act next?"
                " Or should we FINISH? Select one of: {options}",
            ),
        ]
    ).partial(options=str(options), team_members=", ".join(members))
    return (
        prompt
        | llm.bind_functions(functions=[function_def], function_call="route")
        | JsonOutputFunctionsParser()
    )
```

Multi-agent architectures use state machines to coordinate tool use, reflection, and planning within bounded autonomous flows. A *state machine* defines how an agent moves between predefined states based on inputs, logic, and context, with each state representing a specific behavior or function. In LangGraph, this concept maps directly to its node-edge-state structure, where nodes represent actions or agents, edges define conditional transitions, and the state object maintains memory and metadata. Table 9-4 illustrates how LangGraph relates to state machines.

*Table 9-4. State machine versus LangGraph*

| State machine | LangGraph |
|---|---|
| A system where the agent exists in a defined state | State: a shared data structure that represents the current snapshot of an application. It can be any Python type but is typically a TypedDict or Pydantic BaseModel. |
| Each state has an associated function or behavior | Nodes: functions that encode the logic of the agents. They receive the current state as input, perform some computation or side effect, and return an updated state. |
| Based on input and logic, it transitions to the next state | Edges: functions that determine which node to execute next based on the current state. They can be conditional branches or fixed transitions. |
| — | Command: combines control flow (edges) and state updates (nodes) to facilitate multi-agent communication. For example, a node can update state and decide the next node to visit. LangGraph enables this by returning a Command object from node functions. |
| Explicit control flow where transitions are predetermined | Explicit control flow (normal edges): define the application's control flow explicitly via graph edges, ensuring a deterministic sequence where the next agent is known in advance. |
| Dynamic control flow where transitions can change at runtime | Dynamic control flow (command): allows an LLM to decide parts of the control flow using Command. A special case is the supervisor tool-calling architecture, where the supervisor's LLM determines the order in which tools (agents) are invoked. |

Example 9-3 shows how a state object can be implemented in LangGraph to support a supervisor-based multi-agent setup. This state maps directly to the concepts outlined in Table 9-4, storing the application's snapshot, coordinating agent execution, and maintaining shared context across nodes.

*Example 9-3. Defining a state object for a supervisor-based MAS*

```
class ResearchTeamState(TypedDict):
    messages: Annotated[List[BaseMessage], operator.add] ❶
    team_members: str ❷
    next: str ❸
    current_files: str ❹
```

❶ Tracks the team's conversation.

❷ Provides each worker with context on the other agents skill sets.

❸ Supervisor tells LangGraph which agents have to work next.

❹ Tracks the shared directory state.

Example 9-4 builds a supervisor-routed LangGraph for the market research team. Each specialist agent is added as a node, while a supervisor node decides which agent works next by reading the next field in the shared `ResearchTeamState`. Fixed edges send every worker back to the supervisor after each action. A conditional edge on the supervisor maps next to the selected agent or to `FINISH` to end execution. The small

`enter_chain` helper initializes the top level state with the user prompt and the available team members, and a `functools.partial` wrapper creates `authoring_chain` so callers can pass a message and run the compiled graph without touching internal state.

*Example 9-4. Creating a supervisor-routed graph*

```
authoring_graph = StateGraph(ResearchTeamState) ❶
authoring_graph.add_node("DocWriter", doc_writing_node)
authoring_graph.add_node("Search", search_node)
authoring_graph.add_node("ExaSearch", exa_search_node)
authoring_graph.add_node("PatentSearch", patent_search_node)
authoring_graph.add_node("supervisor", research_writing_supervisor)

authoring_graph.add_edge("DocWriter", "supervisor") ❷
authoring_graph.add_edge("PatentSearch", "supervisor")
authoring_graph.add_edge("Search", "supervisor")
authoring_graph.add_edge("ExaSearch", "supervisor")

authoring_graph.add_conditional_edges( ❸
    "supervisor",
    lambda x: x["next"],
    {
        "DocWriter": "DocWriter",
        "Search": "Search",
        "PatentSearch": "PatentSearch",
        "ExaSearch": "ExaSearch",
        "FINISH": END,
    },
)

authoring_graph.add_edge(START, "supervisor")
chain = authoring_graph.compile()

def enter_chain(message: str, members: List[str]): ❹
    results = {
        "messages": [HumanMessage(content=message)],
        "team_members": ", ".join(members),
    }
    return results

authoring_chain = ( ❺
    functools.partial(enter_chain, members=authoring_graph.nodes)
    | authoring_graph.compile()
)
```

❶ Create the graph.

❷ Add the edges.

❸ Add the edges where routing applies.

❹ Interoperate between the top-level graph state and the state of the research sub-graphs.

❺ Reuse the enter/exit functions to wrap the graph.

With the graph constructed, you can execute it in streaming mode. The call in Example 9-5 sends a single high-level prompt to the supervisor and lets the team iterate until completion. Streaming yields incremental state updates from nodes as they act, which you can print for visibility while skipping the final event. The `recursion_limit` bounds the number of agent steps to protect against *runaway loops*: situations where agents keep passing control back and forth without ever reaching a finishing condition. For example, you might have a runaway loop if the supervisor repeatedly reassigns tasks or a worker keeps requesting retries indefinitely. The prompt instructs the team to search, collect sources and patent links, and then write the final report to disk as a *.txt* file.

*Example 9-5. Run the supervisor architecture*

```
for s in authoring_chain.stream(
    """Write an 800-word-long research report white paper on semiconductor
    development which includes an executive summary at the beginning about
    your findings. Search also for relevant recent patents and include the
    links to it. IMPORTANT: Provide the links to all your sources, and then
    write the report to disk as .txt file.""",
    {"recursion_limit": 100},
):
    if "__end__" not in s:
        print(s)
        print("---")
```

By combining specialized agents, explicit routing, and state machine control this way, you enable AI systems to handle tasks that would overwhelm a single agent. This approach not only improves accuracy and efficiency through domain specialization but also provides a clear, expandable and maintainable structure for complex workflows. Whether you apply it to research, analysis, or planning, this design gives you fine-grained control over your execution flow while agents remain flexible enough to adapt their actions dynamically.

# Agentic Communication: The Right Context Is All You Need

This section gives you guidance about why more information does not always mean better answers in agentic systems. Just as personas shape how an agent performs and responds, context determines how effectively it can carry out a task. Even the most capable human can't perform their job well without first understanding the context of what they are being asked to do, and the same is true for AI agents. The term *prompt engineering* describes the craft of writing instructions in the optimal format for an LLM chatbot. *Context engineering* takes this further by dynamically generating and adapting that context within an agentic system. This requires a deeper understanding of both the task and the system's evolving state, making it one of the most important responsibilities for anyone building AI agents. In practice, this means designing systems that can recognize, assemble, and refine the information an agent needs before it acts.

To understand why this matters, imagine asking an AI travel assistant, "Can you plan my next holiday?" A basic prototype might see only that one question and respond with something like, "Sure, where would you like to go?" The agent might run without errors, but without richer context, the answer is generic and forces the user to repeat information they've already provided in the past.

Now consider the same request in an agent designed with full context engineering. Before responding, the agent recalls the user's travel history from previous interactions, including preferred destinations, budget ranges, and typical travel times. It retrieves loyalty program details and checks the user's calendar for available dates. It then uses a live search tool to gather current news and weather conditions and finds that a hurricane is forecast to hit the user's usual vacation region next month. With this richer picture, the agent can reply:

```
I remember you usually spend early September in the Caribbean, but there is a
hurricane warning for that period. Based on your past trips and budget, I found
a similar beachfront resort in Madeira with availability for the same dates.
It's part of your hotel loyalty program and has excellent reviews. Would you
like me to book the reservation?
```

The difference between the two agents is not in the complexity of the model but in the completeness and relevance of the context. Prompt engineering focuses on crafting a single well-formed instruction, while context engineering is about building systems that dynamically assemble the right mix of historical knowledge, real-time information, and available tools so that the agent can act with precision and foresight.

To make context engineering tangible, it helps to look at the key components an agent can draw on when shaping its response. These elements work together inside the broader context space, each influencing how the agent interprets the user's request

and decides on the best course of action. As shown in Figure 9-5, the user prompt sits at the center, with all other components surrounding and informing it.

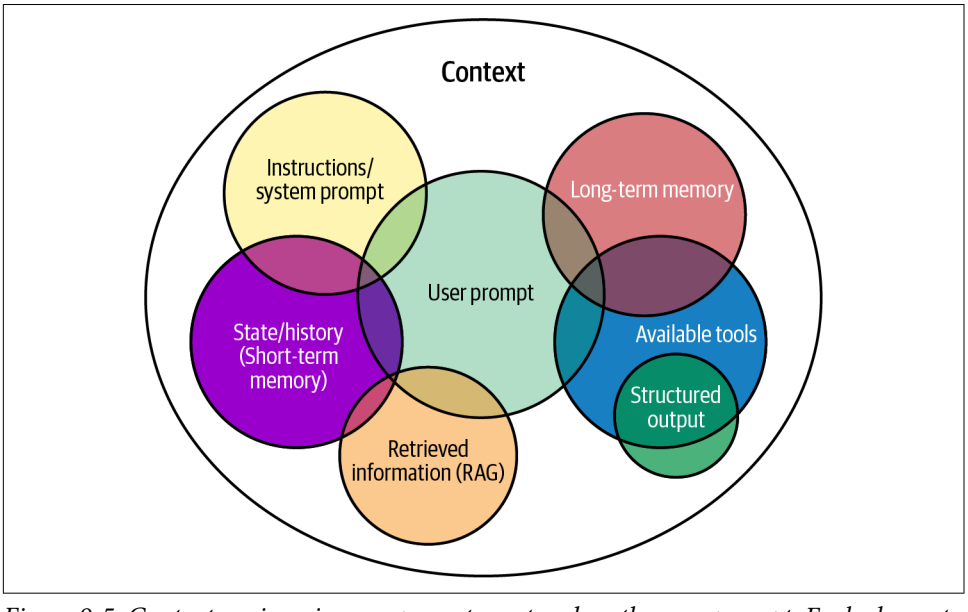


*Figure 9-5. Context engineering components centered on the user prompt. Each element contributes to shaping the agent's understanding and response within the overall context space.*

Below are short explanations for what each context engineering component entails:

*Instructions/system prompt*
: The foundational guide that shapes how the model behaves throughout the conversation. This can include explicit rules, constraints, style preferences, and worked examples. Well-crafted system prompts align the model's responses with the agent's intended persona and objectives.

*User prompt*
: The immediate request or task provided by the user. It's the focal point around which all other context elements revolve, ensuring that the system's output directly addresses the user's needs.

*State/history (short-term memory)*
: The running record of the current conversation, including both user inputs and model outputs. This temporary memory enables continuity, prevents repetition, and maintains coherence over multi-turn interactions.

*Long-term memory*
: A persistent knowledge store accumulated over multiple sessions. It can contain user preferences, summaries of past projects, recurring workflows, or other facts the agent has been explicitly instructed to remember for future use.

*Retrieved information (RAG)*
: External, up-to-date knowledge pulled from documents, databases, APIs, or the web at the time of the query. This retrieval step enriches the agent's knowledge base with context that is specific, timely, and beyond its static training data.

*Available tools*
: The set of functions, APIs, or capabilities the agent can invoke to take action. Examples range from checking inventory and sending emails to running data analyses or booking reservations. This can also include integrations built on the Model Context Protocol (MCP), which provides a standardized way for agents to access external systems and services. The toolset determines what actions are possible in response to the user's prompt.

*Structured output*
: Explicit specifications for how the model should return its answer, such as a JSON object, HTML snippet, or table. Structured formats are essential when outputs must be machine-readable or integrated into downstream workflows.

Understanding these components is only the first step. Effective context engineering also involves deciding how and when each element is brought into the model's active reasoning space. A common pitfall when building agentic systems is to provide every possible piece of context in every interaction. This can quickly overwhelm the model's context window and dilute the relevance of the information that actually matters for the task. Effective context engineering is therefore as much about restraint as it is about completeness. The goal is to present the model with only the most useful and targeted information at the right moment.

This process begins with selecting the most relevant information for the task at hand. A well-designed agent does not simply load all possible context at once. Instead, it identifies what's needed for the current request, which may involve retrieving relevant memories, narrowing the available toolset to avoid confusion, or pulling the most relevant knowledge from external sources. The aim is to ensure that the user prompt is surrounded by information that's timely, accurate, and directly related to the question being asked.

Once relevant context is selected, it must often be compressed to fit within the model's available context window. Long conversations, detailed tool outputs, or extensive retrieval results can quickly exceed capacity or lead to the haystack problem covered in Chapter 1. Summarization offers one way to retain the essence of this information while removing redundancy and can be applied in stages to preserve important

details across many turns. Trimming uses a more selective approach, removing less relevant content according to fixed rules or trained filtering mechanisms. Both approaches aim to give the model as much useful input as possible without overwhelming it with unnecessary detail.

There are also cases where it's more effective to isolate context entirely. In MASs, each agent may have its own dedicated instructions, memory, and tools, focusing only on a specific part of the task. This separation allows each agent to work without interference from unrelated information while still contributing to a shared goal through structured communication and coordination.

In practice, context engineering is therefore as much about management as it is about collection. Selecting, compressing, and isolating information ensures that when the model interprets the user's request, it does so within a context that is both rich and precisely tuned to the task at hand. This balance is what allows an agent to act with accuracy, efficiency, and adaptability.

# Beyond Context: How to Help Agents Remember

Context engineering determines what an agent knows at the moment of decision. Memory determines how that knowledge is retained, organized, and recalled over time. To make this concrete, it helps to start with the basic memory categories that underlie both human cognition and agent system design.

## Agent Memory Types

In cognitive science, memory is often grouped into three broad types, which we can directly map to memory design in agents. Figure 9-6 shows an overview of these types in the context of coding agents.

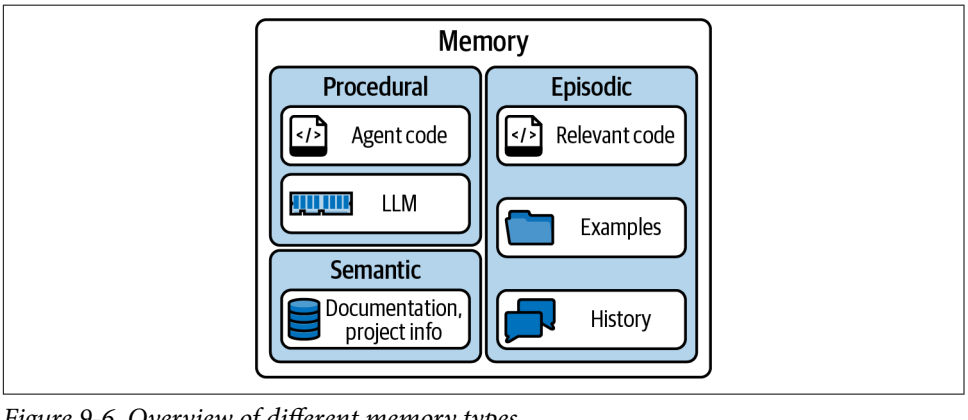


*Figure 9-6. Overview of different memory types.*

These categories are practical for agents because they help clarify what information is stored and how it's used. A coding agent, for example, may rely on procedural memory for its own internal code and the LLM it runs on, episodic memory for recalling relevant code snippets or examples from past work, and semantic memory for retrieving documentation or project details. The categories include:

*Procedural memory*
: "How-to" knowledge. This encodes skills, workflows, and action patterns. For agents, procedural memory might include learned tool invocation sequences, optimized query strategies, or a preferred order of reasoning steps. It's often updated incrementally through fine-tuning or reinforcement learning and retrieved automatically during execution.

*Episodic memory*
: Memory of specific events or interactions. In an AI agent, this could be the log of a past troubleshooting session, the sequence of actions taken in a customer support case, or the steps followed in a successful trade execution. Episodic memory supports reflection, replay, and postmortem analysis of task outcomes.

*Semantic memory*
: Factual and conceptual knowledge, independent of the event in which it was learned. This might be a vector database of patents, a curated financial knowledge base, or embeddings of technical documentation. Semantic memory can be searched and combined with current observations to answer complex questions.

These categories are not mutually exclusive. A single workflow can draw on all three: procedural memory to know how to act, episodic memory to recall when a similar case happened, and semantic memory to fetch what facts are relevant. In practice, these types of memory are not just about what's stored but also about how widely that information can be shared and how long it's kept. This is where memory scope comes in, moving us from the question of what an agent remembers to how long and with whom it remembers it.

## Going Global and Lifelong

In a single agent setup, all memory is local. It lives with that agent alone. *Local memory* lets the agent specialize and keep its own notes without anyone else looking over its shoulder. In MAS, you can go wider. A *global or shared memory* acts like a common whiteboard all agents can see and write on. It's where knowledge can flow without the agents having to send direct messages to one another. Then there's *lifelong memory*, which sticks around across sessions and projects, letting the agent or the whole team build up experience over time instead of starting from scratch every morning.

Letta is a stateful agents framework providing lifelong memory, reasoning, and context management. It treats an agent's context window like the RAM in a computer, fast but limited, and then adds an external persistent store that works like a hard drive. When the agent's working context starts to fill up, Letta automatically moves less relevant information out to long-term storage and can pull it back later if needed. This allows the agent to operate as if it has an almost unlimited memory, even when it runs on a model with a fixed context size. Example 9-6 shows how to create a shared memory block and attach it to a supervisor agent and a worker agent. This is an example from Letta's API documentation.

*Example 9-6. Create a shared memory block*

```
client = Letta(base_url="http://127.0.0.1:8283")


shared_block = client.blocks.create(label="organization",
                                    value="Organization: Letta") ❶

supervisor_agent = client.agents.create( ❷
    model="anthropic/claude-3-5-sonnet-20241022",
    embedding="openai/text-embedding-3-small",
    # blocks created for this agentj
    memory_blocks=[{"label": "persona", "value": "I am a supervisor"}],
    # pre-existing shared block that is "attached" to this agent
    block_ids=[shared_block.id],
)

worker_agent = client.agents.create( ❸
    model="anthropic/claude-3-5-sonnet-20241022",
    embedding="openai/text-embedding-3-small",
    # blocks created for this agent
    memory_blocks=[{"label": "persona", "value": "I am a worker"}],
    # pre-existing shared block that is "attached" to this agent
    block_ids=[shared_block.id],
)
```

❶ Create a shared memory block.

❷ Create a supervisor agent.

❸ Create a worker agent.

The important part is that the agent manages this process itself. It decides what to keep immediately available, what to archive, and when to go searching through past records. In practice, this means an agent built with Letta can remember details from conversations weeks ago, reference older documents while working on a new task, or

keep track of evolving strategies without a human having to re-upload the same background material again and again.

With Letta, you can keep a fast and finite local memory for active reasoning, a larger shared or persistent memory for the information you might need later, and a smart way of moving information between them. Whether you're running a single agent that needs to remember its entire history or a group of agents that share one common knowledge base, the result is the same. Agents work better together when they aren't forced to operate with goldfish memory.

# The Human Factor: Steering Agent Actions

As you saw in "Autonomy: What's Possible at the Moment?" on page 244, agents have a certain amount of autonomy in how they carry out tasks. But even with that limited freedom, you might be wondering, "Can I really trust an agent to act the right way without me watching?" The answer is…not always. You can put safeguards in place, like a judge or verifier LLM, and you can test how accurate its outputs are. But sometimes agents still misread a prompt or take an action they shouldn't. And if that action affects something sensitive, like access control, financial operations, or customer data, that isn't a risk worth taking. This is where a human-in-the-loop (HITL) setup makes all the difference, giving the agent a chance to pause and get your approval before making an important move.

Inserting humans' approval at key decision points is not just about controlling the agent. It's about protecting you and your organization from costly mistakes. With a HITL protocol, every critical action passes through a human reviewer or approver before it's executed. This ensures that irreversible errors are caught in time, that there is clear accountability for every decision, and that you remain compliant with audit requirements such as SOC 2 policies and internal governance rules. It also turns your AI from a black box into a supervised assistant, building trust with both users and stakeholders. Therefore, this isn't a nice-to-have. In many agentic workflows, especially those touching high stakes operations, HITL is the only responsible path.

## Common Patterns for Human-in-the-Loop

Once you decide to integrate HITL into your workflow, the question becomes how to design those decision points so they fit naturally into your agent's flow. The approach you choose will depend on the type of task, the sensitivity of the action, and the degree of oversight you want to maintain.

In LangGraph, the HITL designs are built on a single core idea: pause the graph at the right moment and resume it only after a human provides input. LangGraph's `interrupt()` function is the key to doing this. When called inside a node, it stops execution and saves the graph state. The human can then review or edit whatever you surface, and you resume execution by passing the input back in using the `Command` primitive.

To help you to see how the pieces fit together, I'll walk you through a LangGraph workflow that generates LinkedIn posts using an LLM, with an HITL feedback mechanism. The flow includes three main components:

*Model node*
: Uses a language model (Llama 3.3 via Nebius) to generate a LinkedIn post based on a user-provided topic and optional feedback.

*Human node*
: Pauses execution to collect user feedback on the generated post. The user can iteratively refine the output or use a predefined keyword to finalize it.

*End node*
: Outputs the final post and feedback summary.

The code uses LangGraph's `interrupt()` and `command()` mechanism to handle real-time input and supports continuous, feedback-driven improvement of the generated content. The flow is structured using a StateGraph, and all interactions are managed with a `while True loop` that handles streaming and user input.

You start with the state as shown in Example 9-7. Here, you define exactly what the graph carries from node to node. In this example, the state holds the LinkedIn topic, a running list of generated drafts, and a running list of human feedback entries. The lists let you keep a full trail rather than overwriting values.

*Example 9-7. Set state for HITL loop*

```
class State(TypedDict):
    linkedin_topic: str ❶
    generated_post: Annotated[List[str], add_messages] ❷
    human_feedback: Annotated[List[str], add_messages] ❸
```

❶ Topic to write about. Immutable across iterations.

❷ List of drafts across the loop so you can inspect the full trail later.

❸ List of feedback messages to condition future drafts.

Next comes the model node (Example 9-8). It reads the current topic and the most recent human feedback and then drafts the next version of the post. The prompt includes both the topic and the last piece of feedback, so each iteration can refine the text. The node returns the new draft and passes the feedback list through unchanged.

*Example 9-8. Set the model state*

```
def model(state: State) -> State:
    """Draft a LinkedIn post using the topic and the most
        recent human feedback."""
    topic = state["linkedin_topic"] ❶
    feedback_list = state.get("human_feedback", []) ❷
    feedback_summary = _summarize_feedback(feedback_list) ❸

    prompt = f"""
    You are an expert LinkedIn content writer. Create a strong post that:
    - Opens with a clear hook in 1 to 2 short sentences
    - Uses short paragraphs with concrete points and a practical example
    - Avoids fluff and clichés
    - Ends with a simple call to action
    - Keeps a professional but conversational tone

    Topic: {topic}
    Latest feedback to consider: {feedback_summary}
    Return plain text only.
    """.strip() ❹

    response = llm.invoke([ ❺
        SystemMessage(content="Write crisp, useful LinkedIn
                        posts with substance and clarity."),
        HumanMessage(content=prompt)
    ])

    generated = response.content ❻
    print("\n[model] Draft ready. Handing over for review...\n") ❼
    print("----- DRAFT START -----")
    print(generated)
    print("------ DRAFT END ------\n")

    return {
        "linkedin_topic": topic, ❽
        "generated_post": [AIMessage(content=generated)], ❾
        "human_feedback": feedback_list, ❿
    }
```

❶ Read invariant topic from state.

❷ Carry the existing feedback list forward.

❸ Reduce many feedback items to a concise summary for the next prompt.

❹ Build a tight instruction that embeds topic and feedback summary.

❺ Call the model with system and human messages. Assumes `llm` and `message` classes are available.

❻ Extract generated text from the model response.

❼ Print to console to aid review during the loop.

❽ Pass the topic through unchanged to preserve state.

❾ Append the latest draft as an AIMessage so downstream nodes can access content consistently.

❿ Pass feedback through unchanged here. The human node will update it.

Then you need to set up the human intervention node (Example 9-9). This is where you pause execution. The node calls `interrupt(...)` and surfaces the current draft along with a message that asks for feedback. The graph saves its state and stops. When you resume, the node receives the human's input as the return value of `interrupt(...)`. If the input indicates you're done, the node routes to the end. Otherwise, it appends the feedback and loops back to the model for another draft.

*Example 9-9. Set up human node*

```
def human_node(state: State) -> Command:
    """Pause for human feedback. Default to 'okay' if the user presses Enter."""
    latest = state["generated_post"][-1].content if
             state["generated_post"] else "" ❶
    print("[human_node] Review the draft above.")
    print("Tip: press Enter for 'okay'. Type 'done' to finish.
            Or write feedback to iterate.\n")

    user_feedback = interrupt({ ❷
        "generated_post": latest,
        "message": "Your feedback [okay/done/custom]:"
    })

    feedback_list = state.get("human_feedback", []) ❸
    if isinstance(user_feedback, str):  ❹
        raw = user_feedback.strip()
        if not raw:
            raw = "okay"
        low = raw.lower()
```

```
        if low in DONE_SET: ❺
            return Command(
                update={"human_feedback": feedback_list + ["Finalized"]},
                goto="end_node"
            )

        if low in OKAY_SET: ❻
            feedback_list = feedback_list + ["okay"]
        else:
            feedback_list = feedback_list + [raw]
    else:
        feedback_list = feedback_list + [str(user_feedback)] ❼

    return Command(update={"human_feedback": feedback_list}, goto="model") ❽
```

❶ Grab the most recent draft text so the reviewer sees the right content.

❷ Signal `interrupt` to pause the graph and persist state. Use the return value as human input on resume.

❸ Copy the current feedback list so new input can be appended immutably.

❹ Normalize user input. Map empty input to the default "okay."

❺ If the user says "done," route to the terminal node and mark the list as finalized.

❻ If the user says "okay," record a lightweight approval token. Otherwise, record free-form feedback.

❼ Handle non-string inputs by converting everything to a string entry in the audit trail.

❽ Continue the loop by routing back to the model node with updated feedback.

To make pausing work, you need to compile the graph with a `checkpointer` and run it with a `thread id`, as shown in Example 9-10. When an interrupt occurs, you'll see the special `__interrupt__` marker in the stream. Resume by passing the human input through `Command(resume=...)`.

*Example 9-10. Enable interrupt mechanism*

```
checkpointer = MemorySaver() ❶
app = graph.compile(checkpointer=checkpointer) ❷

thread_config = {"configurable": {
    "thread_id": uuid.uuid4() ❸
```

```
}}

linkedin_topic = input("Enter your LinkedIn topic: ") ❹
initial_state = { ❺
    "linkedin_topic": linkedin_topic,
    "generated_post": [],
    "human_feedback": []
}

stream = app.stream(initial_state, config=thread_config) ❻

while True:
    try:
        chunk = next(stream)
        # An interrupt arrives as a special event
        if "__interrupt__" in chunk: ❼
            # Interactive loop until we resume
            while True:
                try:
                    user_feedback = input("Your feedback
                                          [okay/done/custom]: ").strip()
                except KeyboardInterrupt:
                    user_feedback = "done"

                if not user_feedback:
                    user_feedback = "okay"

                if user_feedback.lower() in DONE_SET: ❽
                    app.invoke(Command(resume="done"), config=thread_config)
                    break
                else:
                    app.invoke(Command(resume=user_feedback),
                               config=thread_config)
    except StopIteration:
        break

print("Finished.") ❾
```

❶ Enable checkpointing so the graph can pause and later resume with full state.

❷ Compile the graph with the `checkpointer` attached.

❸ Unique thread ID isolates this conversation. Use a stable ID to resume later runs.

❹ Collect the topic interactively. You can also seed this from upstream logic.

❺ Seed the state with empty lists so `add_messages` can append cleanly.

❻ Start streaming execution to receive events, including interrupts.

❼ Detect the interrupt marker and enter a local input loop until you resume.

❽ Map user choice to the correct `resume` command. `Done` exits the loop. Any other string continues the iteration.

❾ Exit banner for a clean end state.

This multi-turn conversation example is just one of several patterns you can use. The right pattern depends on the type of task, the level of risk, and the kind of oversight you want to maintain. Following are some of the most common approaches, each with a brief description followed by a code example you can adapt for your own LangGraph workflow:

*Approve or reject*
: Pause before a sensitive action, such as sending an API request or initiating a financial transaction. Review the action and either approve it, reject it, or choose an alternative path:

```
if low in {"approve", "yes", "ok", "okay"}:
    return Command(update={"human_feedback": feedback_list + ["approved"]},
    goto="model")
if low in {"reject", "no"}:
    return Command(update={"human_feedback": feedback_list + ["rejected"]},
    goto="end_node")
```

*Review and edit*
: Instead of answering with a simple yes or no, you can make adjustments to the data or instructions before allowing the agent to continue:

```
payload = {"draft": latest, "instructions": "Edit text or press Enter"}
user_feedback = interrupt(payload)

if isinstance(user_feedback, dict) and "draft" in user_feedback:
    edited = user_feedback["draft"]
    return Command(
        update={
            "generated_post": [AIMessage(content=edited)],
            "human_feedback": feedback_list + ["edited"]
        },
        goto="model"
    )
# fall back to normal feedback handling if needed
```

*Review tool calls*

If a tool call could affect external systems, inspect its arguments before execution. For example, you might confirm booking details before a travel reservation is made:

```
def tool_gate(state: State) -> Command:
    proposed = {"tool": "book_flight", "args": state["proposed_args"]}
    decision = interrupt({"review": proposed, "message":
                          "Approve or edit args"})

    if isinstance(decision, dict) and decision.get("approve"):
        return Command(goto="execute_tool")
    if isinstance(decision, dict) and "args" in decision:
        return Command(update={"proposed_args": decision["args"]},
                                goto="execute_tool")
    return Command(update={"human_feedback": feedback_list +
                    ["rejected tool call"]}, goto="end_node")
```

*Validate human input*

In some cases, the human is providing data to the system. HITL can validate that data before the agent proceeds, ensuring accuracy and completeness:

```
def validate_input(state: State) -> Command:
    errors = find_errors(state["form_data"])  # your validation logic
    if errors:
        fixes = interrupt({"errors": errors, "message": "Please correct fields"})
        merged = {**state["form_data"], **(fixes or {})}
        return Command(update={"form_data": merged}, goto="validate_input")
    return Command(goto="model")
```

LangGraph provides extensive documentation on how you can apply these different patterns in your own application.

As a rule of thumb, when implementing HITL, place any code with side effects such as API calls or database writes after the approval step to avoid repeating irreversible actions when a paused step is resumed.

# Solving GitHub Issues with Coding Agents

Just as human developers benefit from integrated development environments (IDEs) like PyCharm, coding agents can benefit from purpose-built interfaces that match their strengths and mitigate their weaknesses. SWE-agent[3] introduces an *agent-computer interface* (ACI), a specialized interaction layer that abstracts away the complexity of raw Linux shells and instead offers a streamlined set of language

3 John Yang et al. "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering" (2024).

model–friendly commands for navigating, searching, and editing code. Figure 9-7 shows a simplified overview of SWE-agent.

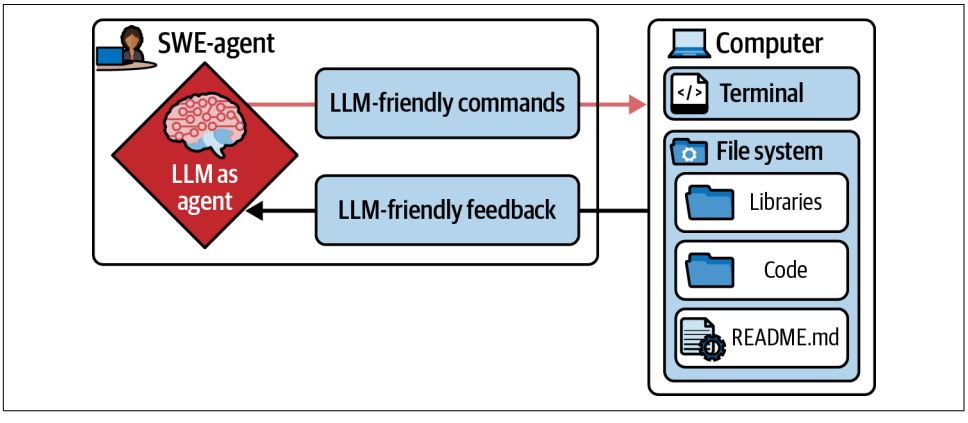


*Figure 9-7. Overview of SWE-agent and how it interacts with a computer via an agent-computer interface (ACI) defining its commands and feedback format.*

Rather than expecting the model to construct verbose, error-prone shell commands, the ACI provides a compact and well-documented action space with commands like `find_file`, `search_file`, `goto`, and `edit`, each designed for reliability and ease of use. Guardrails such as syntax checking and automated error recovery prevent invalid edits from propagating through the workflow. The environment feedback is concise yet informative, giving the model only the context it needs to make its next decision without overloading its token window.

With this interface in place, SWE-agent allows your chosen language model (such as GPT-4o or Qwen3-Coder-480B-A35B) to autonomously use tools to fix issues in your GitHub repositories, detecting cybersecurity vulnerabilities or carrying out any other custom-defined task for you.

### Mini SWE-agent

The developers of SWE-agent released a 100 times smaller version of SWE-agent, called mini SWE-agent, which still works nearly as well. This version is perfect for developers who like their tools like their scripts: short, sharp, and readable. It's also useful for engineers who want something trivial to sandbox and to deploy anywhere. You can easily install it via PyPI with `pip install mini-swe-agent`. You can run it locally or with docker execution, and with models like GPT-5, Claude Sonnet, or local models. The framework supports HITL and customized execution. More information on how to run mini SWE-agent can be found in their cookbook.

To run SWE-agent to solve your GitHub issues, you need to install it from the source and then prepare the environment:

```
git clone https://github.com/SWE-agent/SWE-agent.git

SWE-agent $ python3.11 -m venv venv-sweagent
SWE-agent $ source venv-sweagent/bin/activate
(venv-sweagent) SWE-agent $ pip install --editable .
```

After that, you can simply run:

```
sweagent run \
--agent.model.name=gpt-5-mini \
--agent.model.per_instance_cost_limit=2.00 \ ❶
--env.repo.github_url=https://github.com/SWE-agent/test-repo \ ❷
--problem_statement.github_url=https://github.com/SWE-agent/test-repo/issues/1 ❸
```

❶ Limit cost.

❷ Repo link.

❸ GitHub issue to be solved.

You aren't limited to a single model when running SWE-agent. By changing the `--agent.model.name` parameter, you can select from a range of LLMs, such as GPT 4o or Anthropic's Claude Sonnet, depending on your performance, cost, or preference requirements. The same interface can also be paired with multimodal models, enabling the agent to go beyond text-based reasoning and handle visual debugging tasks, such as identifying issues in frontend layouts or fixing problems in data visualizations. What makes this possible is the ACI, which reduces the complexity of the environment into a clear and consistent set of actions, allowing the agent to focus on reasoning rather than low-level command construction.

This illustrates a broader principle of agent design. Autonomy is shaped as much by the tools and feedback an agent receives as by the model's internal capabilities. Purpose-built interfaces such as ACI give agents the structure, guardrails, and concise context they need to act reliably, adapt to feedback, and carry out complex problem solving.

# Conclusion

In this chapter, you explored what autonomy means in today's AI agents and how it's expressed through orchestrated decision making, tool use, and adaptive workflows. While these systems are not yet fully self-directed, they can plan, act, and refine their behavior in real time within the boundaries of a defined architecture.

You saw how autonomy is shaped by design, from routers that make single-step choices to MASs where specialized agents collaborate under a supervisor. State machines and frameworks like LangGraph provide the structure for these workflows, supporting dynamic control flow, memory management, and coordination between agents.

Context engineering emerged as a critical paradigm, ensuring that agents can act with exactly the information they need, while human-in-the-loop patterns give you the ability to guide decisions at key moments, balancing efficiency with safety and trust.

Memory proved to be as critical as planning or tool use. By combining procedural, episodic, and semantic memory, agents can retain skills, recall past events, and retrieve factual knowledge at the right moment. With mechanisms for local, shared, and lifelong memory, agents can accumulate experience across sessions.

Finally, SWE-agent illustrated how purpose-built environments can amplify agent performance by constraining the action space, standardizing feedback, and integrating guardrails, making complex problem solving more reliable.

With these principles in place, you're ready to move into Chapter 10, where the focus turns to optimizing the transformer itself for problem solving, tuning its architecture, reasoning strategies, and inference methods to push agent performance even further.

CHAPTER 10

# Smarter, Better, Faster, Stronger: Optimizing LLMs and AI Agents

As you've seen throughout the book, transformers are generalists. They can generate text, compose music, write code, and create or classify images. You've learned that end-to-end reinforcement learning combined with chain of thought can spark internal reasoning and help models solve more complex problems. You've also gained an understanding that dividing problems into subproblems and allowing language models to collaborate in a multi-agent system can turn them into robust tools for challenging tasks.

Yet as task complexity grows, these capabilities are not enough. Extended reasoning, multi-step decision making, and production-grade reliability expose weak points. Without optimization, even the largest models or the most advanced agentic systems can have problems with difficult reasoning chains, waste computation on trivial problems, or fail to adapt in real-world deployments.

Optimization is the missing piece. If you think about it, world-class athletes don't rely on talent or regular practice alone. They depend on an ecosystem that includes targeted training, recovery, nutrition, and mental preparation. In the same way, transformers can't rely only on pretraining or supervised fine-tuning. They need reinforcement signals, adaptive reasoning strategies, efficient test-time scaling, and strong systems-level infrastructure. In practice, this means going beyond the baseline with post-training frameworks that refine policies and rewards, teaching models when to think and when to act, scaling compute at inference to search effectively, and updating weights quickly in production so policies improve while serving real workloads. Table 10-1 summarizes this optimization ecosystem.

*Table 10-1. Components of the optimization ecosystem for agentic LLMs*

| Concept | Component(s) | Role | Example |
|---|---|---|---|
| Training-time optimization | ART, GRPO, RULER | Refine reasoning, tool use, and decision policies with RL | Relative trajectory ranking with RULER |
| Inference-time scaling | AdaptThink, search methods (e.g., ETS) | Allocate compute by difficulty; decide when to think versus act | Difficulty-gated CoT with AdaptThink |
| Open RL frameworks | rLLM, verl | Distributed rollouts, training, and evaluation | Train coding agents with GRPO |
| Systems-level infrastructure | Checkpoint Engine | Distribute updated weights from learner to actors rapidly | Policy refresh in production |

The following sections walk through each part of this optimization ecosystem. Training-time optimization shows how RL methods like ART, GRPO, and RULER make agents more reliable. Inference-time scaling covers strategies such as AdaptThink that cut wasted compute by deciding when extended reasoning is worthwhile. Open RL frameworks like rLLM and verl illustrate how distributed infrastructure brings these ideas into practice. And systems-level infrastructure with the Checkpoint Engine shows how policies keep improving efficiently in production.

## Training-Time Intelligence: Reinforcement Learning for Agents

Training-time optimization is the first lever you have for making transformers more reliable problem solvers. Pretraining and supervised fine-tuning establish broad competence, but RL provides the feedback loop that refines behavior against real objectives. Rather than relying on fixed labels or static datasets, RL evaluates the model's behavior dynamically: you score its outputs relative to desired goals and then update weights accordingly.

Modern frameworks extend this principle beyond simple preference learning. Agent reinforcement training (ART) structures rollouts around tool use and decision making. *ART* is an open source framework that trains agentic LLMs to enhance their performance and reliability through experience. It uses GRPO, which scales preference learning to groups of candidate solutions and removes the need for absolute scores. *Relative universal LLM-elicited rewards* (RULER) is a general-purpose reward function that uses an LLM-as-judge to rank agent trajectories. It requires no labeled data, expert supervision, or manually designed rewards, and it consistently improves agent performance. Combine ART with RULER, and you can judge multiple trajectories side by side, rewarding those that follow the best reasoning path or produce the most useful final answer.

Together, these frameworks turn RL into a scalable engine for agent alignment, enabling models to refine reasoning strategies, tool usage patterns, and decision policies at training time.

## Beyond Hand-Crafted Rewards: How RULER Works

One of the biggest bottlenecks in RL for agents is reward design. Hand-crafted reward functions are brittle, domain-specific, and expensive to maintain. LLM-as-judge approaches have emerged as a flexible alternative, but they often suffer from calibration problems: scores are inconsistent across runs, difficult to interpret, and not always reliable enough to drive stable training. RULER addresses this gap by reframing the problem in relative rather than absolute terms.

RULER works by ranking multiple candidate trajectories against one another, instead of assigning absolute scores in isolation. It generates N trajectories for the same prompt, each capturing the full sequence of inputs, outputs, and tool calls. RULER deduplicates any shared prefixes (such as the system prompt) and passes the unique suffixes to an LLM-as-judge, along with a rubric for evaluation. The judge scores each trajectory from 0 to 1 based on whether it achieved its goal and how efficiently it did so. These scores are then normalized within the group and used directly as rewards in a GRPO update. The judge can also provide explanations for its scores, which you can use to analyze faults and cluster low-performing runs. Figure 10-1 shows a high-level overview of how RULER works.

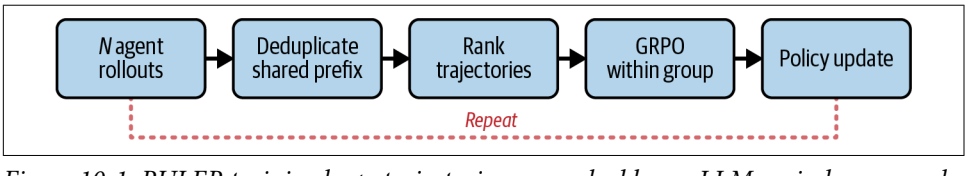


*Figure 10-1. RULER training loop: trajectories are ranked by an LLM-as-judge, normalized by GRPO, and used to update the policy iteratively.*

The design uses two key insights:

- It's easier for an LLM to rank multiple candidate solutions than to score them individually. Side-by-side comparison highlights deficiencies that might not be obvious in isolation.
- With GRPO, rewards only need to be consistent within a group of candidates. Global calibration is unnecessary, which sidesteps one of the most persistent challenges in LLM-as-judge setups.

In practice, its design makes RULER highly stable and scalable. It removes the need for labeled data or expert-crafted rules while still producing signals strong enough to

guide complex policy updates. It also converges faster than hand-tuned baselines, since it can give partial credit to incomplete but promising solutions.

#### Customizing RULER

Although RULER accepts custom rubrics for domain-specific tasks, experience shows that the default rubric often works best. It prioritizes goal completion, efficiency, and proportional scoring for close versus large performance gaps. Rather than over-customizing the rubric, it's usually more effective to provide clear instructions in the system prompt of the agent itself. Because RULER sees that prompt as part of the trajectory, it can use it to align rankings with the agent's intended goals.

RULER's relative scoring design also makes it suitable for inference-time scaling. Instead of producing a single best answer, a model can generate multiple candidates and let RULER select the most promising one on the fly. This bridges the gap between training-time optimization and test-time reasoning; it allows online learning systems to adapt continuously as agents are deployed.

## Training in Practice: ART in a Market Scenario

To make this concrete, I'll walk you through an example of how you can combine ART and RULER in a trading environment. I'll use ART here because it provides you the conceptual framework for RL with agents, and it's designed to be developer-friendly. It offers you wrappers that make it straightforward to use RL training with your existing applications without requiring you to rewrite large parts of your code. The training server is abstracted as a modular service, so your application doesn't need to interface with it directly.

You can train from almost anywhere: run the ART client on your laptop and let the server launch a GPU-enabled environment, or train on a local GPU if available. Integrations with hosted platforms such as Weights and Biases, Langfuse, and OpenPipe provide flexible observability, logging, and debugging out of the box. ART also comes with intelligent defaults for training parameters and inference settings, optimized for efficiency and stability, while still letting you customize configurations for your specific needs.

The example I'll show is a simple agent that calculates a short-term moving average (SMA) for stock prices using LangGraph for rollouts and RULER for reward scoring. What matters is not the finance context but the pattern. Table 10-2 shows you some ideas for adapting my example to support other functionality.

*Table 10-2. What you can swap in the ART training loop*

| Component | Market SMA example | How to adapt for other domains |
|---|---|---|
| Environment | Stock price history | Email corpus, support tickets, codebase, medical records |
| Tools | `latest_closes_tool`, `calculator`, `return_final_answer_tool` | Search APIs, query tools, unit test runners, document readers |
| Reward | Deterministic numeric SMA check | Relative scoring with RULER, task-specific rubrics, pass/fail unit tests |
| Agent loop | LangGraph ReAct with ART + GRPO | Same loop reused across domains with different prompts and toolsets |

You can use the same loop for pretty much any agentic task where an LLM needs to work with tools, create an output, and then be checked for success. The market SMA agent is just an easy example to show you the patterns in action, so you can see how they work and add them to your own projects.

### Additional Example

There's also a full example from the ART authors that you can run from this book's repo. It trains an email search agent over the Enron corpus. The agent searches, reads, and returns a grounded answer with cited message IDs. As with the market SMA agent, it uses LangGraph for rollouts and RULER for relative rewards, then updates the policy with GRPO.

Now, let me walk you through how to combine ART and RULER in a financial trading environment. You'll build an agent that computes SMAs for stocks using real price data. The training process uses ART's rollout abstraction and RULER's trajectory ranking to optimize the agent's behavior. I omit the tools and some other code here, but you'll find everything working together in the accompanying notebook.

First, you need to choose a model (Example 10-1) that ART can train. In this case, I'll use a Qwen 2.5 model, though you could just as well point to an Unsloth-optimized 4-bit variant if you want faster training on limited hardware, or any other model you'd like.

*Example 10-1. Choose the model*

```
model = art.TrainableModel(
    name="market-sma-langgraph-001",
    project="market-sma-agent",
    base_model="Qwen/Qwen2.5-7B-Instruct",
)
```

Because we're targeting GPUs with different memory profiles, it's often necessary to override some configuration defaults. For example, when training on a T4, you need to reduce sequence length and adjust memory utilization. ART exposes these low-level configs directly. Example 10-2 shows you how you can adjust these configurations.

*Example 10-2. Override configuration defaults*

```
model._internal_config = art.dev.InternalModelConfig(
    init_args=art.dev.InitArgs(
        max_seq_length=8192, ❶
    ),
    engine_args=art.dev.EngineArgs(
        enforce_eager=True, ❷
        gpu_memory_utilization=0.8, ❸
    ),
)
```

❶ Reduce the max sequence length to fit within GPU limits.

❷ Force eager execution for reproducibility.

❸ Cap GPU usage so training doesn't run out of memory.

Once the model is configured, you need to spin up a backend, by implementing the code from Example 10-3. The `LocalBackend` is the simplest option; it launches the training server on your machine.

*Example 10-3. Spin up the backend*

```
backend = LocalBackend(
    in_process=True, ❶
    path="./.art",
)
```

❶ Normally, you'd run ART as a separate server. On Colab, set `in_process=True` so logs stream inline.

Finally, you register the model with the backend with `await model.register(backend)`. This step wires together the training engine, inference service, and observability stack, so tools like Weights & Biases or Langfuse can plug in. Note that the `await` is just for using this in Colab notebook, since async runs won't work in Colab.

Next, define a schema (Example 10-4) and a tool for the final answer, since agents need a structured way to output results.

*Example 10-4. Final answer setup*

```
class FinalAnswer(BaseModel):
    answer: str
    source_ids: List[str]

nonlocal_final = {"value": None}

@tool
def return_final_answer_tool(answer: str, reference_ids: List[str]) -> dict:
    nonlocal_final["value"] = FinalAnswer(answer=answer, source_ids=reference_ids)
    return nonlocal_final["value"].model_dump()
```

To train the agent, you need a reward. Example 10-5 uses a numeric reward function. This function compares the agent's SMA prediction against the ground truth and assigns a score.

*Example 10-5. Numeric reward function*

```
def judge_sma_numeric(ticker: str, window: int, period_days: int,
                      y_pred: str, tol: float=0.02) -> tuple[float, float, float]:

    df = fetch_prices(ticker, period_days)
    closes = df["Close"].tail(window).tolist()
    if len(closes) < window:
        return 0.0, float("nan"), float("nan")
    truth = sum(closes)/window
    try:
        pred = float(re.findall(r"-?\d+(?:\.\d+)?", y_pred)[0]) ❶
    except Exception:
        return 0.0, truth, float("nan")
    err = abs(pred - round(truth, 2))
    reward = 1.0 if err <= tol else 0.0
    return reward, truth, pred
```

❶ Extracts the number string from the agent's output, tolerating formatting noise.

Now you tie everything together in a rollout, as implemented in Example 10-6. The *rollout* is the "unit of experience" for ART. Each rollout is a complete episode: the agent receives a task, interacts with tools, produces an answer, and receives a reward. This abstraction is domain-agnostic: whether the task is computing stock SMAs, answering emails, or running tests, the rollout always follows the same structure.

*Example 10-6. Rollout with LangGraph agent*

```
@weave.op
async def rollout(model: art.Model,
                  market_scenario: MarketScenario) -> ProjectTrajectory:
    scn = market_scenario.scenario

    nonlocal_final["value"] = None ❶

    traj = ProjectTrajectory(
        reward=0.0,
        messages_and_choices=[],
        metadata={"scenario_id": scn.id, "ticker": scn.ticker,
                  "window": scn.window}, ❷
    )

    system_prompt = dedent(f"""
        You are a market data agent.

        Use tools to fetch prices and compute the {scn.window}-day SMA:
        - Call latest_closes_tool(ticker="{scn.ticker}",
          n={scn.window}, period_days={scn.period_days})
        - Compute SMA = average of those closes.
        - Use calculator for exact arithmetic, e.g. '(x+y+z)/{scn.window}'
        - Round to 2 decimals.
        - MUST call return_final_answer_tool with ONLY the number
          string (e.g., "240.79").

        Constraints:
        - Do not guess. Always call latest_closes_tool first.
        - Do NOT add text; final answer tool gets only the number string.
    """) ❸

    tools = [latest_closes_tool, calculator, return_final_answer_tool]
    chat = init_chat_model(model.name, temperature=0.2, max_tokens=200)
    agent = create_react_agent(chat, tools) ❹

    try:
        config = {
            "configurable": {"thread_id": str(uuid.uuid4())},
            "recursion_limit": MAX_TURNS, ❺
        }
        await agent.ainvoke(
            {"messages": [SystemMessage(content=system_prompt),
             HumanMessage(content=scn.question)]},
            config=config,
        )

        if nonlocal_final["value"]: ❻
            traj.final_answer = nonlocal_final["value"]
            reward, truth, pred = judge_sma_numeric(
                scn.ticker, scn.window, scn.period_days,
```

```
                traj.final_answer.answer, tol=scn.tolerance
            )
            traj.reward = reward
            traj.metrics["truth"] = round(truth, 6) if truth == truth else None
            traj.metrics["pred"] = pred
            traj.metrics["error"] = (abs(round(truth,2)-pred) if
                                     truth==truth and pred==pred else None)
            traj.metrics["correct"] = float(reward)
    except Exception as e:
        print("Rollout error:", e)
        traj.messages_and_choices.append({"role": "assistant",
                                          "content": f"Error: {e}"})
    return traj
```

❶ Resets the final answer holder at the start of each rollout.

❷ Attaches metadata so you can later slice results by scenario or ticker.

❸ Encodes workflow constraints in the prompt to enforce tool discipline.

❹ Binds the model to only the tools you expose.

❺ Gives each rollout its own thread ID and recursion limit for clean logs and safety.

❻ Attaches rewards and metrics directly to the trajectory so training and evaluation remain coupled.

The same rollout design works beyond finance:

- Replace `latest_closes_tool` with a search API or a document reader.
- Replace the numeric SMA reward with, for example, a pass/fail unit test.

The agent still follows the same structure: prompt → tools → answer → reward.

Once you have rollouts, you can generate batches, assign rewards, and train the model. You can see how to implement this in Example 10-7. The training objective then turns these rollouts into gradient updates via GRPO, which essentially starts by sampling prompts and generating completions. The reward model then evaluates these completions to compute individual rewards $r_i$. These rewards are group-normalized across completions for each prompt, where $r_i$ values are centered and scaled to produce relative comparisons that serve as the policy update signal.

Next, the Kullback–Leibler (KL) divergence term, introduced in "Diffusion Models: What's That Noise About?" on page 99, is estimated between the current and reference policies. It approximates how much the probability distribution $Q$ of the updated policy differs from the true or reference distribution $P$. In practice, this KL term

acts as a regularizer that penalizes the model for drifting too far from its reference distribution while still allowing meaningful improvement. Finally, the model optimizes the mean GRPO loss, defined as the reward minus the KL penalty, which combines both terms into a single objective. Figure 10-2 illustrates a high-level overview of this flow.

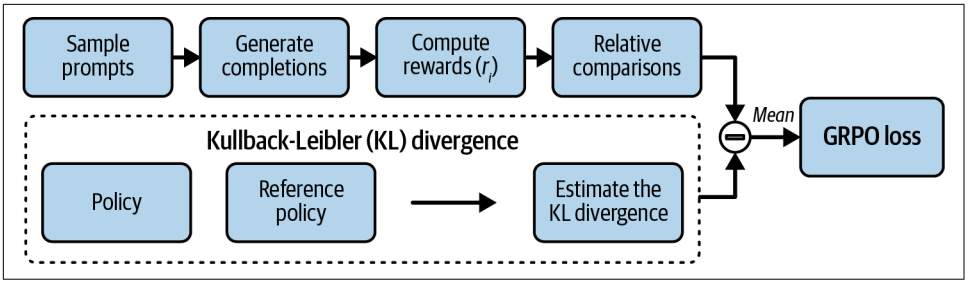


*Figure 10-2. High-level GRPO mechanics: group-normalized rewards plus reference-policy KL as a mean loss.*

*Example 10-7. Training loop with numeric rewards*

```
training_config = { ❶
    "groups_per_step": 1,
    "num_epochs": 2,
    "rollouts_per_group": 4,
    "learning_rate": 1e-5,
    "max_steps": 3,
}

training_iterator = iterate_dataset( ❷
    scenarios,
    groups_per_step=training_config["groups_per_step"],
    num_epochs=training_config["num_epochs"],
    initial_step=await model.get_step(),
)

for batch in training_iterator: ❸
    print(f"Training step {batch.step}, epoch {batch.epoch}")
    groups = []
    for scn in batch.items:
        group = art.TrajectoryGroup(
            [wrap_rollout(model, rollout)(model,
                          MarketScenario(step=batch.step, scenario=scn))
              for _ in range(training_config["rollouts_per_group"]) ]
        )
        groups.append(group)

    finished = await art.gather_trajectory_groups( ❹
        groups, pbar_desc="gather",
        max_exceptions=training_config["rollouts_per_group"] * len(batch.items),
```

```
    )

    print_group_summary(finished, "Pre judge group summary")

    judged = finished ❺

    print_group_summary(judged, "Post judge group summary")

    await model.train( ❻
        judged,
        config=art.TrainConfig(learning_rate=training_config["learning_rate"]),
        _config={"logprob_calculation_chunk_size": 8},
    )
    print(f"Completed training step {batch.step}")

    if batch.step >= training_config["max_steps"]:
        break
```

❶ Training is mini-batched into groups per step. Each group bundles several rollouts for stability.

❷ `iterate_dataset` handles sampling and ensures reproducibility across epochs.

❸ Each scenario spawns multiple rollouts → turned into a `TrajectoryGroup`. This makes GRPO-style relative scoring possible.

❹ `gather` executes rollouts concurrently and collects results safely, handling partial failures.

❺ You've already computed numeric rewards in rollout.

❻ Each batch ends with a gradient update, informed by the reward signal.

## GRPO Intuition

GRPO works by collecting multiple candidate answers for the same task and ranking them. The learning signal comes from these relative rankings, not from absolute scores. This makes training more stable, because the model only needs to improve within each group of candidates.

The update also includes a regularization term that keeps the new policy close to a reference, so learning is steady and safe.

The result is a reinforcement learning method that scales well, avoids brittle hand-crafted rewards, and steadily improves reasoning and tool use.

Finally, you can swap out the numeric reward for RULER's relative scoring, as shown in Example 10-8. This turns your setup into a scalable, general-purpose RL loop.

*Example 10-8. RULER scoring integration*

```
judged_groups = []
for group in finished:
    judged_group = await ruler_score_group(group, "openai/o4-mini", debug=True) ❶
    judged_groups.append(judged_group)

await model.train(
    judged_groups, ❷
    config=art.TrainConfig(learning_rate=training_config["learning_rate"]),
    _config={"logprob_calculation_chunk_size": 8}, ❸
)
```

❶ RULER deduplicates shared prefixes and applies relative scoring using an LLM-as-judge.

❷ The ranked trajectory groups are then used directly for policy updates.

❸ Chunk size is lowered for stable log-prob calculations on long sequences.

**Prevent Leakage**

Guard against leakage by prohibiting tools from fetching the gold answer during judging to keep the judge prompt deterministic. *Leakage* occurs when the evaluation prompt or an external tool indirectly exposes the ground-truth solution to the model. This can lead the LLM-as-judge to overfit or memorize answers instead of genuinely assessing reasoning quality, producing inflated scores and unstable training signals. The *gold answer* is the ground-truth solution used for evaluation. To avoid leakage, ensure that the judge prompt is fully deterministic and that all inputs available to the LLM are limited to the agent's raw trajectories and the evaluation rubric.

As you've seen, the ART-LangGraph integration makes it simple to build AI agents that get smarter with experience, helping you turn rough prototypes into reliable, real-world systems. And tracking the experiments with Weights & Biases or LangFuse, this gives you a good way to improve your agents and the underlying LLMs. However, strong policies still should not overthink easy inputs. *Overthinking* is when an LLM performs unnecessary reasoning steps on easy inputs, which increases computational cost without improving accuracy. The good news is LLMs can be trained

for selective reasoning, enabling them to allocate reasoning only when the task requires it.

# Reason Smarter, Not Harder: Adaptive Compute Allocation

While systems like ART and RULER teach models how to reason effectively, they still apply complex, multi-step processes to every task. This leads to an efficiency bottleneck: forcing an LLM to perform an extended CoT reasoning path for a simple problem wastes compute, increases latency, and can even introduce "overthinking" that could degrade the final answer.

An easy way to address this and help your models decide when to think is AdaptThink.[1] AdaptThink teaches the model to allocate compute strategically and adapt for difficulty. It moves the model from being a purely reactive reasoner to a strategic resource manager that adapts its inference strategy according to task complexity. It has two modes:

*Thinking mode*
: A verbose, CoT-style process for complex problems, where the computational cost is justified by the higher accuracy

*NoThinking mode*
: Provides a direct, concise final answer and is often more efficient and performant for straightforward problems

The core insight from the paper is that the benefits of CoT are not always given; instead, they become helpful only when the problem is actually hard.

## The Delta Incentive: Enforcing Efficiency

To teach the model this strategic allocation, AdaptThink uses a specialized RL objective that encodes a cost-benefit analysis directly into the reward signal. This mechanism relies on two components:

- The policy is penalized if its overall accuracy drops below that of a reference model.
- The model receives a structural bonus, delta, for choosing the highly efficient `NoThinking` mode.

1 Jiajie Zhang et al. "AdaptThink: Reasoning Models Can Learn When to Think" (2025).

This creates a competitive dynamic where the model is mathematically pushed toward the cheapest answer unless the expected performance gain from thinking outweighs the efficiency bonus. The policy must at least match the accuracy of a frozen reference model while earning a structural δ-bonus whenever it selects `NoThinking`. In practice, δ is a dial: higher δ favors efficiency (more `NoThinking`), while lower δ favors accuracy (more `Thinking`), allowing you to adjust the policy to match your latency or token-cost requirements. Figure 10-3 illustrates the concept.

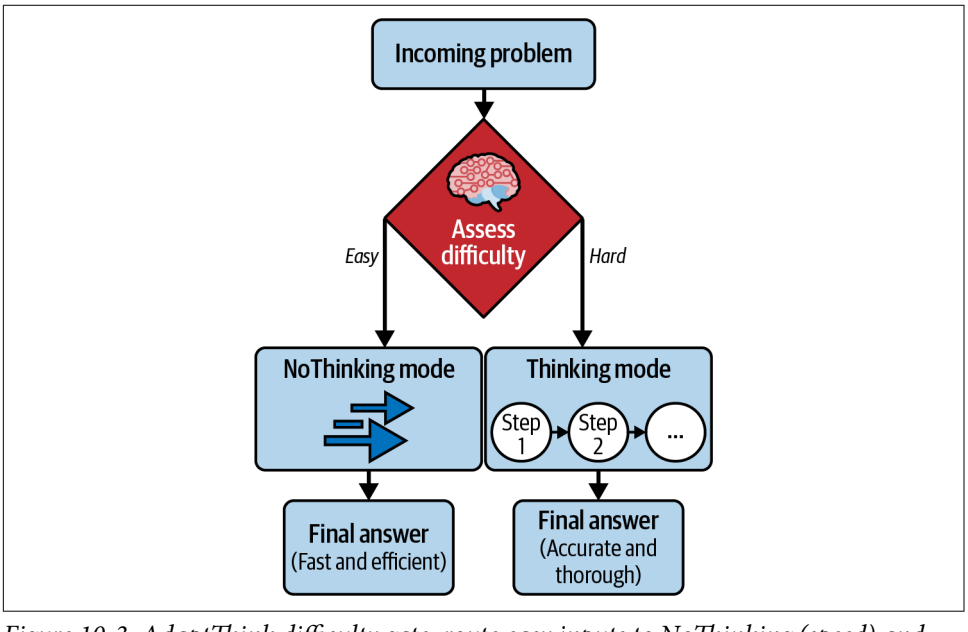


*Figure 10-3. AdaptThink difficulty gate: route easy inputs to NoThinking (speed) and hard ones to Thinking (accuracy).*

The official repository provides end-to-end training scripts, but it's helpful to understand what each stage actually does. The first step is to check the tokenizer templates. Both `<think>` and `</think>` must appear at the correct positions in `tokenizer_config.json`; otherwise, the gating mechanism that distinguishes `Thinking` and `NoThinking` modes will silently fail. Once you've verified the template, the next step is to presample reference results. Here, you run the frozen reference model multiple times on each training problem to compute an instance-level accuracy baseline. This baseline is important: it anchors the optimization by ensuring that the trained model can't go below the reference accuracy already achieved.

Training itself is launched using either single-node or multi-node scripts, depending on model size, with a chosen value of δ determining how aggressively the policy is biased toward `NoThinking` responses.

### How to Tune $\delta$

As a rule of thumb, set δ according to how much efficiency you're willing to trade for accuracy:

- Around 0.02, the model remains conservative, favoring thinking and closely preserving accuracy.
- Near 0.05, you usually get the best balance, with shorter responses and stable accuracy.
- Beyond 0.075, the policy becomes aggressive, maximizing `NoThinking` and efficiency but often at the expense of performance on harder tasks.

After training, checkpoints are converted to Hugging Face format for evaluation, and the supplied scripts allow you to measure the accuracy, response length, and `NoThinking` ratio consistently.

By using AdaptThink, you shift your model from applying uniform reasoning to applying reasoning selectively, based on task difficulty. The reinforcement signal you set ensures that efficiency gains never come at the expense of overall accuracy, while the $\delta$-incentive lets you control the cost/performance balance. The outcome is a model that can maintain high accuracy on complex problems while avoiding unnecessary computation on simpler ones.

## Open Innovation: Community-Driven RL Frameworks

RL for agents is not just advancing inside corporate labs; it's actively being shaped by open source frameworks like RL for language agents (rLLM). rLLM is a high-throughput RL infrastructure giving researchers and engineers access to the same architectural setups as proprietary models so they can run distributed sampling, integrate custom tools, and scale post-training for language agents.

rLLM's interface addresses the full lifecycle of agent development, the training phase and the real-world deployment for production workloads. The Python-based framework focuses on distributed RL infrastructure to deliver efficient trajectory generation and scalable RL training through specialized execution mechanisms. Table 10-3 maps the classic sampler-trainer loop into rLLM's modular components.

*Table 10-3. rLLM components in the RL training loop*

| Concept | rLLM component | Role | Notes | Example |
|---|---|---|---|---|
| Sampler (trajectory generator) | AgentExecutionEngine | Runs agents in environments to produce trajectories in parallel | Async, OpenAI messages protocol, handles tool calls | Batch pass@k evaluation |
| Trainer (policy updater) | AgentTrainer | Coordinates policy updates via a pluggable RL backend | Commonly paired with verl for PPO or GRPO | GRPO over reasoning tasks |
| Agent policy | ToolAgent | Reasoning loop with tool use | Parser configurable, compact prompts | Python tool calls |
| Environment | ToolEnvironment | Safe tool execution and reward computation | Time limits, output caps, deterministic rewards | Numeric judge or task-specific reward |
| Dataset layer | DatasetRegistry | Standardizes datasets and splits for reuse | Stores Parquet, load by key | Train and test splits |

rLLM provides recipes that train coding agents like DeepCoder, which reached 60.6% pass@1 on LiveCodeBench with only open-weight models. Its math-specialized sibling, DeepScaleR, exceeded proprietary baselines on AIME, a difficult reasoning benchmark. And in software engineering, SWE-agent achieved leaderboard results on SWE-Bench, a benchmark grounded in real GitHub issues.

rLLM provides utilities for your entire workflow. With `DatasetRegistry`, you can register datasets once, standardize preprocessing, and store them in Parquet format for both inference and training. For example, math or coding benchmarks can be normalized into LiveCodeBench-style fields with a small preprocessing function, as shown in Example 10-9, and then plugged directly into agents for training or evaluation.

*Example 10-9. DeepCoder preprocessing step*

```
def _preprocess_fn(example, idx):
    tests, metadata, starter_code = _normalize_tests(example) ❶
    problem = example.get("problem") or example.get("prompt") or
              example.get("question") or "" ❷
    question = fetch_live_code_bench_system_prompt(problem, starter_code if
               starter_code else None) ❸
    return {
        "question": question,
        "ground_truth": json.dumps(tests), ❹
        "data_source": "livecodebench",
        "uid": f"deepcoder_{idx}", ❺
        "index": idx,
        "starter_code": starter_code,
        "metadata": json.dumps(metadata),
    }
```

❶ Normalize raw test cases into a consistent structure.

❷ Fallbacks ensure that a valid problem statement is always used.

❸ Wraps problem and starter code into a LiveCodeBench-style prompt.

❹ Store functional tests as JSON for the reward function.

❺ Each row gets a unique ID for tracking across tasks.

With preprocessing in place, you can register your train splits and your test splits via `DatasetRegistry` so rLLM can load them by key. The function in Example 10-10 and the CLI in Example 10-11 take care of preparing the data for rLLM.

*Example 10-10. Register and shard the dataset*

```
def prepare_deepcoder_data(train_size: int = 0, test_size: int = 0,
                           num_proc: int = 8):
    name = "agentica-org/DeepCoder-Preview-Dataset"

    train_parts = [ds for cfg in TRAIN_CONFIGS if
                   (ds := _safe_load(name, cfg, "train")) is not None]
    if not train_parts:
        raise RuntimeError("No train splits found among primeintellect, taco, lcbv5")

    test_parts  = [ds for cfg in TEST_CONFIGS  if
                   (ds := _safe_load(name, cfg, "test"))  is not None]
    if not test_parts:
        # fallback: carve a small test slice out of train concat
        test_parts = [train_parts[0].select(range(min(100, len(train_parts[0]))))]

    train_ds = concatenate_datasets(train_parts)
    test_ds  = concatenate_datasets(test_parts)

    if train_size and train_size > 0:
        train_ds = train_ds.select(range(min(train_size, len(train_ds))))
    if test_size and test_size > 0:
        test_ds = test_ds.select(range(min(test_size, len(test_ds))))

    train_ds = train_ds.map(_preprocess_fn, with_indices=True,
                            writer_batch_size=16, num_proc=num_proc) ❶
    test_ds  = test_ds.map(_preprocess_fn,  with_indices=True,
                            writer_batch_size=16, num_proc=num_proc)

    train_reg = DatasetRegistry.register_dataset("deepcoder", train_ds, "train") ❷
    test_reg  = DatasetRegistry.register_dataset("deepcoder_test", test_ds, "test")
    return train_reg, test_reg
```

❶ Apply preprocessing in parallel for efficiency.

❷ Register datasets under keys used later in evaluation.

*Example 10-11. CLI for dataset registration*

```
if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--train_size", type=int,
                    default=0, help="cap train rows for quick runs")
    ap.add_argument("--test_size",  type=int,
                    default=0, help="cap test rows for quick runs")
    ap.add_argument("--num_proc",   type=int, default=8)
    args = ap.parse_args()

    train_dataset, test_dataset = prepare_deepcoder_data(
                                  args.train_size, args.test_size, args.num_proc)
    print(f"- Train rows: {len(train_dataset.get_data())}")
    print(f"- Test rows:  {len(test_dataset.get_data())}")
    print(train_dataset.get_data()[0])
```

To create and register a small dataset, run `python prepare_code_data.py --train_size 200 --test_size 50` in your terminal. This writes the splits into rLLM's registry. At evaluation time, the loader in Example 10-12 selects either the humaneval test split or a livecodebench-style split you registered.

*Example 10-12. Load a registered split*

```
def load_eval_dataset(name: str):
    if name == "humaneval":
        return DatasetRegistry.load_dataset("humaneval", "test")
    elif name == "livecodebench":
        # expects you have registered a LiveCodeBench test split
        return DatasetRegistry.load_dataset("livecodebench", "test")
    else:
        raise ValueError(f"unknown eval split {name}")
```

Next, you need to configure the evaluator. The CLI flags in Example 10-13 define the model, endpoint, dataset key, and pass at k settings.

*Example 10-13. Evaluator CLI arguments*

```
def parse_args():
    p = argparse.ArgumentParser()
    p.add_argument("--model", type=str, default="Qwen/Qwen3-4B") ❶
    p.add_argument("--base_url", type=str, default="http://localhost:30000/v1") ❷
    p.add_argument("--api_key", type=str, default="None")
    p.add_argument("--dataset", type=str, default="deepcoder_test")  ❸
```

```
    p.add_argument("--split", type=str, default="test")
    p.add_argument("--parallel", type=int, default=8) ❹
    p.add_argument("--temperature", type=float, default=0.2) ❺
    p.add_argument("--top_p", type=float, default=0.95)
    p.add_argument("--max_response_length", type=int, default=32768)
    p.add_argument("--max_prompt_length", type=int, default=8192)
    p.add_argument("--limit", type=int, default=0)
    p.add_argument("--repeat", type=int, default=4) ❻
    return p.parse_args()
```

❶ Model identifier to load tokenizer and route inference requests.

❷ Base URL of your OpenAI-compatible inference server.

❸ Dataset key registered via `DatasetRegistry`.

❹ Number of parallel agents to run at once.

❺ Sampling temperature for stochastic decoding.

❻ Number of independent attempts per task to compute pass@k.

The loader in Example 10-14 pulls either the DeepCoder registry entry or a HumanEval fallback.

*Example 10-14. Load a registered split*

```
def load_eval_dataset(dataset_name: str, split: str):
    if dataset_name.startswith("deepcoder"):
        return DatasetRegistry.load_dataset(dataset_name, split) ❶
    if dataset_name == "humaneval":
        return DatasetRegistry.load_dataset("humaneval", "test") ❷
    raise ValueError(f"unknown dataset {
                   dataset_name}. Expected deepcoder* or humaneval")
```

❶ Loads the DeepCoder dataset you registered earlier.

❷ Fallback to HumanEval if requested.

Now you create the engine and environment. The block in Example 10-15 wires the `ToolAgent` to the Python tool and the code reward.

*Example 10-15. Build the evaluation engine*

```
if __name__ == "__main__":
    os.environ["TOKENIZERS_PARALLELISM"] = "true"
    args = parse_args()

    tokenizer = AutoTokenizer.from_pretrained(args.model) ❶

    agent_args = {
        "tools": ["python"], ❷
        "parser_name": "qwen",
        "system_prompt": "You are a coding assistant that writes and
                          executes Python functions to solve programming problems.",
    }
    env_args = {"tools": ["python"], "reward_fn": code_reward_fn} ❸

    engine = AgentExecutionEngine(
        agent_class=ToolAgent,
        agent_args=agent_args,
        env_class=ToolEnvironment,
        env_args=env_args,
        engine_name="openai", ❹
        rollout_engine_args={"base_url": args.base_url, "api_key": args.api_key},
        tokenizer=tokenizer,
        sampling_params={"temperature": args.temperature, "top_p": args.top_p,
                         "model": args.model}, ❺
        max_response_length=args.max_response_length,
        max_prompt_length=args.max_prompt_length,
        n_parallel_agents=args.parallel, ❻
    )

    test_dataset = load_eval_dataset(args.dataset, args.split)
    if test_dataset is None:
        raise RuntimeError(f"dataset {
              args.dataset}:{args.split} not found.
              Register it via DatasetRegistry first.")
```

❶ Load the tokenizer for consistent input/output packing.

❷ Assign Python execution as the ToolAgent's tool.

❸ Validate code correctness with the reward function.

❹ Use the OpenAI messages protocol in the engine.

❺ Forward decoding settings to the backend.

❻ Specify the number of agents running in parallel.

Create the repeated task list for pass@k. The task assembly in Example 10-16 mirrors your math example.

*Example 10-16. Create repeated tasks for pass@k*

```
    if not args.limit or args.limit <= 0:
        tasks = test_dataset.repeat(n=args.repeat)  ❶
    else:
        base = getattr(test_dataset, "get_data", lambda: None)()
        if base is None:
            one_pass = test_dataset.repeat(n=1)
            base_slice = one_pass[: args.limit]
        else:
            k = min(args.limit, len(base))
            base_slice = [base[i] for i in range(k)]
        tasks = base_slice * max(1, args.repeat) ❷
```

❶ Default: repeat the whole dataset n times.

❷ For quick tests, slice a subset and multiply for repetitions.

Finally, you need to call the engine to run and compute metrics, as shown in Example 10-17.

*Example 10-17. Entry point to run pass@k metrics*

```
results = asyncio.run(engine.execute_tasks(tasks)) ❶
compute_pass_at_k(results) ❷
```

❶ Execute the evaluation asynchronously across agents.

❷ Report pass@1 and pass@k accuracy.

Before running the evaluation, you need a model server to handle inference. You can launch one locally with vLLM, as shown in Example 10-18, or with SGLang, as shown in Example 10-19. Both options expose an OpenAI-compatible API at `http://localhost:30000/v1`.

*Example 10-18. Start a vLLM server*

```
python -m vllm.entrypoints.openai.api_server \
    --model Qwen/Qwen3-4B \ ❶
    --host 0.0.0.0 \ ❷
    --port 30000 \ ❸
    --dtype bfloat16 ❹
```

❶ Specify the Hugging Face model name or local checkpoint path.

❷ Bind the server to all interfaces.

❸ Set the port where the OpenAI-compatible endpoint will be served.

❹ Use `bfloat16` to reduce memory usage without significant loss of accuracy.

You can also start a server with SGLang, a lightweight and efficient inference framework designed for running large models with low latency. It supports the same OpenAI-compatible API, so you can easily switch between vLLM and SGLang without changing your evaluation scripts.

*Example 10-19. Start an SGLang server*

```
python -m sglang_router.launch_server \
    --model-path Qwen/Qwen3-4B \ ❶
    --dp-size 1 \ ❷
    --dtype bfloat16 ❸
```

❶ Path to the model, same as for vLLM.

❷ Data parallel size, set to 1 for a single GPU.

❸ Mixed precision for efficient inference.

This creates an OpenAI-compatible endpoint that you can access via `http://localhost:30000/v1`.

Once the server is up, you can run the evaluation script. The command in Example 10-20 evaluates 20 problems with four parallel agents, producing 80 agent trajectories in total.

*Example 10-20. Run the evaluation*

```
python run_code_with_tool.py
--dataset deepcoder_test ❶
--limit 20 ❷
--parallel 4 ❸
```

❶ Register the dataset key in DatasetRegistry.

❷ Restrict the run to 20 problems.

❸ Spawn four parallel agents so each problem is repeated 4 times for pass@k.

This will run the evaluation with 20 unique problems and create in total 80 agent trajectories. This process will take about an hour to run. I get the following result:

```
Progress: 80/80 trajectories completed
Total unique problems: 20
Average Pass@1 Accuracy: 0.3375
Average Pass@k Accuracy: 0.55
```

To improve the agent's ability to use its tool more effectively, you can train it with GRPO or other RL methods. rLLM integrates with verl,[2] an open source, production-ready RL training library for LLMs that supports multiple algorithms and is designed for efficiency at scale. The training loop in Example 10-21 shows how to connect a `ToolAgent`, tool environment, and the registered DeepCoder dataset into an RL pipeline, while Figure 10-4 visualizes this flow.

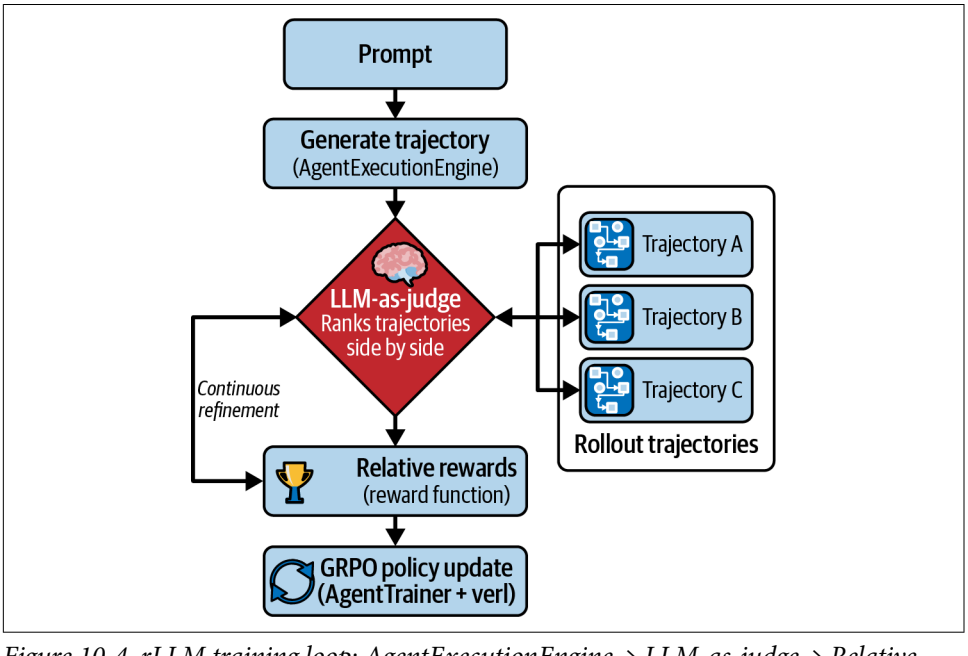


*Figure 10-4. rLLM training loop: AgentExecutionEngine → LLM-as-judge → Relative rewards → GRPO via AgentTrainer (verl) → continuous refinement.*

*Example 10-21. Train a coding agent with GRPO (rLLM + verl)*

```
@hydra.main(config_path="pkg://rllm.trainer.config",
            config_name="ppo_trainer", version_base=None)
def main(config):
```

2 Guangming Sheng et al. "HybridFlow: A Flexible and Efficient RLHF Framework" (2024).

```
train_dataset = DatasetRegistry.load_dataset("deepcoder", "train")
val_dataset   = DatasetRegistry.load_dataset("deepcoder_test", "test")

agent_args = {
    "tools": ["python"], ❶
    "parser_name": "qwen",
    # keep prompt compact to save tokens
    "system_prompt": (
        "Write a correct Python function for the task. "
        "Use the python tool to run tests. Return only the final function."
    ),
}

env_args = {
    "tools": ["python"],
    "reward_fn": code_reward_fn, ❷
    "max_steps": 8,                ❸
    "tool_time_limit_sec": 10,     ❹
    "max_tool_output_chars": 4000, ❺
}

trainer = AgentTrainer(
    agent_class=ToolAgent, ❻
    env_class=ToolEnvironment,
    agent_args=agent_args,
    env_args=env_args,
    config=config,              ❼
    train_dataset=train_dataset,
    val_dataset=val_dataset,
)
trainer.train()
```

❶ Configure the ToolAgent with the Python tool for code execution.

❷ Check the correctness of generated solutions with the reward function.

❸ Allow multiple think execute cycles per tool call.

❹ Per tool call.

❺ Trim `stdout` to avoid context blowup.

❻ Coordinate training of the agent inside the tool environment with AgentTrainer.

❼ Inject the training configuration (PPO or GRPO) from Hydra.

In this example, the different pieces of rLLM came together to form a full training and evaluation loop. The `ToolAgent` provided the reasoning layer with access to Python execution, while the `ToolEnvironment` ensured that this execution happened

in a safe and controlled sandbox. Datasets were prepared and managed centrally through the `DatasetRegistry`, making them easy to register once and reuse for both evaluation and training. For evaluation, the `AgentExecutionEngine` enabled efficient, parallel rollouts that produced pass@k metrics across many tasks. Finally, the `AgentTrainer` orchestrated the RL process itself, applying GRPO on top of the agent and environment to steadily improve its tool use. Together, these components give you an easy way to refine the agent's performance through RL.

## The Checkpoint Engine: Systems-Level Optimization for LLM Policy Updates

The success of a scalable RL system relies on a high-throughput, low-latency loop for policy updates. The open source middleware Checkpoint Engine[3] is designed to solve this bottleneck of transferring updated model weights between the training step and the generation step, enabling faster iteration for RL. The engine facilitates a hybrid colocated architecture where the entire RL process is contained within the same worker nodes: a training engine (the learner) and an inference engine (the actor) reside side by side. This physical proximity minimizes data transfer overhead, which is important to optimize training of larger models. Figure 10-5 shows an overview of this training process via the checkpoint engine.

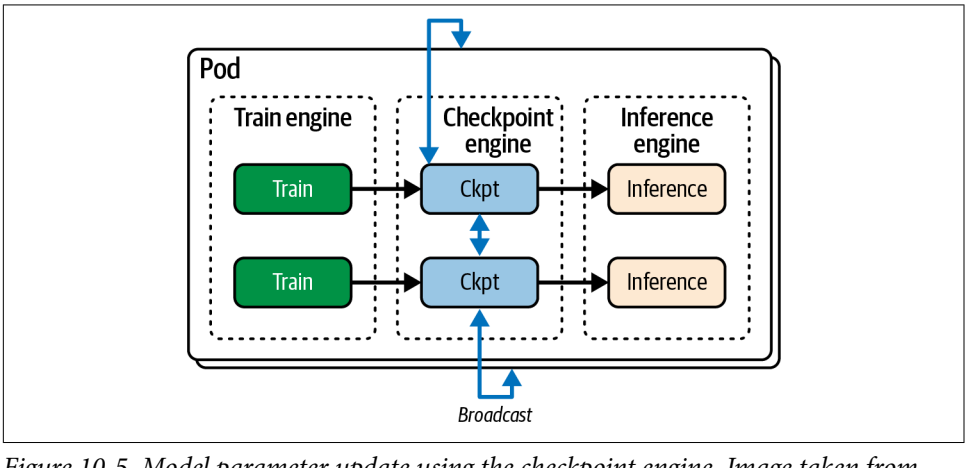


*Figure 10-5. Model parameter update using the checkpoint engine. Image taken from Yifan Bai et al. (2025).*

In modern RL for agentic systems, the *inference engine* generates new agent trajectories using the current policy (acting as the actor). The *training engine* then uses this

3 Yifan Bai et al. "Kimi K2: Open Agentic Intelligence" (2025).

data to compute a new, improved policy (acting as the learner). The system must efficiently shift between these two engines, updating the model's weights in the process. The checkpoint engine facilitates this fast transition, enabling an efficient, high-frequency RL loop.

The core of the checkpoint engine is the `ParameterServer` service, which is designed to run colocated with the LLM inference engine (such as vLLM) on the same GPU workers. This service offers two primary, optimized methods for distributing model weights. The *broadcast method* is the fastest and default, intended for synchronous updates across a cluster. It uses a high-throughput, pipelined data transfer strategy, including host-to-device (H2D) transfer, inter-worker broadcast, and reloading from a shared CUDA IPC buffer, to apply the new policy. However, due to hardware constraints like PCIe saturation on devices such as NVIDIA H800 clusters, concurrent H2D and broadcast operations are often not possible. The engine therefore adopts a practical two-stage scheme: all devices perform a single, synchronous H2D transfer, followed by the broadcast and reload proceeding in parallel, as illustrated in Figure 10-6.

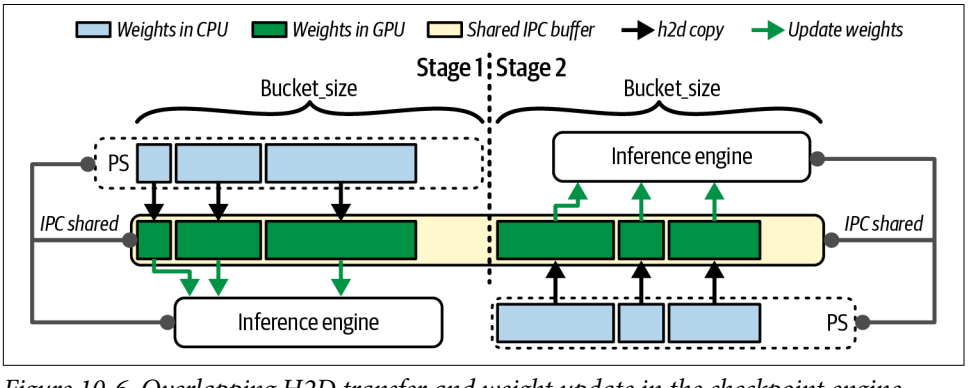


*Figure 10-6. Overlapping H2D transfer and weight update in the checkpoint engine. Image taken from Yifan Bai et al. (2025).*

The alternative method, *peer-to-peer* (P2P) is designed for greater robustness, allowing new instances to asynchronously join the cluster without disrupting the ongoing rollouts of existing workers. By abstracting this, the checkpoint engine supports effective RL for LLMs and agentic systems.

In practice, this means that models are not only trained more efficiently but can also be kept fresh in deployed environments. With rapid distribution of updated weights, actors run on the latest policy almost immediately, closing the loop between learning and deployment. This ability to iterate continuously without interrupting service is what turns RL from a research workflow into a production-grade system.

# Conclusion

In this chapter, you saw how RL converts objectives into behavior, with ART providing a developer-friendly rollout loop, GRPO enabling stable group-based updates, and RULER turning LLM-as-judge into a reliable relative reward. Together, these methods align reasoning, tool use, and decision policies so agents improve through experience.

You learned that speed and accuracy are not enemies if the model learns when to think. AdaptThink introduces selective reasoning with a simple idea: prefer the cheapest valid path, switching into deliberate chains of thought only when the expected gain justifies it. By tuning the δ, you can match policy behavior to your latency or cost budget while preserving accuracy on hard problems.

Open frameworks make RL practical. With rLLM, you can register datasets, run high-throughput rollouts, score trajectories, and train agents with PPO or GRPO. Benchmarks for coding, math, and software engineering provide grounded evaluation so progress is measurable and repeatable.

Finally, you looked at systems-level plumbing. Checkpoint engine closes the loop between acting and learning by moving updated weights to inference servers quickly and safely. This keeps the policy fresh in production settings and enables continuous improvement without service disruption.

With these principles and tools, you're ready to move from optimization to deployment. Chapter 11 focuses on putting transformer models into production. You'll learn how to serve models efficiently, monitor quality and drift, manage safety and guardrails, and set up rollbacks and cost controls for real workloads.

# CHAPTER 11
# Deploying Transformer Models

Now that you know about different transformer models, it's time to put them to work. Deploying transformers with tools like vLLM or SGLang has become straightforward on most major cloud platforms, such as GCP or AWS. Therefore, this chapter doesn't focus on cloud deployment instructions but on the objectives that kept me up at night when I first deployed transformer-based application, such as cost and data control, runtime engineering, cache behavior, architecture-specific constraints, and infrastructure strategy.

With this in mind, one of the first choices that shapes how you set up your application is whether you rely on open or closed models. This choice is not a purely technical question; it determines your organization's ability to control costs, scale flexibly, audit behavior, customize models, and maintain long-term independence.

It's also important to understand that deploying a transformer isn't simply a matter of taking an open source checkpoint and calling `.generate`. In production, a model becomes just one component in a larger system that includes routing, storage, monitoring, guardrails, and infrastructure. And even if you choose open source models, this does not automatically mean self-hosting. Most open models can also be accessed through managed cloud APIs, allowing fast experimentation before eventually moving to self-hosted deployments for cost efficiency or compliance.

Once you decide how you want to access your model—closed API, managed open source API, or fully self-hosted—the next step is understanding the architecture you're deploying. Different transformer families have different runtime, memory, and concurrency characteristics, which directly affect cost, latency, and operational behavior. Finally, you may want to optimize them through quantization.

This chapter provides a structured way to think about these decisions. The following list acts as a simple roadmap that applies across modalities and helps you move transformers from ideation to production in an organized way:

1. Define the task, constraints, and modality.
2. Choose an appropriate architecture and model family.
3. Decide on open versus closed deployments.
4. Optimize the model for inference (quantization, pruning, sharding).
5. Engineer the inference runtime, including cache and thinking modes.
6. Integrate monitoring, benchmarking, and guardrails.
7. Choose and operate the infrastructure layer.

In the remainder of this chapter, I'll guide you through the open-versus-closed choice, explain what to watch for in different transformer architectures, and show you how to optimize models for real-world production environments. The goal is to give you guidance that helps you navigate deployment.

## Choosing Between Open and Closed Source

Closed models offer strong baseline performance and immediate usability but at the cost of opacity and vendor dependence. Open models provide visibility and control yet traditionally required infrastructure experience to deploy. However, the distinction is no longer binary. Most open source models can be accessed through managed cloud APIs, such as Together.ai, Novita.ai, or Nebius, allowing teams to experiment quickly without running their own GPU clusters.

In practice, the deployment spectrum ranges from fully closed models accessed via proprietary APIs, to open models accessed via managed cloud providers, to self-hosted open models that give you full control over cost, compliance, and optimization. Both open and closed models coexist in many organizations: closed models for rapid prototyping or broad general tasks and open models for scalable, auditable, domain-specific systems that need to evolve over time. In practice, many teams start with open models via managed cloud providers and later migrate workloads that require direct data access to self-hosted environments. Table 11-1 gives you an orientation on comparing open source and closed source models.

*Table 11-1. Open versus closed models: a practical comparison*

| Factor | Closed source models | Open source models |
|---|---|---|
| Speed of innovation | Depends on vendor release cycles; immediate API access | Global research drives rapid iteration; usually available via both cloud APIs and self-hosting |
| Accessibility | Simple API usage; limited visibility into data, weights, or architecture | Full visibility and auditability; accessible via Hugging Face, cloud APIs, or local deployment |
| Customization | Fine-tuning possible inside vendor environment; no control over weights or runtime | Full control over weights, training workflow, and inference runtime |
| Performance | Often achieve top benchmarks due to proprietary compute/training advantages | Performance gap narrowing quickly; modern models (Qwen-3, Kimi K2) reach near-parity |
| Cost | High per-token pricing; predictable but potentially expensive at scale | Low per-token cost via managed OSS providers; lowest long-term cost when self-hosted |
| Data security and compliance | Relies on vendor certifications; limited control over data residency | Full control when self-hosted; configurable compliance for regulated industries |
| Operational burden | Very low; vendor handles infrastructure | Varies: zero when using managed APIs; higher for self-hosting |

This often leads to a hybrid approach. Early in a project's lifecycle, managed closed APIs or managed open source providers allow you to iterate quickly without committing to infrastructure. Later, as your team expands and your use cases demand more control or customization, it becomes clear that it's better to bring open source deployments in-house. At that stage, owning your infrastructure, fine-tuning models, and optimizing inference becomes a core competence rather than an operational burden, and can, in fact, lower costs over time.

# Understanding the Architecture You're Deploying

Another important part in moving from proof of concept to actual deployment is to understand that different transformer architectures exhibit different runtime, memory, and cost behaviors. This is why you need to be aware about these constraints and deploy accordingly. In general, your deployment strategy must align with the underlying architecture:

*Decoder-only models (LLMs, code models, some time series transformers)*
: These models generate outputs autoregressively and rely heavily on *key-value (KV) caching* to avoid recomputing attention over previously processed tokens. Their runtime behavior is dominated by context length, batching strategy, and scheduling efficiency. They also support different thinking modes, such as chain-of-thought, multi-sample decoding, or tool calls, all of which increase inference cost by triggering additional forward passes. These model architectures don't just

exist for LLMs but also for time series. TimesFM[1] is, for instance, a decoder-only time series foundation model.

*Encoder-only models (vision transformers, retrieval encoders, some anomaly detectors)*
: These models operate in a feed-forward, stateless manner, making them naturally batch-friendly and high throughput. Since there's no temporal state to maintain across requests, latency is predictable, memory usage is easier to control, and scaling is straightforward. They're typically used for embedding extraction, classification, and retrieval.

*Encoder-decoder models (machine translation, diffusion pipelines, video transformer stacks, end-to-end multimodal)*
: These systems perform multi-stage inference, with an encoder producing a latent representation that conditions the decoder. They often rely on latent caching, cross-attention, or repeated denoising steps, depending on the modality. Their memory footprint is higher than encoder-only models, but many parts of their computation can be parallelized across devices or stages.

*Specialized architectures (latent diffusion, DiT, ViT backbones, PatchTST, video diffusion transformers)*
: These models introduce modality-specific runtime characteristics. Latent diffusion models reuse text embeddings and latent states across sampling steps, time series transformers reuse rolling hidden states, and video models cache temporal features across frames. Their performance and memory behavior are shaped less by token length and more by spatial and temporal resolution, making memory planning and batching critical for stable deployment.

Understanding these architectural differences is essential because each family imposes its own constraints on latency, memory usage, batching efficiency, and cost. Once you know which architecture you're working with, you can design a deployment strategy that fits its behavior rather than treating all transformers as interchangeable.

With that in mind, in the following sections I'll cover more deeply some specialties you need to think about when you're deploying different transformer architectures.

1 Abhimanyu Das et al. "A Decoder-Only Foundation Model for Time-Series Forecasting" (2024).

# Deploying Decoder-Only Models

Decoder-only models, such as LLMs and code models, operate autoregressively and therefore have a very distinct runtime profile. Figure 11-1 shows an overview of this autoregressive process.

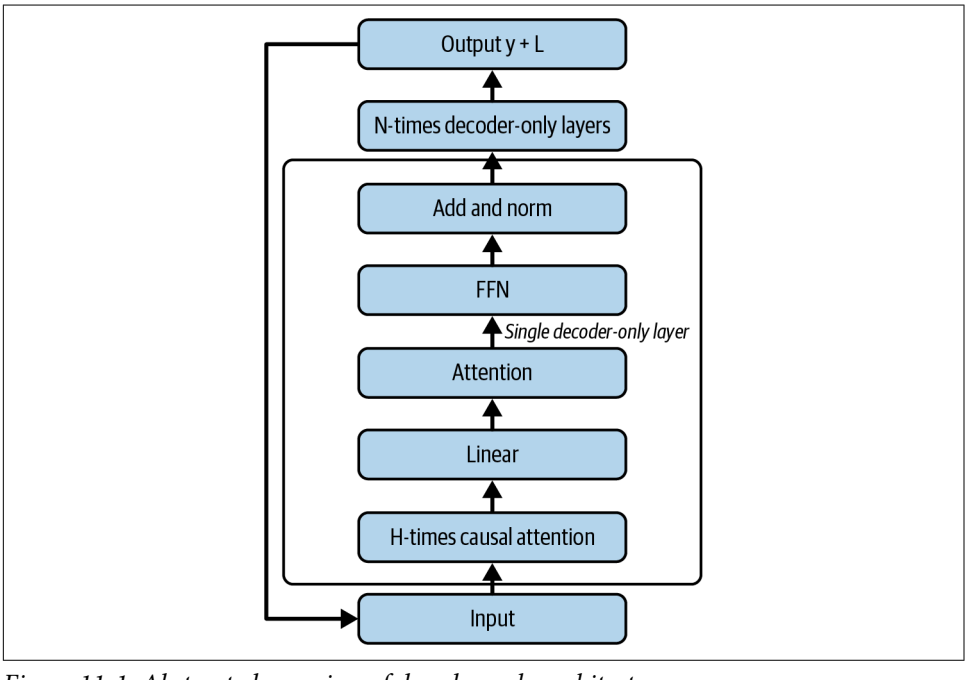


*Figure 11-1. Abstracted overview of decoder-only architecture.*

Since these models rely heavily on KV caching to avoid recomputing attention over the entire sequence, their overall performance and cost are dominated by context length, scheduling decisions, and how efficiently the cache is managed. From a deployment perspective, the key property of decoder-only models is that they reuse previously computed attention projections. Without additional optimizations, each step would require recomputing all earlier attention states.

KV caching improves latency and throughput but also introduces a growing memory footprint. Cache size scales with sequence length × layers × heads, which means memory pressure becomes one of the primary bottlenecks when deploying large decoder-only models.

Modern models mitigate this through architectural choices such as *grouped-query attention (GQA)*, where many query heads share a smaller number of KV heads. You can inspect this configuration easily via `model.config` in Hugging Face. For example, `Qwen2.5-7B-Instruct-1M` uses:

```
"num_attention_heads": 28,
"num_key_value_heads": 4,
```

This configuration reduces KV cache size significantly, as you do serve 7 query heads for each KV pair, which is ~ 7× less KV memory.

In practice, KV cache management is one of the main indicators of whether a system scales gracefully or collapses under load. For example, poorly controlled cache growth will increase memory fragmentation, reduce scheduling efficiency, and decrease concurrency.

#### Production Gotcha of Increasing Batch Size

The hidden bottleneck here isn't compute; it's memory bandwidth. Increasing batch size looks free on paper until you hit the KV cache limit, at which point performance falls off a cliff.

You can also reduce memory usage through *prefix reuse*. During beam search or multi-sample decoding, models often share long initial prefixes. Reusing their KV caches rather than recomputing them results in substantial performance gains. Likewise, once the context grows beyond its useful range, techniques such as prefix trimming or sliding-window attention prevent KV caches from ballooning to the model's maximum context size.

## Runtime Engineering for Decoder-Only Models

In deployment, you should monitor cache behavior continuously. The cache hit ratio reveals whether the system is recomputing tokens unnecessarily or invalidating caches too aggressively. Memory pressure exposes when KV usage approaches device limits and signals that paging or trimming strategies may be required. Fragmentation is another recurring concern, since unmanaged KV growth can reduce GPU memory locality and degrade throughput. Frameworks such as vLLM provide optimized mechanisms to manage KV caches efficiently. Table 11-2 summarizes the most important ones.

*Table 11-2. vLLM cache management features*

| Mechanism | Description | Benefit |
|---|---|---|
| Paged attention | Splits the KV cache into fixed-size memory blocks managed by a `free_block_queue` | Prevents fragmentation; supports long-running sessions |
| Prefix caching | Hash-based reuse of shared prefixes | Eliminates redundant prefill computation |
| Chunked prefill | Breaks long prefills into incremental chunks | Improves concurrency; avoids cache monopolization |
| Continuous batching | Joint scheduling of prefill and decode | Maintains consistent throughput across mixed workloads |
| Hash-based verification | Ensures that KV blocks remain correct after context changes | Prevents reuse of stale or mismatched KV blocks |
| Dynamic cache reclamation | Returns KV blocks as soon as a request ends | Keeps GPU utilization predictable and stable |
| Disaggregated KV transfer | Shares KV across workers via shared storage | Enables distributed inference setups |

Another thing you'd want to look at is that any structural change in the input context can invalidate portions of the KV cache. Token insertions, document injection, reordered segments, or trimmed prefixes all misalign the cached keys and values with the model's positional encodings. This leads to incoherent outputs unless the cache is selectively invalidated.

Modern systems therefore combine the following:

- Sliding-window caching for bounded attention
- Segment-level caching to separate system prompts, retrieved chunks, and user turns
- Context hashing to detect mismatches

This preserves correctness while retaining the performance benefits of caching.

The key insight I would like you to take away from this section is that efficient deployment of decoder-only models isn't all about retraining or quantizing them or adding more GPUs; it's mostly about engineering the inference runtime.

## Security Considerations for Decoder-Only Deployments

When deploying decoder-only models at scale, especially with distributed inference, security becomes an important concern. vLLM and similar inference servers rely heavily on inter-node communication, and these communication paths are insecure by default. This affects PyTorch Distributed traffic, KV cache transfers, tensor parallelism, and pipeline parallelism.

None of these channels include built-in encryption or authentication. Messages are sent in clear text, no authorization checks are performed, and services often bind to all network interfaces unless explicitly configured otherwise. For this reason, distributed inference must never be exposed to untrusted networks.

To deploy decoder-only models safely, follow these guidelines:

*Network isolation*
: Run all vLLM nodes on a private, firewalled network. Only the public API port should ever be exposed externally. Internal ports used for distributed training, data-parallel coordination, or KV cache transfer must remain unreachable from untrusted sources.

*Explicit configuration*
: Set communication IPs explicitly using `VLLM_HOST_IP`, `--kv-ip`, `data_parallel_master_ip`, and related options instead of relying on defaults. Defaults often bind to `0.0.0.0` and expose internal services unintentionally.

*Firewall rules*
: Allow only necessary internal ports between trusted nodes. Block all other inbound connections. Never expose PyTorch Distributed ports or KV cache transfer ports to the public internet.

*Restrict media domain access*
: Use `--allowed-media-domains` to restrict external URLs the model can fetch during tool calls or image retrieval. Disable redirects with `VLLM_MEDIA_URL_ALLOW_REDIRECTS=0` to avoid server-side request forgery (SSRF).

*Least privilege access*
: Limit shell, Kubernetes, or machine-level access to only those who require it. Distributed inference frameworks assume trusted environments; your infrastructure must enforce that trust boundary.

In short, distributed inference workloads are powerful but inherently unsafe when exposed directly, regardless of whether they serve decoder-only, encoder-only, or multimodal architectures. Proper network segmentation and strict firewall configuration are essential components of any secure inference deployment.

# Building Applications with Coding Models

Coding models introduce a fundamentally different deployment challenge than language, vision, or audio models. While they often share the same decoder-only architecture with LLMs, their risk profile changes the moment the model is allowed to inspect, modify, or execute code. At that point, the primary concern is no longer

decoding speed but the safety and containment of the execution environment in which the model operates.

From a deployment perspective, coding models behave less like text generators and more like agents with system-level capabilities. They write files, generate scripts, invoke compilers, run tests, interact with package managers, or access external tools. Deploying such a model therefore becomes a question of granting the model the minimum necessary capabilities while constraining its operating environment in a way that prevents damage, privilege escalation, or system compromise.

Safe deployment begins with strict sandboxing. Any code produced by a model must run inside an isolated environment with restricted filesystem access, network controls, resource limits, and non-root user permissions. The Dockerfile in Example 11-1 creates a controlled sandbox for executing model-generated code with an MCP server to run Python code in a sandbox (see GitHub). Every instruction supports one of three goals: minimize the attack surface, ensure deterministic behavior, and enforce least privilege.

**Production Gotcha: The "Internal Network" Fallacy**

A common mistake is assuming that because a model is running inside a VPC or a private cluster, it's safe. It isn't. If a coding model can execute code, it can scan your internal network, access metadata services (like AWS Instant Metadata Service [IMDS]), or hit internal admin dashboards. Zero trust is the only safe default for agentic execution environments.

In the Dockerfile, you start from a slim Python base image to reduce installed software and to keep the runtime predictable. Environment variables disable pip caching and define install paths so the image remains self-contained. Only the required OS packages are installed, and all apt metadata is removed immediately to keep the footprint small.

Deno, an open source JavaScript runtime, is installed as a fixed version and validated at build time using a small inline script. This guarantees that the interpreter works before the container reaches production. The MCP server is then installed along with additional Python dependencies using binary wheels whenever possible to avoid fragile compilation paths.

The image switches to a dedicated non-root user, which is essential because model-generated code will execute inside the container. This prevents access to system directories and limits potential damage from malicious or faulty scripts. A Deno cache directory is created under that user to ensure correct permissions during runtime.

Finally, you add a minimal entrypoint. The result is a lightweight, deterministic, and secure environment that isolates model-produced code and enforces strict safety boundaries during deployment.

*Example 11-1. Set up Dockerfile*

```
FROM python:3.11-slim

ARG DENO_VERSION=2.1.0
ENV PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    DENO_INSTALL=/usr/local

RUN apt-get update \
  && apt-get install -y --no-install-recommends \
       curl ca-certificates bash git tini unzip libgfortran5 \
  && rm -rf /var/lib/apt/lists/*

RUN curl -fsSL https://deno.land/install.sh | sh -s -- -q \
  && chmod 0755 /usr/local/bin/deno \
  && /usr/local/bin/deno --version \
  && /usr/local/bin/deno run --node-modules-dir=auto -A - <<'JS'
console.log("deno OK with --node-modules-dir=auto");
JS

RUN pip install --no-cache-dir mcp-run-python==0.0.21

COPY requirements.txt /tmp/requirements.txt
RUN pip install --no-cache-dir --prefer-binary -r /tmp/requirements.txt


RUN useradd -m -u 10001 mcp ❶
USER mcp
WORKDIR /home/mcp

ENV DENO_DIR=/home/mcp/.deno_dir
RUN mkdir -p "$DENO_DIR"

COPY --chown=mcp:mcp entrypoint.sh /usr/local/bin/entrypoint.sh
RUN chmod +x /usr/local/bin/entrypoint.sh

EXPOSE 3333
ENTRYPOINT ["/usr/bin/tini","--"]
CMD ["/usr/local/bin/entrypoint.sh","streamable-http"]
```

❶ Non-root user.

You can build the image with `docker build -t mcp-run-python:latest .` in the project directory.

As a next step, you need to start the MCP server (Example 11-2). This runs the Python MCP server inside the container.

*Example 11-2. Start MCP server*

```
set -euo pipefail

MODE="${1:-streamable-http}"
PORT="${PORT:-3333}"
DEPS="${DEPS:-}"

ARGS=()
if [ -n "${DEPS}" ]; then
  ARGS+=(--deps "${DEPS}")
fi

if [ "$MODE" = "stdio" ]; then
  exec python -m mcp_run_python stdio "${ARGS[@]}"
elif [ "$MODE" = "streamable-http" ]; then
  exec python -m mcp_run_python streamable-http --port "${PORT}" "${ARGS[@]}"
elif [ "$MODE" = "example" ]; then
  exec python -m mcp_run_python ${ARGS:+--deps "${DEPS}"} example
else
  echo "Unknown mode: $MODE (use stdio | streamable-http | example)"
  exit 1
fi
```

To start the server from this script, you run simply `docker run --rm -it mcp-run-python:latest /usr/local/bin/entrypoint.sh stdio`. This starts up the server, and you can execute Python code in a safe environment.

**Full Running Code**

The full code example can be found in this repository.

Another way to safeguard your coding models is to use E2B. Example 11-3 shows how to create a coding sandbox with E2B.

*Example 11-3. Create coding sandbox*

```
def create_sandbox() -> Sandbox:
    api_key = os.environ.get("E2B_API_KEY", "")
    if not api_key:
        raise RuntimeError("E2B_API_KEY is not set")
    return Sandbox.create()
```

Example 11-4 shows how to run a single sandbox for a multi-agent system.

*Example 11-4. Run a single sandbox*

```
if __name__ == "__main__":
    sb = create_sandbox()
    try:
        app = build_app(sb)

        initial_state: FibState = {
            "messages": [
                HumanMessage(
                    content="""Write an efficient Fibonacci
                            implementation fib(n: int) -> int."""
                )
            ],
            "iteration": 0,
            "max_iterations": 3,
        }

        final_state = app.invoke(initial_state)

        print("Done:", final_state.get("done"))
        print("Iterations:", final_state.get("iteration"))
        print("\nFinal code:\n")
        print(final_state.get("code"))
        print("\nRationale:\n")
        print(final_state.get("draft_rationale"))

        tests = final_state.get("tests")
        if isinstance(tests, TestResult):
            print("\nTests passed:", tests.passed)
            print("Details:", tests.details)
            if tests.failed_cases:
                print("Failed cases:", tests.failed_cases)

    finally:
        sb.kill()
```

Another way to safeguard the access to the rest of the system, and specifically if you have to scale your system, is to use microVM layers such as gVisor or Firecracker runtimes. gVisor, for instance, can easily be activated for your Kubernetes cluster on GCP. At the end, whatever setup you're choosing, the goal is always the same: treat model-generated code as untrusted, and ensure that its execution can't affect the surrounding system or other users.

## Evaluating LLM Deployments in Production

Another important part you want to look into is how to evaluate your LLM deployment under real production conditions. A model that performs well in isolated tests may still fail once it encounters concurrent traffic, long contexts, or variable workloads. For this reason, you should treat deployment not as a single decision but as a continuous process of measurement, iteration, and optimization. The goal is to determine whether your deployment meets your latency, throughput, reliability, and cost targets. LLM inference has several observable performance dimensions:

*Time to first token (TTFT)*
: The user-perceived responsiveness

*Inter-token latency*
: The rate at which the model streams output

*End-to-end latency*
: The total time a user waits for a complete response

*Throughput in tokens per second (TPS)*
: The measure of your serving stack's efficiency

*Capacity*
: The number of requests per minute the system can sustain before degrading

*Error rate*
: The indicator of system stability under load and concurrency

All of these can be measured with a simple workload harness ("Cost Efficiency and Hardware Comparison" on page 314) that generates realistic input and output token distributions and submits requests at configurable concurrency levels. This allows you to evaluate not only managed APIs (OpenAI, Anthropic, Nebius) but also your own deployments on vLLM or SGLang. With these metrics, you can perform three essential categories of production evaluation:

*Capacity testing*
: *Capacity testing* determines how much load a deployment can sustain before violating service-level objectives by holding the workload constant and gradually increasing concurrency. The goal is to find the *sweet spot* where throughput is maximized while latency and error rates remain stable. It tracks metrics such as mean and p95 TTFT, mean and p95 end-to-end latency, tokens per second, completed requests per minute, and failure patterns like timeouts or 5xx responses. The capacity point is the highest load at which performance remains predictable; beyond it, systems begin to queue, degrade, or fail.

*Stress testing*
: *Stress testing* intentionally pushes the system beyond the sweet spot to reveal how it behaves under unsafe or extreme load. Rather than optimizing performance, it seeks to understand failure modes, whether latency degrades gradually or collapses, whether the system can recover automatically, which error codes dominate, and whether internal components such as the KV cache fragment become memory-constrained. Stress tests expose architectural weaknesses in scheduling, batching, caching, and rate limiting.

*Soak testing*
: *Soak testing* evaluates whether the system can sustain the sweet spot over long durations by running steady load for tens of minutes or hours. It uncovers slow-burn degradation patterns such as drift in TTFT or end-to-end latency, rising error rates, GPU memory leaks or fragmentation, increasing cache reclamation overhead, and throttling behavior from managed APIs. Soak tests are especially important for self-hosted deployments and GPU-backed microservices, where fragmentation and resource pressure accumulate over time.

## Cost Efficiency and Hardware Comparison

Performance evaluation is only one dimension of deploying transformer models. In practice, every deployment decision is coupled to a cost model: the price of managed APIs, the hourly cost of GPU hardware, and the efficiency of the underlying runtime. The following benchmarking framework is from Ray, a Python-native framework for distributed computing, slightly adapted for later Python versions, so I'd recommend to use the version from my repo.

To illustrate how you can use this framework, let's evaluate two variants of the Qwen3-32B model provided through the Nebius managed API: the fast tier and the base tier. Both are benchmarked under identical workloads, allowing direct comparison of latency, throughput, and error behavior. You'll find a notebook with the complete code in the book's repo.

Example 11-5 shows how to initiate a benchmark run for the Qwen3-32B-fast API endpoint.

*Example 11-5. Run benchmark against the Qwen3-32B-fast API endpoint*

```
%%bash
export OPENAI_API_KEY="${NEBIUS_API_KEY}"
export OPENAI_API_BASE="https://api.tokenfactory.nebius.com/v1"

python token_benchmark_ray.py \
  --model "Qwen/Qwen3-32B-fast" \
  --mean-input-tokens 1024 \
  --stddev-input-tokens 200 \
```

```
  --mean-output-tokens 256 \
  --stddev-output-tokens 32 \
  --max-num-completed-requests 40 \
  --num-concurrent-requests 4 \
  --timeout 600 \
  --llm-api openai \
  --results-dir nebius-qwen32b-fast \
  --additional-sampling-params '{
    "temperature": 0.0,
    "extra_body": {
      "chat_template_kwargs": {"enable_thinking": false}
    }
  }'
```

Example 11-6 shows you how you can extract the aggregated performance metrics from the generated summary file of the benchmark run.

*Example 11-6. Get aggregated performance metrics*

```
summary_path = next(Path("nebius-qwen32b-fast").glob("*_summary.json"))

with open(summary_path, "r") as f:
    summary = json.load(f)

summary_metrics = {
    "Model": summary.get("model"),
    "Mean input tokens": summary.get("mean_input_tokens"),
    "Mean output tokens": summary.get("mean_output_tokens"),
    "Mean TTFT (s)": summary.get("results_ttft_s_mean"),
    "Mean inter token latency (s)": summary.get(
    "results_inter_token_latency_s_mean"),
    "Mean output throughput (tokens/s)": summary.get(
    "results_mean_output_throughput_token_per_s"),
    "Completed requests": summary.get("results_num_completed_requests"),
    "Error rate": summary.get("results_error_rate"),
}

for k, v in summary_metrics.items():
    print(f"{k}: {v}")
```

Running the same procedure for the Qwen3-32B base model produces a second summary file. Table 11-3 compares the two variants side-by-side.

*Table 11-3. Qwen3-32B fast versus base API performance (Nebius)*

| Metric | Qwen3-32B-fast | Qwen3-32B |
|---|---|---|
| Mean input tokens | 1,024 | 1,024 |
| Mean output tokens | 256 | 256 |
| Mean TTFT (s) | 0.5072 | 0.5270 |
| Mean inter-token latency (s) | 0.0194 | 0.0258 |
| Mean output throughput (tokens/s) | 344.46 | 129.36 |
| Completed requests | 40 | 40 |
| Error rate | 0.0 | 0.0 |

These results show a clear difference between the two service tiers: the fast variant more than doubles throughput and significantly reduces inter-token latency while keeping TTFT nearly identical. Such measurements form the basis for cost-efficient model selection, helping you compare API tiers, GPU types, quantization strategies, or self-hosted deployments using the same workload and the same evaluation harness.

#### Production Gotcha: The "Avg Latency" Lie

Never optimize solely for average latency (mean). In production, your user's experience is defined by the tail latency (p95 or p99). A system with a low mean but high p99 can feel "broken" to 1 in 20 users. Always run your capacity tests until the p95 metric violates your *service level objective* (SLO), the specific latency target you promised your users (such as 500 ms max per task), rather than just watching the average.

The numerical comparison in Table 11-3 is informative, but the performance gap becomes even clearer when visualized. Figure 11-2 plots mean output throughput and mean end-to-end latency for both service tiers. Although the two variants have nearly identical TTFT, their streaming performance differs dramatically.

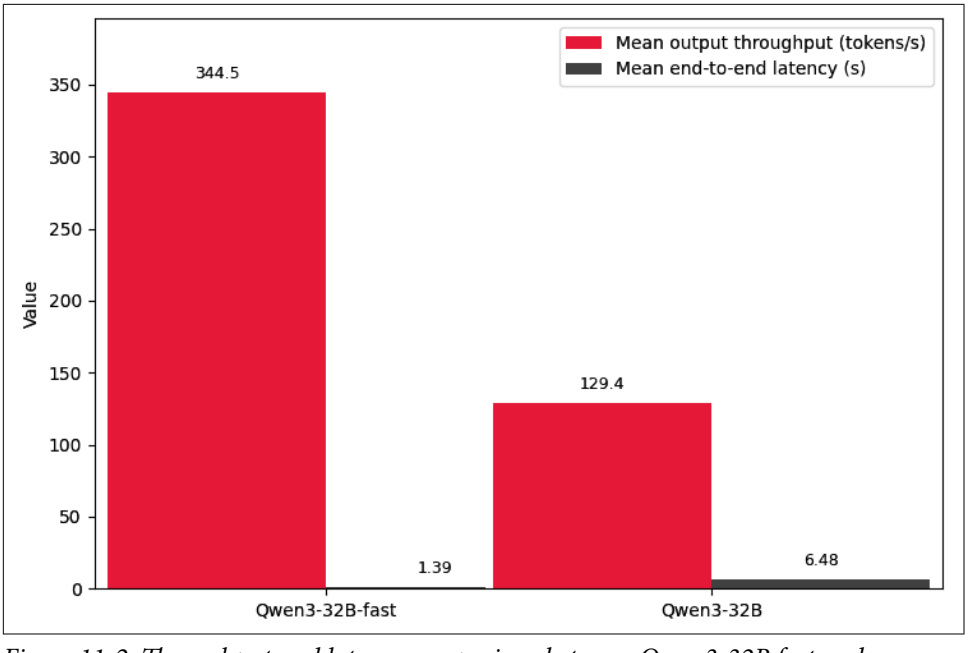


*Figure 11-2. Throughput and latency comparison between Qwen3-32B fast and base tiers.*

The fast tier achieves more than 2.6× higher throughput and reduces end-to-end latency from 6.48 seconds to 1.39 seconds, a difference users will perceive immediately. This illustrates that the performance characteristics of a deployment are shaped not only by the model architecture but also by the serving stack, batching strategy, and hardware tier behind the API.

To understand how these latency differences translate into practical capacity, Figure 11-3 shows the number of completed requests per minute under identical workload and concurrency settings.

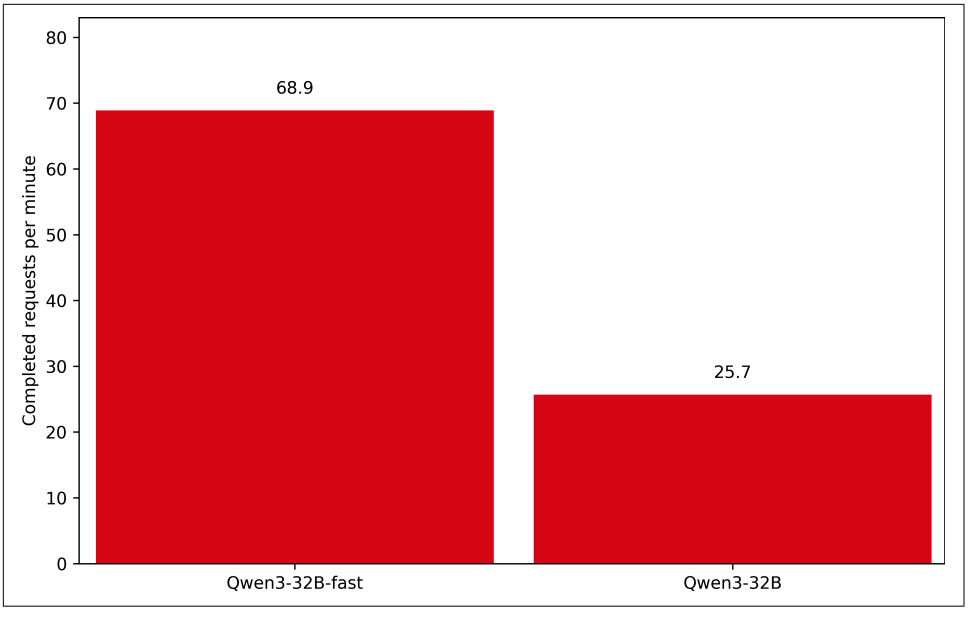


*Figure 11-3. Completed requests per minute for Qwen3-32B fast versus base.*

Here, the difference becomes operationally significant. The fast tier sustains almost 69 requests per minute, compared to 26 requests per minute for the base tier. In a production environment, this gap directly affects how many concurrent users can be supported, how quickly a system recovers from bursts of traffic, and ultimately how much the deployment costs at scale.

### Calculating GenAI Prices

GenAI Prices is a library by Pydantic that provides a compact way to compute inference costs across providers. It resolves model and provider identifiers through an internal matching engine, applies historic and time varying price schedules, and handles more complex structures such as DeepSeek's off-peak pricing or the tiered context billing used by Gemini models. The library can also identify discrepancies between vendor documentation and observed prices. It's available as both a Python and JavaScript package with a simple CLI.

Taken together, these two visualizations illustrate an important lesson: model performance is inseparable from infrastructure performance. A model that appears identical on paper can behave very differently depending on the inference tier, hardware class, and runtime implementation used by the provider. Evaluating these factors is essential for choosing the right balance between speed, scalability, and cost.

Table 11-4 compares the results in terms of cost versus latency. Interestingly, the fast tier is more expensive per token but significantly cheaper per unit of throughput and per user served.

*Table 11-4. Qwen3-32B fast versus base API costs (Nebius)*

| Metric | Qwen3-32B (base) | Qwen3-32B-fast |
|---|---|---|
| Input cost per 1M tokens | $0.10 | $0.20 |
| Input cost per 1K tokens | $0.00010 | $0.00020 |
| Output cost per 1M tokens | $0.30 | $0.60 |
| Output cost per 1K tokens | $0.00030 | $0.00060 |
| Effective throughput (tokens/s) | 129.4 | 344.5 |
| Approx. requests/min (benchmark) | 25.7 | 68.9 |
| Quantization | fp8 | fp8 |
| Context length | 41K | 41K |

When deploying models on self-hosted hardware, cost per token follows directly from two quantities: the GPU hourly price (`C_usd_per_hour`) and the measured output throughput from the benchmark (`throughput_tokens_per_s`). The number of tokens produced per hour is:

$$\text{tokens/ hour} = T_{\text{tokens /}s} \times 3600$$

The infrastructure cost per 1,000 tokens is then computed as:

$$\text{cost}_{1k} = 1000 \times \frac{C_{\text{usd / hour}}}{\text{tokens / hour}} = 1000 \times \frac{C_{\text{usd / hour}}}{T_{\text{tokens /}s} \times 3600}$$

This gives a simple, general-purpose formula:

Take the throughput from the evaluation, plug in the GPU price, and get the dollar cost per 1,000 tokens for that deployment.

If you want to compute the total token (input + output) you can use:

$$T_{\text{total}} = T_{\text{input}} + T_{\text{output}}$$

This allows you to compare a wide range of deployment options under a unified framework, including:

- Managed closed source APIs such as OpenAI or Cohere.
- Managed open source providers such as Nebius, deepinfra, or Together.ai.
- Self-hosted inference using vLLM, TGI, or SGLang on GPUs such as L4, A100, or H100.
- Quantized versus full-precision deployments.
- Single-node versus multi-node inference setups.

All deployments become comparable on a single axis: USD per 1,000 tokens, combined with the latency and throughput measurements from the framework.

## Quantization

This section doesn't aim to explain different quantization methods but to give you a way of validating which adapter strategy might be the best choice. The following code will compare LoRA,[2] QLoRA,[3] DoRA,[4] and LoftQ[5] on a single base model (Qwen3-4B-Base) and a single domain (step-by-step math reasoning) and then vary only the adapter strategy. Each configuration trains on the same GPU, supervised fine-tuning, and reinforcement learning setup, so any difference in behavior comes from the adapter and quantization choice. The supervised fine-tuning phase uses a chain-of-thought–style dataset to teach the model to separate its working from its final answer with explicit tags. The GRPO phase then applies a simple reward function that checks both the output format and the correctness of the final solution. The training loop records three signals that matter for deployment:

2 Edward J. Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models" (2021).

3 Tim Dettmers et al. "QLoRA: Efficient Finetuning of Quantized LLMs" (2023).

4 Shih-Yang Liu et al. "DoRA: Weight-Decomposed Low-Rank Adaptation" (2024).

5 Yixiao Li et al. "LoftQ: LoRA-Fine-Tuning-Aware Quantization for Large Language Models" (2023).

- How quickly each adapter converges during supervised fine-tuning, through the Cross-Entropy Loss curve
- How well each setup learns the reward in the GRPO phase, through the reward trajectory
- How fast the adapted model can generate tokens during inference, measured as tokens per second for a simple prompt

Before you test the individual methods, it helps to understand how the different adapter families approach parameter-efficient fine-tuning. Each technique modifies only a small portion of the model, but they differ in how they introduce low-rank structure, how they interact with quantization, and how much expressive power they recover relative to full fine-tuning. The following summaries highlight the core idea behind each adapter strategy and what makes it distinct:

*LoRA: Low-Rank Adaptation*
: LoRA injects a small pair of low-rank matrices into selected linear layers and updates only those matrices during training. The base weights remain untouched and stay in full precision. This is the classic parameter-efficient baseline: stable, simple, and a good reference point for all other methods.

*QLoRA: Quantized LoRA*
: QLoRA loads the base model directly in four-bit precision and trains LoRA adapters on top of those quantized weights. Only the adapters get updated, but gradients still flow "through" the quantized weights. This isolates the effect of quantization: you keep the LoRA structure unchanged while drastically reducing memory usage.

*DoRA: Direction and Magnitude Decomposition*
: DoRA follows the same low-rank philosophy as LoRA but decomposes each target weight into a magnitude and a direction. The low-rank update adjusts the direction, while a small scalar pathway adjusts the magnitude. This mimics more closely full fine-tuning without increasing inference cost, often giving the adapter more expressive power under the same parameter budget.

*LoftQ: Quantization-Aware LoRA Initialization*
: LoftQ couples quantization and adapter initialization. Instead of quantizing the model first and then attaching randomly initialized adapters, it computes quantized weights and a matching low-rank initialization jointly. This "builds in" quantization effects, so the adapters begin aligned with the model you actually train. The result is LoRA on a quantized base but with an initialization that wastes less capacity counteracting quantization error.

Before running the benchmark, you need a minimal configuration that fixes the base model, dataset size, and training schedules for both supervised fine-tuning and GRPO. Example 11-7 defines these constants so that every adapter variant is evaluated under identical conditions.

*Example 11-7. Setup for the quantization benchmark*

```
MODEL_ID = "unsloth/Qwen3-4B-Base" ❶
SFT_STEPS = 60 ❷
GRPO_STEPS = 200 ❸
GRPO_GENERATIONS = 4 ❹
DATASET_SIZE = 500 ❺
```

❶ Controls the single base model used for all experiments.

❷ Defines how long you train in each phase. Higher values produce smoother curves and more stable rewards, at the cost of more compute.

❸ Specifies how many sampled completions you use per update step in GRPO, which reduces variance in the reward signal.

❹ Caps how many math problems you use from each dataset, so you can scale the benchmark up or down to fit your hardware budget.

❺ Use an adjusted, smaller dataset size.

With the global configuration in place, the next step is to specify the adapter variants and wrap them in a simple loop that trains, evaluates, and logs their performance. Example 11-8 allows the benchmark to cycle through LoRA, QLoRA, DoRA, and LoftQ using the same data, same training phases, and the same hardware budget.

*Example 11-8. Adapter configurations and benchmark loop*

```
ADAPTERS = {
    "LoRA":  {"4bit": False,
    "peft": {"r": 16, "lora_alpha": 16,
    "target_modules": ["q_proj", "k_proj", "v_proj", "o_proj"]}},
    "QLoRA": {"4bit": True,
    "peft": {"r": 16, "lora_alpha": 16,
    "target_modules": ["q_proj", "k_proj", "v_proj", "o_proj"]}},
    "DoRA":  {"4bit": False,
    "peft": {"r": 16, "lora_alpha": 16, "use_dora": True,
    "target_modules": ["q_proj", "k_proj", "v_proj", "o_proj"]}},
    "LoftQ": {"4bit": False,
    "peft": {"r": 16, "lora_alpha": 16,
    "target_modules": ["q_proj", "k_proj", "v_proj", "o_proj"],
    "init_lora_weights": "loftq",
    "loftq_config": LoftQConfig(loftq_bits=4, loftq_iter=1)}}
}
```

Each entry in `ADAPTERS` describes one strategy, with two pieces of information: whether the base model is loaded in four-bit mode ("4bit") and the parameter-efficient fine-tuning (PEFT) configuration passed into `FastLanguageModel.get_peft_model`. LoRA and QLoRA share the same low-rank structure and target modules. QLoRA differs only by enabling four-bit loading, which isolates the effect of quantizing the base weights. DoRA adds `use_dora=True`, which decomposes the weight updates, so you can see whether this extra flexibility buys you better loss or reward under the same training budget. LoftQ uses the same LoRA shape but initializes adapter weights with LoftQ via `init_lora_weights="loftq"` and a `LoftQConfig`. This lets you test different quantization adapters without changing the rest of the pipeline.

### DenseMixer to Improve MoE Training

DenseMixer gives MoE models more accurate router gradients without requiring changes to your training code. It works with any Top-k MoE trained via backpropagation and can be enabled as a plug-and-play addition. DenseMixer integrates with common training stacks (transformers, llama-factory, open-instruct, verl) and is compatible with parameter-efficient approaches such as LoRA.

This setup will run for about 5 hours on an A100 GPU. The following charts are from this training, and they tell you a very clear story about the trade-offs for different adapters. Figure 11-4 shows the results from the supervised fine-tuning with different adapters.

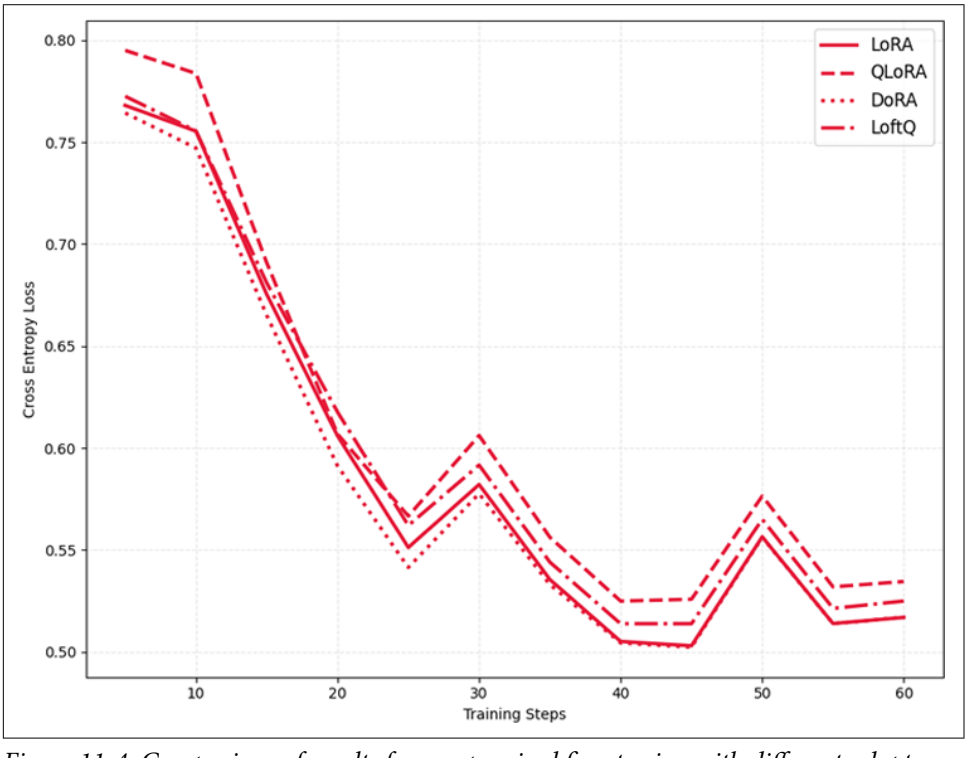


*Figure 11-4. Comparison of results from supervised fine-tuning with different adapters.*

DoRA (dotted line) is the "best student": the dotted line is the deepest from all four. Since DoRA separates the magnitude and direction of weights, this allows it to learn the syntax more efficiently. The "worst student" is QLoRA (dashed line). This is due to the quantization tax, because the model is struggling to learn precise formatting through the noise of 4-bit quantization.

### LoRAX for Large-Scale Multi-Adapter Serving

LoRAX is a multi-LoRA inference server that can host thousands of fine-tuned variants on a single GPU. It uses dynamic adapter loading to keep memory usage low while preserving throughput and latency, making it a cost-effective solution for serving large fleets of personalized or domain-specialized models.

If you now look at the plot from the GRPO training (Figure 11-5), you see that LoftQ (dash-dot) spikes up to a reward of 0.05 around step 75.

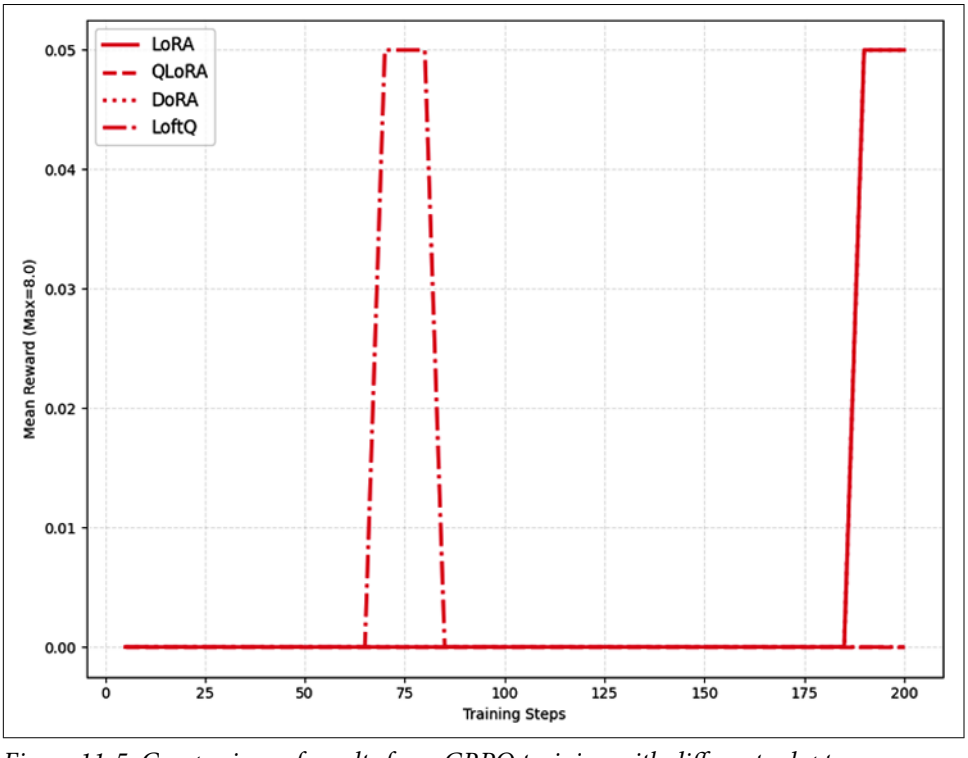


*Figure 11-5. Comparison of results from GRPO training with different adapters.*

The reason why LoftQ performs better is that it initializes the weights intelligently—that is, canceling out quantization error. Therefore, the model is still "smart" enough to stumble upon the correct answer/reasoning much earlier than the others. LoRA (solid line) finally found a reward around step 190, while QLoRA and DoRA need 400+ steps to start learning reasoning strategies.

Figure 11-6 shows the inference speed comparison across the different adapters.

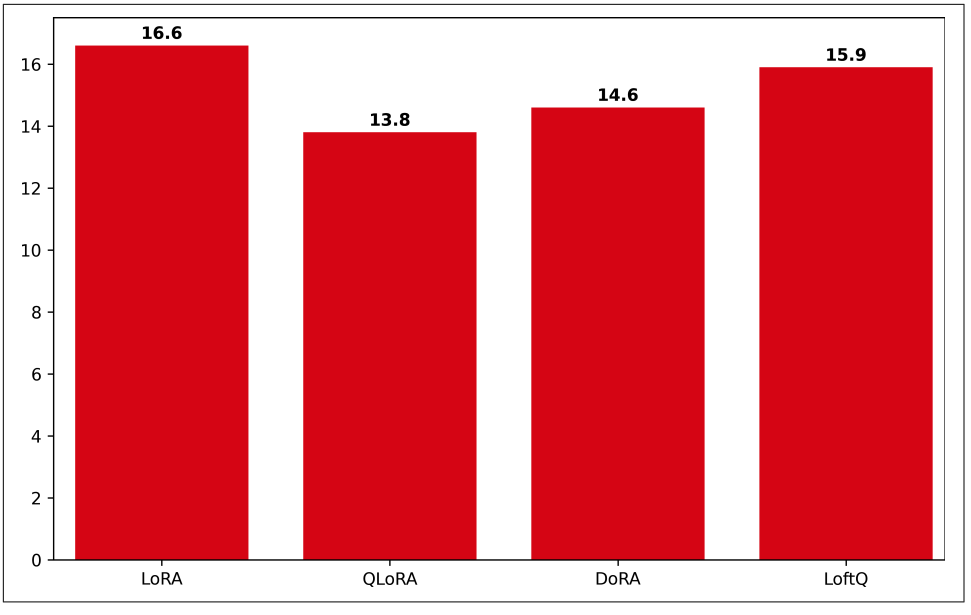


*Figure 11-6. Inference speed comparison across the different adapters.*

The standard 4-bit QLoRA introduces an ~17% latency penalty due to the on-the-fly dequantization (unpacking 4-bit to 16-bit during generation), while LoftQ is almost as fast as LoRA (because it merges well) but uses the low VRAM of QLoRA. Given this data, LoftQ would be a reasonable choice for deployment, as it bridges the gap: it fits in memory (4-bit), infers quickly (15.9 TPS), and initializes "smart enough" to find the reasoning rewards early.

## Test-Time Low-Rank Adaptation in Vision-Language Models

So far, I've treated deployment in this chapter as a mostly static decision. You pick a model, possibly fine-tune it, quantize it, then put it behind an inference server. In practice, many real-world systems must cope with slow domain drift. Image distributions change, lighting conditions shift, camera hardware is upgraded, and the test distribution no longer matches the data the model was originally trained on.

One way to react to this is periodic fine-tuning with new labeled data. A more agile alternative is test-time adaptation. In this setting, you allow a tiny number of parameters to adjust at inference time, driven by a self-supervised objective on the incoming test samples themselves.

Test Time Low Rank Adaptation (TTL)[6] allows this dynamic adaption for Contrastive Language-Image Pretraining (CLIP)-style encoders. Instead of tuning prompts, TTL inserts low-rank adapters into the attention layers of the visual encoder and updates only those adapters during test-time, while the rest of CLIP stays frozen.

The adaptation signal comes from confidence rather than labels. For each test image, the system creates several augmented views and passes them through CLIP with the usual class text prompts. TTL then applies a weighted entropy loss over the predicted class probabilities for those augmentations. The objective encourages consistent, high-confidence predictions across all views. The gradient updates only the low-rank adapters in the visual encoder. Prompts and backbone weights remain unchanged.

From a deployment perspective, TTL is interesting because:

- It respects strict zero-shot assumptions. No target domain labels and no access to the original training data are required.
- The number of trainable parameters is a tiny fraction of the full model, which makes adaptation cheap and fast.
- Updates can be performed in a single optimization step per test sample or per small batch of samples.

#### Modality-Grounded Audio Reasoning

Step-Audio-R1 is an audio reasoning model that can be used with test-time compute scaling. The model overcomes the *textual surrogate reasoning problem*, where models initialized from text rely on transcript-style abstractions instead of acoustic cues. The model is available via Hugging Face.

The important point isn't the exact loss formulation but the pattern. You can treat adapters as a deployment time control surface that lets you track domain shift without ever touching the backbone. You could apply a similar pattern to other modalities, such as in audio classification. Here, you could adapt low-rank blocks in the spectrogram encoder using a consistency objective across augmentations, such as time masking or frequency masking.

6 Raza Imam et al. "Test-Time Low Rank Adaptation via Confidence Maximization for Zero-Shot Generalization of Vision-Language Models" (2024).

The backbone remains a stable, pretrained foundation model, but your deployment now becomes a composition of three elements:

- A frozen foundation model
- A small, trainable adapter layer
- A self-supervised test-time objective that nudges the adapter toward the current input distribution

When you design a production system for non-text modalities, it's worth deciding explicitly whether the model will remain static in deployment or whether you want to use this kind of dynamic adaptation. The latter introduces new operational overhead around monitoring, rollback, and stability, but it can significantly improve robustness without the cost of full retraining.

# Conclusion

In this chapter, you saw that deployment is less about wiring up an endpoint and more about making structural choices. The spectrum from closed APIs to managed open models to self-hosted deployments determines who controls cost, data, observability, and long-term independence. Understanding the architecture family you serve, from decoder-only LLMs to encoder-only perception models and encoder-decoder or diffusion stacks, lets you align deployment strategy with the runtime, memory, and concurrency behavior of the model itself.

You learned that most headaches in practice come from the inference runtime, not from the checkpoint. Decoder-only models live and die by KV cache behavior, batching, and scheduling, which is why mechanisms such as paged attention, prefix reuse, and continuous batching matter as much as the underlying model weights. At the same time, distributed inference is insecure by default. Secure deployments treat intra-cluster traffic, media access, and machine-level permissions as untrusted and rely on isolation, explicit configuration, and least privilege to keep powerful models from becoming powerful attack surfaces.

You also saw how deployment changes once models can execute code. Coding models behave like agents with system capabilities, so safe deployment begins with strict sandboxing. MCP-based containers, E2B-style sandboxes, and microVM layers such as gVisor or Firecracker provide the isolation needed to run generated code without risking the rest of your environment. On the evaluation side, capacity, stress, and soak tests turn LLM serving into an empirical process, while benchmarking frameworks and tools such as GenAI Prices let you compare providers, hardware tiers, and runtimes in terms of both latency and cost per tokens.

Finally, you explored quantization and adaptation through the lens of deployment trade-offs. You saw how LoRA, QLoRA, DoRA, and LoftQ differ in convergence, reward learning, and inference speed, with LoftQ offering an attractive balance between VRAM usage and runtime performance. Test-time low-rank adaptation extends this idea into production, using tiny adapter surfaces and self-supervised objectives to track slow domain shift without retraining the backbone.

With these principles and tools, you're ready to think about deployment as its own design space. Chapter 12 takes a look at where to go next and where the field is heading.

CHAPTER 12

# Where to Go Next: From Models to Intelligent Systems

This is the last chapter of the book, and by now you've learned how diverse and adaptable the transformer architecture is. You started from first principles and saw how a single architecture can power models for time series, vision, image and video generation, audio, reinforcement learning, planning, and coding. You also saw how test-time compute, such as tree search or reranking, and reinforcement learning can enhance performance and steer behavior. And you saw how agentic systems create loops of planning, action, feedback, and self-correction.

This was never meant to be a collection of isolated techniques. It was meant to show that transformers have become a general interface between data, decisions, and tools. Treat this book as your foundation to move from individual models to larger systems of specialist components working together to solve real tasks. By now, you understand the architecture, the patterns, and the reasoning that make transformers work across domains. Your next step is not learning another model or chasing another benchmark. The next step is learning how to wire these components into something coherent, reliable, maintainable, and capable of improvement through experience.

The shift happening in the field reflects this. Progress is no longer defined by a single checkpoint or a bigger context window. It's defined by systems that integrate multiple models, allocate compute at test-time, coordinate tool usage, manage memory, and learn from their own errors. Table 12-1 shows a snapshot of the number of papers published in 2024 and 2025 across these domains.

*Table 12-1. rLLM components in the RL training loop (2024 versus 2025)*

| Topic | 2024 papers | 2025 papers |
|---|---|---|
| Agentic and multi-agent systems | 820 | 2,520 |
| RL plus transformers or LLMs | 1,200 | 3,082 |
| Self-improving systems plus transformers or LLMs | 209 | 458 |
| Test-time compute plus transformers or LLMs | 395 | 1,053 |

The table shows that this direction isn't theoretical. It's visible in the research distribution itself. I scraped arXiv for 2024 and 2025 papers across the domains shown in the table. The scrape is not exhaustive, since some papers will fall through the cracks due to naming conventions, missing keywords, or uploads outside the query window. The spikes around May/June and November are to be expected, as this is influenced by ICML and NeurIPS submission cycles, which naturally increase volume around those months. Even with that context, the trend is clear. The jump from 2024 to 2025 for agentic/multi-agent systems suggests that these systems are the primary focus of the world's top research labs. The rise in test-time compute (increasing by nearly 3×) indicates that the industry has realized it's more efficient to let a model think longer (search, reason, verify) than it is to just make the model bigger. The field is not scaling parameter count; it's scaling coordination, alignment, and improvement across components. Models are no longer endpoints. They are parts of systems. Figures 12-1 and 12-2 illustrate the flow and increase over the two years.

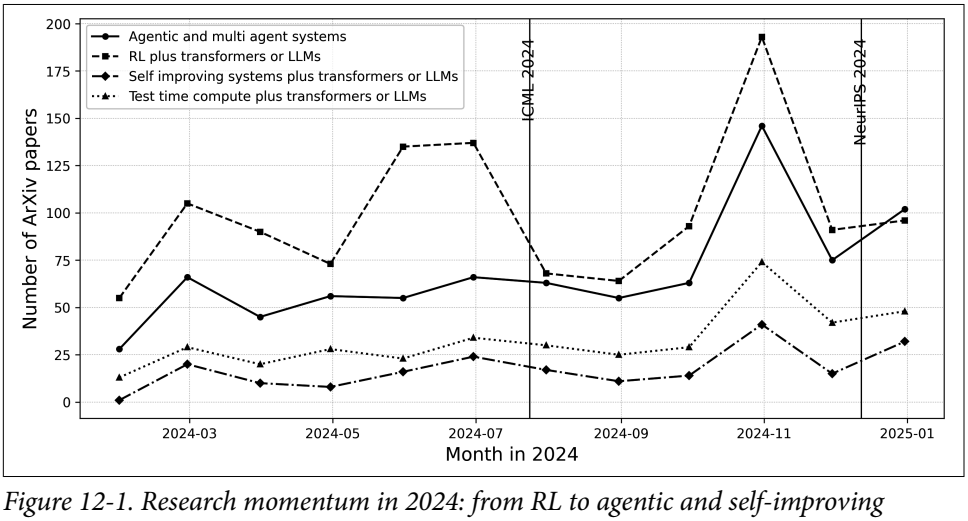


*Figure 12-1. Research momentum in 2024: from RL to agentic and self-improving systems.*

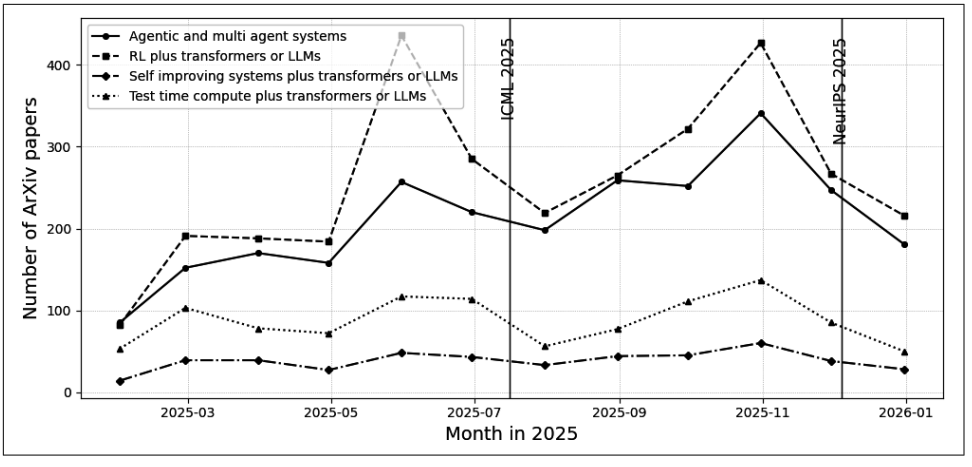


*Figure 12-2. Research momentum in 2025: from RL to agentic and self-improving systems.*

This is why my intent for this last chapter is to transition your thinking from a model-focused practitioner to a system thinker. Someone who can connect multiple models with intention. Someone who understands that each model in a system introduces strengths and weaknesses. Someone who recognizes that the system is only as reliable as the weakest point in its design. This is how it all connects; the fundamentals you learned in this book matter. They are the difference between building a fragile demo and building an intelligent system you can rely on in production. Recent research reflects this shift, and the next section demonstrates how.

# Combining Capabilities: SAM 3 Agent

Performance is no longer the product of a stronger model alone but of the interaction between model capability, coordination structure, and task properties. A clear example appears in the research published alongside SAM 3. The team combined a multimodal large language model (MLLM) with SAM 3[1] as a tool and let the system iterate: propose a query, call SAM 3, inspect the masks, refine, and repeat until the instruction is satisfied. The improvement does not come from stronger vision or stronger language. It comes from the loop that connects them. The gain is architectural, not parametric.

1 Nicolas Carion et al. "SAM 3: Segment Anything with Concepts" (2025).

In Chapter 3, you saw how SAM 3 can segment visual content, but the workflow remained manual. Even with SAM 3 ("Segment Videos and Images with Concept Prompts" on page 88), where text prompts can guide segmentation, the model still acts alone. It waits for input, executes a single task, and returns a mask. It's a capable component, but it's not a system yet. To answer more complex segmentation prompts, such as "the leftmost red yarn ball," when applied to an input image (Figure 12-3), SAM 3 must be connected to an MLLM in an agent loop.

*Figure 12-3. Example image for more difficult text query.*

In this setup, segmentation is no longer a single request. It becomes a step in a plan. The MLLM interprets the query, proposes an action, and generates a tool call. SAM 3 executes the segmentation task as a specialist. The agent evaluates the result, selects the correct mask, and decides whether to continue or stop. The workflow becomes iterative, reflective, and goal-oriented. Figures 12-4 through 12-6 visualize this workflow.

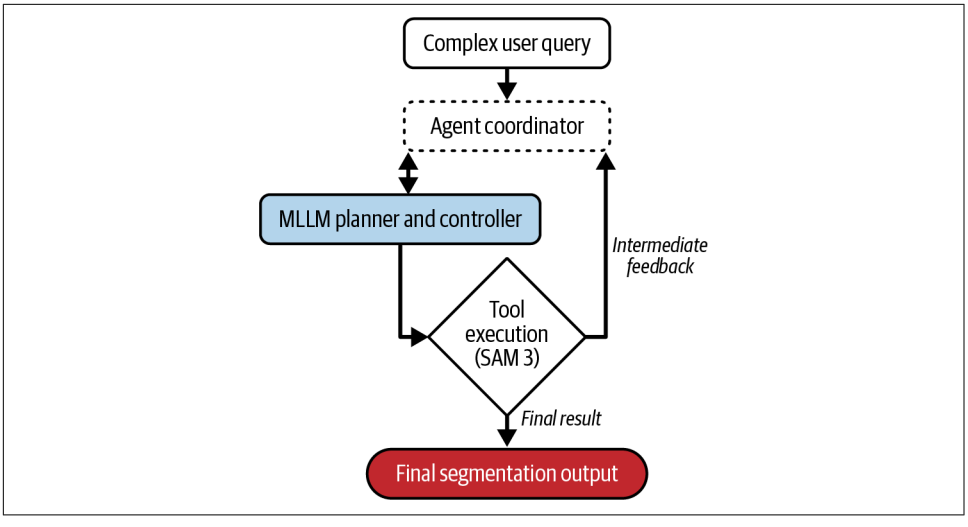


*Figure 12-4. Bird's-eye view of workflow.*

Once the user query is received, the agent coordinator (AC) initializes the session. It acts as the central hub, keeping track of the segmentation process. From here, the state is passed to the MLLM planner, which enters a cognitive loop to decide how to handle the request. Here's a step-by-step breakdown:

1. Analyze: the model looks at the current visual state and any previous feedback.
2. Think: it breaks the user's request into logical subtasks.
3. Remind: it reviews system constraints (such as "Don't overwrite specific masks").
4. Plan: it selects the most efficient path forward.
5. Decide: it commits to a specific tool call.

Figure 12-5 illustrates this decision process.

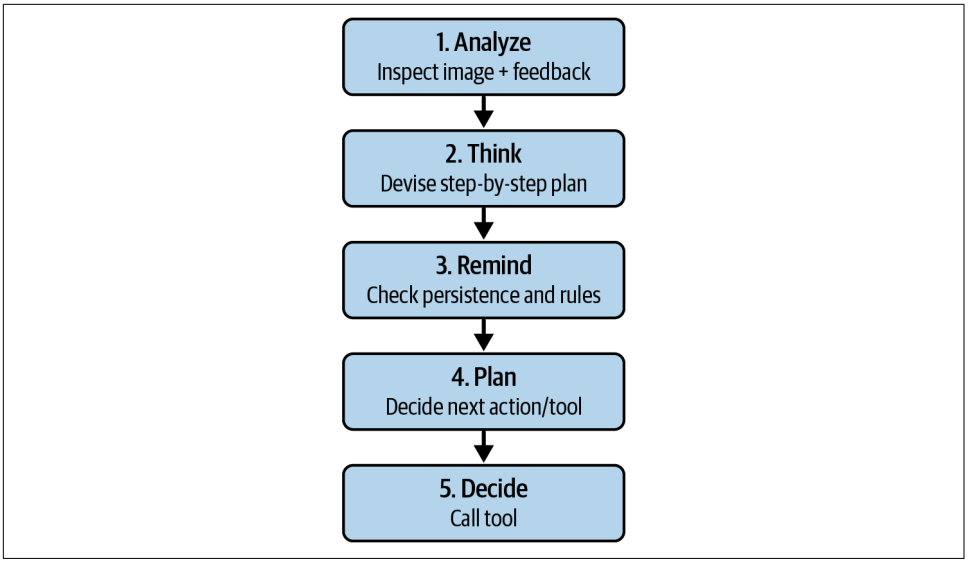


*Figure 12-5. Cognitive loop of the system.*

After the MLLM makes a decision, it exits the loop to execute a tool. This leads to a branching path (Figure 12-6): either we need more information or the loop can terminate.

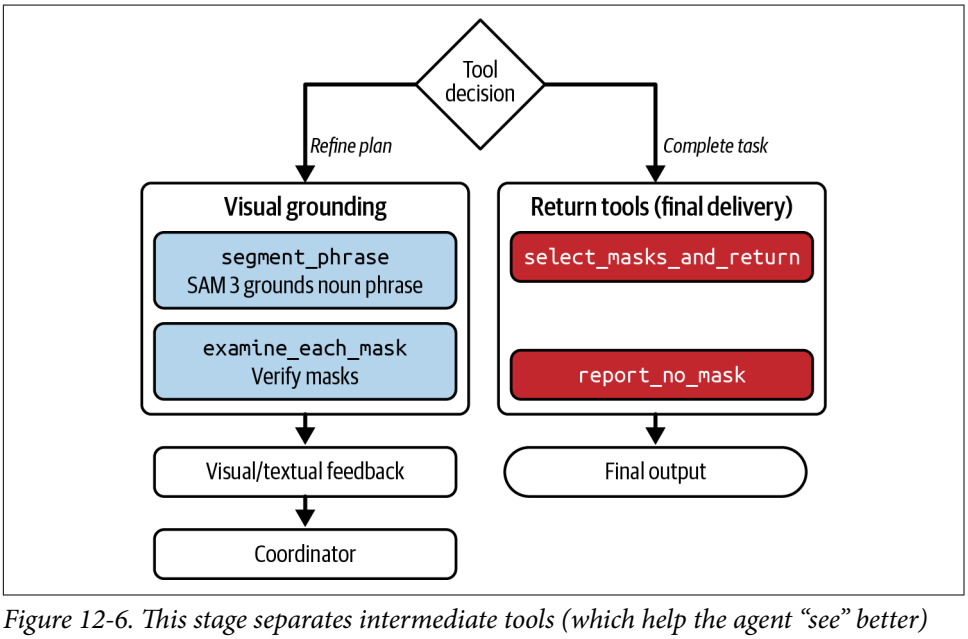


*Figure 12-6. This stage separates intermediate tools (which help the agent "see" better) from Terminal Tools (which end the session).*

This is the systems-level moment of the book I want to leave you with as an overview of where to go next or how to think about your next AI project. An MLLM alone can't isolate the correct object. SAM 3 alone can't interpret the instruction. The agent alone can't ground the decision. The result appears only when different models cooperate. The system validates, selects, and continues the loop until the goal is reached. This is orchestration. This is the shift from operating models to designing systems.

Using SAM 3 in an agentic setup is fairly straightforward. As usual, you first load the model and processor. Example 12-1 shows how to do this.

*Example 12-1. Load SAM 3 model and processor*

```
bpe_path = "/content/sam3/sam3/assets/bpe_simple_vocab_16e6.txt.gz"
model = build_sam3_image_model(bpe_path=bpe_path)
processor = Sam3Processor(model, confidence_threshold=0.5)
```

The code from the SAM 3 repo works with OpenAI-compatible APIs, so you can just start an vLLM server while allowing local media. Example 12-2 illustrates how to do this.

*Example 12-2. Start vLLM server in background*

```
nohup vllm serve Qwen/Qwen3-VL-8B-Thinking \
  --served-model-name qwen3_vl_8b_thinking \
  --enforce-eager \
  --port 8001 \
  --host 0.0.0.0 \
  --allowed-local-media-path / \
  > vllm_server.log 2>&1 &
```

### Security Risk with Media Path

The `--allowed-local-media-path /` flag grants the model server read access to your entire root file system. While this is convenient for local development and Google Colab environments, it's a significant security risk in a production environment. If a malicious actor gains access to your agent loop, they could use the MLLM to "inspect" and exfiltrate sensitive files (like *.env* files or SSH keys) under the guise of image processing. Always restrict this path to a specific, isolated directory (such as `--allowed-local-media-path /home/user/app/data`), and run your agent in a containerized environment.

Example 12-3 shows how to set up the model configurations correctly, as it's important that the server model name and the server URL match precisely.

*Example 12-3. Simple text generation*

```
LLM_CONFIGS = {
    "qwen3_vl_8b_thinking": {
        "provider": "vllm",
        "model": "qwen3_vl_8b_thinking",
    },
}

model = "qwen3_vl_8b_thinking"
LLM_API_KEY = "DUMMY_API_KEY"

llm_config = LLM_CONFIGS[model]
llm_config["api_key"] = LLM_API_KEY
llm_config["name"] = model

if llm_config["provider"] == "vllm":
    LLM_SERVER_URL = "http://127.0.0.1:8001/v1"
else:
    LLM_SERVER_URL = llm_config["base_url"]
```

Now you can just load your image, in my case the cute kitten surrounded by different-colored yarn balls. To combine the MLLM and SAM 3, you can use the agent functions provided by the SAM 3 team. Example 12-4 shows how to handle more complex prompts.

*Example 12-4. Agent inference with SAM 3 and Qwen3 VL*

```
prompt = "the leftmost red ball"
image = os.path.abspath(image)
send_generate_request = partial(send_generate_request_orig,
                        server_url=LLM_SERVER_URL,
                        model=llm_config["model"],
                        api_key=llm_config["api_key"])
call_sam_service = partial(call_sam_service_orig, sam3_processor=processor)
output_image_path = run_single_image_inference(
    image, prompt, llm_config, send_generate_request, call_sam_service,
    debug=True, output_dir="agent_output"
)
```

Example 12-5 shows how you can display the image with the segmentation mask.

*Example 12-5. Display image with segmentation mask*

```
if output_image_path is not None:
    display(Image(filename=output_image_path))
```

Figure 12-7 displays the final image with the segmentation mask and the number 1.

*Figure 12-7. Image with segmentation mask.*

What you've just seen here, by combining a segmentation model with an MLLM in an agentic system, is exactly what you'll encounter in multi-agent systems as well. That is, once a single model baseline crosses a certain threshold, adding more agents no longer guarantees improvement. Coordination overhead, fragmented context, and error propagation can erase the advantage of scale. In other words, the limitation isn't architectural size but rather architectural alignment. These systems behave like organisms, not collections of isolated parts, and they require design decisions that treat them as such.

## The Science of Scaling Agentic Systems

As you've seen in the previous section, you can't just "prompt your way out" of system-level failure. In fact, in a comment on LinkedIn, I was asked, "Is it really worth investing time to learn prompt engineering techniques? Does a very well crafted prompt lead to meaningfully better responses from an LLM than a reasonably good one?" This section here is my long answer to that, as I believe the future lies in the combination of systems and how they evolve over time.

This is where recent research on system-level scaling becomes useful, because it gives you a vocabulary to reason about why some systems improve and others collapse. In a paper[2] published by Google Research and Google DeepMind, the authors showed that the difference between a system that gets better with every loop and one that falls apart isn't the number of agents or the size of the model. It's how the system is wired. The coordination structure, the flow of information, and the topology of decision making determine whether the system amplifies capability or amplifies error. In other words: performance is an architectural outcome.

If an agentic system loops, stalls, or amplifies its own errors, it's often not a prompt or fine-tuning issue. It's a topology issue. Some tasks benefit from centralized coordination, where a supervisor validates and routes decisions to prevent cascading mistakes. Others benefit from decentralized patterns, where specialists explore in parallel and exchange information peer-to-peer. Some tasks will break down in multi-agent settings, because sequential reasoning with strict dependencies is often more stable when handled by a single agent supported by tools. System design becomes the optimization step, not parameter count.

#### Taking the Next Step

If you want to deepen the system-level mindset and explore building AI agents more deeply, I go further into this system-level thinking and skill set approach in my book *AI Agents: The Definitive Guide* (O'Reilly).

This aligns with the early principles of collaborative scaling.[3] The process of increasing the number of agents doesn't behave like the process of increasing parameter count in a transformer. Neural scaling[4] is continuous; collaborative scaling is conditional. It grows, plateaus, and collapses depending on communication costs, memory constraints, and how much context each agent actually sees. This is why topologies matter: chains, trees, and different designs behave differently under load.

As you move toward production, system planning becomes essential. When I design AI systems or agentic systems, I think of the models, or agents, as dynamic, collaborative systems rather than isolated components. Every model size and model family has a distinct skill profile. If you build your system with that in mind, you'll not only get better results; you'll also save money.

2 Yubin Kim et al. "Towards a Science of Scaling Agent Systems" (2025).

3 Chen Qian et al. "Scaling Large Language Model-Based Multi-Agent Collaboration" (2025).

4 Jared Kaplan et al. "Scaling Laws for Neural Language Models" (2020).

# Conclusion

In this final chapter, you learned that the real frontier is no longer a single model. It's the space between models. The moment you connect components, you move from isolated capability to coordinated behavior, and every design choice begins to matter. Intelligence is becoming modular. Architecture becomes intent. Topology becomes a lever. Tooling becomes part of the cognitive loop rather than an accessory attached to the side.

You now understand that system performance doesn't emerge from scale alone. It comes from structure. Supervisor patterns, centralized planners, peer-to-peer specialists, and single-agent loops with tools all behave differently under pressure. Some amplify strengths. Some amplify failure. Coordination shapes outcome more than parameter count. The wiring determines whether a system stabilizes, improves, or collapses.

With that, you've reached the point where you see that thinking like a model practitioner is no longer enough. Thinking like a systems designer is the next step. Someone who treats architecture, memory, interaction, and evaluation as part of a system. Someone who understands that capability is not only in the model but in the space between models.

This Definitive Guide ends here, but your work in the era of experience and systems design is just beginning.

# Index


## A

AB-MCTS (Adaptive Branching Monte Carlo Tree Search), 232-238
  compared to MCTS, 233
  variants, 234-235
AC-CPC (action-conditioned contrastive predictive coding), 205-208
ACIs (agent-computer interfaces), 268-270
ACs (agent coordinators), 335
action-value function, 184
adaLN (adaptive layer normalization) blocks
  adaLN-single module, 110
  adaLN-Zero blocks, 104
  defined, 104
AdamW-style weight decay, 229
Adaptive Branching Monte Carlo Tree Search (see AB-MCTS)
adaptive input synthesis, 240
adaptive layer normalization blocks (see adaLN blocks)
AdaptThink, 285-287
  delta incentive for enforcing efficiency, 285-287
  modes, 285
agent coordinators (ACs), 335
agent reinforcement training (see ART)
agent-computer interfaces (ACIs), 268-270
agentic systems, 243
  (see also AI agents; RL)
  autonomy, 245
  context engineering, 255
  scaling, 339-340
  traditional workflow versus, 243
AI agents, 243-271
  autonomy, 244-246
  context engineering, 255-258
  human-in-the-loop, 261-268
  memory, 258-261
    global, 259-261
    lifelong, 259-261
    types of, 258-259
  multi-agent architectures, 249-254
  optimization, 273-299
  solving GitHub issues with coding agents, 268-270
  workflow comparison, 243
  workflow design, 246-254
ALE (Arcade Learning Environment), 203
amplitude, 152
anomaly detection, 33
AnomalyBERT, 55-58
ARIMA (autoregressive integrated moving average), 38
ART (agent reinforcement training), 276-285
  combining with RULER, 276, 284
  defined, 274
  rollouts, 279
aspect bucketing, 119
ASR (Automatic Speech Recognition), 159-160
attention mechanism, 5-7, 16-25
  attribution, 5
  bidirectional attention, 7
  cross-attention, 17
  FlashAttention, 20-21
  FlashAttention-2, 21-22
  FlashAttention-3, 23-25
  GQA, 19-20
  linear projection, 5

MHA, 6
MQA, 17-19
PagedAttention, 24
unidirectional attention, 7
attribution and attribution matrix, 5
audio domain, 149-182
modeling in various application domains, 159-160
structure of audio data, 151-158
amplitude, 152
bit depth, 152
Fourier transform, 153
frequency domain, 153
mel spectrograms, 155
Nyquist theorem, 152
perceptual scaling, 155
phase, 155-158
quantization, 152-153
reconstruction, 155-158
sampling rate, 152
sound, defined, 151
spectrograms, 154
STFT, 154
vocoders, 155-158
waveforms, 151-152
transformer architectures, 160-181
audio foundation models, 163-174
MusicGen, 177-181
SAM Audio, 174-176
speech transformers, 160-163
audio event detection, 160
audio foundation models, 163-174
defined, 163
Kimi-Audio, 168-174
Qwen2-Audio, 164-168
audio reasoning models, 327
audio segmentation, 160
AURA, 174
autocorrelation, 28-29
Automatic Speech Recognition (ASR), 159-160
autoregressive integrated moving average (ARIMA), 38
autoregressive models, 98, 190
average latency, 316


## B


batch (offline) reinforcement learning, 185, 196
behavior policy, 187
Bellman equation, 185
BERT (bidirectional encoder representations from transformers), 3, 55
bidirectional attention, 7
bit depth, 152
bootstrapping
errors, 190
temporal difference learning, 188
BPE (byte-pair encoding), 35
broadcast method, 298


## C


capacity testing, 313
causal (unidirectional) attention, 7
chain-of-thought (CoT) reasoning, 214, 216
channel independence, 43-44
channel mixing, 43
character-level tokenization, 2
checkpoint engine, 297-298
chromagrams, 179
Chronos, 38-42
fine-tuning, 40-42
tokenizing time series, 36-38
Clarke, Arthur C., x
classification
audio, 160
image, 60, 68
time series, 33
classifier-free guidance, 101-102
CLIP (Contrastive Language-Image Pretraining), 73
closed source models
compared to open source, 302
deployment, 302-303
CNNs (convolutional neural networks), 59, 61, 66, 200
code generation
Pass@k metric, 215
RethinkMCTS framework, 238-239
S* framework, 239-240
coding models
building applications with, 308-312
internal network fallacy, 309
sandboxing, 309, 311
cointegration, 29
cold boots, 116
cold-start SFT, 217
collaborative scaling, 340
Compound Loss, 65
ConceptAttention, 123-126

confidence bands, 29
context engineering, 255-258
context windows
  defined, 9
  longer, with better performance, 9-16
Contrastive Language-Image Pretraining (CLIP), 73
convolutional neural networks (CNNs), 59, 61, 66, 200
correlograms, 28
CoT (chain-of-thought) reasoning, 214, 216
CPC (contrastive predictive coding), 208
credit assignment problem, 216
cross-attention
  defined, 17
  DiTs, 103
  PIXART-α, 109
cross-correlation (cross covariances), 30
Cross-Entropy Loss, 36-38

## D

DALL·E, 97-98
data augmentation, 65-66
DDPMs (denoising diffusion probabilistic models), 98
decision transformer (see DT)
decoder-only models
  coding models, 308-312
  defined, 303
  deployment, 305-308
  increasing batch size, 306
  runtime engineering, 306-307
  security considerations, 307-308
decoding strategies, 230
decomposition, 31
deep reinforcement learning (DRL), 200
DeepMind, Google, 340
DeepSeek-R1, 217-220
DeepSeek-R1-Zero, 214-215
delta (δ)
  incentive for enforcing efficiency, 285-287
  tuning, 287
Deno, 309
denoising diffusion probabilistic models (DDPMs), 98
dense prompts, 72
DenseMixer, 323
deployment, 301-329
  architecture, 303-304
  decoder-only models
    coding models, 308-312
    runtime engineering, 306-307
    security considerations, 307-308
  evaluating deployments in production, 313-320
    categories of evaluation, 313
    cost efficiency and hardware comparison, 314-320
    performance dimensions, 313
  open source versus closed source models, 302-303
  quantization, 320-326
  stepwise process, 302
  TTL in vision-language models, 326-328
Designing an Intelligence (Silver and Sutton), 214
DFT (discrete Fourier transform), 153
Dice Focal Loss, 66
Dice Loss, 65
DiffiT (Diffusion Vision Transformers for Image Generation), 120-123
diffusion, 97
  (see also DiTs)
  defined, 97
  latent, 130-131
diffusion models, 99-105
  (see also DiTs)
  classifier-free guidance in, 101-102
  defined, 98
  image captioning, 123
  scalable diffusion models with transformers, 102-105
diffusion transformers (see DiTs; image generation)
Diffusion Vision Transformers for Image Generation (DiffiT), 120-123
direct preference optimization (DPO), 164
Direction and Magnitude Decomposition (DoRA), 321
discrete Fourier transform (DFT), 153
discriminators, 98
DiTs (diffusion transformers), 97-127
  adaLN blocks, 104
  adaLN-Zero blocks, 104
  configurations of, 105
  cross-attention blocks, 103
  DiffiT, 120-123
  diffusion, defined, 97

generating images with, 106-108
in-context conditioning, 103
interpretable features with, 123-126
PixArt-Σ, 111-120
generating images with, 115-116
memory efficiency, 116-119
training, 119
PIXART-α, 108-110
video generation, 129-148
DoRA (Direction and Magnitude Decomposition), 321
DPO (direct preference optimization), 164
DRL (deep reinforcement learning), 200
dropout, 65
DT (decision transformer), 190-196
autoregressive approach, 190
embeddings, 191
environments, 192
interactions, 194
online, 196-199
dynamic mode switching, 220
dynamic NTK interpolation, 15


## E


early stopping, 65
EDA (exploratory data analysis), 31
embeddings
absolute and relative, 9
decision transformer, 191, 192
overview of, 3-5
vision tasks, 61-64
encoder-decoder structure
decoder-only models, 303, 305-308
encoder-decoder models, 304
encoder-only models, 304
overview of, 7-8
end-to-end latency, 313
environmental models, 186
episodic memory, 259
error rate, 313
event forecasting, 33
experience-driven learning
explicit configuration, 308
exploration-exploitation trade-off, 233
exploratory data analysis (EDA), 31


## F


FID (Fréchet Inception Distance), 123
fine-tuning, 216
cold-start supervised, 217
supervised, 216
firewalls, 308
fixed branching factors, 233
FlashAttention, 20-21
FlashAttention-2, 21-22
FlashAttention-3, 23-25
Focal Loss, 65
Fourier transform, 153
discrete, 153
inverse short-time, 157-158
short-time, 154
Fréchet Inception Distance (FID), 123
frequency domain, 153


## G


GANs (Generative Adversarial Networks), 97
GCP (Google Cloud), 226
GEMM (general matrix-matrix multiplication), 24
GenAI Prices, 318
generators, 98
GitHub, solving issues with coding agents, 268-270
global (shared) memory, 259-261
GN (group normalization), 122
gold answer, 284
Google Cloud (GCP), 226
Google DeepMind, 340
Google Research, 340
GQA (grouped-query attention), 19-20, 306
Griffin-Lim Algorithm, 155
group normalization (GN), 122
GRPO (Group Relative Policy Optimization)
framework, 214, 218-220
ART, 274, 281-283
intuition, 283
RULER, 275
Gymnasium, 192


## H


Hadamard transforms and Hadamard matrix, 25
HalfCheetah, 192
Hanning windows, 154
haystack problem, 14
HER (hindsight return relabeling), 199
heterogeneous time series, 49
hierarchical reinforcement learning, 189

HITL (human-in-the-loop), 261-268
  common patterns for, 261-268
  end node, 262
  flow, 262
  human node, 262, 264
  importance of, 261
  model node, 262-264
Hopper architecture, 23

## I

I2V (image-to-video) models, 129
image captioning, 123
image classification
  defined, 60
  Swin Transformer V2, 68-72
image generation, 97-127
  combining SAM 3 with MLLMs for segmentation, 333-339
  diffusion models, 99-105
    classifier-free guidance in, 101-102
    scalable, with transformers, 102-105
  DiTs, 102-127
    DiffiT, 120-123
    generating images with, 106-108
    interpretable features with, 123-126
    PixArt-Σ, 111-120
    PIXART-α, 108-110
  inference engines, 116
  overview of, 98
image-to-video (I2V) models, 129
in-context conditioning, 103
inference engines, 116, 297
inference latency, 19
instance segmentation, 60
instruction tuning, 216
inter-token latency, 313
interleaved attention layers without positional embeddings (iRoPE), 14
interleaved spatiotemporal attention, 137
internal network fallacy, 309
intrinsic dimensionality, 14
IoU (Intersection over Union), 65, 91
iRoPE (interleaved attention layers without positional embeddings), 14
ISTFT (inverse short-time Fourier transform), 157-158

## K

Kimi K2, 227-230
  inference with, 229-230
  Muon optimizer, 228-229
Kimi-Audio, 168-174
KL (Kullback-Leibler) divergence, 100-101, 218-220, 281
KV (key-value) caching, 19, 303, 305
KV (key-value) token compression, 111-114

## L

L2 regularization (weight decay), 65
LaDiC (Latent Diffusion-based Captioner), 123
Lag-Llama, 34
LALMs (large audio-language models), 150, 163
  (see also audio foundation models)
LangGraph
  autonomy, 244
  HITL feedback mechanism, 262-268
  versus state machines, 251
Language Agent Tree Search (LATS), 249
large audio-language models (LALMs), 150, 163
  (see also audio foundation models)
large language models (see LLMs)
late fusion, 138
latent diffusion, 130-131
Latent Diffusion-based Captioner (LaDiC), 123
LATS (Language Agent Tree Search), 249
Latte, 131, 137-140
LDMs (latent diffusion models), 130, 304
  (see also video generation)
leakage, preventing, 284
least privilege access, 308
length adjustment, 55
Librosa, 154
lifelong memory, 259-261
linear projection, 5
LLaVA, 110
LLMs (large language models)
  evaluating deployments in production, 313-320
  optimization, 273-299
  reasoning models, 1
LLMTime, 35
local memory, 259
LoftQ (Quantization-Aware LoRA Initialization), 322
long sequence time-series forecasting (LSTF), 27

long short-term memory (LSTM) networks, 1
long-term forecasting, 32
long-term memory, 257
look-ahead bias, 32
loop hierarchy, 22
LoRA (Low-Rank Adaptation), 321
LoRAX, 324
loss functions, 65
LSTF (long sequence time-series forecasting), 27
LSTM (long short-term memory) networks, 1
LTX-Video, 132-135

## M

Markov chains, 99
Markov decision processes (MDPs), 183-184
Mars science laboratory (MSL) dataset, 57
MAS (multi-agent systems)
  compared to single-agent systems, 245
  defined, 245
  workflow design, 249-254
mask decoders, 73
masked multi-head self-attention, 7
max-entropy sequence modeling, 196
MCP (Model Context Protocol), 257
MCTS (Monte Carlo Tree Search), 231-239
  Adaptive Branching, 232-238
  RethinkMCTS framework, 238-239
(M)DETR (modulated end-to-end detector), 89
MDPs (Markov decision processes), 183-184
meeting transcription, 168-174
mel spectrograms, 149, 155
MGF (Motion-guidance Fuser), 140
MHA (multi-head attention)
  compared to GQA, 19
  compared to MQA, 17
  masked multi-head self-attention, 7
  overview of, 6
mini SWE-agent, 269
mixture of expert (MoE) models, 223-224
MLLMs (multimodal large language models), 333-339
MMATTN (multimodal attention) layers, 124-126
Model Context Protocol (MCP), 257
model-based reinforcement learning, 186, 189
model-free reinforcement learning, 186
modulated end-to-end detector ((M)DETR), 89
MoE (mixture of expert) models, 223-224
Monte Carlo methods, 100, 188
  (see also MCTS)
Monte Carlo Tree Search (see MCTS)
Motion-guidance Fuser (MGF), 140
MQA (multi-query attention), 17-19
MSL (Mars science laboratory) dataset, 57
multi-agent systems (see MAS)
multi-head attention (see MHA)
multi-query attention (MQA), 17-19
multi-variate time series, 43
multicollinearity, 30
multimodal attention (MMATTN) layers, 124-126
multimodal large language models (MLLMs), 333-339
music composition, 177-181
music generation, 159
MusicGen, 177-181

## N

network isolation, 308
neural tangent kernel (see NTK)
Newton-Schulz iterations, 228
NLP (natural language processing), 1
NLU (natural language understanding), 1
non-Markovian processes, 190
non-stationarity, 189
normalized fractional coordinates, 135
Novita AI, 226
NTK (neural tangent kernel), 14-15
  dynamic NTK interpolation, 15
  NTK-aware interpolation, 14
  NTK-by-parts interpolation, 15
nucleus (top-p) sampling, 230
Nyquist theorem, 152

## O

ODT (online decision transformer), 196-199
off-policy distillation, 223
off-policy reinforcement learning, 186-187
offline (batch) reinforcement learning, 185, 196
on-policy distillation, 223
on-policy reinforcement learning, 186-187
online decision transformer (ODT), 196-199
online learning paradigm, 185
open source models
  compared to closed source, 302
  deployment, 302-303
OpenCV, 75

optimization, 273-299
AdaptThink, 285-287
checkpoint engine, 297-298
rLLM, 287-297
training-time optimization, 274-285
ART, 276-285
RULER, 275-276
ORMs (outcome reward models), 216
overthinking, 284

## P

P2P (peer-to-peer), 298
PagedAttention, 24
panoptic segmentation, 60
parallel attention head splitting, 138
partial autocorrelation, 28-29
Pass@k metric, 215
patching
DiTs, 103
patchifying for video generation, 134
time series, 34
vision tasks, 61
PatchTST, 34, 43-48
channel independence, 43-44
fine-tuning, 45-48
PCS (promptable concept segmentation), 89
peak noise, 55
peer-to-peer (P2P), 298
perceptual scaling, 155
period independence, 31
perplexity (PPL), 13-15
PEs (positional embeddings)
absolute and relative, 9
decision transformer, 192
overview of, 3-5
phase, 155-158
PI (position interpolation), 9, 12-13
pingpong scheduling, 23
PixArt-Σ, 111-120
generating images with, 115-116
memory efficiency, 116-119
training, 119
PIXART-α, 108-110
pixel dependency learning, 108
planning, reasoning, and code generation, 213-241
DeepSeek-R1, 217-220
era of experience, 214-216
Kimi K2, 227-230
inference with, 229-230
Muon optimizer, 228-229
learning to reason, 216-217
Qwen3, 220-225
Qwen3-Coder, 225-227
scaling reasoning at test-time, 230-240
AB-MCTS, 232-238
RethinkMCTS framework, 238-239
S* framework, 239-240
pointwise, defined, 7
policy oscillation (policy chattering), 187
position interpolation (PI), 9, 12-13
positional embeddings (see embeddings; PEs)
PPL (perplexity), 13-15
PPO (proximal policy optimization), 216
prefix reuse, 306
PRMs (process reward models), 217, 230
probabilistic inference, 190
probabilistic time series forecasting, 27
procedural memory, 259
prompt engineering, 255
promptable concept segmentation (PCS), 89
proximal policy optimization (PPO), 216
PVS (promptable visual segmentation), 83, 89

## Q

Q-learning, 187
QLoRA (Quantized LoRA), 321
quantization, 152-153, 320-326
Quantization-Aware LoRA Initialization (LoftQ), 322
Qwen Code, 225
Qwen2-Audio, 164-168
Qwen3, 220-225
Qwen3-32B, 314-320
Qwen3-Coder, 225-227

## R

RAG (Retrieval-Augmented Generation), 244, 257
ReAct pattern, 247-249
reasoning, 1
chain-of-thought, 214, 216
DeepSeek-R1, 217-220
Kimi K2, 227-230
learning to reason, 216-217
Qwen3, 220-225
Qwen3-Coder, 225-227
reconstruction, 155-158

Rectified Linear Unit (ReLU), 67
rectified-flow models, 134
recurrence, defined, 4
regularization, 65
reinforcement learning (see RL)
relative universal LLM-elicited rewards (see RULER)
ReLU (Rectified Linear Unit), 67
reparameterization, 99-100, 110
replay buffers, 196
res-post-norm configuration, 66
residual uncertainty, 134
residual vector quantization (RVQ), 177
ResNet, 66
RethinkMCTS framework, 238-239
Retrieval-Augmented Generation (RAG), 244, 257
returns-to-go, 191
reward hacking, 218
RL (reinforcement learning), 1, 183-211, 213-214
  (see also planning, reasoning, and code generation)
  deep, 200
  defined, 183
  foundational concepts in, 185-188
    model-based and model-free RL, 186
    on-policy versus off-policy RL, 186-187
    online and offline RL, 185
    temporal difference learning, 188
    world models, 188
  goal of, 185
  hierarchical, 189
  model-based, 189
  progressing from pretraining to reasoning, 216-217
  reward hacking, 218
  rLLM, 287-297
  training-time optimization, 274-285
    ART, 276-285
    RULER, 275-276
  transformers in, 189-210
    decision transformer, 190-196
    online decision transformer, 196-199
    STORM, 199-204
    TWISTER, 204-210
rLLM (RL for language agents), 287-297
  building evaluator engine, 291
  components of, 287
  evaluator arguments, 290
  preprocessing, 288
  registering and sharding, 289
  running evaluation, 294
  training loop, 295
Road Runner, 203-204
rollouts, 279
RoPE (rotary positional embedding)
  context windows, 9-12
  video generation, 135
RULER (relative universal LLM-elicited rewards), 275-276
  combining with ART, 276-277, 284
  customizing, 276
  defined, 274
  key insights, 275
  training loop, 275
runaway loops, 254
RVQ (residual vector quantization), 177

## S

S* framework, 239-240
saliency maps, 123
SAM (Segment Anything Model), 72-82, 88
  fine-tuning, 76-82
    preparing data, 76-78
    preparing hyperparameter tuning, 78-79
    setting up model and logging, 78
    using the best hyperparameters, 80-82
  variations of, 75
SAM 2 (Segment Anything Model 2), 83-88
  model sizes, 84
  promptable visual segmentation, 83
  segmentation, 85-87
SAM 3 (Segment Anything Model 3), 88-94
  combining with MLLMs, 333-339
  components of, 90
  segmentation, 93-94
SAM Audio (Segment Anything Model Audio), 174-176
SAM2 Studio, 88
sample inefficiency, 187
sampling rate, 152
sampling strategies, 230
sat (SwissArmyTransformer), 146
scaled cosine attention, 67
scaling law, 48, 130
SDXL (Stable Diffusion XL) models, 119
seasonality, 31

secure water treatment (SWaT) dataset, 57
Segment Anything Model (see SAM; SAM 2; SAM 3; SAM Audio)
segmentation
  audio, 160
  instance, 60
  labeling tools, 72
  panoptic, 60
  SAM, 72-82
  SAM 2, 83-88
  SAM 3, 88-94, 333-339
  SAM Audio, 174-176
  semantic, 60
semantic image segmentation, 60
semantic memory, 259
sequential attention decomposition, 138
SER (Speech Emotion Recognition), 159
server machine dataset, 58
SFT (supervised fine-tuning), 216-217
Share-Captioner, 111
shared (global) memory, 259-261
short-term forecasting, 32
short-term memory, 256
short-time Fourier transform (STFT), 154
Silver, David, 214
SimpleTuner, 119
sliding window perplexity, 14
SMAP (soil moisture active passive) dataset, 57
soak testing, 314
soft replacement, 55
Softmax function, 14, 24
soil moisture active passive (SMAP) dataset, 57
sound, defined, 151
  (see also audio domain)
sparse prompts, 72
spatial blocks, 137
Spatial-Temporal DiT (ST-DiT), 141
spatiotemporal forecasting, 33
speaker diarization, 159, 169
speaker identification, 159
spectrograms, 149, 154
  consistent, 155
  mel, 149, 155
Speech Emotion Recognition (SER), 159
speech transformers, 160-163
speech-native agents, 174
spurious correlations, 29
ST (Speech Translation), 159
ST-DiT (Spatial-Temporal DiT), 141
Stable Diffusion XL (SDXL) models, 119
state machines, 251
stationarity, 31
steepest descent under the spectral norm, 228
Step-Audio-R1, 327
STFT (short-time Fourier transform), 154
STORM (stochastic transformer-based world models for reinforcement learning), 199-204
stress testing, 314
structured output, 257
subword tokenization, 3
summarization, 257
supervised fine-tuning (SFT), 216-217
Sutton, Richard S., 214
SWaT (secure water treatment) dataset, 57
SWE-agent
  ACIs, 268-270
  mini SWE-agent, 269
sweet spot, 313
Swin Transformer V2, 66-72
  image classification, 68-72
  res-post-norm configuration, 66
  scaled cosine attention, 67
Swish activation, 120
SwissArmyTransformer (sat), 146
system prompts, 256
systematic bias, 32
systems-level optimization, 297-298
systems-level thinking, 331-341
  combining SAM 3 with MLLMs, 333-339
  research into, 331-333
  scaling agentic systems, 339-340

## T

T2I (text-to-image) generative models, 97, 108
T2V (text-to-video) models, 129
target policy, 187
TD (temporal difference) learning, 188
TE (Trajectory Extractor), 140
temperature, 230
temporal blocks, 137
temporal point processes (TPP), 33
Test-Time Low-Rank Adaptation (TTL), 326-328
text-image alignment, 108
text-to-image (T2I) generative models, 97, 108
text-to-speech (TTS), 159
text-to-video (T2V) models, 129

textual surrogate reasoning problem, 327
theta (θ), 100
thinking budget, 221-223
Thompson sampling, 233
tiling, 20
time series, 27-58, 304
  AnomalyBERT, 55-58
  Chronos, 38-42
  classification, 33
  forecasting, 32
  LSTF, 27
  modeling in various application domains, 32-33
  PatchTST, 43-48
  preparing datasets, 32
  probabilistic time series forecasting, 27
  properties of, 28-31
    autocorrelation and partial autocorrelation, 28-29
    cointegration, 29
    cross-correlation, 30
    stationarity, 31
    trends and seasonality, 31
  TimesFM, 48-55
  tokenizing, 33-37
time to first token (TTFT), 313
TimesFM, 48-55, 304
TMSA (time-dependent multi-head self-attention), 120
tokenization, 2-5
  audio, 168
  character-level, 2
  defined, 2
  detokenization, 168
  KV token compression, 111-114
  PEs, 3-5
  subword, 3
  time series data, 33-37
  tokenizers, defined, 2-3
  tokens, 3-5
  vision tasks, 61-64
  word-level, 3
top-k sampling, 230
top-p (nucleus) sampling, 230
Tora, 131, 140-148
TPP (temporal point processes), 33
TPS (tokens per second), 313
training engines, 297
training-time optimization, 274-285
  ART, 276-285
  RULER, 275-276
Trajectory Extractor (TE), 140
transformer state-space models (TSSMs), 205
Transformer-based World model wIth contraSTivE Representations (TWISTER), 204-210
transformers, 1-25
  AI agents, 243-271
  in audio domain, 149-182
  components of, 2-8
    attention mechanism, 5-7
    encoder-decoder structure, 7-8
    tokenization, 2-5
  deployment, 301-329
  enhancements in design, 9-25
    attention mechanism variations, 16-25
    context windows, 9-16
  for image generation, 97-127
  optimization, 273-299
  for planning, reasoning, and code generation, 213-241
  reinforcement learning, 183-211
  for time series, 27-58, 304
  versatility and elegance of, ix
  for video generation, 129-148
  for vision tasks, 59-95
transition functions, 186
tree search algorithms, 230-239, 249
trends, 31
trimming, 258
TSSMs (transformer state-space models), 205
TTFT (time to first token), 313
TTL (Test-Time Low-Rank Adaptation), 326-328
TTS (text-to-speech), 159
TWISTER (Transformer-based World model wIth contraSTivE Representations), 204-210

## U

U-Nets, 98, 102
UCT (Upper Confidence Bound for Trees), 231
unidirectional (causal) attention, 7
uniform replacement, 55
update scale adjustment, 229
user prompts, 256

## V

VAEs (variational autoencoders)
  image generation, 106

STORM, 199
training DiTs, 102
video generation, 132-134, 141
value functions, 184-185
VGG, 66
video generation, 129-148
latent diffusion, 130-131
Latte, 137-140
LTX-Video, 132-135
Tora, 140-148
vision tasks, 59-95
embeddings and tokenization, 61-64
improving robustness and effectiveness of, 64-66
SAM, 72-82
SAM 2, 83-88
SAM 3, 88-94
Swin Transformer V2, 66-72
vision transformers (see vision tasks; ViTs)
vision-language models, 110, 326-328
ViTs (vision transformers), 59, 131
(see also video generation)
compared to DiTs, 103
patch embedding, 61
vLLM, 306
vocoders, 155-158

## W

WADI (water distribution) dataset, 57
Walker2D, 192
warm boots, 116
waveforms, 151-152
weak-to-strong training, 111
weight decay (L2 regularization), 65
Whisper, 150, 160-163
word-level tokenization, 3
world models, in reinforcement learning, 188

## X

Xie, Saining, 102

## Y

YaRN (Yet another RoPE extensioN method), 9, 12, 15

## About the Author

**Nicole Koenigstein** is an AI researcher and practitioner in agentic systems, working across research, consulting, teaching, and direct system implementation to build reliable, production-ready AI systems. Her work focuses on multi-agent architectures, evaluation, safety, and long-term system behavior.

She served as an external evaluator for a European Commission AI Grand Challenge and has advised IOSCO on generative AI in regulated environments. She also serves on advisory boards for leading AI and quantitative finance conferences. Nicole regularly delivers invited talks and technical workshops across academia, industry, and international events.

She is the author of *Math for Machine Learning* and *Transformers in Action* with Manning Publications. Her forthcoming books, *Transformers: The Definitive Guide—Applications Beyond NLP* and *AI Agents: The Definitive Guide*, will be published by O'Reilly Media.

## Colophon

The animal on the cover of *Transformers: The Definitive Guide* is a wattled jacana (*Jacana jacana*).

The wattled jacana is a tropical bird known for its remarkable ability to walk on floating vegetation, thanks to its long toes and claws that distribute its weight over lily pads and other aquatic plants. Native to Central and South America, this bird inhabits freshwater wetlands, marshes, and ponds, where it forages in slow-moving water.

The wattled jacana is recognized by its chestnut-brown body, black head and neck, and yellow flight feathers. It sports a red frontal shield and wattle on the face, which give the species its name. The bird can live up to seven years in the wild.

According to the IUCN Red List, the wattled jacana's conservation status is of Least Concern. Its adaptability to various wetland habitats and its widespread distribution contribute to its secure conservation status, although wetland drainage and habitat degradation can pose localized threats.

Many of the animals on O'Reilly covers are endangered; all of them are important to the world.

The cover illustration is by José Marzan Jr., based on a black-and-white engraving from *Riverside Natural History*. The series design is by Edie Freedman, Ellie Volckhausen, and Karen Montgomery. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.

O'REILLY®

# Learn from experts. Become one yourself.

60,000+ titles | Live events with experts | Role-based courses
Interactive learning | Certification preparation | Verifiable skills

**Try the O'Reilly learning platform free for 10 days.**

©2026 O'Reilly Media, Inc. O'Reilly is a registered trademark of O'Reilly Media, Inc. 1035600_7x9.1875